

分治法回顾

- 4.1 一般方法
 - 掌握分治法适用的问题特点、求解思想和计算时间等基本内容。能掌握分治法求解问题的一般过程。
- 4.2 二分查找
 - 二分查找成功、不成功情况下最好、最坏、平均时间复杂度
- 4.3 归并排序
 - 掌握分治算法的一般设计思想，算法正确性证明、时间复杂度分析和求解的一般方法。
- 4.4 快速排序
 - 随机化快速排序



分治法回顾

- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
 - 深入理解分治法适用的问题特征，掌握复杂问题求解办法。分析分治法时间复杂度的影响因素，掌握分治算法的优化技巧。掌握分治算法优化的一般原理和策略。
- 4.8 递归式分析方法
 - 掌握递归式求解的一般方法

能够识别出适合分治法的可计算性问题、独立设计算法和分析算法复杂度。

主定理

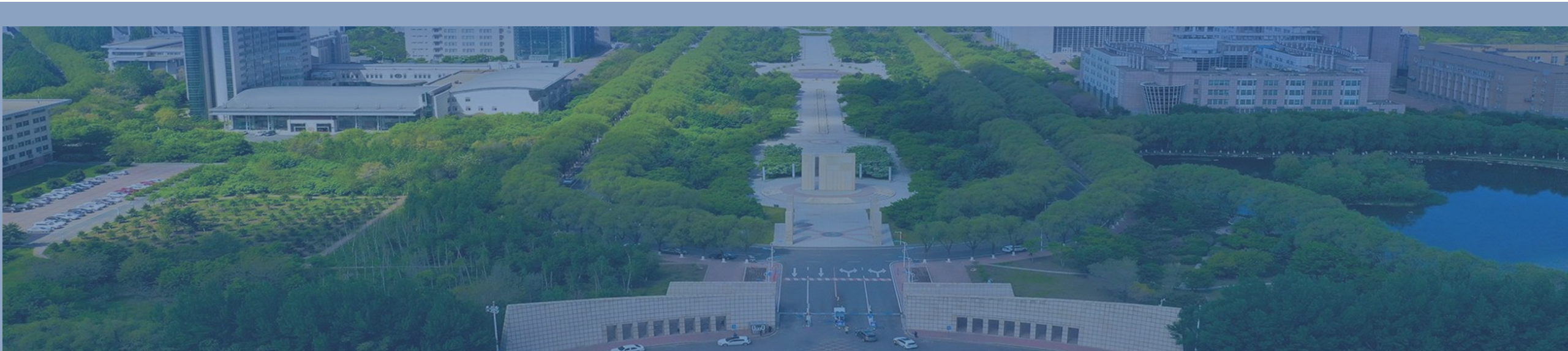
- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

- 例: $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$
 - $a=b=2$, $\log_b a = 1$
 - $f(n) = n \log n = \Omega(n)$
 - $\nexists c > 0$, 使得 $f(n) = \Omega(n^{1+\epsilon})$
 - 不符合任何条件



吉林大学



第五章 贪心方法



目录

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 货郎担问题
- 5.5 小结



目录

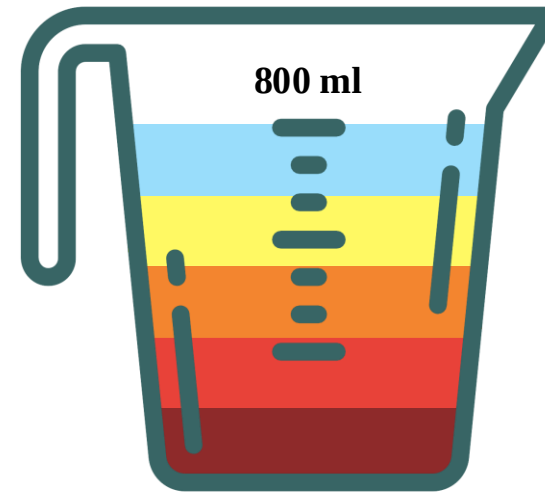
- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 货郎担问题
- 5.5 小结



调制饮品比赛

- 参赛者拥有容量为**800ml**的杯子，可任选不超过体积上限的饮料进行混合
- 调制饮品价格为各所使用饮料的价格之和，所得饮品价格之和最高者获胜

饮料	价格(元)	体积(ml)
 苏打水	60	600
 汽水	10	250
 橙汁	36	200
 苹果汁	16	100
 西瓜汁	45	300



思考：如何使调制的饮品价格最高？

背包问题

- 输入:

- n 个物品组成的集合 O , 每个物品有两个属性 v_i 和 p_i , 分别表示体积和价格
- 背包容量为 C

选取物品的比例

x_i 只能取 0 或 1 时变为
0-1 背包问题

- 输出: 求解一个解决方案 $S = \{x_i \mid 1 \leq i \leq n, 0 \leq x_i \leq 1\}$, 使得

$$\max \sum_{x_i \in S} x_i \cdot p_i$$

优化目标

$$s. t. \sum_{x_i \in S} x_i \cdot v_i \leq C$$

约束条件

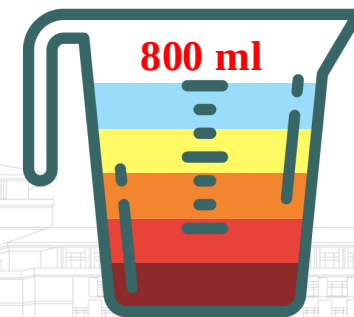


贪心策略

- 价格优先
 - 600ml 苏打水 + 200 ml 西瓜汁
 - 价格：60+30=90
- 体积优先（降序）
 - 100ml 苹果汁+200ml橙汁+300ml 西瓜汁+200ml 汽水
 - 价格：16+36+45+8=105

饮料	价格 (元)	体积 (ml)
 苏打水	60	600
 汽水	10	250
 橙汁	36	200
 苹果汁	16	100
 西瓜汁	45	300

思考：是否存在更好的贪心策略？



贪心策略

- **最高性价比**优先

- 性价比 = 价格/体积
- 优先选择**高性价比**饮料全部装入，尽可能装满杯子

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 苏打水	60	600	0.10
 汽水	10	250	0.04
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15

算法实例

- 最高性价比优先

饮料	价格 (元)	体积 (ml)	总价格 (元)
 橙汁	36	200	

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15
 苏打水	60	600	0.10
 汽水	10	250	0.04



算法实例

- 最高性价比优先

饮料	价格 (元)	体积 (ml)	总价格 (元)
 橙汁	36	200	
 苹果汁	16	100	

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15
 苏打水	60	600	0.10
 汽水	10	250	0.04



算法实例

- 最高性价比优先

饮料	价格 (元)	体积 (ml)	总价格 (元)
 橙汁	36	200	
 苹果汁	16	100	
 西瓜汁	45	300	

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15
 苏打水	60	600	0.10
 汽水	10	250	0.04

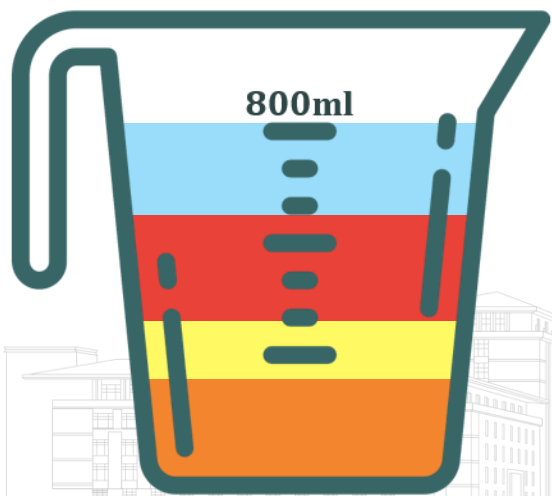


算法实例

• 最高性价比优先

饮料	价格 (元)	体积 (ml)	总价格 (元)
 橙汁	36	200	
 苹果汁	16	100	
 西瓜汁	45	300	
 苏打水	20	200	

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15
 苏打水	60	600	0.10
 汽水	10	250	0.04

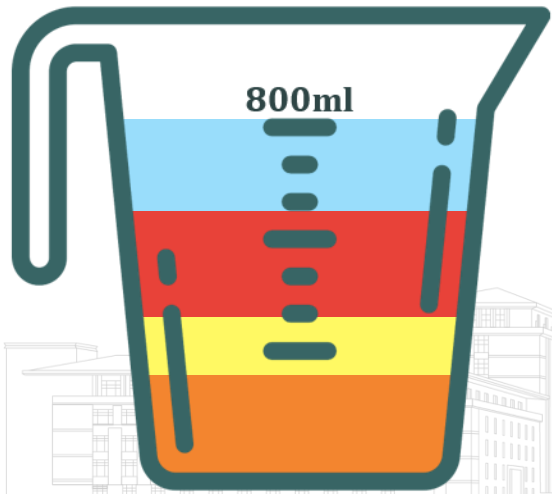


算法实例

• 最高性价比优先

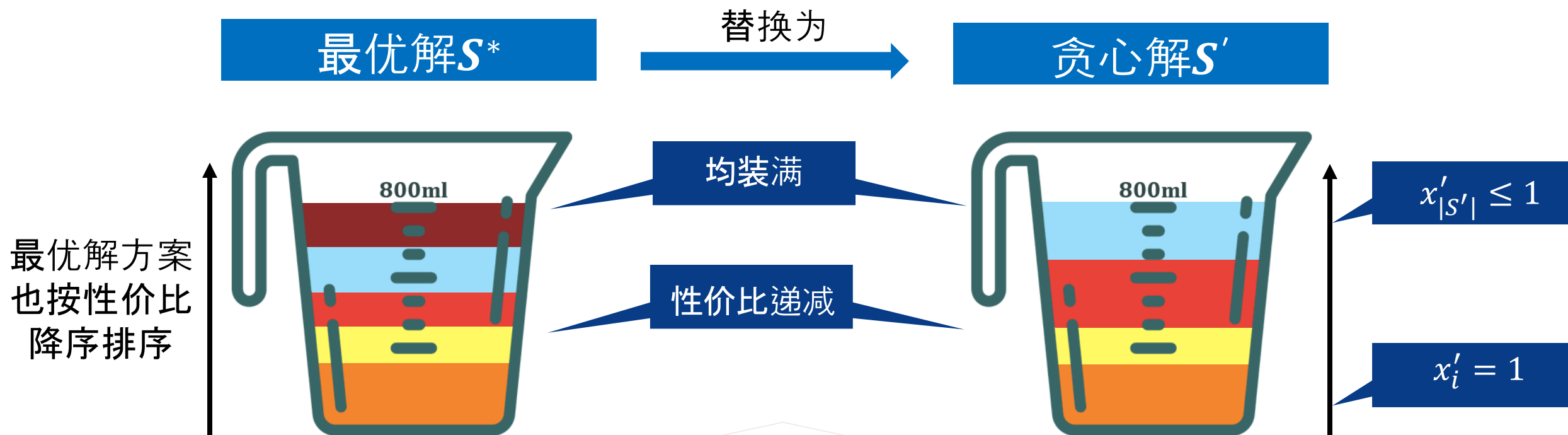
饮料	价格 (元)	体积 (ml)	总价格 (元)
 橙汁	36	200	117
 苹果汁	16	100	
 西瓜汁	45	300	
 苏打水	20	200	

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15
 苏打水	60	600	0.10
 汽水	10	250	0.04



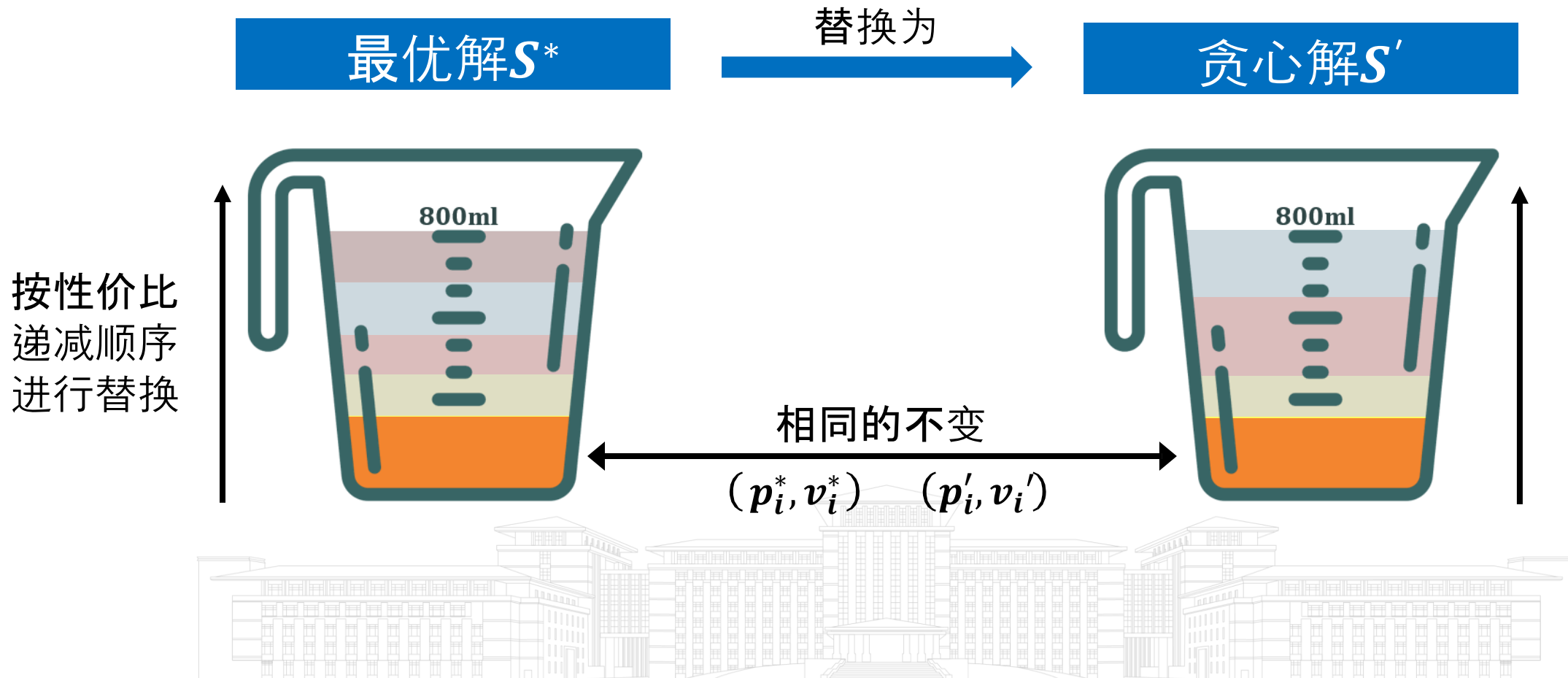
正确性证明

- 最高性价比优先
- 证明：贪心解不劣于最优解



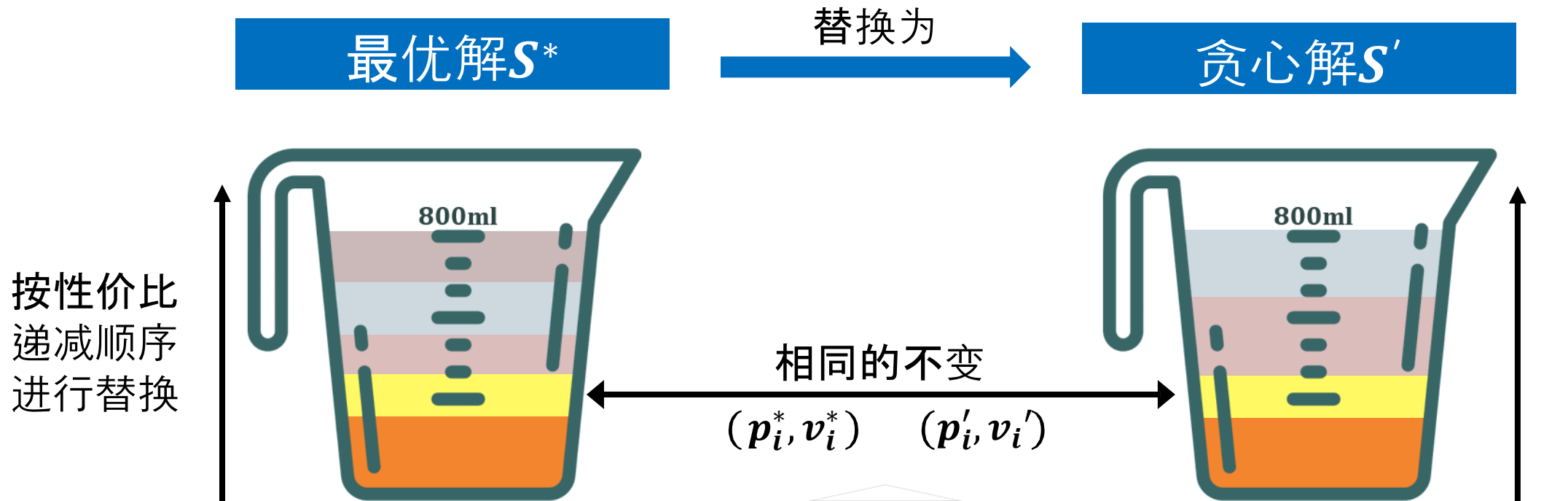
正确性证明

- 最高性价比优先
- 证明：贪心解不劣于最优解



正确性证明

- 最高性价比优先
- 证明：贪心解不劣于最优解



正确性证明

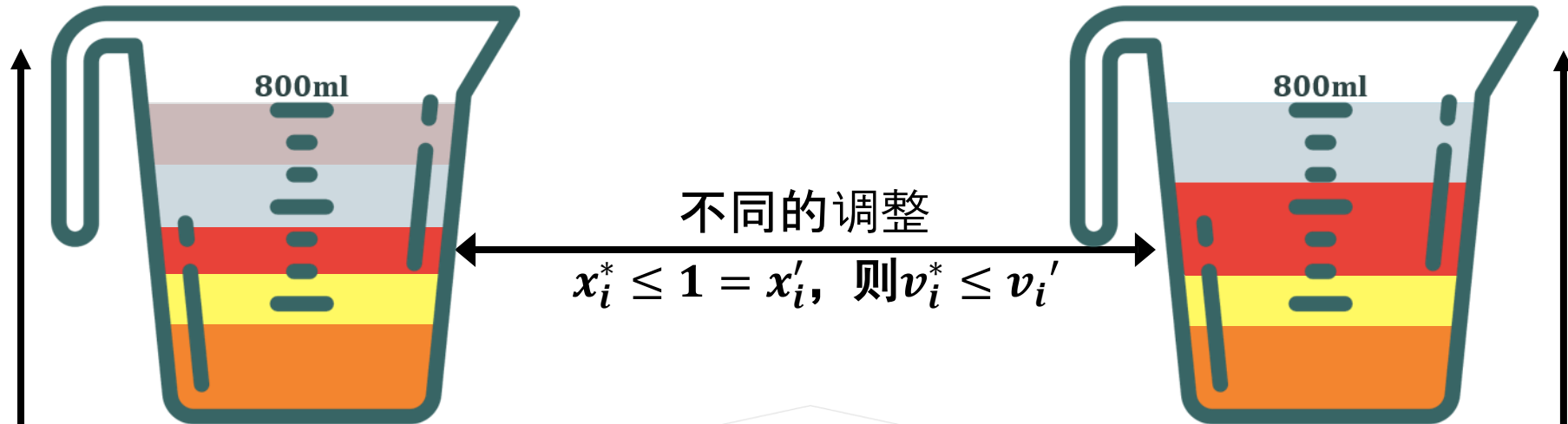
- 最高性价比优先
- 证明：贪心解不劣于最优解

最优解 S^*

替换为

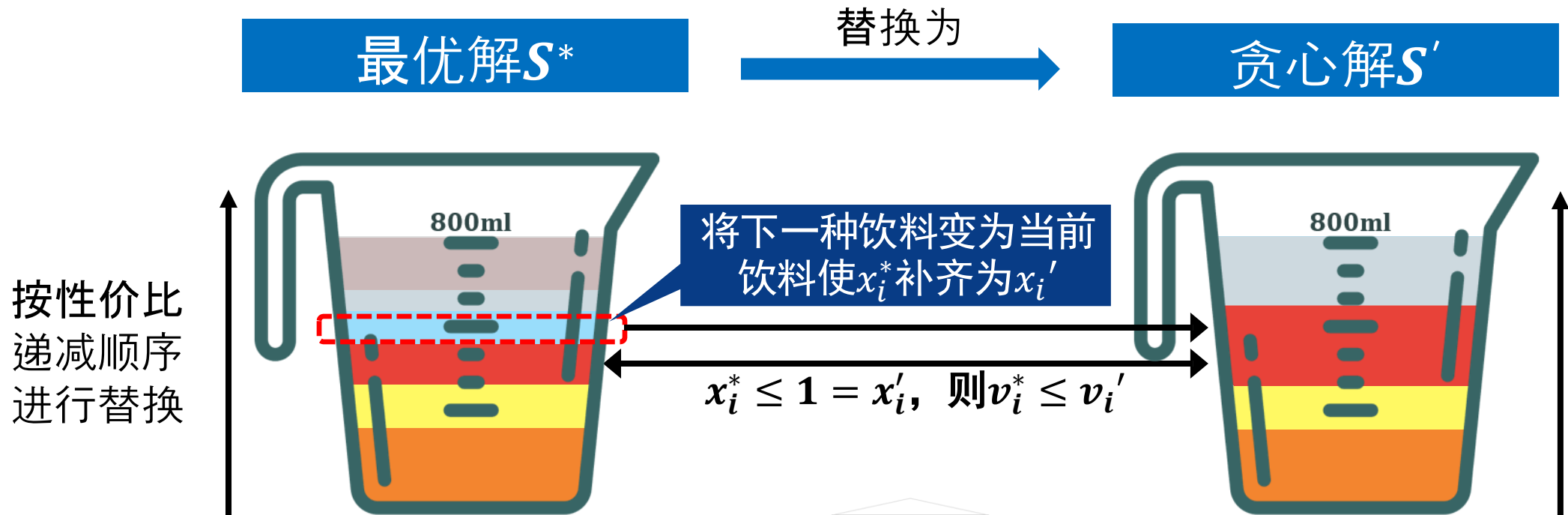
贪心解 S'

按性价比
递减顺序
进行替换



正确性证明

- 最高性价比优先
- 证明：贪心解不劣于最优解



正确性证明

- 最高性价比优先
- 证明：贪心解不劣于最优解

最优解 S^*

替换为

贪心解 S'

按性价比
递减顺序
进行替换



性价比递减： $p'_i/v'_i \geq p^*_{i+1}/v^*_{i+1}$
则替换后总价格不会减少

正确性证明

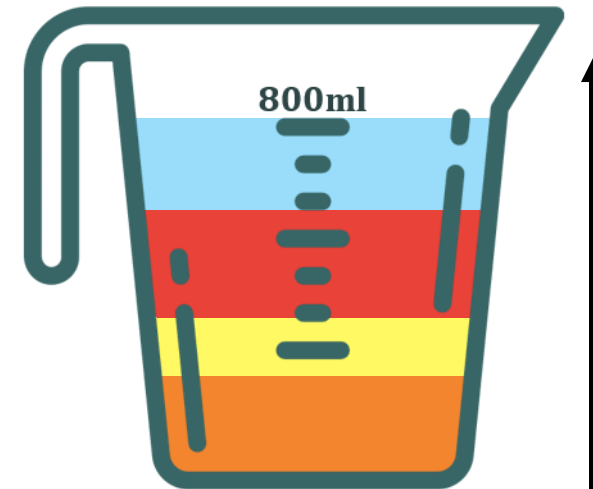
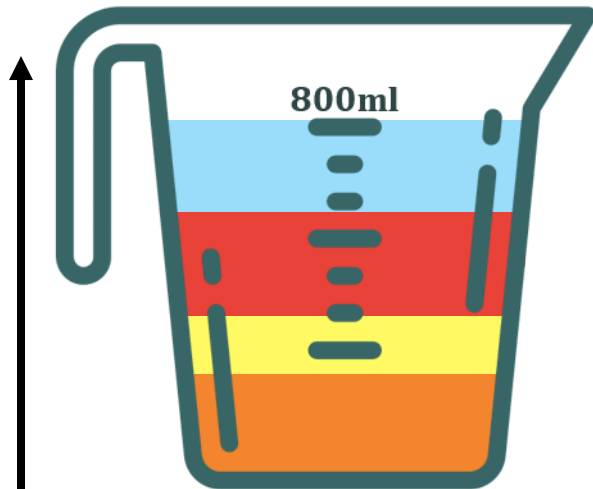
- 最高性价比优先
- 证明：贪心解不劣于最优解

最优解 S^*

替换为

贪心解 S'

按性价比
递减顺序
进行替换



替换后单位体积价值均不减少
故贪心解不劣于最优解

伪代码

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的极大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

// $Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

//根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

按性价比排序, 并初始化



伪代码

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的最大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

// $Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

// 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

当背包未装满且商品未装完时



伪代码

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的极大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

$// Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

$//$ 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

商品体积不大于容量则全部装入



伪代码

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的最大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

// $Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

// 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

否则装入部分商品填满背包



复杂度分析

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的最大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

$// Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

$//$ 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

$O(n \log n)$



复杂度分析

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的极大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

// $Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

// 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

$O(n \log n)$

$O(n)$



复杂度分析

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的最大值, 最优解方案

计算商品性价比 $Ratio[1..n]$ 并按降序排序

// $Ratio[i], p[i], v[i]$ 分别表示性价比第 i 大的商品的性价比、价格和体积

$i \leftarrow 1$

$ans \leftarrow 0$

// 根据贪心策略求解

while $C > 0$ and $i \leq n$ do

 if $v[i] \leq C$ then

 选择商品 i

$ans \leftarrow ans + p[i]$

$C \leftarrow C - v[i]$

 end

 else

 选择 C 体积的商品 i

$ans \leftarrow ans + p[i] \cdot \frac{C}{v[i]}$

$C \leftarrow 0$

 end

$i \leftarrow i + 1$

end

return ans

$O(n \log n)$

$O(n)$

时间复杂度: $O(n \log n)$

目录

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 货郎担问题
- 5.5 小结



贪心策略：一般步骤

提出贪心策略

观察问题特征，构造贪心选择



证明策略正确

假设最优方案，通过替换证明

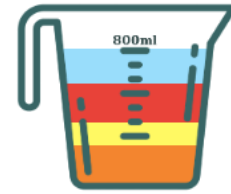


贪心方法适用的问题特点

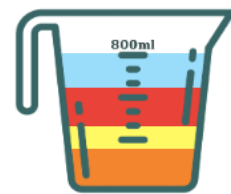
各饮料体积

- 有这样一类问题：它有 n 个输入, 而它的解就是这 n 个输入的某个子集, 这些子集必须满足某些事先给定的条件。
- 约束条件：必须满足的条件
- 可行解：满足约束条件的子集
- 目标函数：为了衡量可行解的优劣, 预先给定衡量标准, 以函数形式给出
- 最优解：使目标函数取极值(极大值或极小值)的可行解

总体积不能
大于800ml



饮品价格



思考：上述术语和背包问题如何对应？



贪心法的求解思想

- 贪心方法是一种“只顾眼前”的分级处理方法：
 - 根据题意选取一种量度标准；
 - 按该标准一次选中一个输入；
 - 如果这个输入和当前的部分解加在一起满足约束条件，则将其加入到部分解中；否则舍弃掉。

思考：贪心法得到的可行解是否一定是问题的最优解？
取决于贪心法中的哪个环节？

量度标准



算法5.1 贪心法的抽象化控制

Procedure GREEDY(A, n)

//A(1:n)包含n个输入

solution $\leftarrow \Phi$ //解向量solution初始化为空

for i $\leftarrow 1$ to n do

 x $\leftarrow$ SELECT(A)

 if FEASIBLE(solution, x)

 then solution $\leftarrow$ UNION(solution, x)

 end

end

return (solution)

end GREEDY

- SELECT: 按某种最优量度标准从A中选择一个输入赋值给x,并从A中消去它
- FEASIBLE: 判定x是否可以包含在解向量中。
- UNION: 将x与解向量结合并修改约束判定。



贪心法的特点

- 贪心法设计求解的核心问题是
 - 选择能产生问题最优解的量度标准，即**最优量度标准**
- 缺点：
 - 不是对所有问题都能得到最优解
 - 基于目标函数制定的度量标准不一定是最优的
 - 最优量度标准需要经过证明
- 优点：
 - 一旦证明成立后，它就是一种高效的算法
 - 策略的构造简单易行
 - 对许多问题都能产生整体最优解或者近似最优解



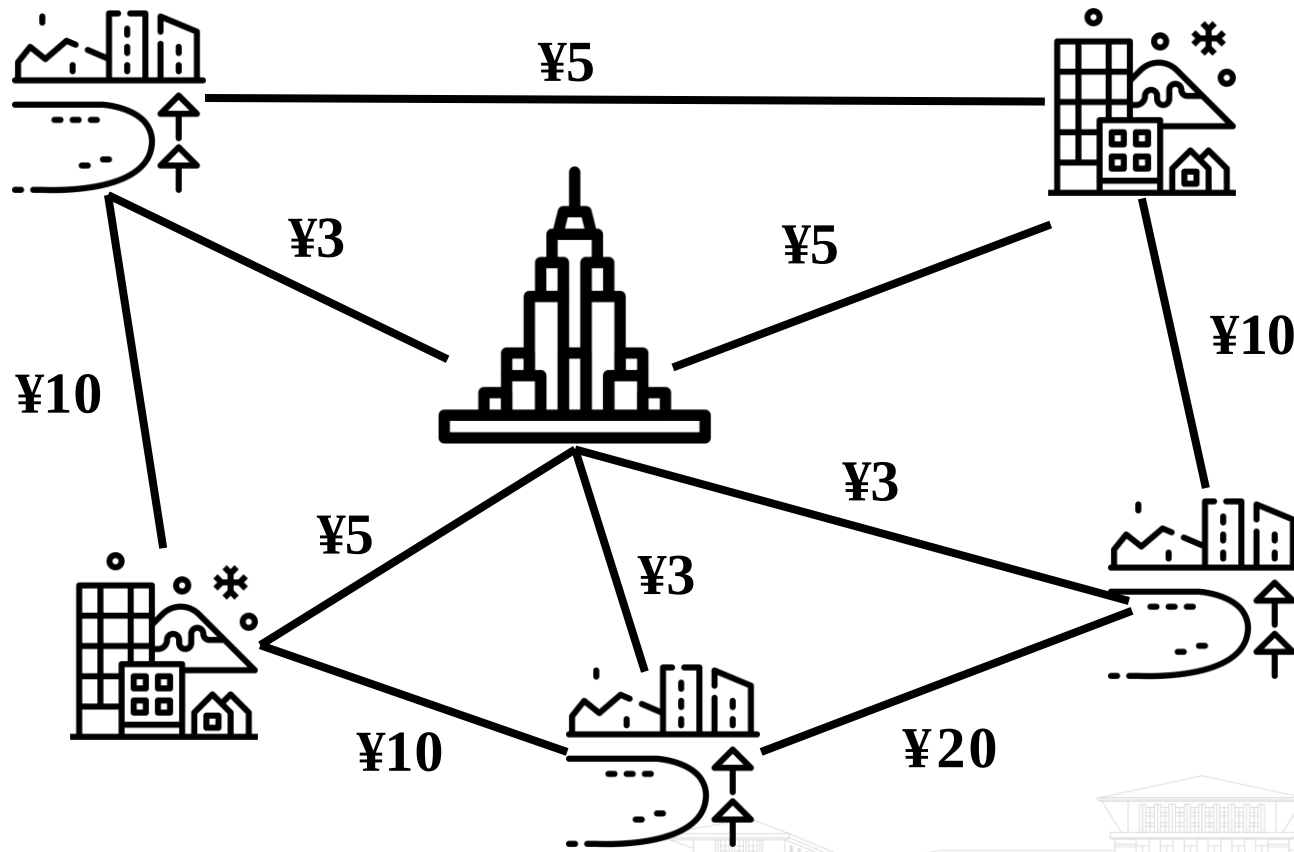
目录

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 货郎担问题
- 5.5 小结



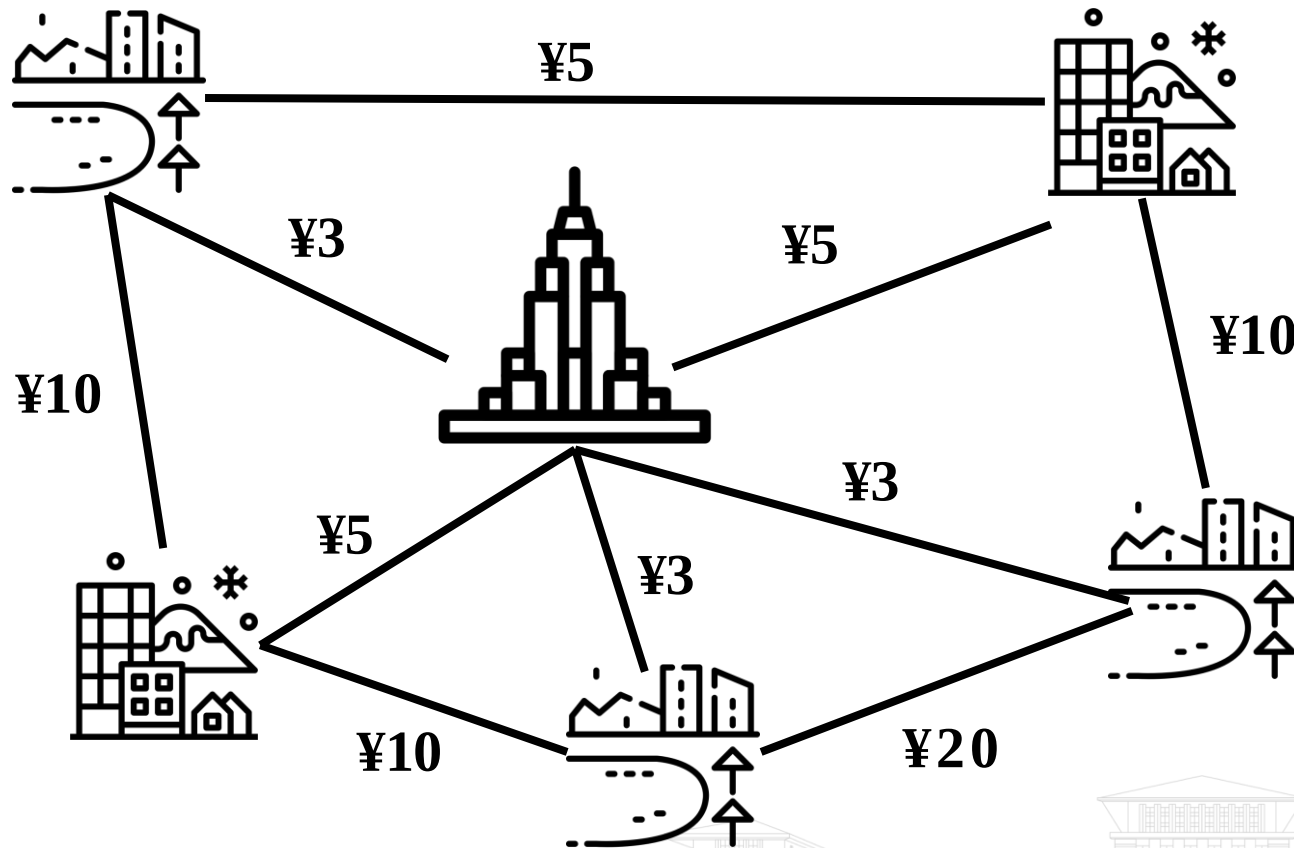
问题背景：道路修建

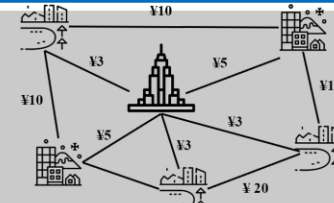
- 需要修建道路连通城市，各道路花费不同



问题背景：道路修建

- 需要修建道路连通城市，各道路花费不同

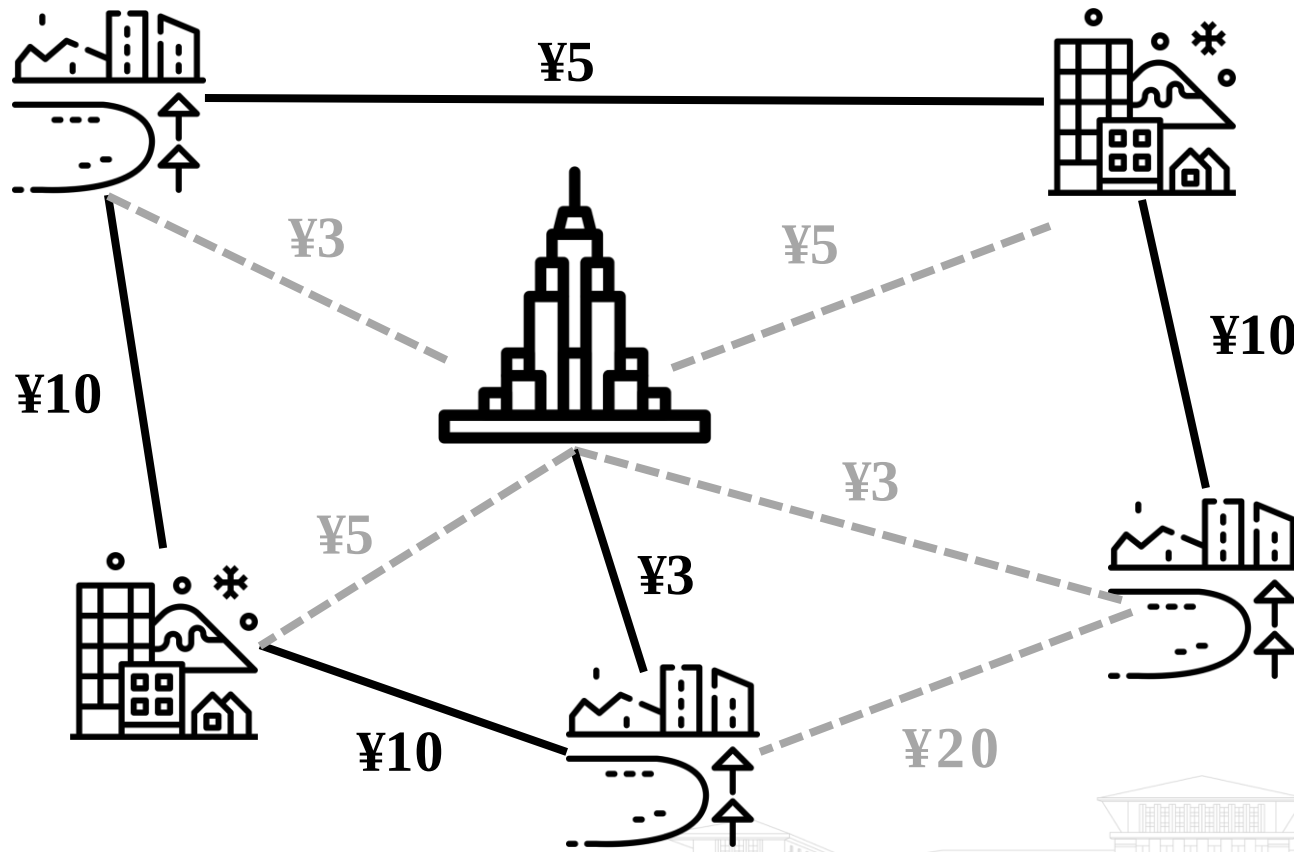


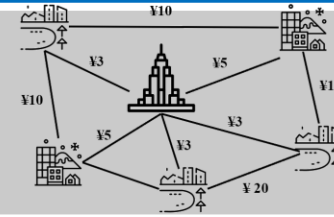
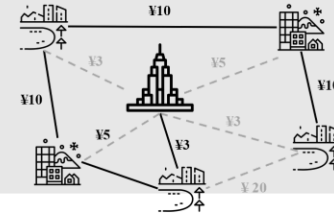
方案	花费
	¥74

$$\text{花费} : 10 + 10 + 10 + 5 + 20 + 3 + 3 + 5 + 3 + 5 = 74$$

问题背景：道路修建

- 需要修建道路连通城市，各道路花费不同

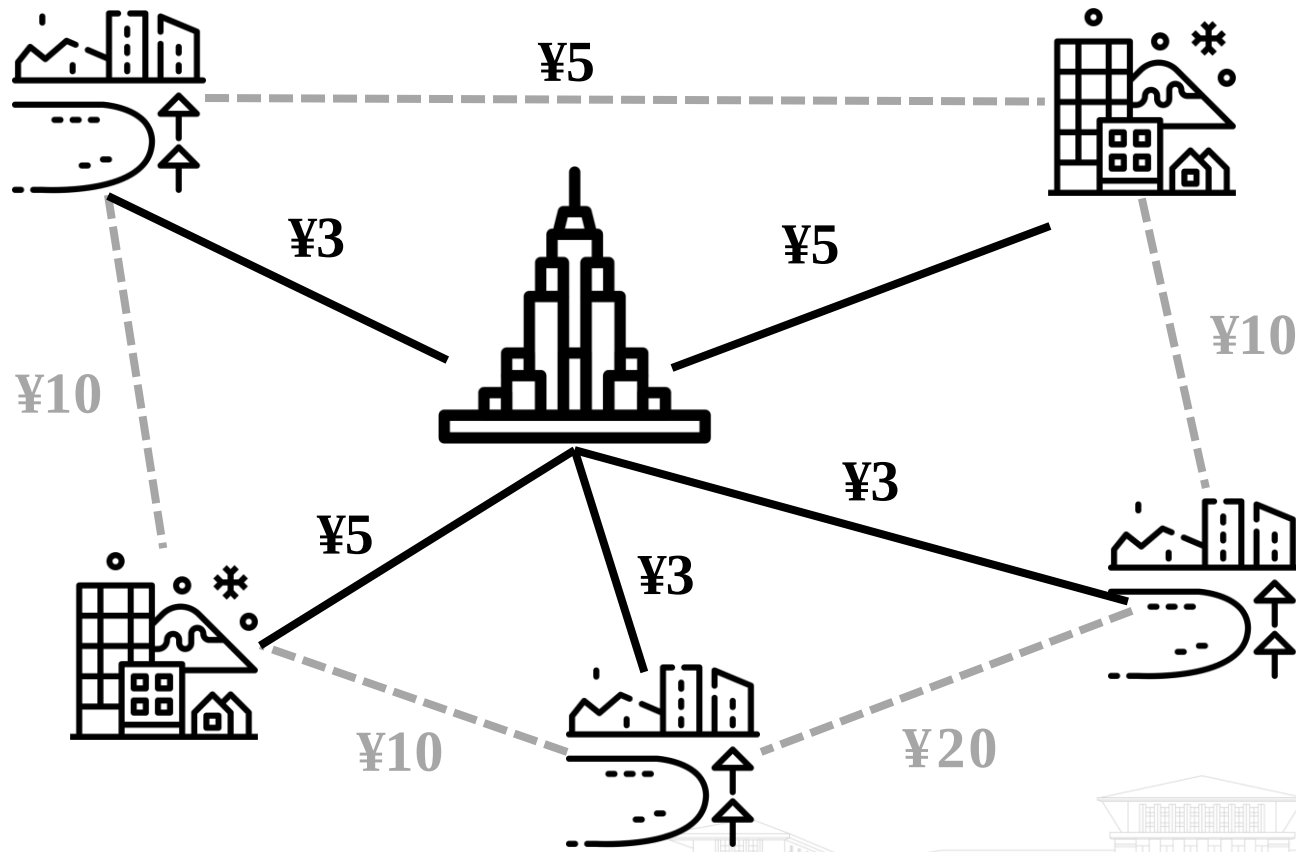


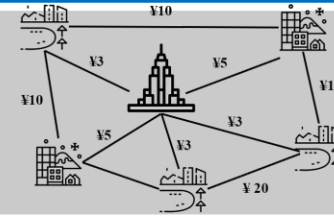
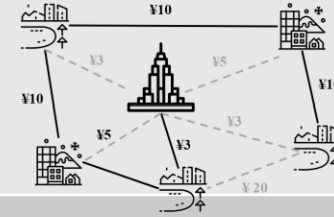
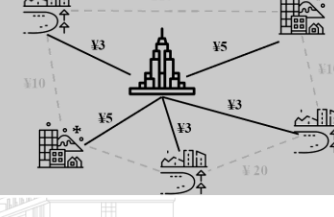
方案	花费
	¥74
	¥38

花费：10 + 10 + 10 + 5 + 3 = 38

问题背景：道路修建

- 需要修建道路连通城市，各道路花费不同

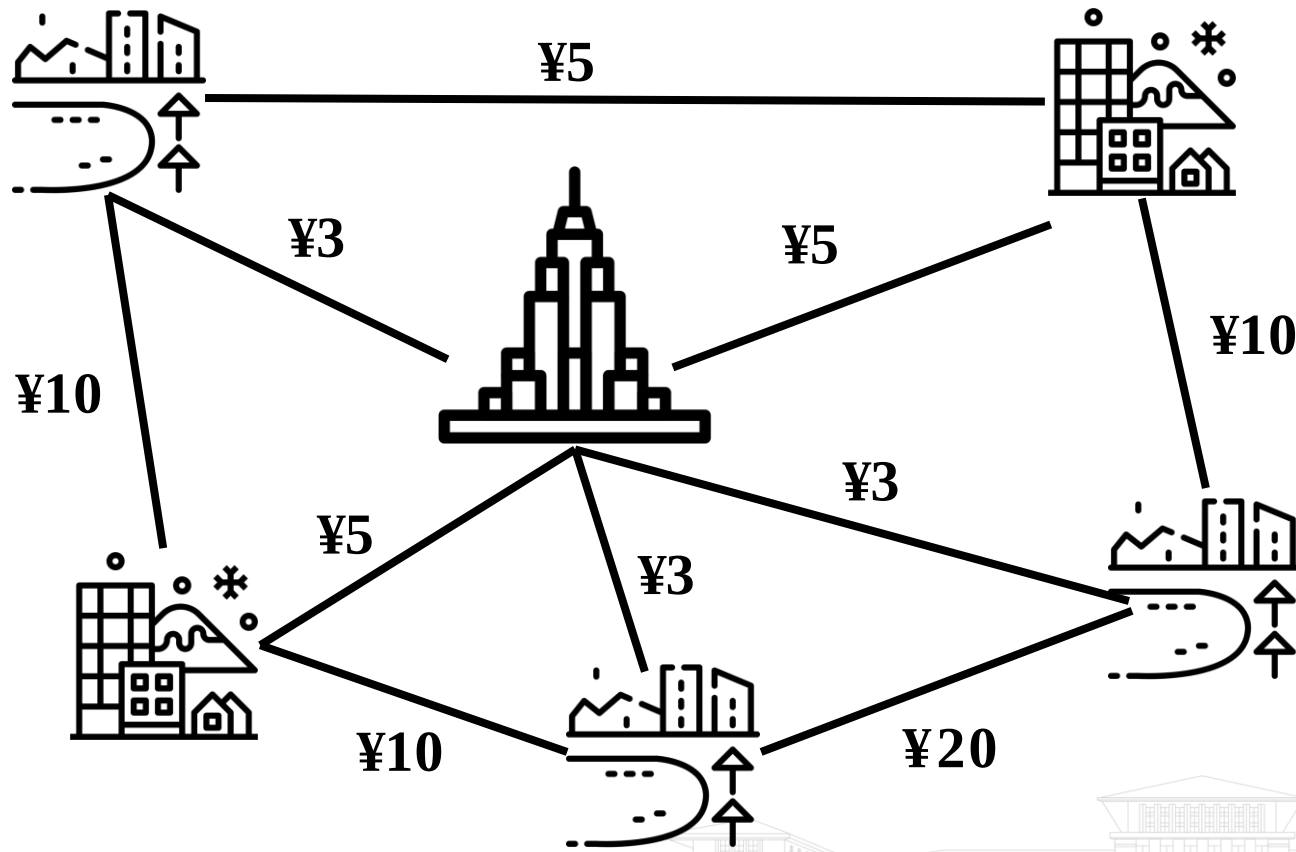


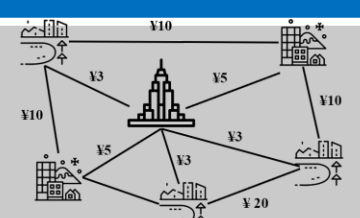
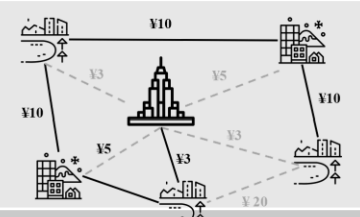
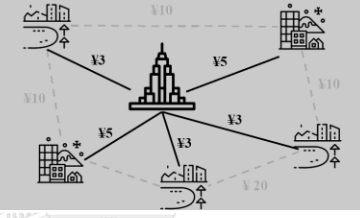
方案	花费
	¥74
	¥38
	¥19

$$\text{花费} : 3 + 3 + 5 + 3 + 5 = 19$$

问题背景：道路修建

- 需要修建道路连通城市，各道路花费不同

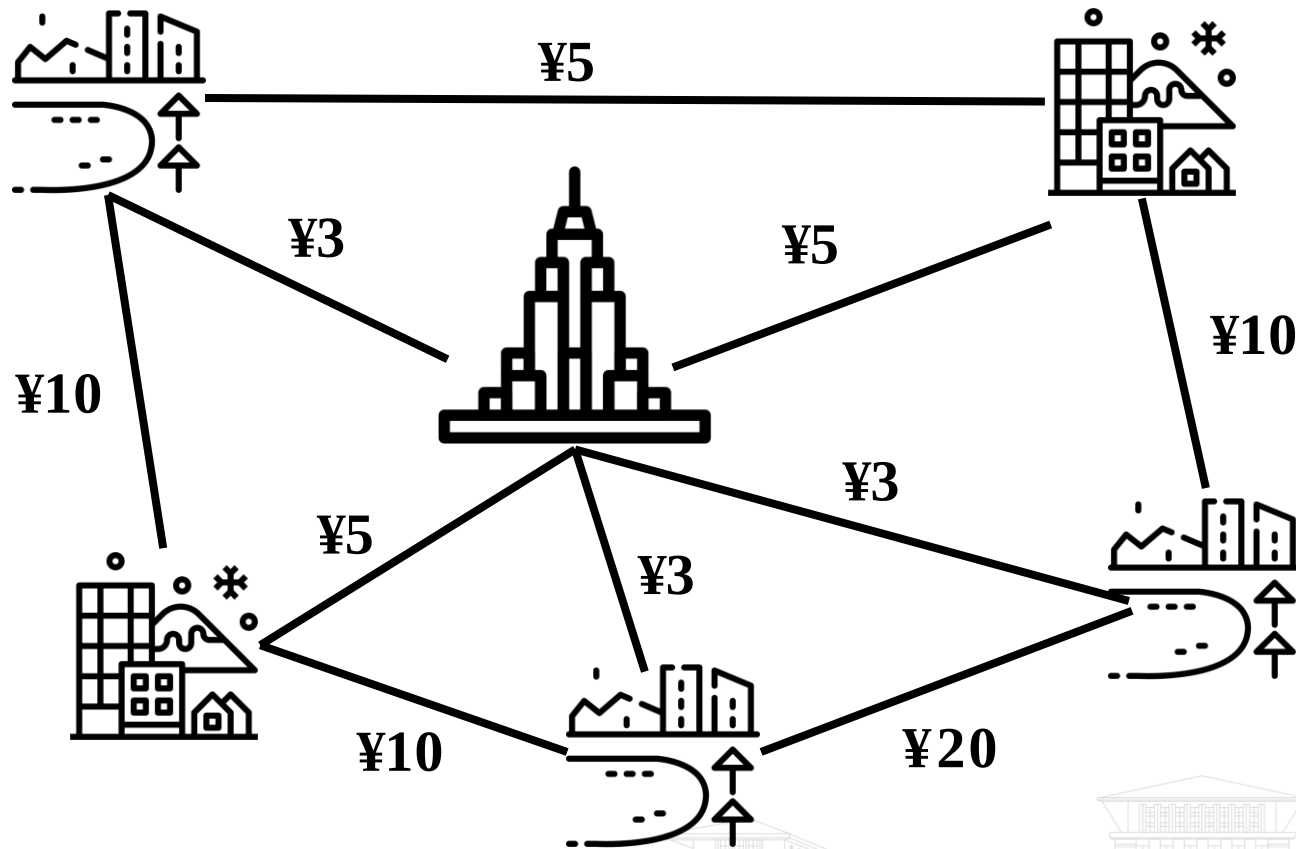


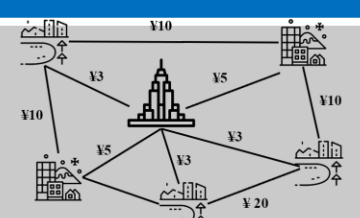
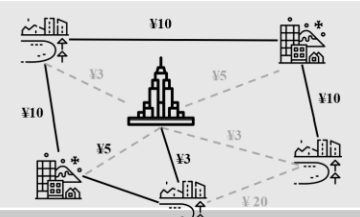
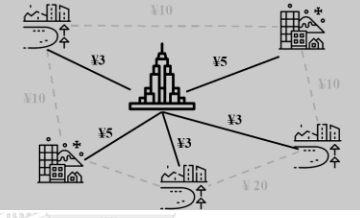
方案	花费
	¥74
	¥38
	¥19

问题：连通各城市的最小花费是多少？

问题背景：道路修建

- 需要修建道路连通城市，各道路花费不同



方案	花费
	¥74
	¥38
	¥19

问题：连通各城市的最小花费是多少？

权重最小的连通生成子图

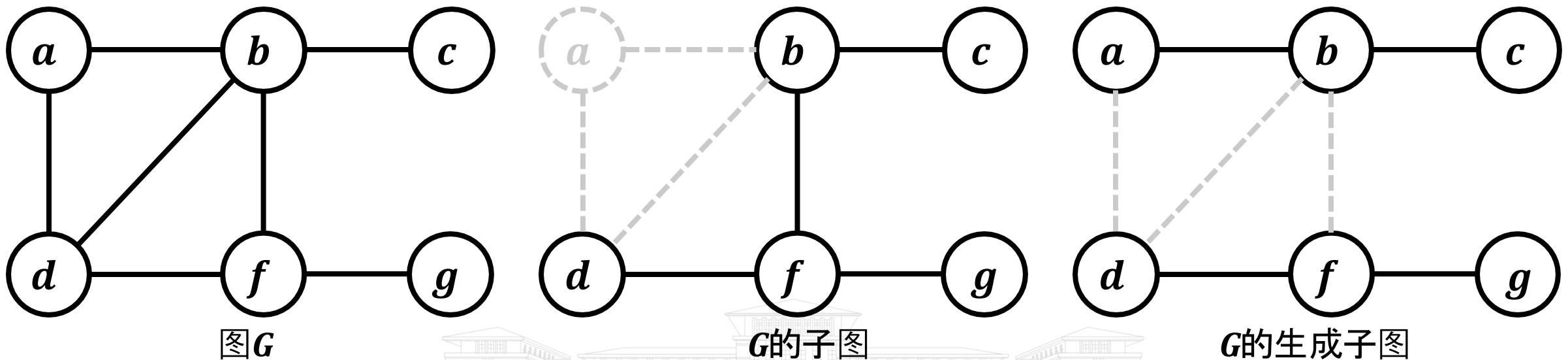
图的概念回顾：生成子图

● 子图(Subgraph)

● 如果 $V' \subseteq V, E' \subseteq E$ ，则称图 $G' = \langle V', E' \rangle$ 是图 G 的一个子图

● 生成子图(Spanning Subgraph)

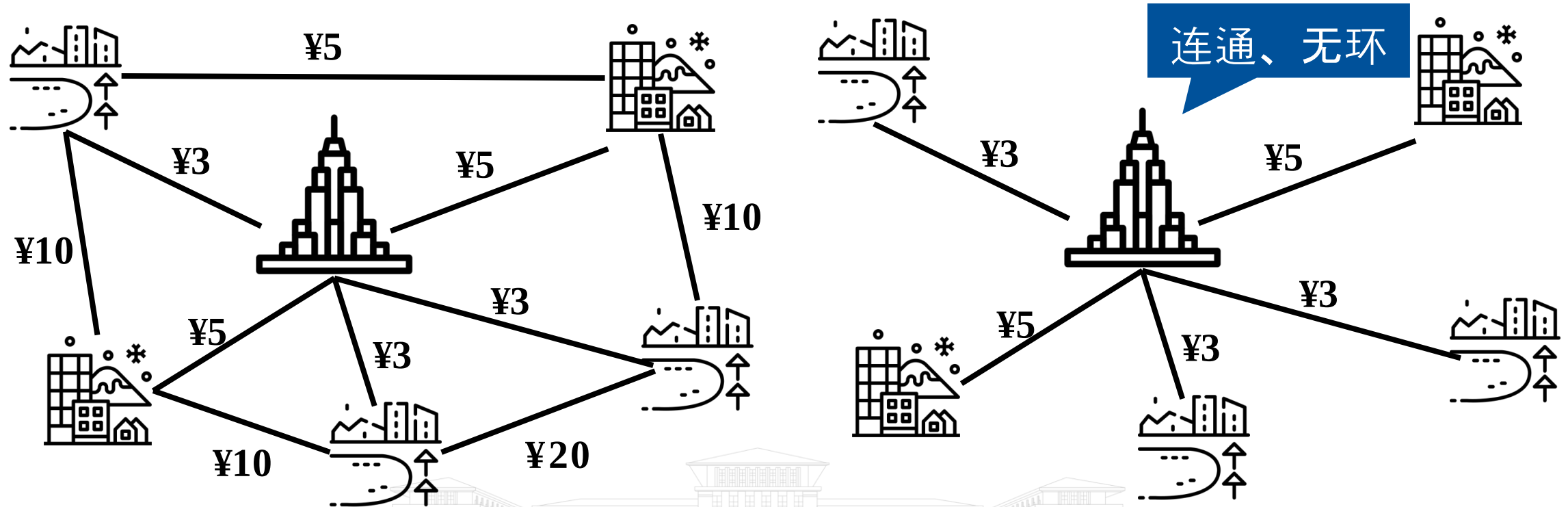
● 如果 $V' = V, E' \subseteq E$ ，则称图 $G' = \langle V', E' \rangle$ 是图 G 的一个生成子图



图的概念：生成树

●生成树(Spanning Tree)

- 图 $T' = \langle V', E' \rangle$ 是无向图 G 的一个生成子图，并且是连通、无环路的(树)



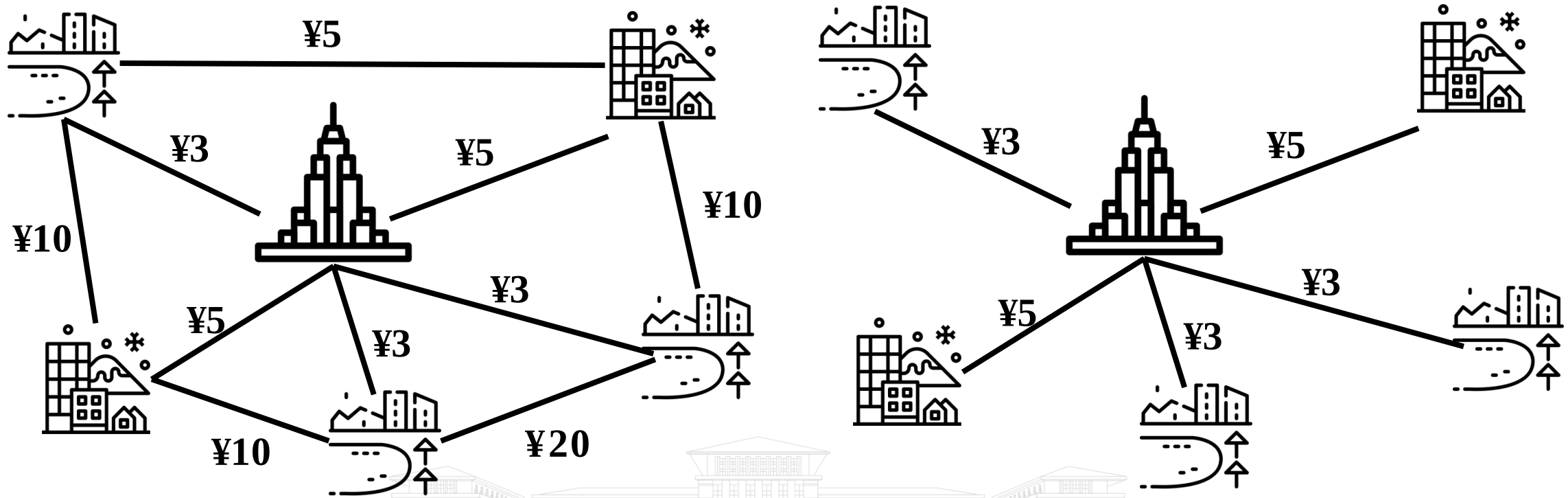
问题：连通各城市的最小花费是多少？

权重最小的连通生成子图

图的概念：生成树

●生成树(Spanning Tree)

● 图 $T' = \langle V', E' \rangle$ 是无向图 G 的一个生成子图，并且是连通、无环路的(树)



权重最小的生成树可能**不唯一**！

问题定义

- 输入：连通无向图 $G = \langle V, E, W \rangle$ ，其中 $w(u, v) \in W$ 表示边 (u, v) 的权重
- 输出：图 G 的最小生成树 $T = \langle V_T, E_T \rangle$

$$\min \sum_{e \in E_T} w(e)$$

优化目标

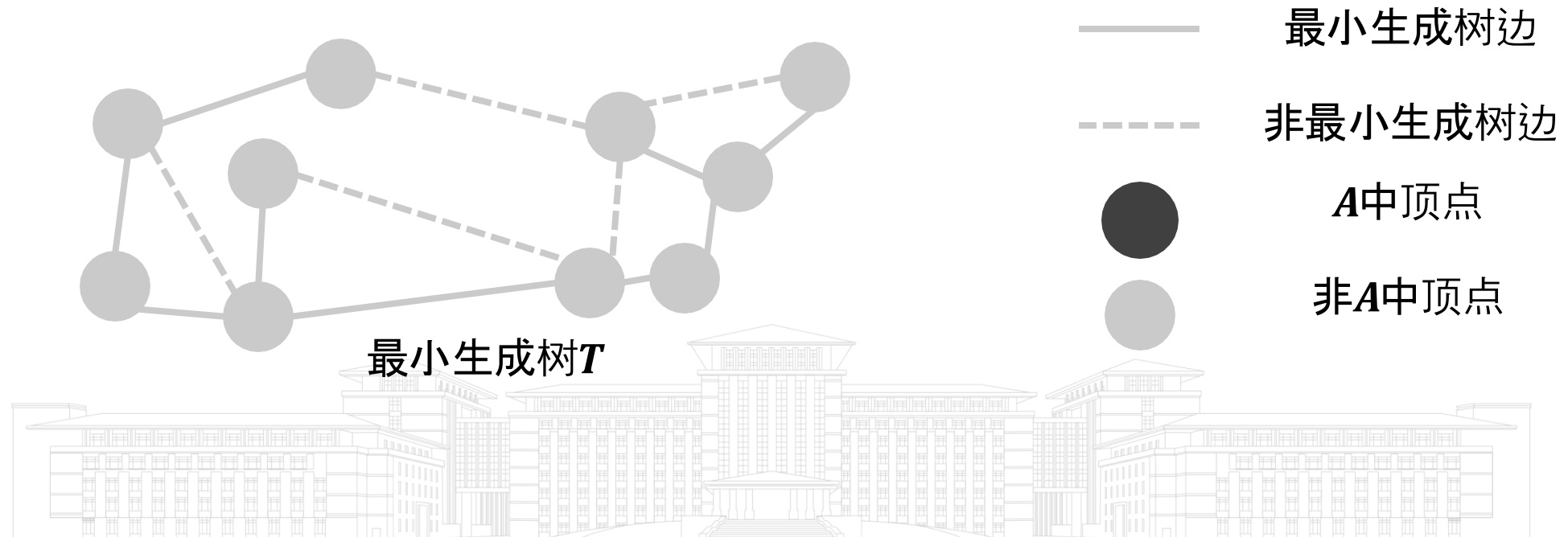
$$s. t. \quad V_T = V, E_T \subseteq E$$

约束条件



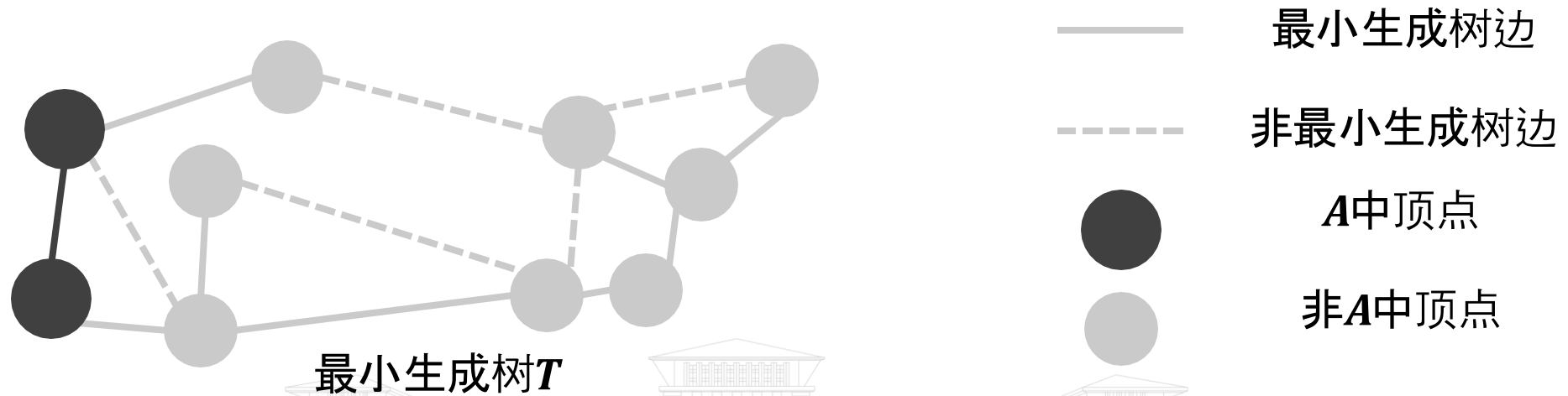
通用框架

- 生成树是一个无向图中的连通、**无环**的生成子图
- 新建一个空边集 A ，边集 A 可逐步扩展为最小生成树



通用框架

- 生成树是一个无向图中的连通、**无环**的生成子图
 - 新建一个空边集 A ，边集 A 可逐步扩展为最小生成树
 - 每次向边集 A 中新增加一条边
 - 需保证边集 A 仍是一个**无环图**
 - 需保证边集 A 仍是**最小生成树的子集**

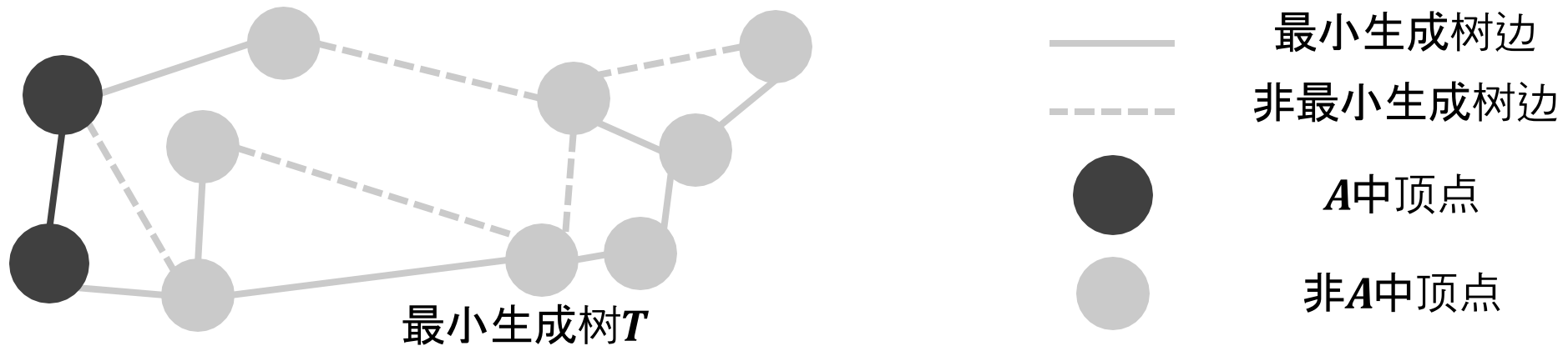


思考：如何保证边集 A 仍是**最小生成树的子集**？

相关概念

- 安全边(Safe Edge)

- A 是某棵最小生成树 T 边的子集, $A \subseteq T$
- $A \cup \{(u, v)\}$ 仍是某棵最小生成树边的一个子集, 则称 (u, v) 是 A 的**安全边**



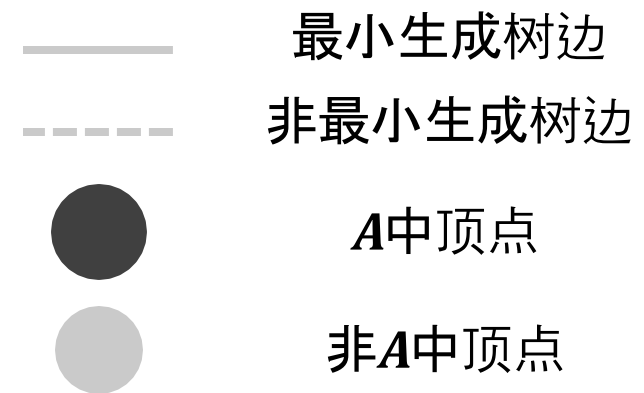
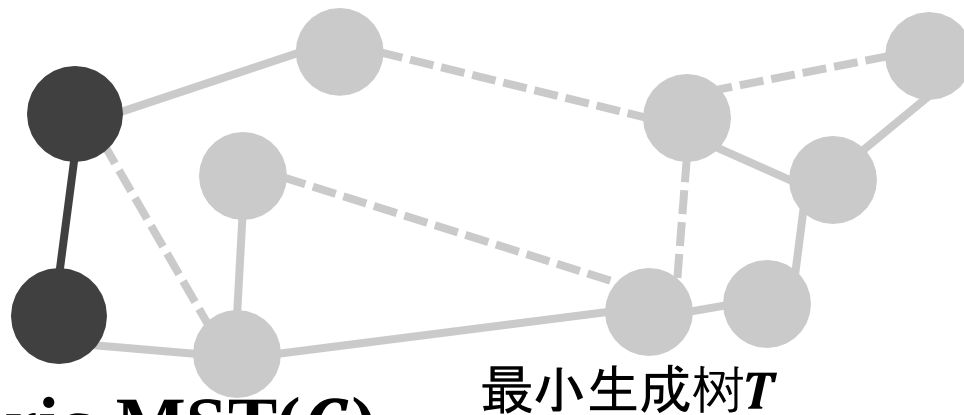
若每次向边集 A 中新增**安全边**, 可保证边集 A 是最小生成树的子集



相关概念

- 安全边(Safe Edge)

- A 是某棵最小生成树 T 边的子集, $A \subseteq T$
- $A \cup \{(u, v)\}$ 仍是某棵最小生成树边的一个子集, 则称 (u, v) 是 A 的**安全边**



- Generic-MST(G)**

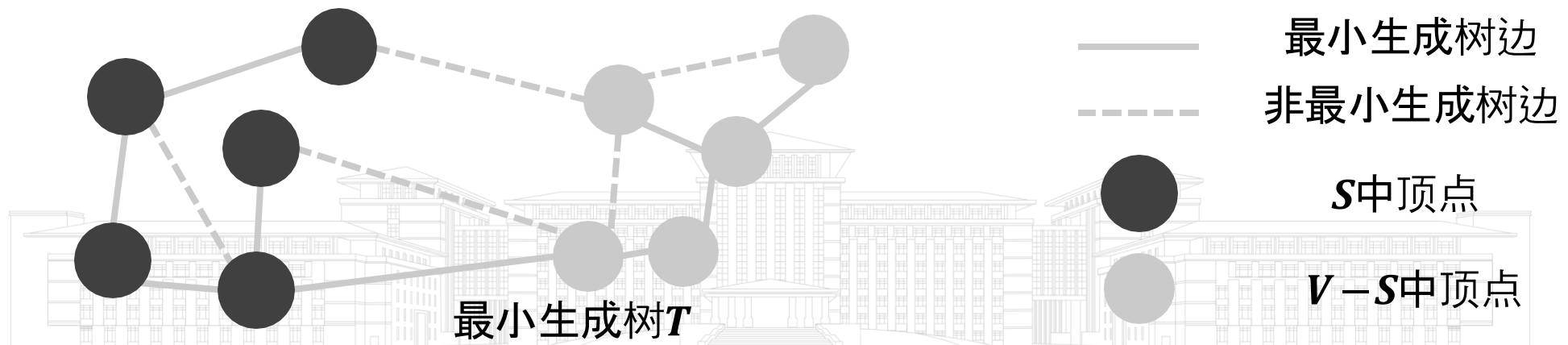
```
A ← ∅  
while 没有形成最小生成树 do  
    | 寻找 $A$ 的安全边 $(u, v)$   
    |  $A \leftarrow A \cup (u, v)$   
end  
return  $A$ 
```

问题：如何有效辨识安全边？

相关概念

- 割(Cut)

- 图 $G = \langle V, E \rangle$ 是一个连通无向图, **割** $(S, V - S)$ 将图 G 的顶点集 V 划分为两部分



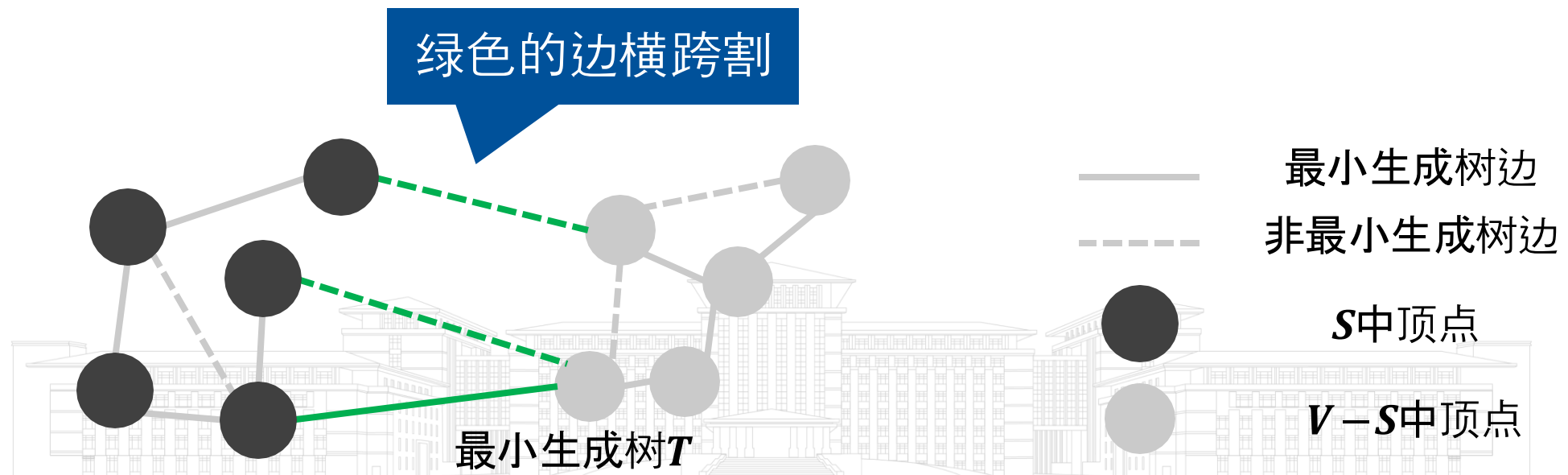
相关概念

- 割(Cut)

- 图 $G = \langle V, E \rangle$ 是一个连通无向图，割 $(S, V - S)$ 将图 G 的顶点集 V 划分为两部分

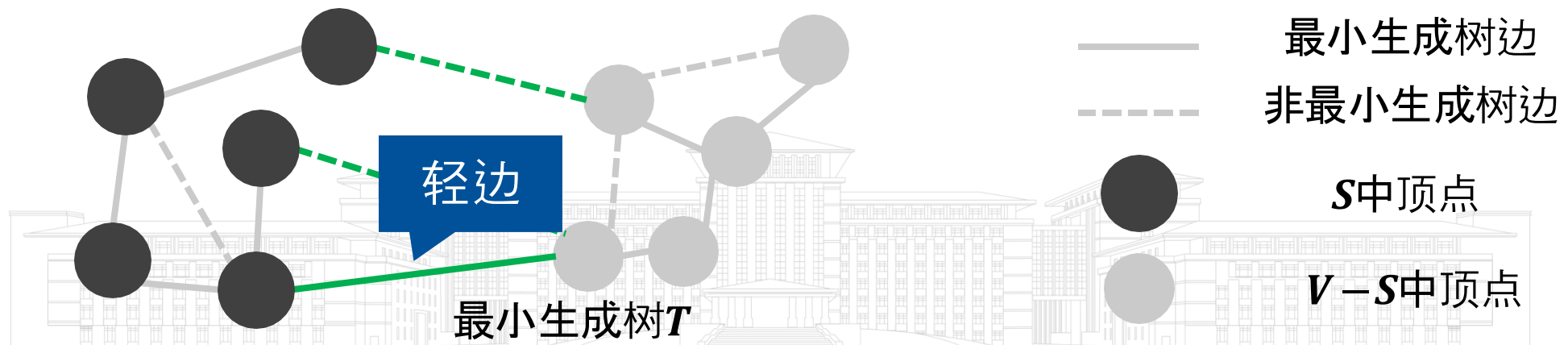
- 横跨(Cross)

- 给定割 $(S, V - S)$ 和边 (u, v) , $u \in S, v \in V - S$, 称边 (u, v) 横跨割 $(S, V - S)$



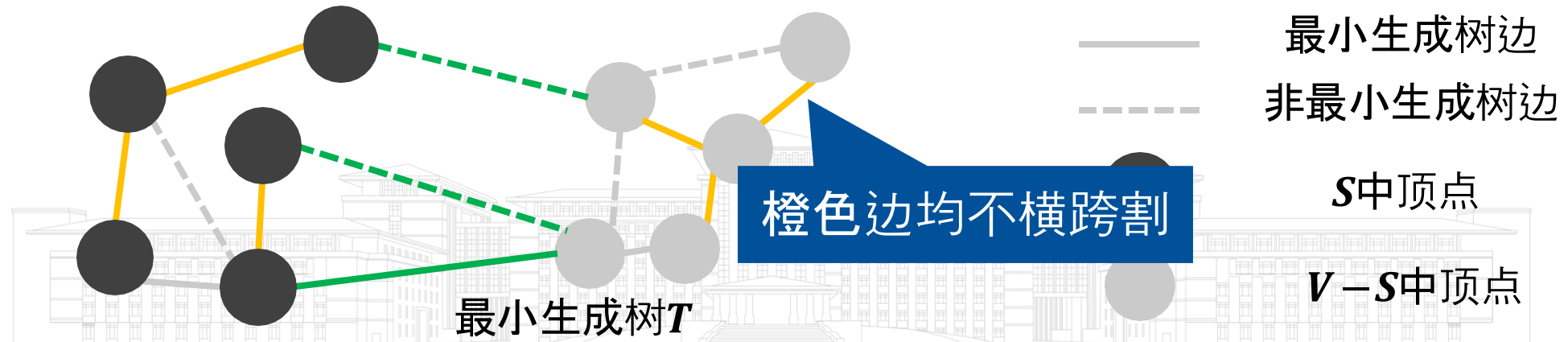
相关概念

- 割(Cut)
 - 图 $G = \langle V, E \rangle$ 是一个连通无向图，割 $(S, V - S)$ 将图 G 的顶点集 V 划分为两部分
- 横跨(Cross)
 - 给定割 $(S, V - S)$ 和边 (u, v) ， $u \in S, v \in V - S$ ，称边 (u, v) 横跨割 $(S, V - S)$
- 轻边(Light Edge)
 - 横跨割的所有边中，权重最小的称为横跨这个割的一条轻边



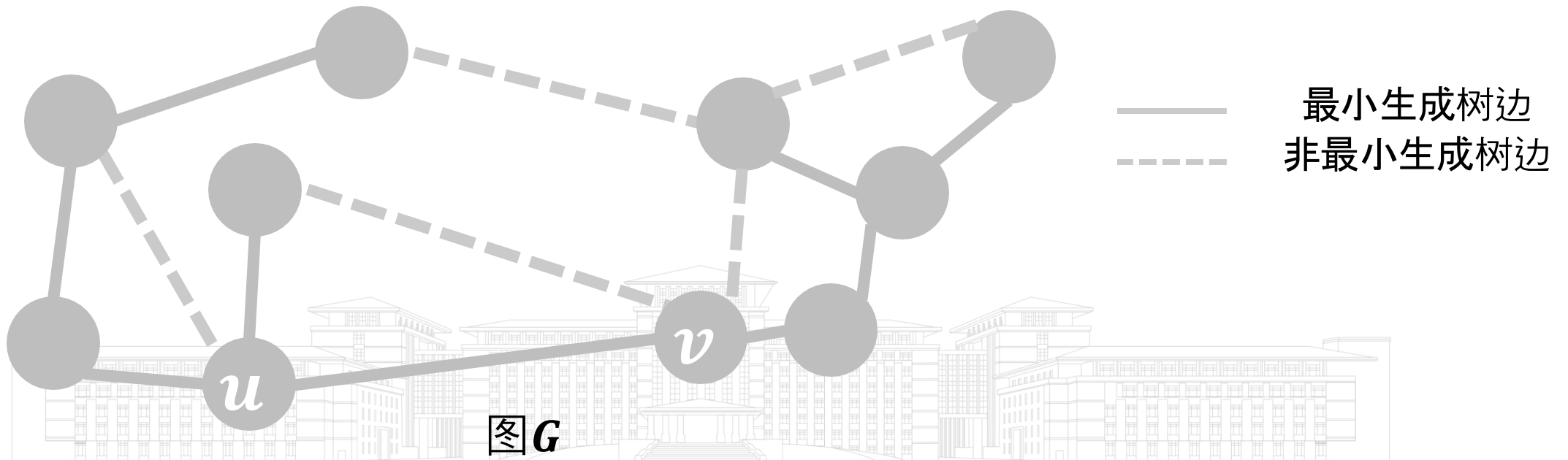
相关概念

- 割(Cut)
 - 图 $G = \langle V, E \rangle$ 是一个连通无向图，割 $(S, V - S)$ 将图 G 的顶点集 V 划分为两部分
- 横跨(Cross)
 - 给定割 $(S, V - S)$ 和边 (u, v) ， $u \in S, v \in V - S$ ，称边 (u, v) 横跨割 $(S, V - S)$
- 轻边(Light Edge)
 - 横跨割的所有边中，权重最小的称为横跨这个割的一条轻边
- 不妨害(Respect)
 - 如果一个边集 A 中没有边横跨某割，则称该割**不妨害**边集 A



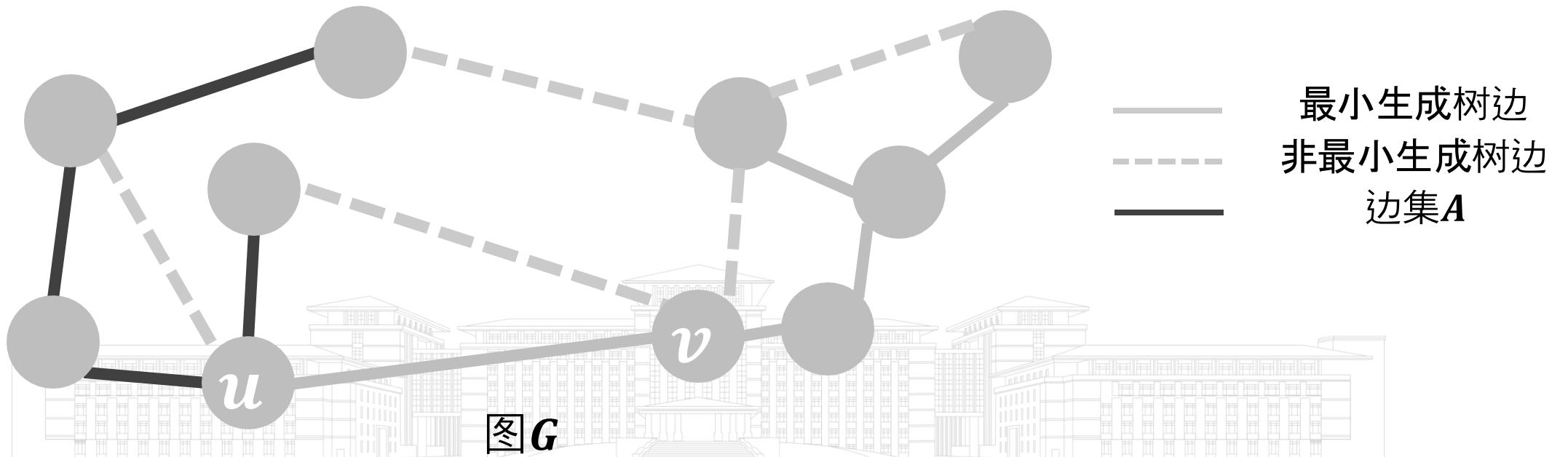
安全边辨识定理

- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图



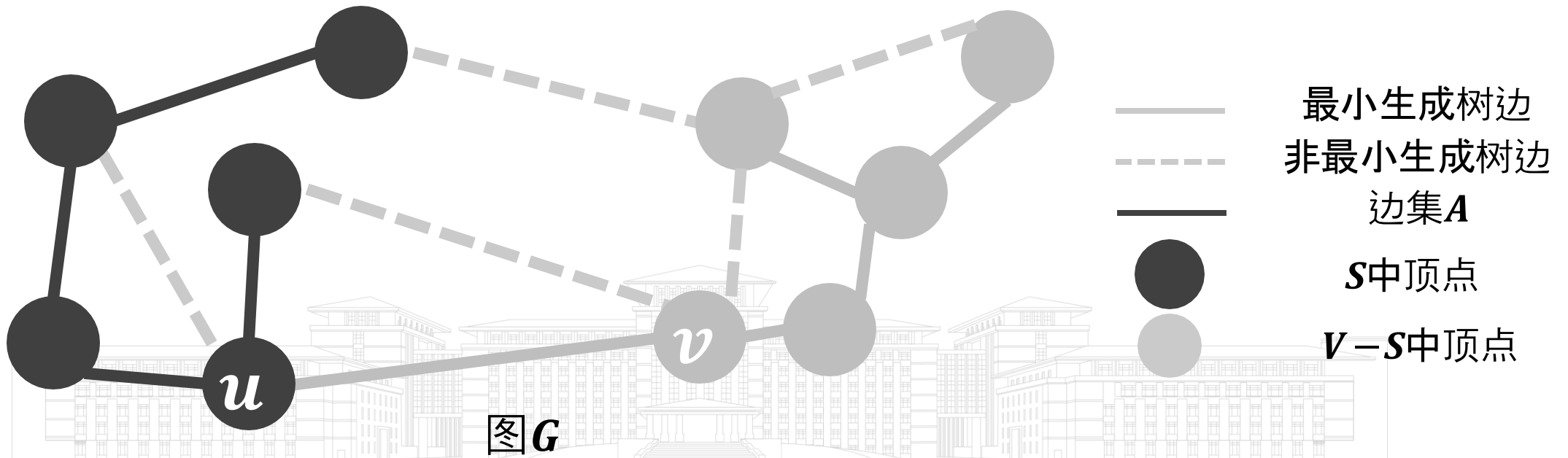
安全边辨识定理

- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图，令 A 为边集 E 的一个子集，且 A 包含在图 G 的某棵最小生成树中



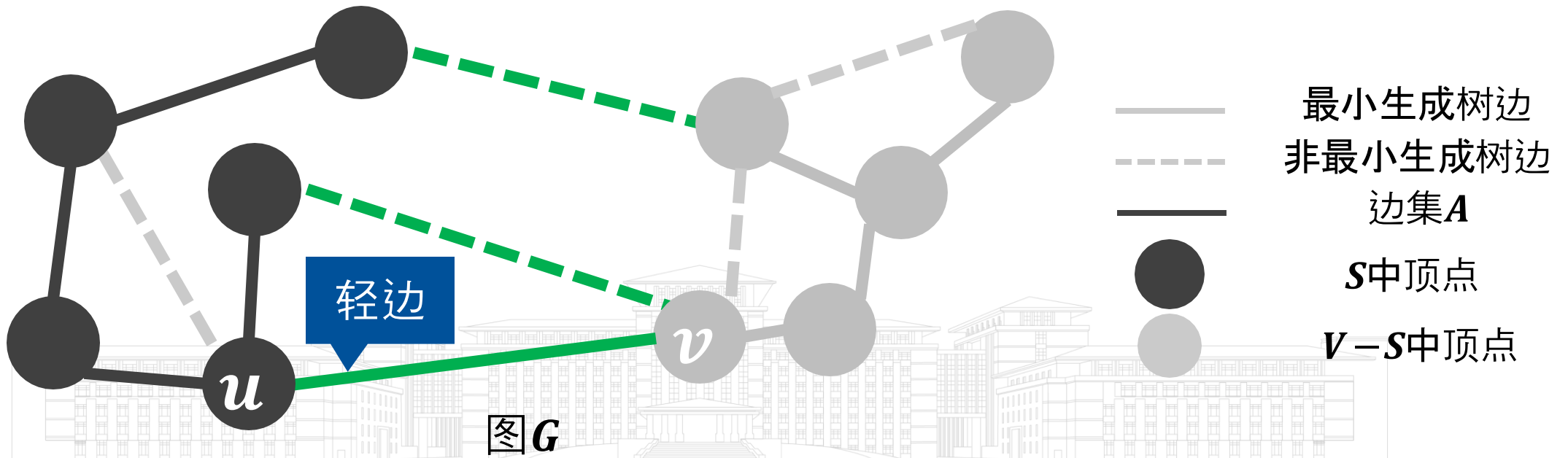
安全边辨识定理

- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图，令 A 为边集 E 的一个子集，且 A 包含在图 G 的某棵最小生成树中
 - 若割 $(S, V - S)$ 是图 G 中不妨害边集 A 的任意割



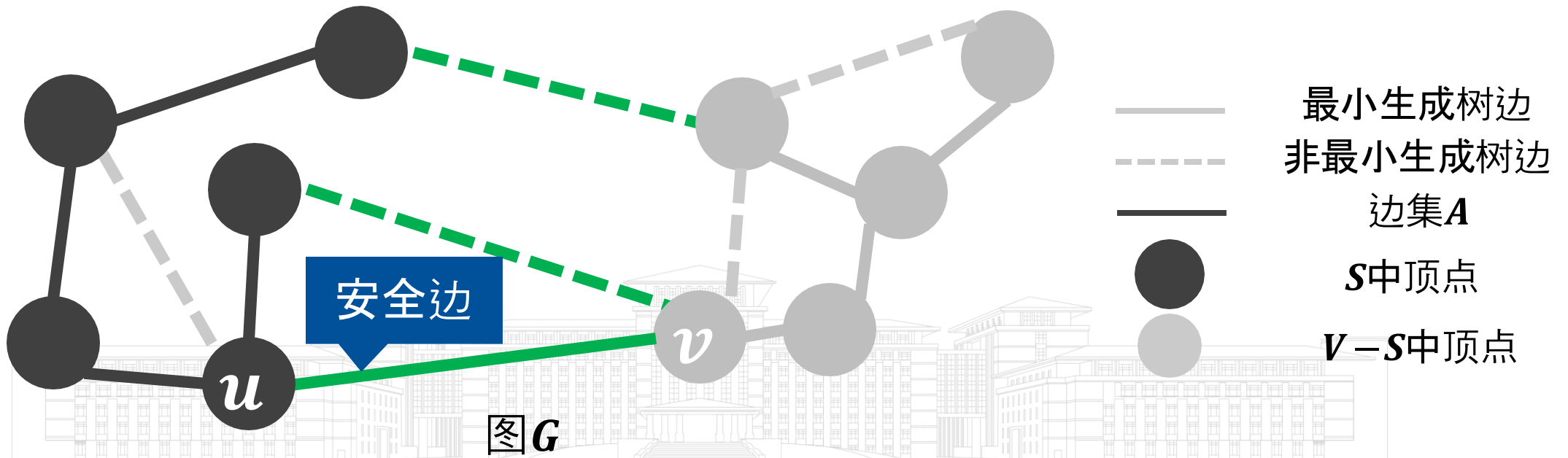
安全边辨识定理

- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图，令 A 为边集 E 的一个子集，且 A 包含在图 G 的某棵最小生成树中
 - 若割 $(S, V - S)$ 是图 G 中不妨害边集 A 的任意割，且 (u, v) 是横跨该割的轻边



安全边辨识定理

- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图，令 A 为边集 E 的一个子集，且 A 包含在图 G 的某棵最小生成树中
 - 若割 $(S, V - S)$ 是图 G 中不妨害边集 A 的任意割，且 (u, v) 是横跨该割的轻边
 - 则对于边集 A ，边 (u, v) 是其安全边

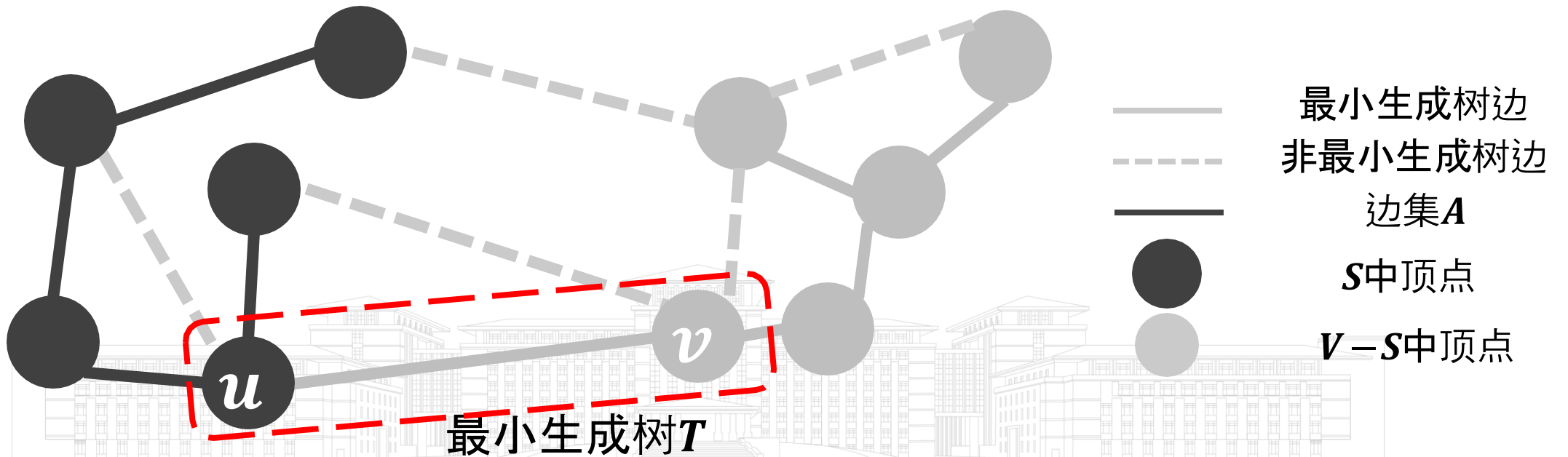


安全边辨识定理

- 证明

T 是任意最小生成树

- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证

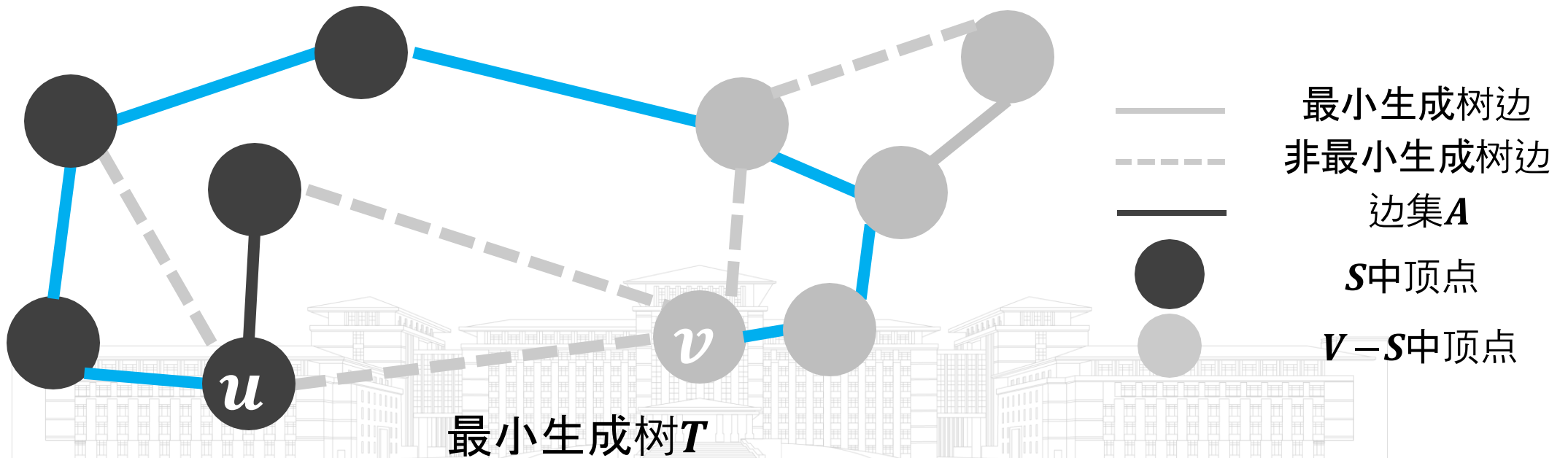


安全边辨识定理

• 证明

T 是任意最小生成树

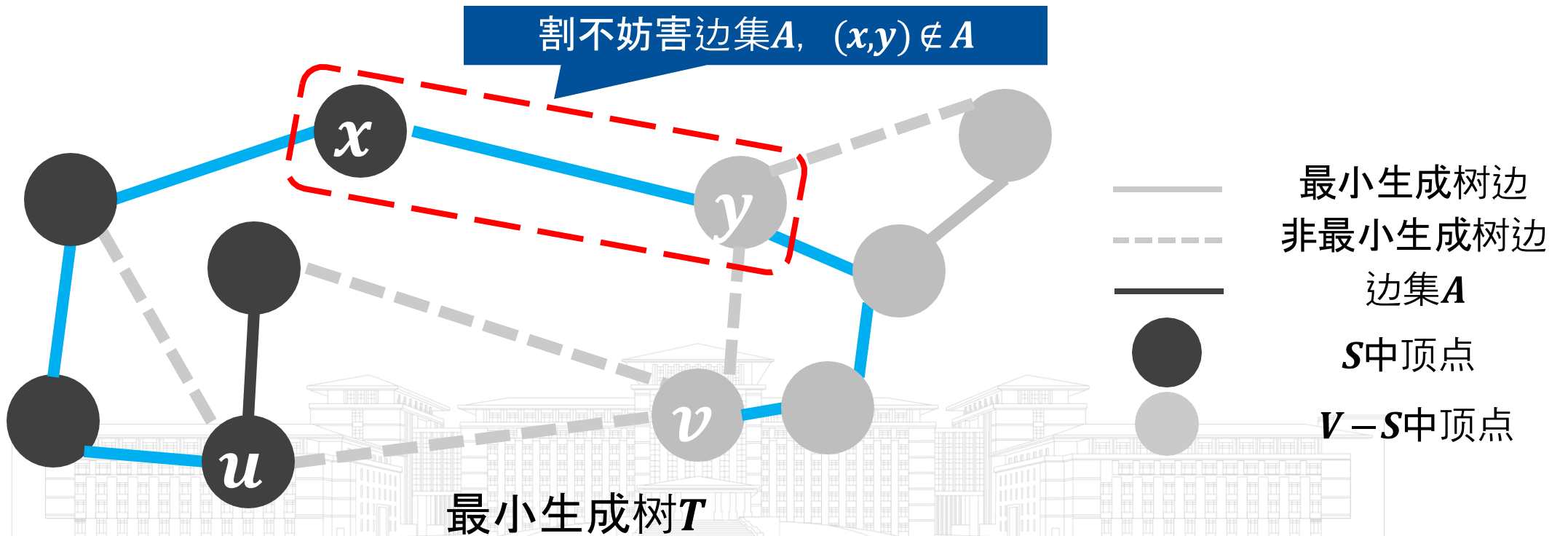
- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P



安全边辨识定理

- 证明

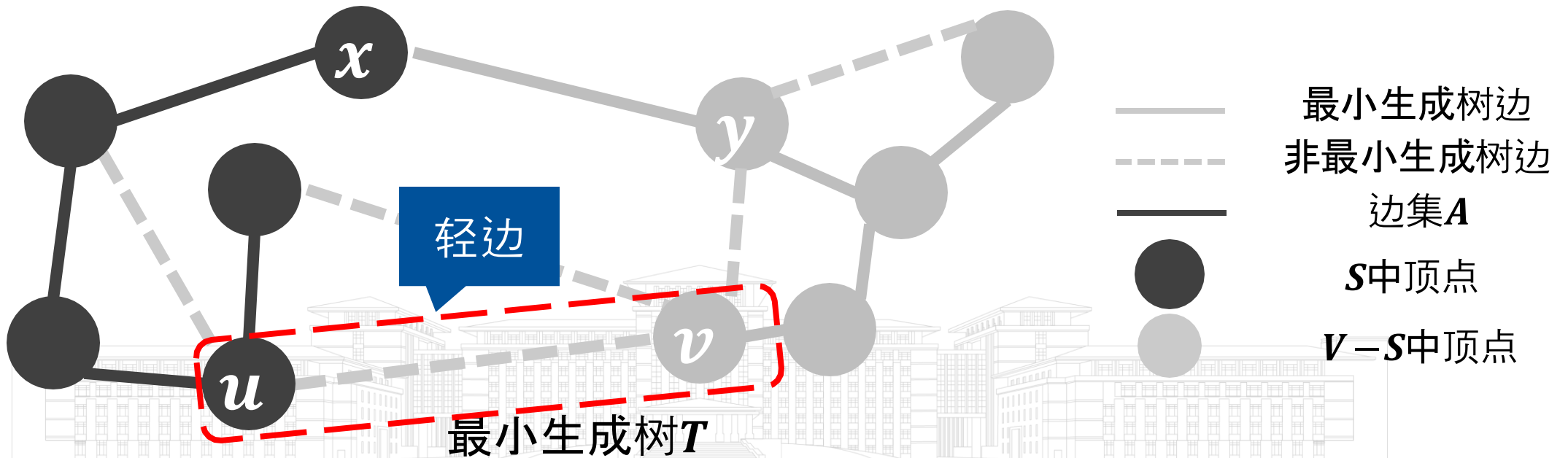
- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P
 - 不妨设路径 P 中, 横跨割 $(S, V - S)$ 的一条边为 (x, y)



安全边辨识定理

- 证明

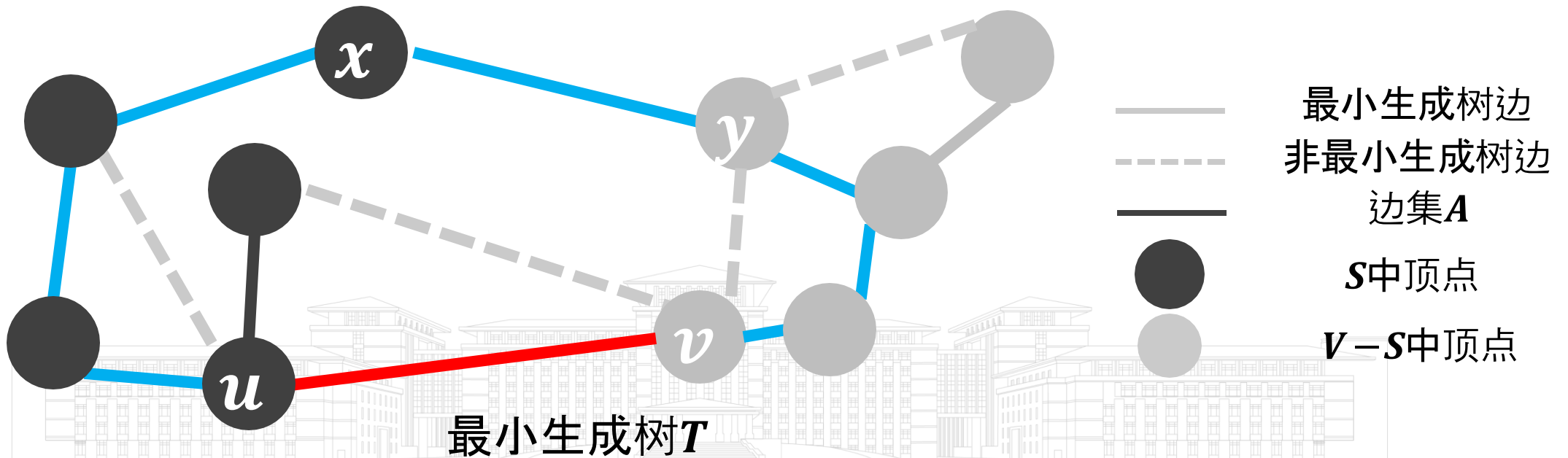
- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P
 - 不妨设路径 P 中, 横跨割 $(S, V - S)$ 的一条边为 (x, y)
 - 因为 (u, v) 是横跨割的轻边, 所以 $w(u, v) \leq w(x, y)$



安全边辨识定理

- 证明

- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P
 - 不妨设路径 P 中, 横跨割 $(S, V - S)$ 的一条边为 (x, y)
 - 因为 (u, v) 是横跨割的轻边, 所以 $w(u, v) \leq w(x, y)$
 - 由于 T 是生成树, 将 (u, v) 加入 T 中会形成环路

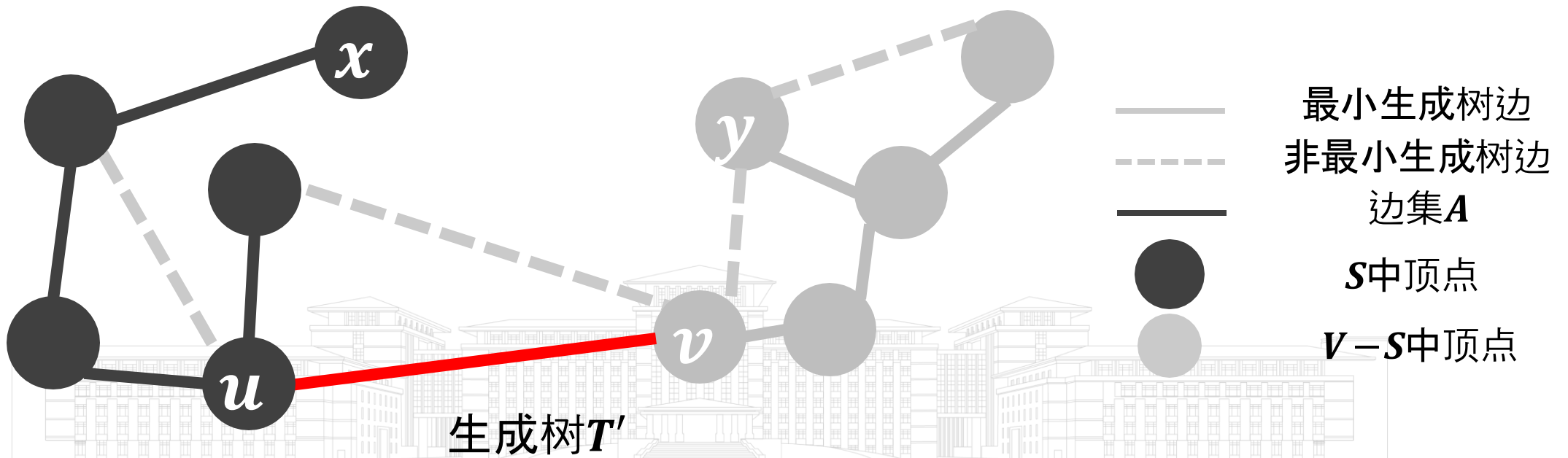


安全边辨识定理

• 证明

- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P
 - 不妨设路径 P 中, 横跨割 $(S, V - S)$ 的一条边为 (x, y)
 - 因为 (u, v) 是横跨割的轻边, 所以 $w(u, v) \leq w(x, y)$
 - 由于 T 是生成树, 将 (u, v) 加入 T 中会形成环路, 再去掉 (x, y) 形成另一棵树 T'

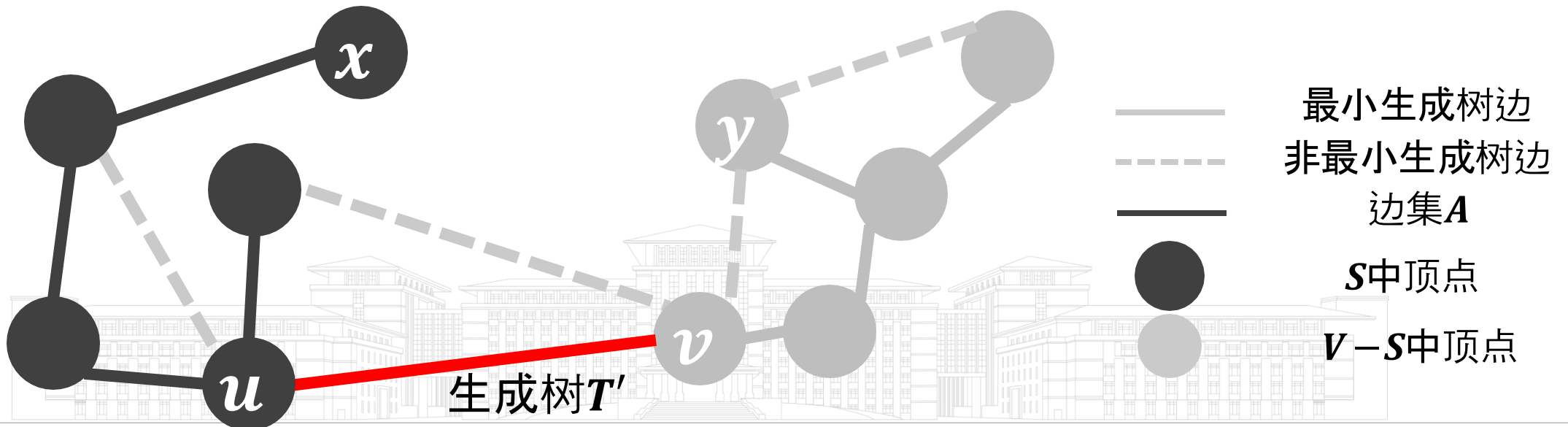
思考为什么?



安全边辨识定理

• 证明

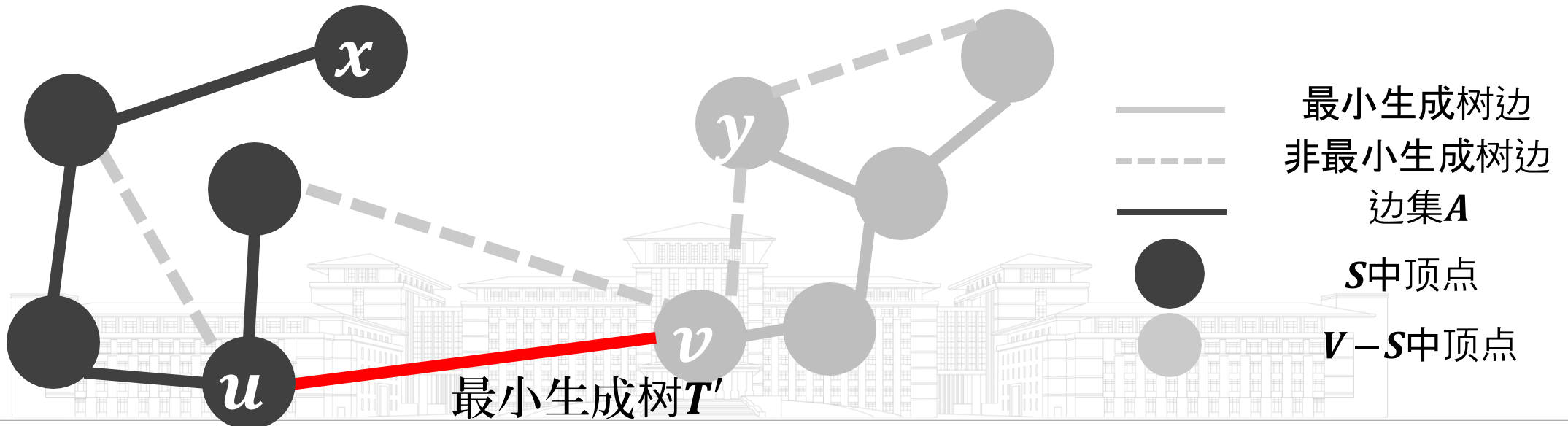
- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P
 - 不妨设路径 P 中, 横跨割 $(S, V - S)$ 的一条边为 (x, y)
 - 因为 (u, v) 是横跨割的轻边, 所以 $w(u, v) \leq w(x, y)$
 - 由于 T 是生成树, 将 (u, v) 加入 T 中会形成环路, 再去掉 (x, y) 形成另一棵树 T'
 - $w(T') = w(T) + w(u, v) - w(x, y) \leq w(T)$



安全边辨识定理

• 证明

- 若 $(u, v) \in T$, 由于 $A \subseteq T$, 则 $A \cup \{(u, v)\} \subseteq T$, 由安全边定义得证
- 若 $(u, v) \notin T$, 则 T 中必存在 u 到 v 的路径 P
 - 不妨设路径 P 中, 横跨割 $(S, V - S)$ 的一条边为 (x, y)
 - 因为 (u, v) 是横跨割的轻边, 所以 $w(u, v) \leq w(x, y)$
 - 由于 T 是生成树, 将 (u, v) 加入 T 中会形成环路, 再去掉 (x, y) 形成另一棵树 T'
 - $w(T') \leq w(T)$, T' 也是最小生成树, 所以 (u, v) 是安全边



通用框架

- 生成树是一个连通、无环的生成子图
 - 新建一个空边集 A ，边集 A 可逐步扩展为最小生成树
 - 每次向边集 A 中新增加一条边
 - 需保证边集 A 仍是一个无环图
 - 需保证边集 A 仍是最小生成树的子集

思考：如何有效地实现此贪心策略？

Prim算法

Kruskal算法



通用框架

- 生成树是一个连通、无环的生成子图
 - 新建一个空边集 A ，边集 A 可逐步扩展为最小生成树
 - 每次向边集 A 中新增加一条边
 - 需保证边集 A 仍是一个无环图
 - 需保证边集 A 仍是最小生成树的子集

添加一条轻边

思考：如何有效地实现此贪心策略？

Prim算法

Kruskal算法



目录

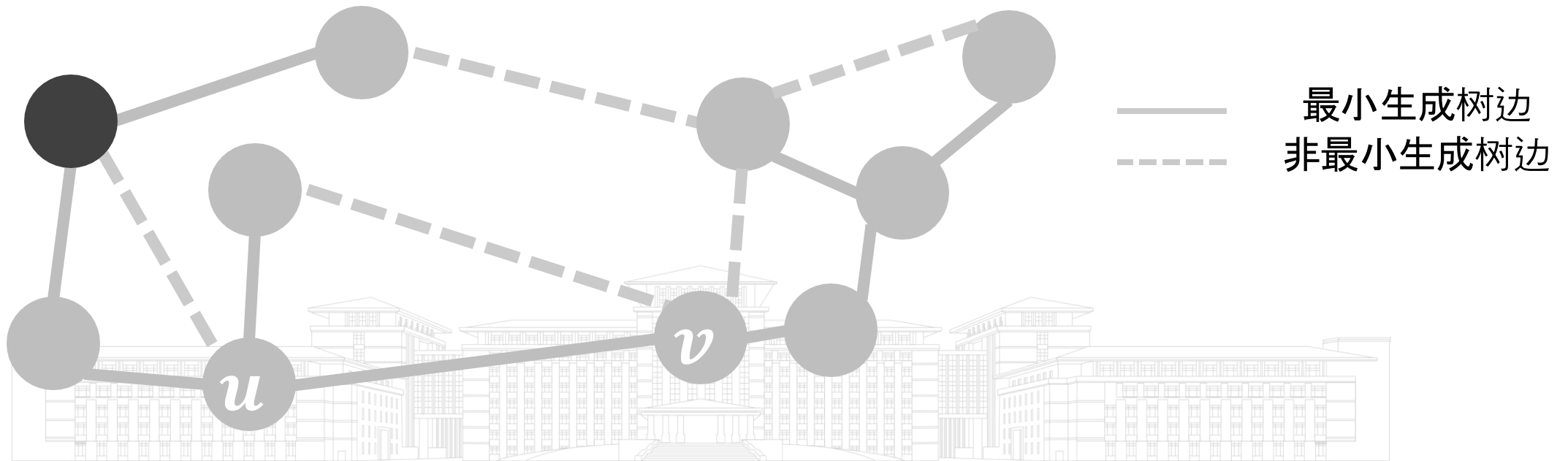
- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 货郎担问题
- 5.5 小结



Prim算法

- 算法思想

- 步骤1：选择任意一个顶点，作为生成树的起始顶点

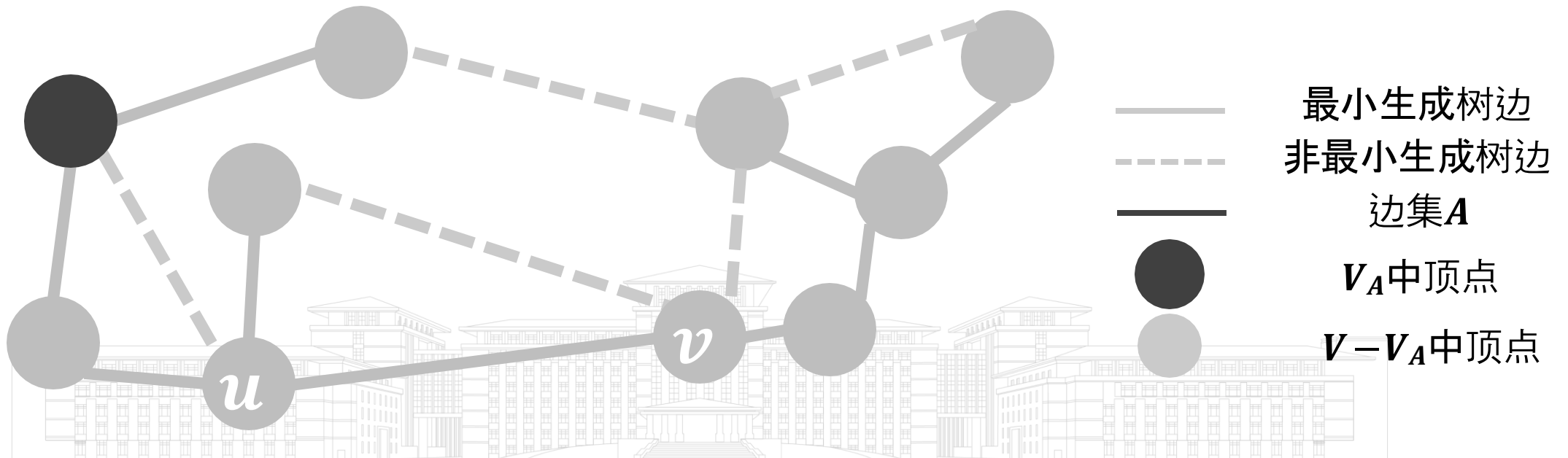


Prim算法

- 算法思想

- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$

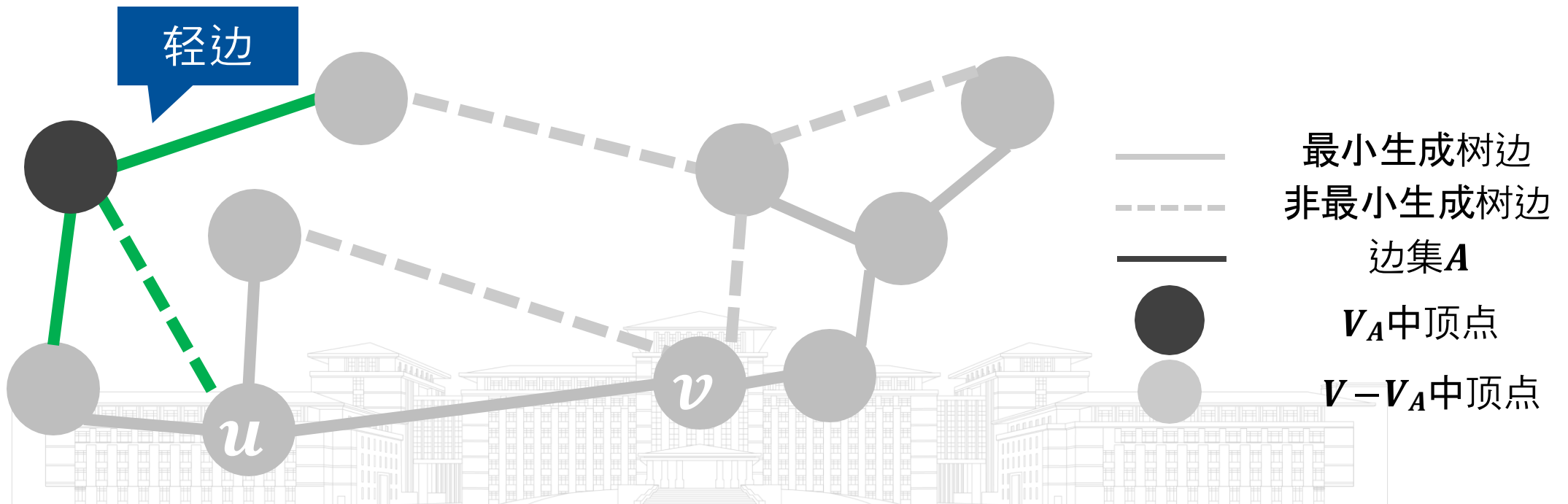
边集 A 的顶点



Prim算法

• 算法思想

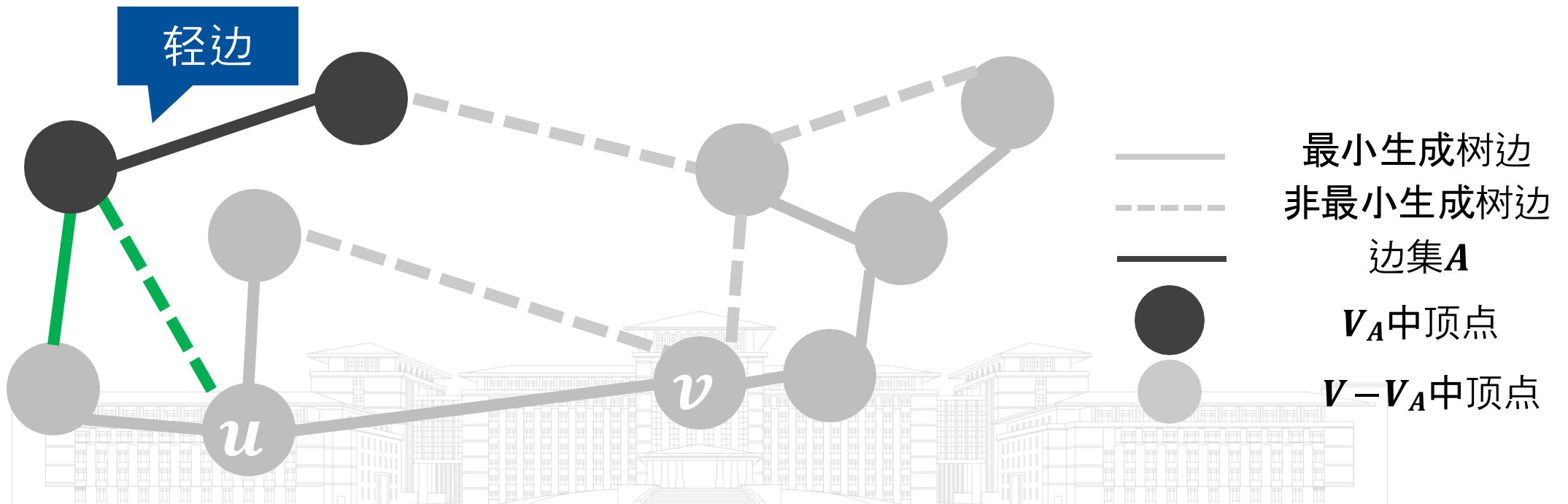
- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$
- 步骤3：选择横跨割 $(V_A, V - V_A)$ 的轻边



Prim算法

• 算法思想

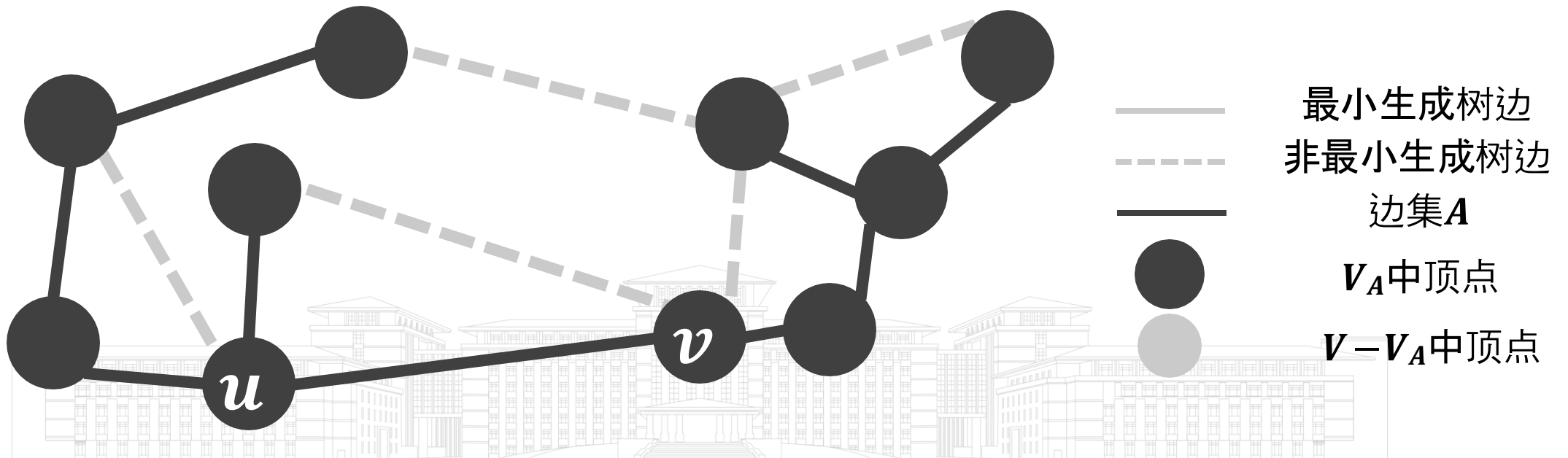
- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$
- 步骤3：选择横跨割 $(V_A, V - V_A)$ 的轻边，添加到边集 A 中



Prim算法

• 算法思想

- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$
- 步骤3：选择横跨割 $(V_A, V - V_A)$ 的轻边，添加到边集 A 中
- 步骤4：重复步骤2和步骤3，直至覆盖所有顶点

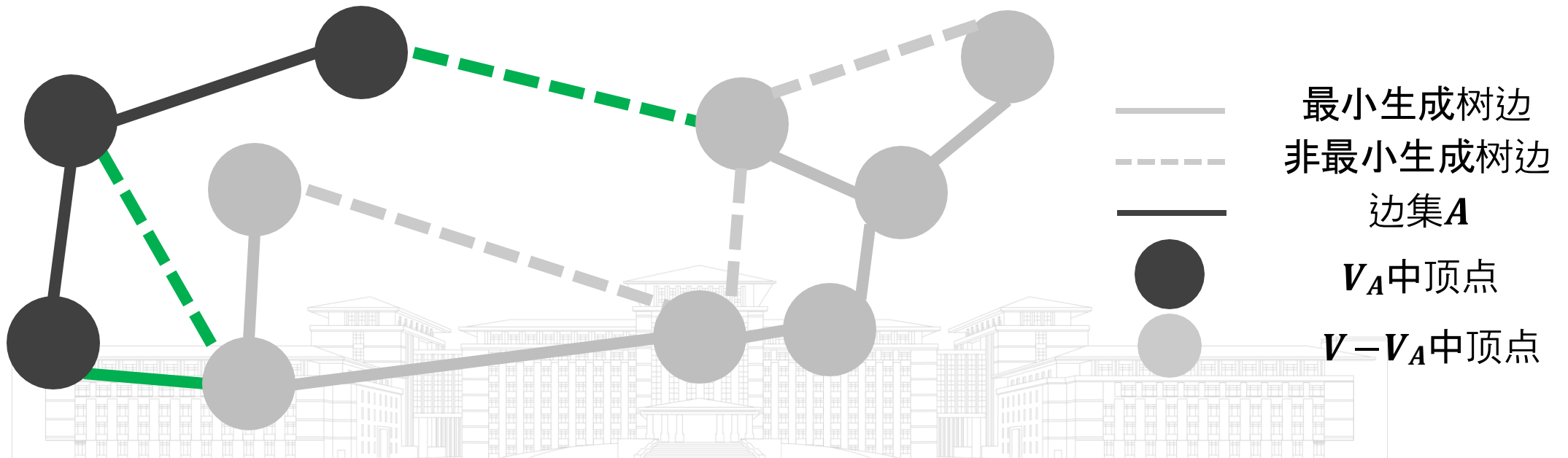


Prim算法

- 辅助数组

- **color**表示顶点状态

- 黑色顶点 u 已覆盖, $u \in V_A$, 白色顶点 u 未覆盖, $u \in V - V_A$



Prim算法

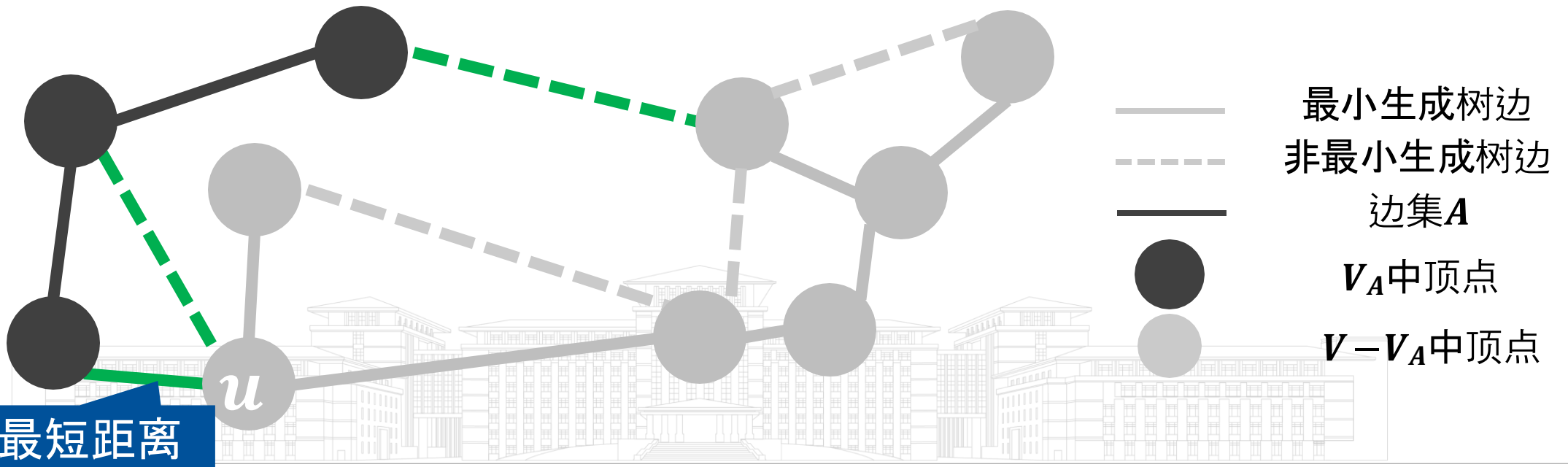
- 辅助数组

- **color**表示顶点状态

- 黑色顶点 u 已覆盖, $u \in V_A$, 白色顶点 u 未覆盖, $u \in V - V_A$

- **dist**记录横跨 $(V_A, V - V_A)$ 边的权重

- 顶点集 V_A 到顶点 u 的最短距离: $dist[u] = \min_{x \in V_A} w(x, u)$



Prim算法

- 辅助数组

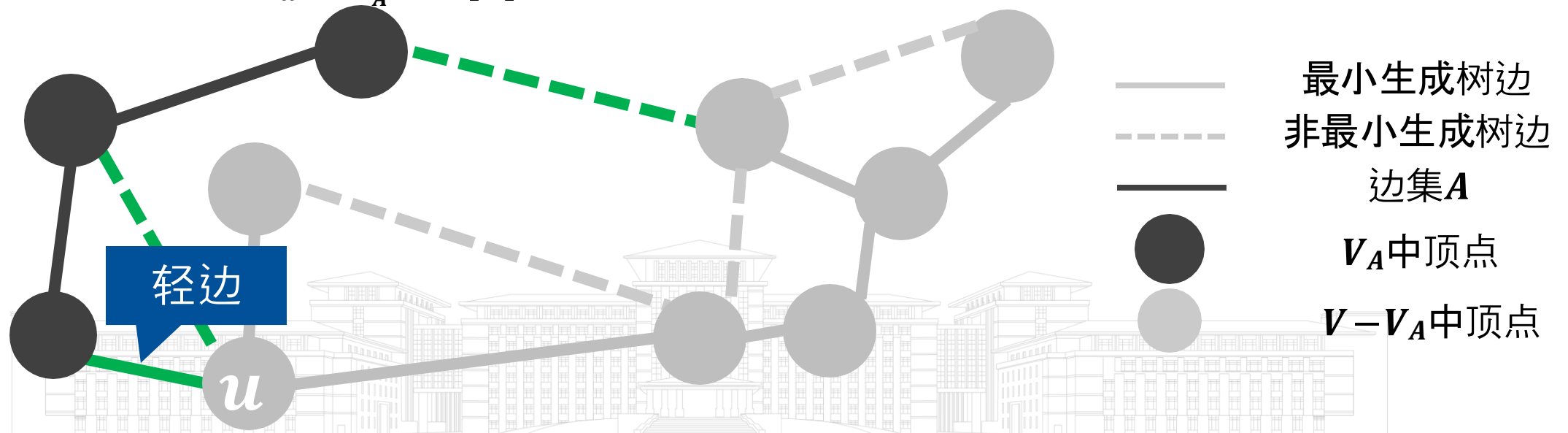
- **color**表示顶点状态

- 黑色顶点 u 已覆盖, $u \in V_A$, 白色顶点 u 未覆盖, $u \in V - V_A$

- **dist**记录横跨 $(V_A, V - V_A)$ 边的权重

- 顶点集 V_A 到顶点 u 的最短距离: $dist[u] = \min_{x \in V_A} w(x, u)$

- 轻边: $\min_{u \in V - V_A} dist[u]$



Prim算法

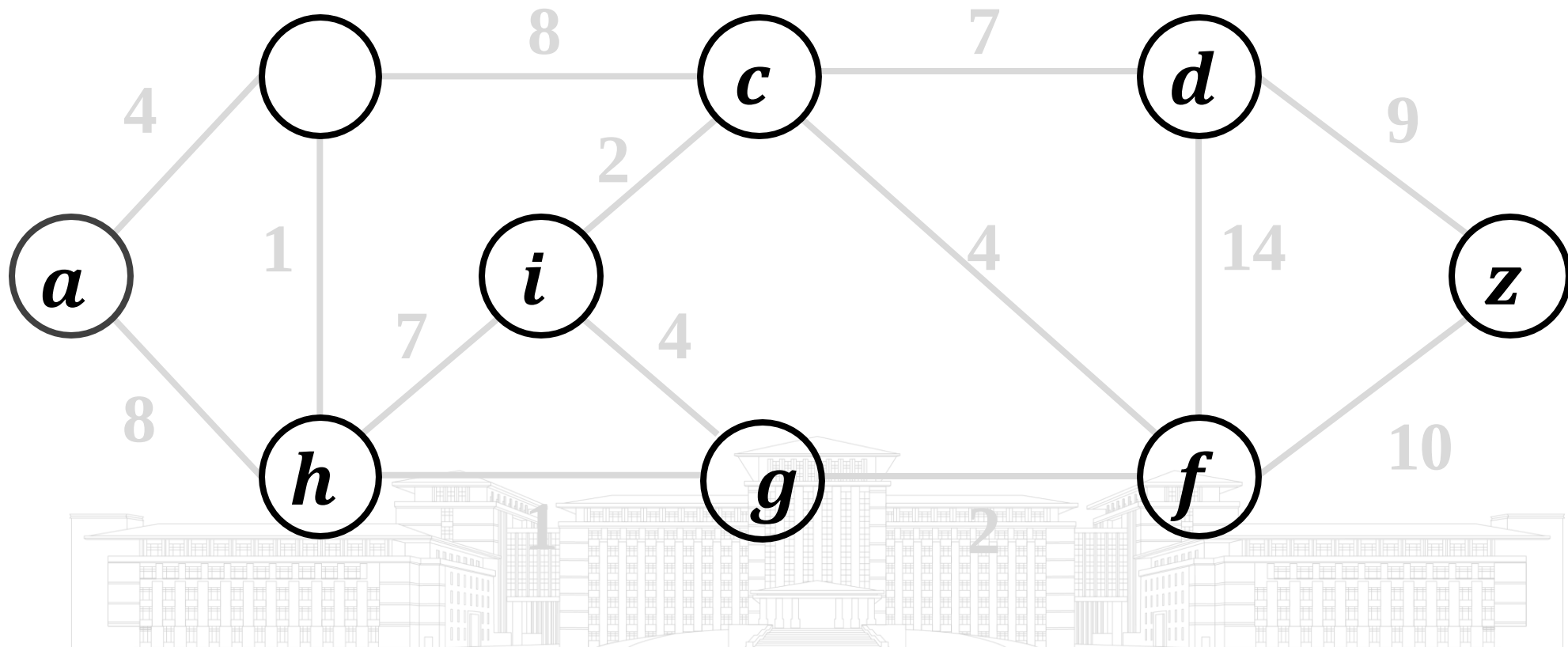
- 辅助数组
 - **color**表示顶点状态
 - 黑色顶点 u 已覆盖, $u \in V_A$, 白色顶点 u 未覆盖, $u \in V - V_A$
 - **dist**记录横跨 $(V_A, V - V_A)$ 边的权重
 - 顶点集 V_A 到顶点 u 的最短距离: $dist[u] = \min_{x \in V_A} w(x, u)$
 - 轻边: $\min_{u \in V - V_A} dist[u]$
 - **pred**表示前驱顶点
 - $(pred[u], u)$ 为最小生成树的边



算法实例



<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	W	W	W	W	W	W	W	W	W
<i>dist</i>	∞	∞	∞	∞	∞	∞	∞	∞	∞
<i>pred</i>	N	N	N	N	N	N	N	N	N

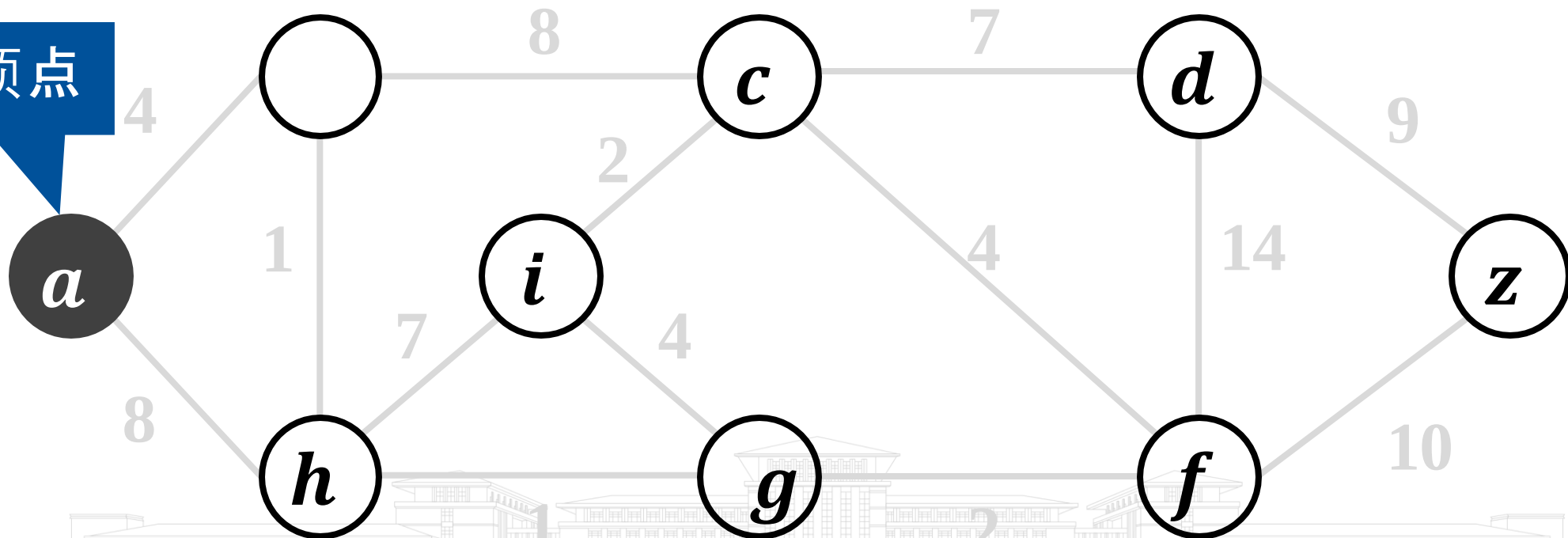


算法实例



<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	W	W	W	W	W	W	W	W
<i>dist</i>	0	∞	∞	∞	∞	∞	∞	∞	∞
<i>pred</i>	N	N	N	N	N	N	N	N	N

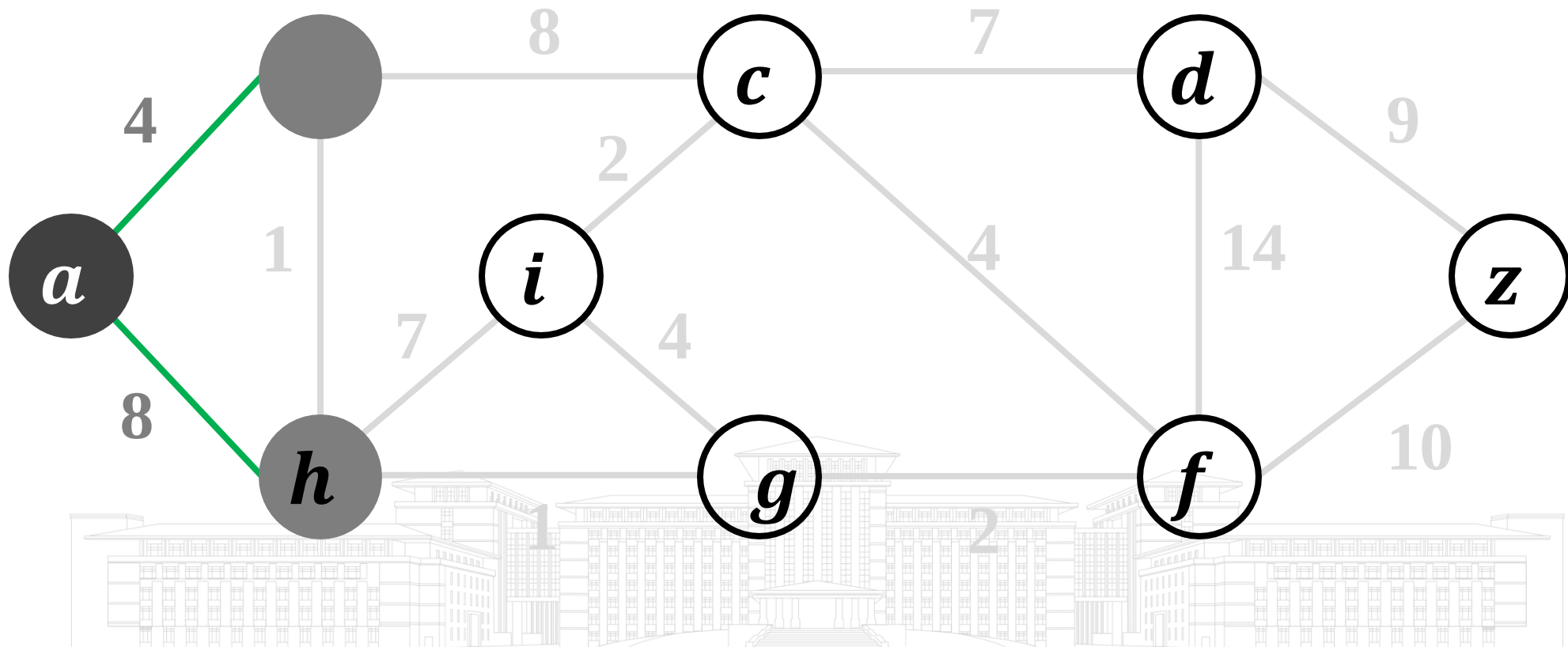
起始顶点



算法实例



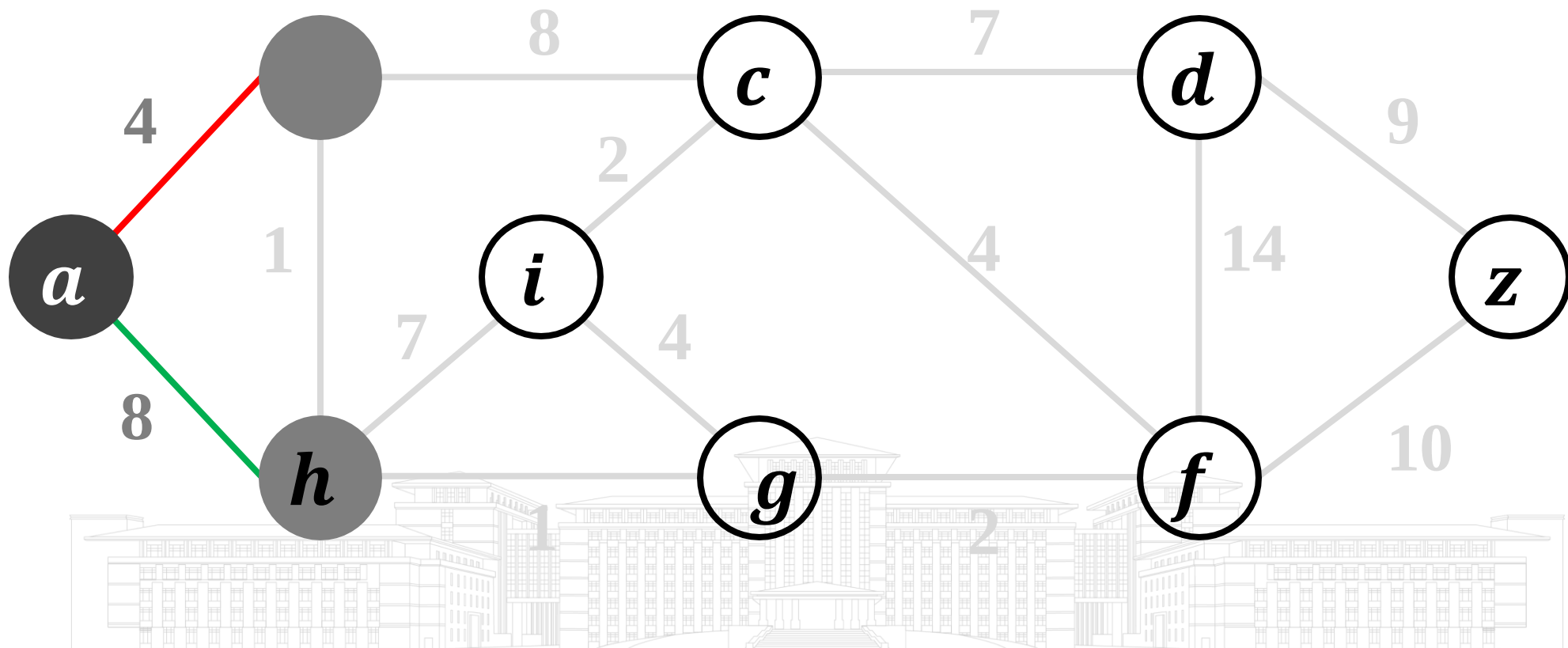
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	W	W	W	W	W	W	W	W
<i>dist</i>	0	4	∞	∞	∞	∞	8	∞	∞
<i>pred</i>	N	<i>a</i>	N	N	N	N	<i>a</i>	N	N



算法实例



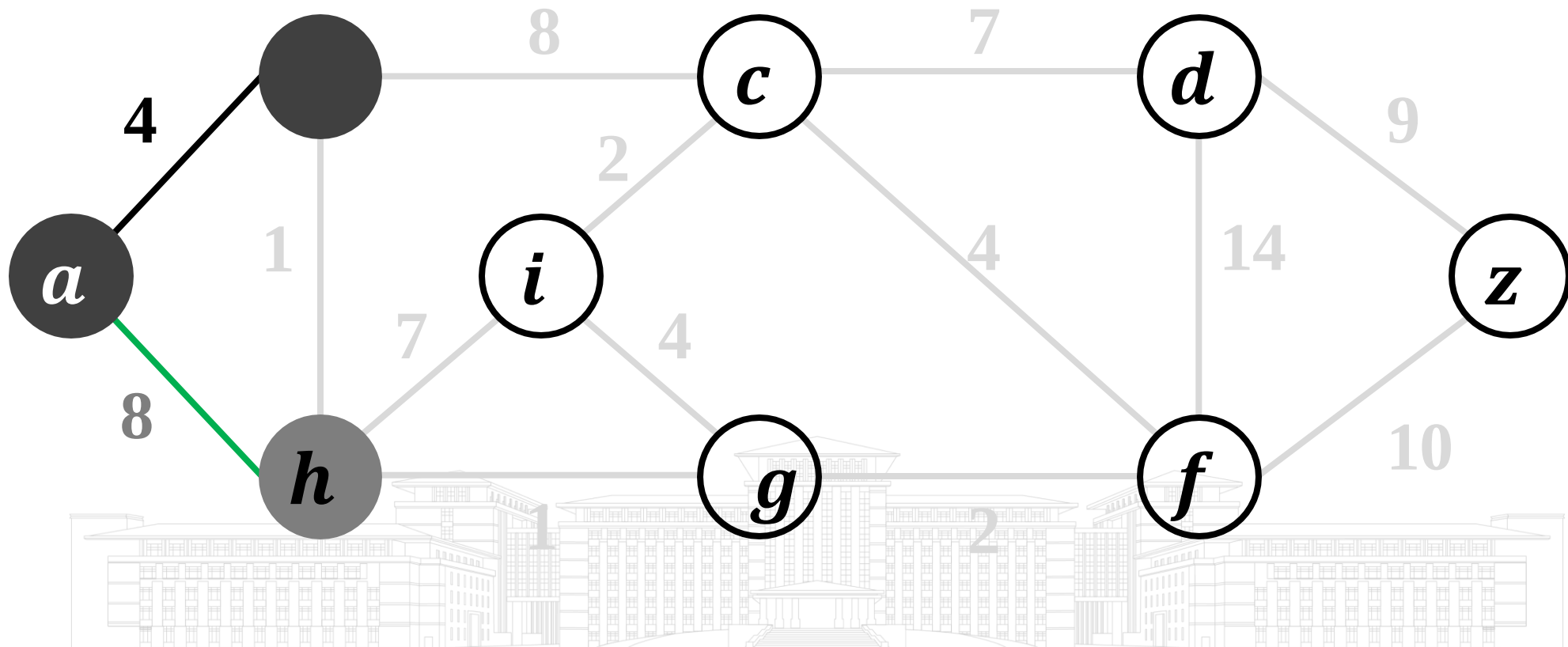
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	W	W	W	W	W	W	W	W
<i>dist</i>	0	4	∞	∞	∞	∞	8	∞	∞
<i>pred</i>	N	<i>a</i>	N	N	N	N	<i>a</i>	N	N



算法实例



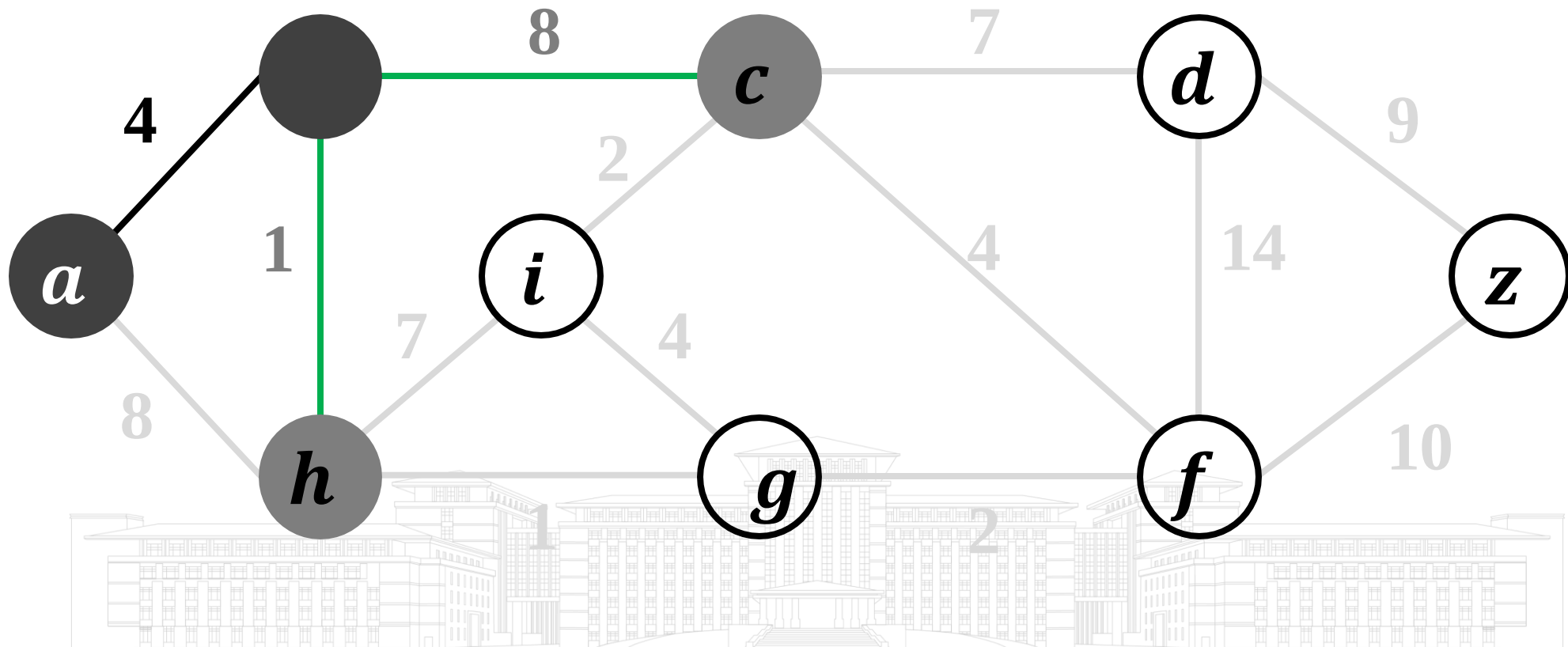
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	W	W	W	W
<i>dist</i>	0	4	∞	∞	∞	∞	8	∞	∞
<i>pred</i>	N	<i>a</i>	N	N	N	N	<i>a</i>	N	N



算法实例



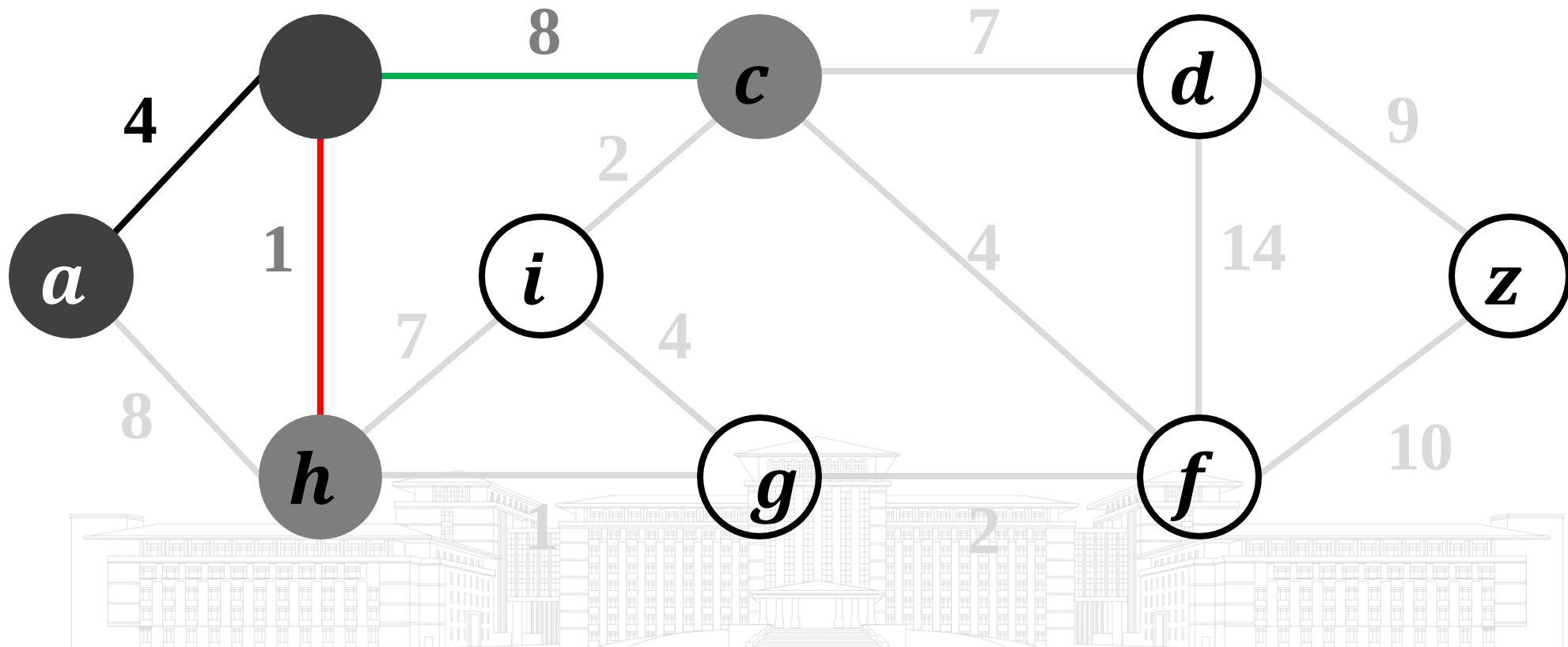
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	W	W	W	W
<i>dist</i>	0	4	8	∞	∞	∞	1	∞	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	N	N	<i>b</i>	N	N



算法实例



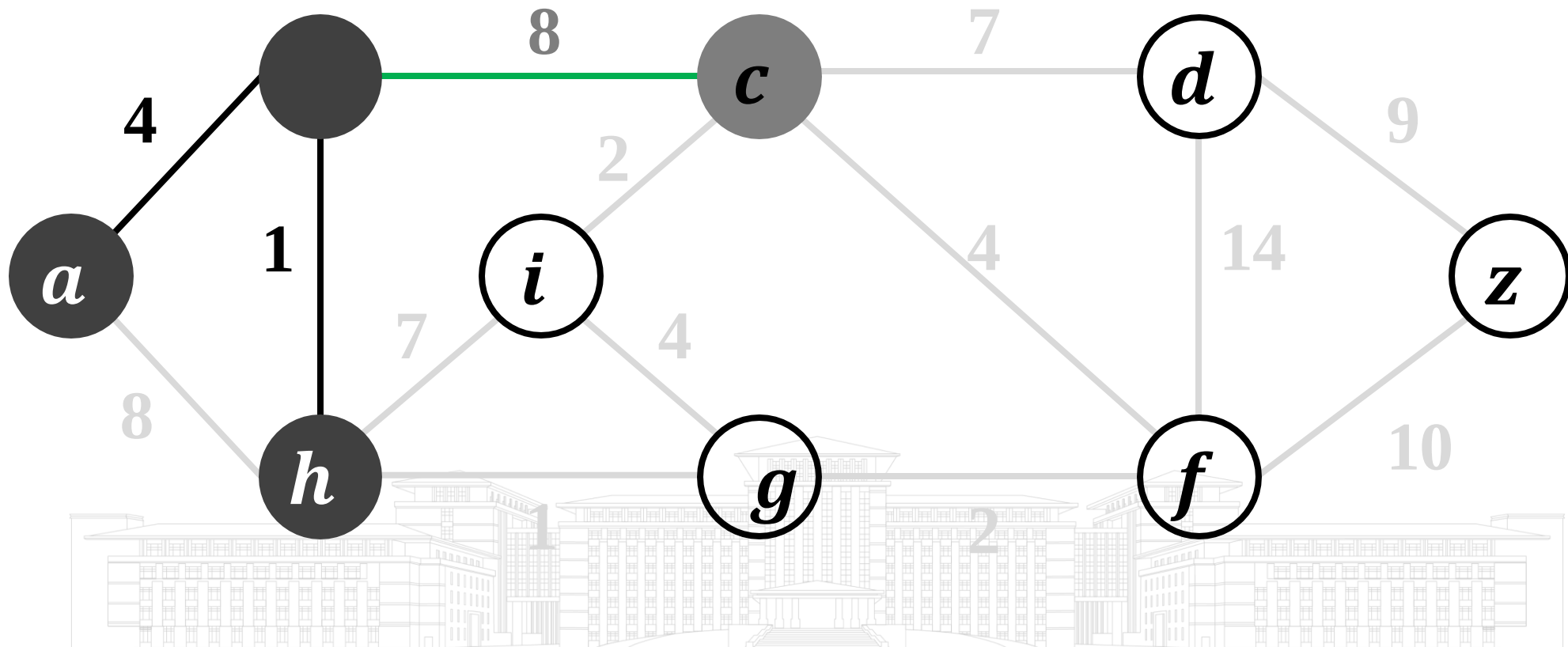
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	W	W	W	W
<i>dist</i>	0	4	8	∞	∞	∞	1	∞	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	N	N	<i>b</i>	N	N



算法实例



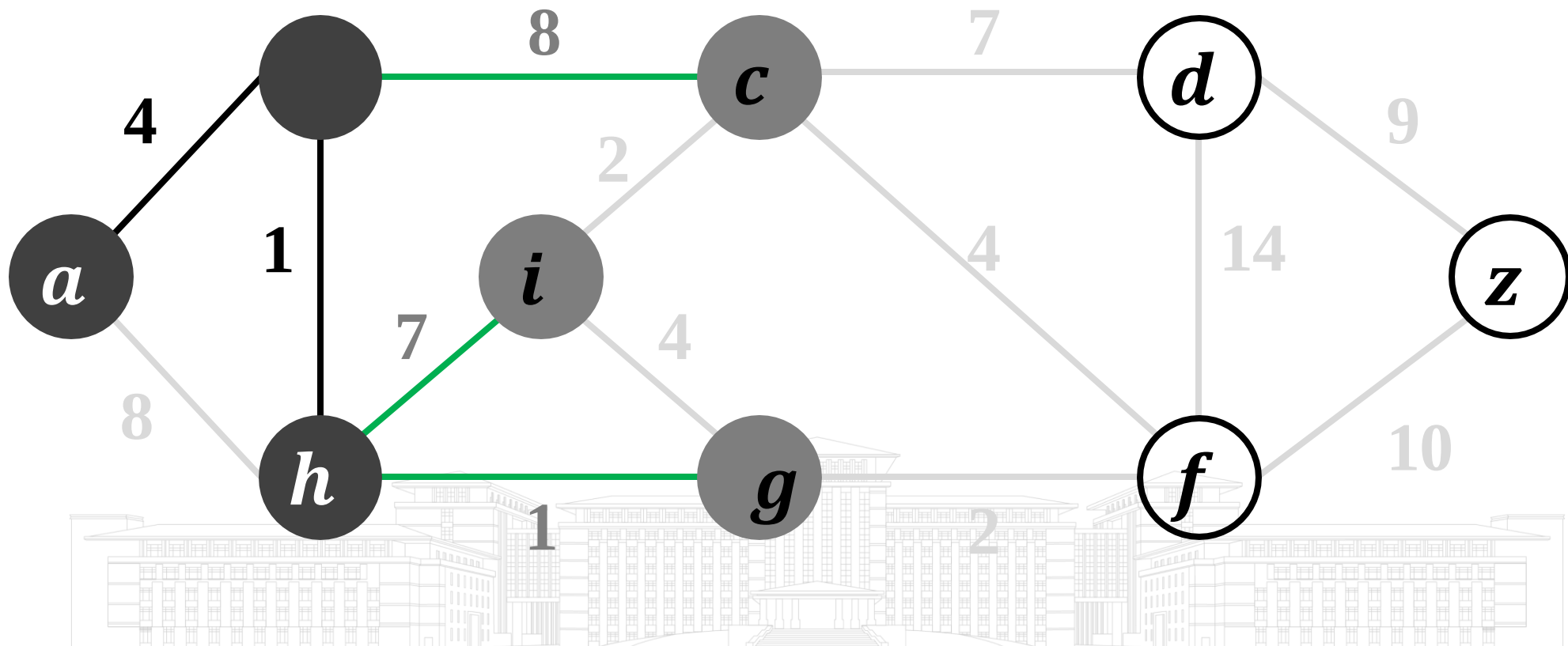
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	W	B	W	W
<i>dist</i>	0	4	8	∞	∞	∞	1	∞	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	N	N	<i>b</i>	N	N



算法实例



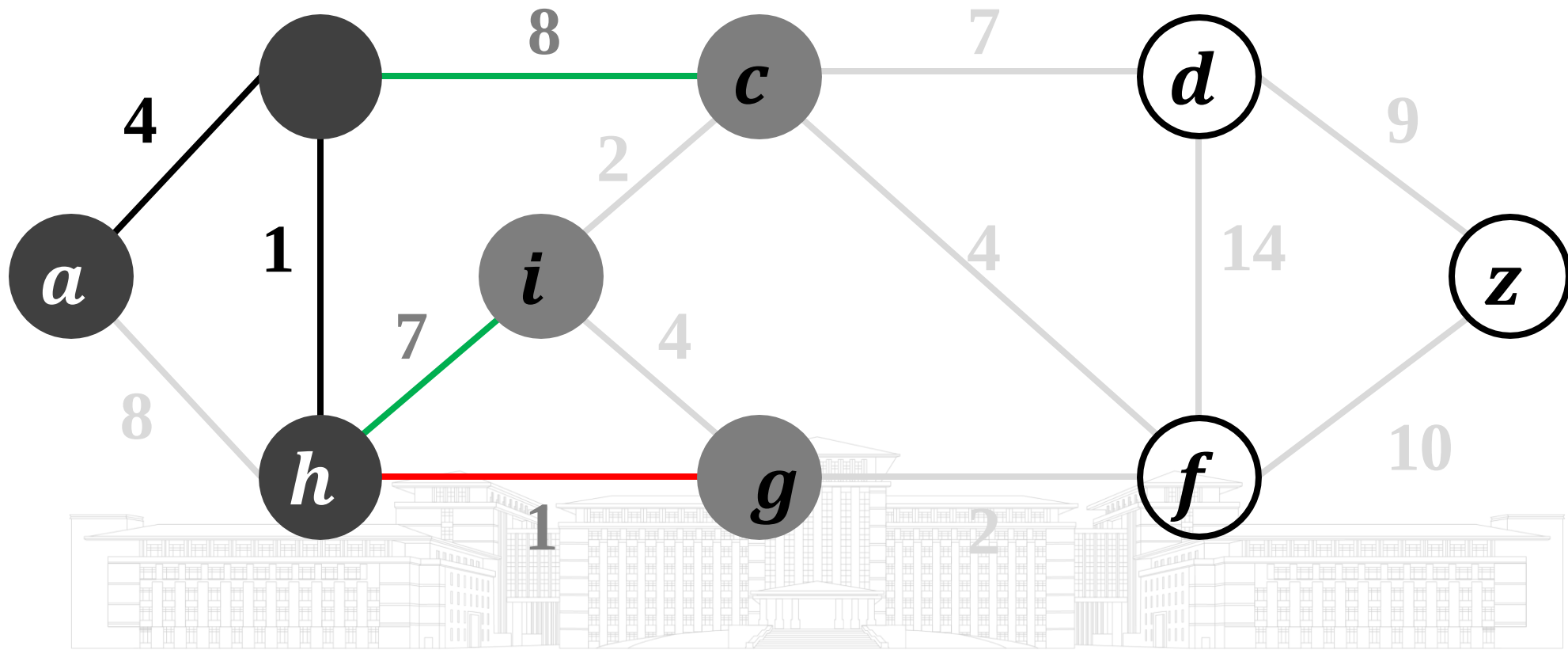
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	W	B	W	W
<i>dist</i>	0	4	8	∞	∞	1	1	7	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	N	<i>h</i>	<i>b</i>	<i>h</i>	N



算法实例



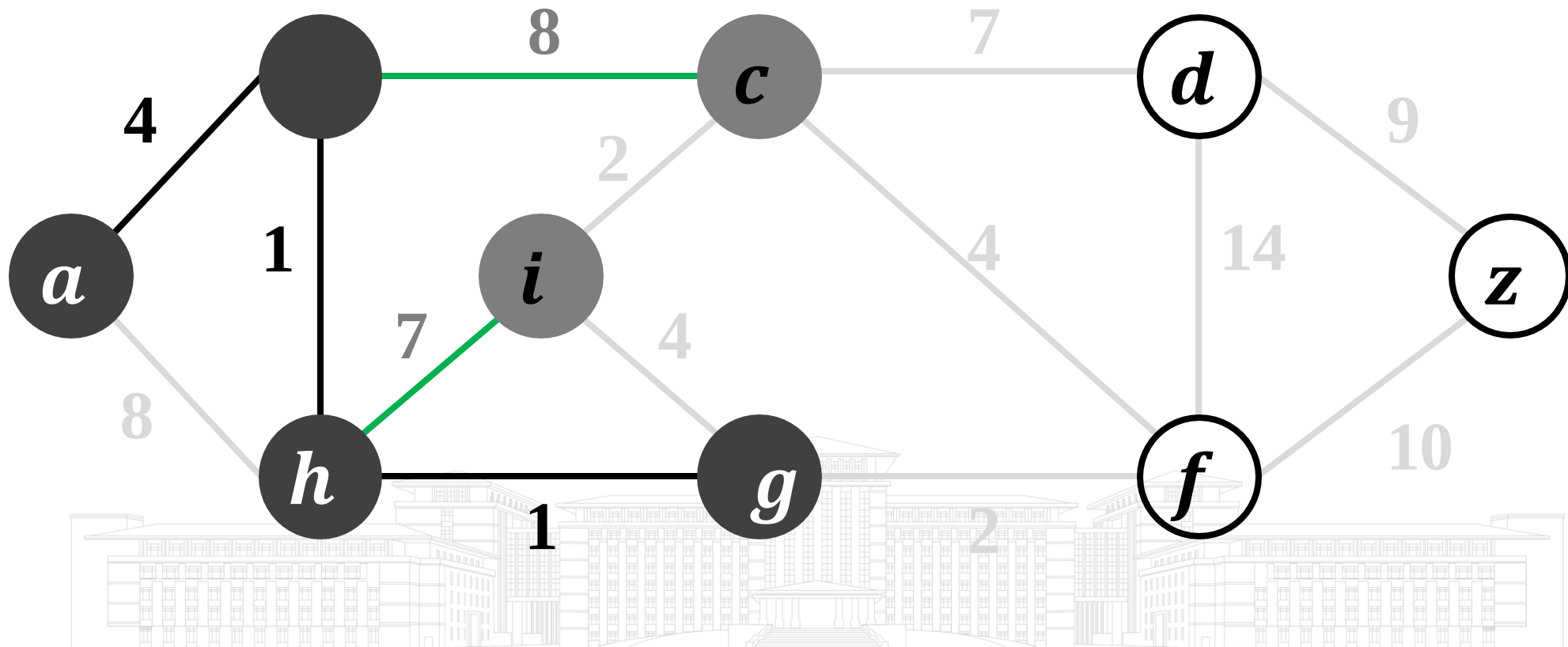
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	W	B	W	W
<i>dist</i>	0	4	8	∞	∞	1	1	7	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	N	<i>h</i>	<i>b</i>	<i>h</i>	N



算法实例



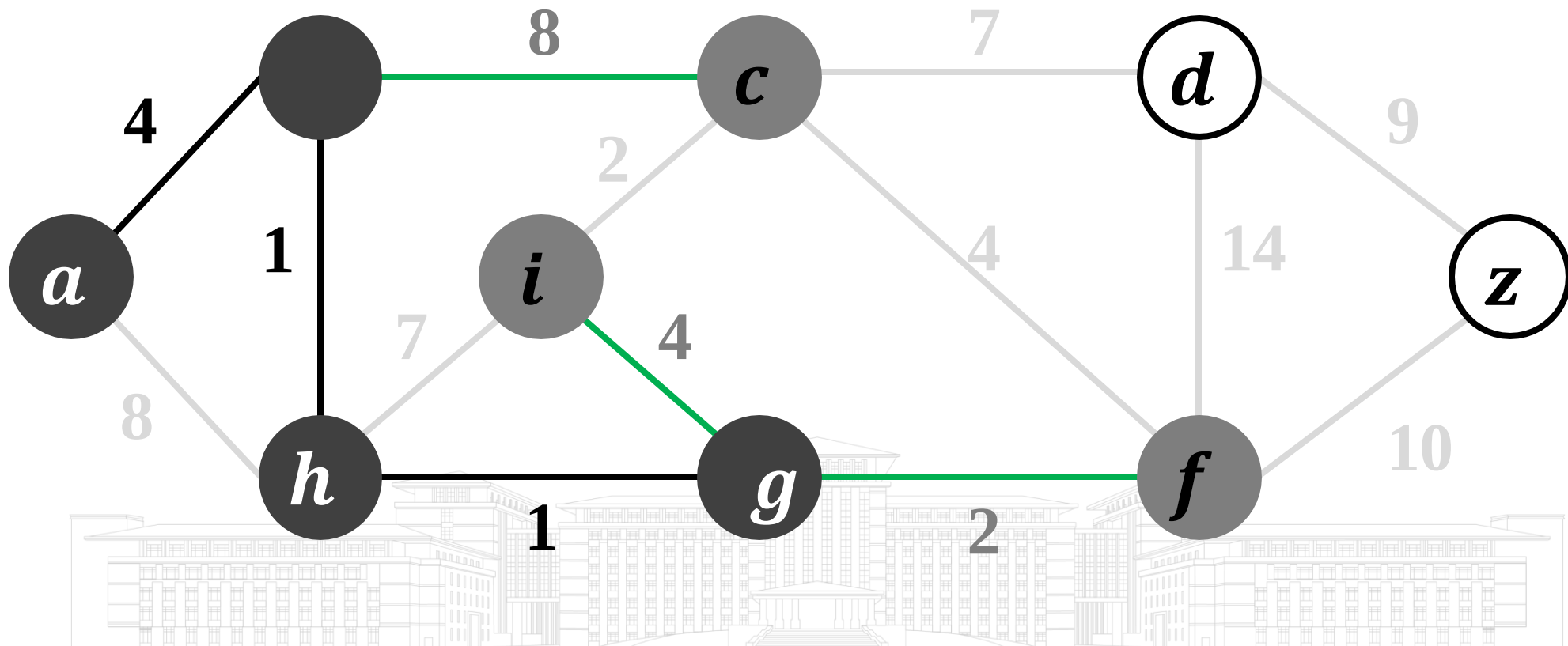
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	B	B	W	W
<i>dist</i>	0	4	8	∞	∞	1	1	7	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	N	<i>h</i>	<i>b</i>	<i>h</i>	N



算法实例



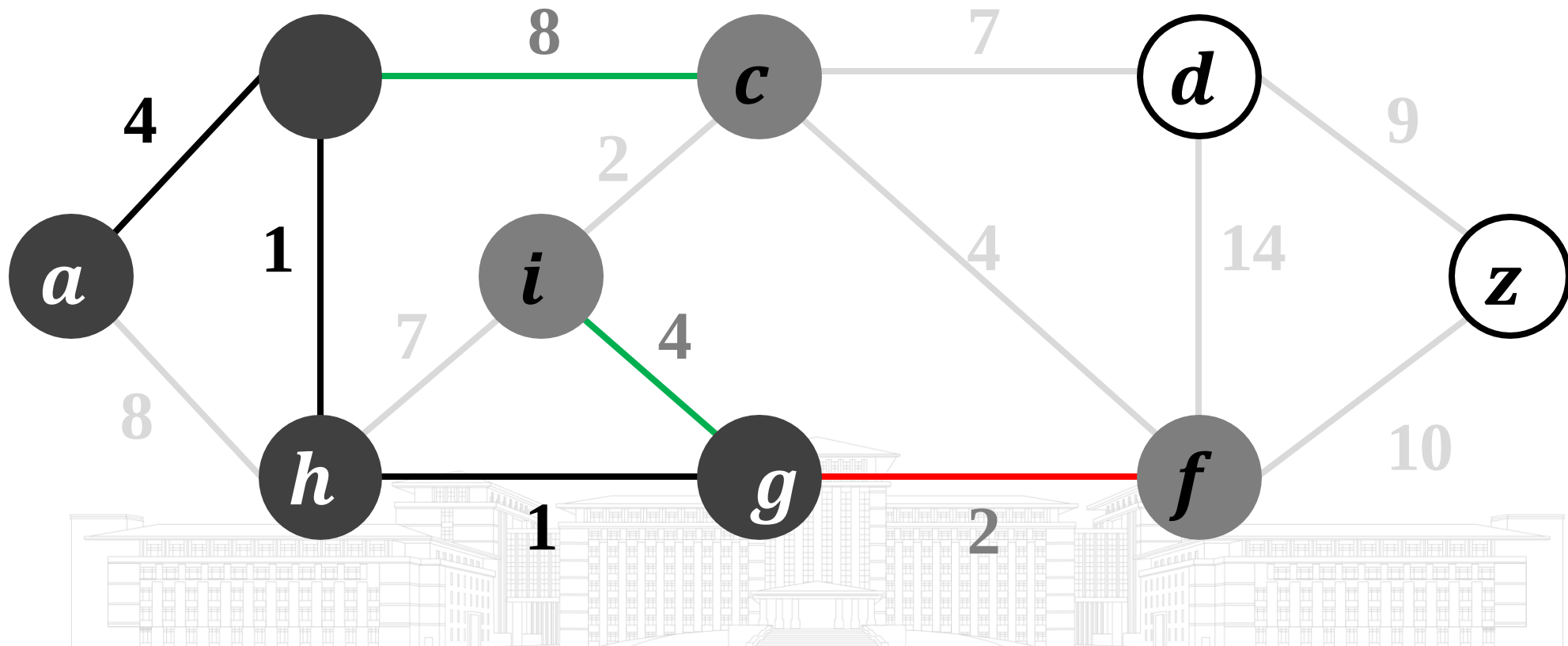
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	B	B	W	W
<i>dist</i>	0	4	8	∞	2	1	1	4	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	N



算法实例



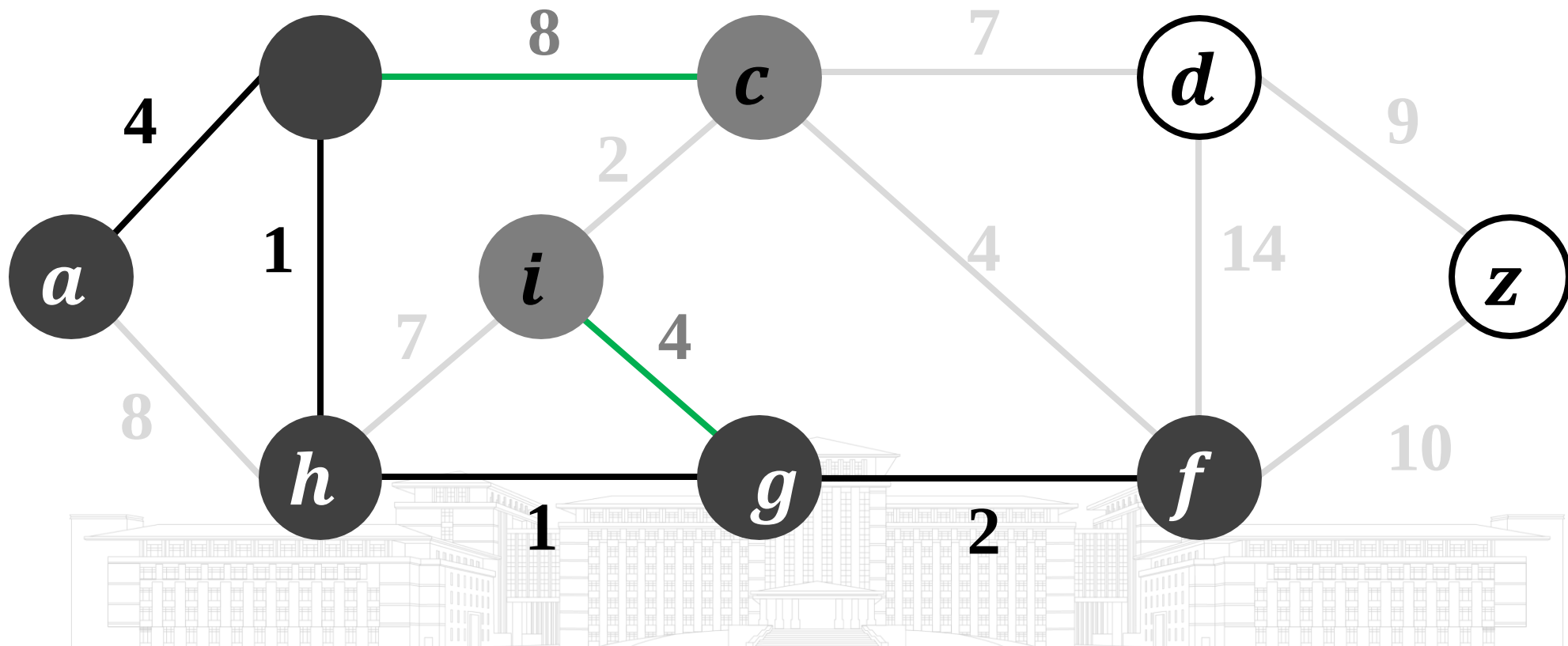
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	W	B	B	W	W
<i>dist</i>	0	4	8	∞	2	1	1	4	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	N



算法实例



<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	B	B	B	W	W
<i>dist</i>	0	4	8	∞	2	1	1	4	∞
<i>pred</i>	N	<i>a</i>	<i>b</i>	N	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	N

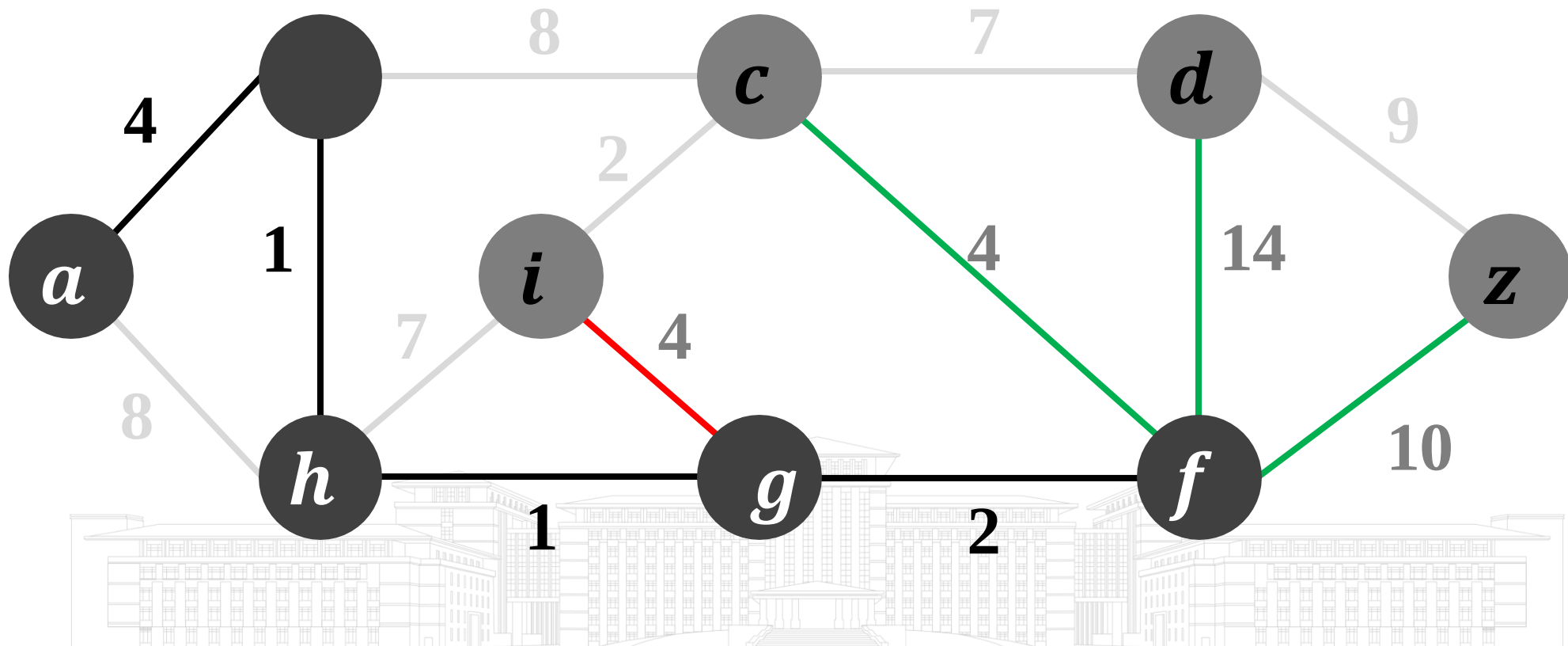




算法实例



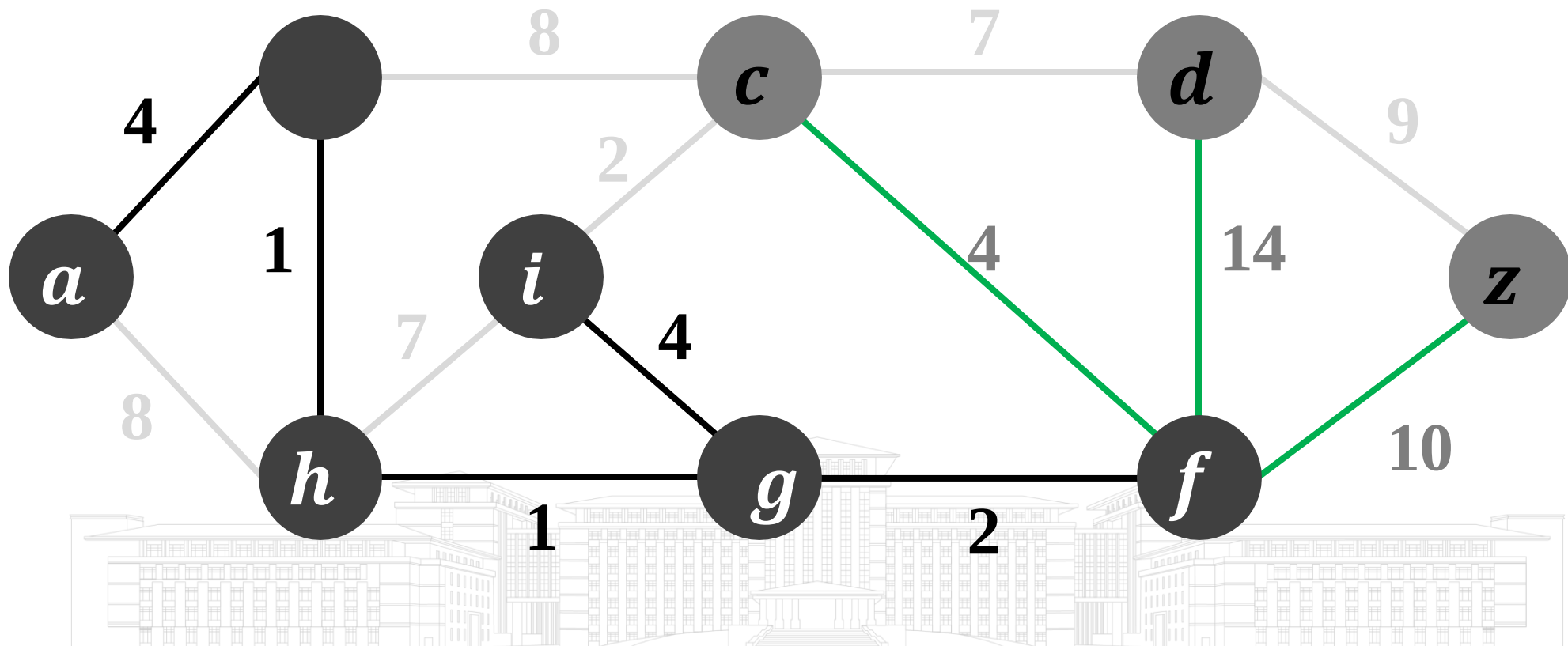
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	B	B	B	W	W
<i>dist</i>	0	4	4	14	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>f</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



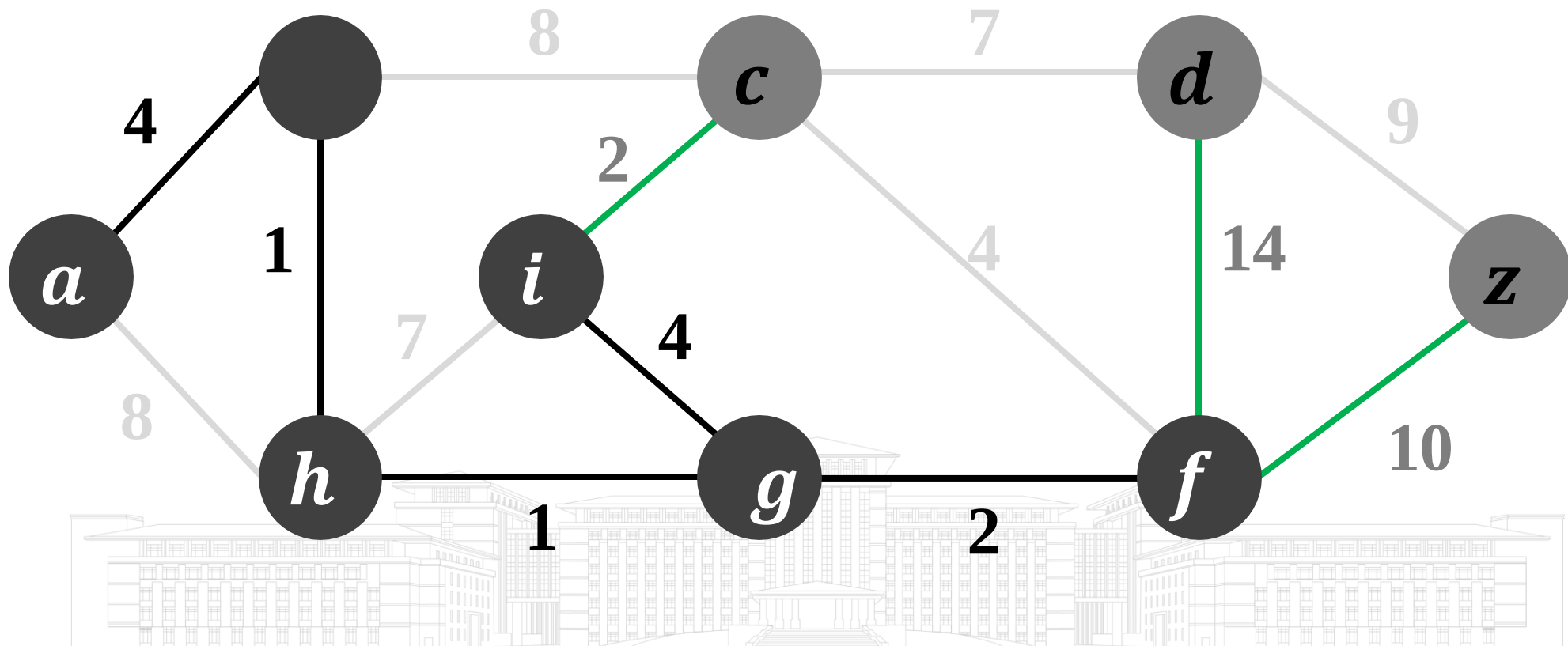
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	B	B	B	B	W
<i>dist</i>	0	4	4	14	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>f</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



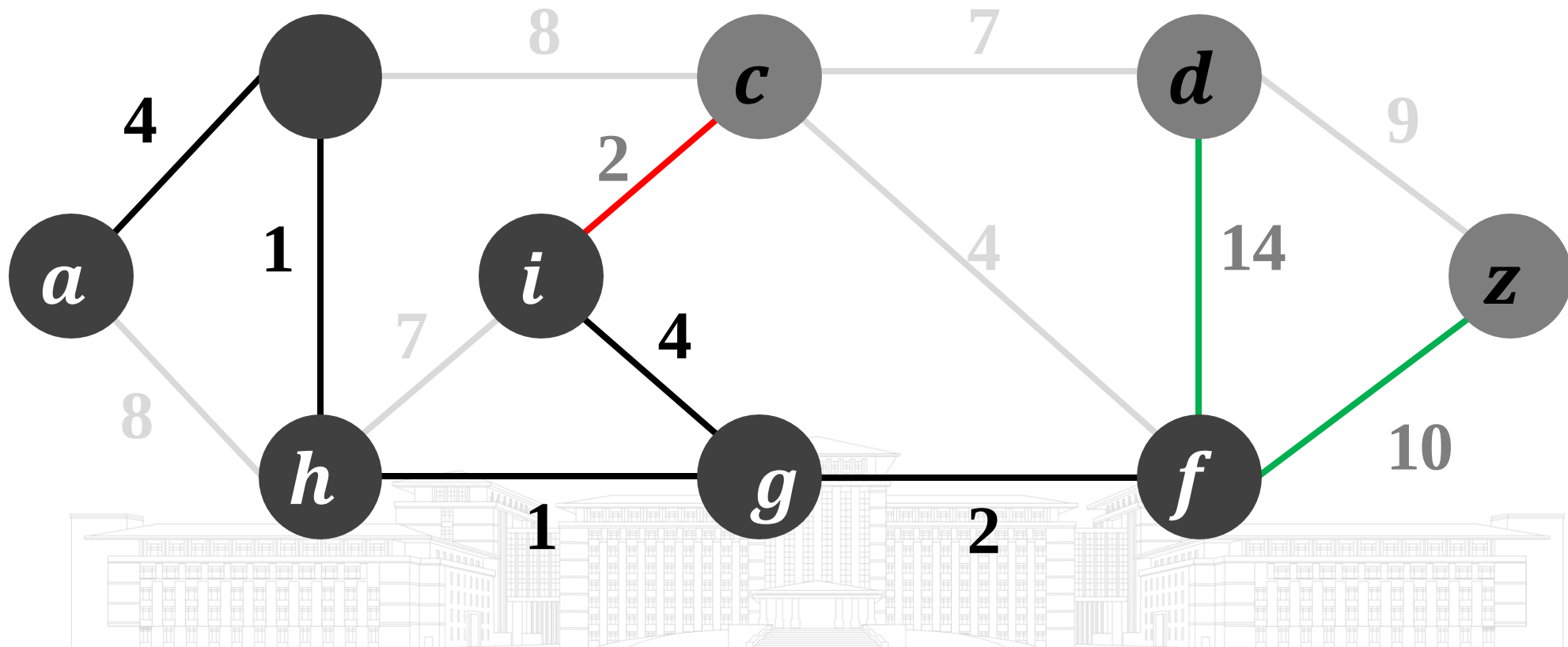
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	B	B	B	B	W
<i>dist</i>	0	4	2	14	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



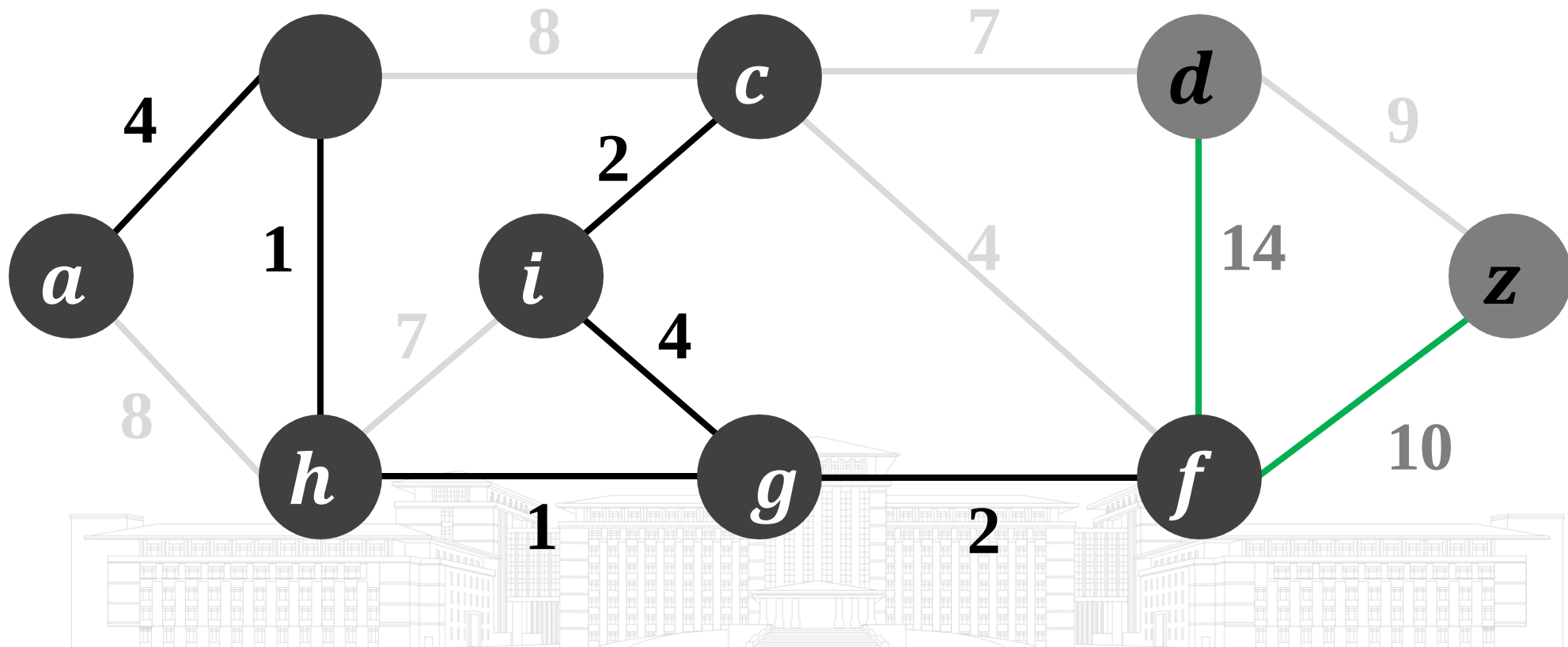
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	W	W	B	B	B	B	W
<i>dist</i>	0	4	2	14	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



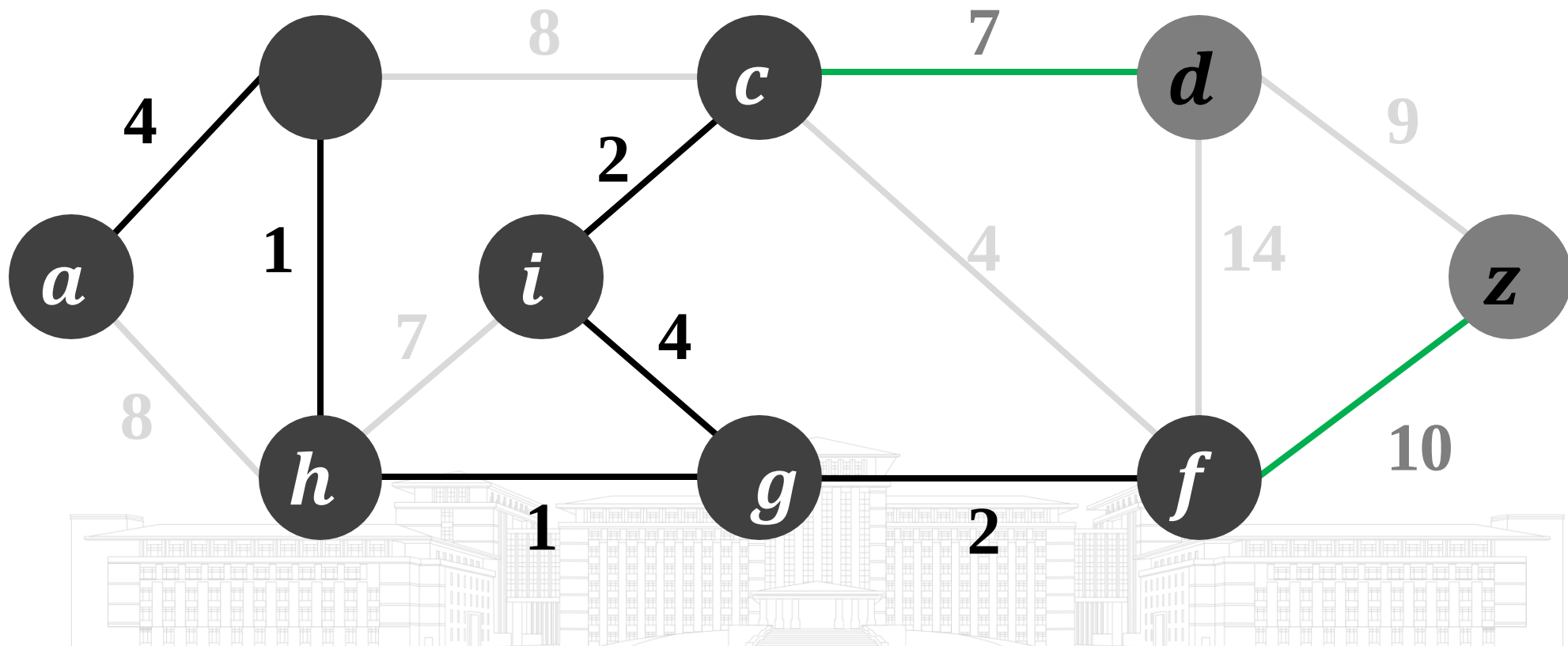
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	W	B	B	B	B	W
<i>dist</i>	0	4	2	14	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



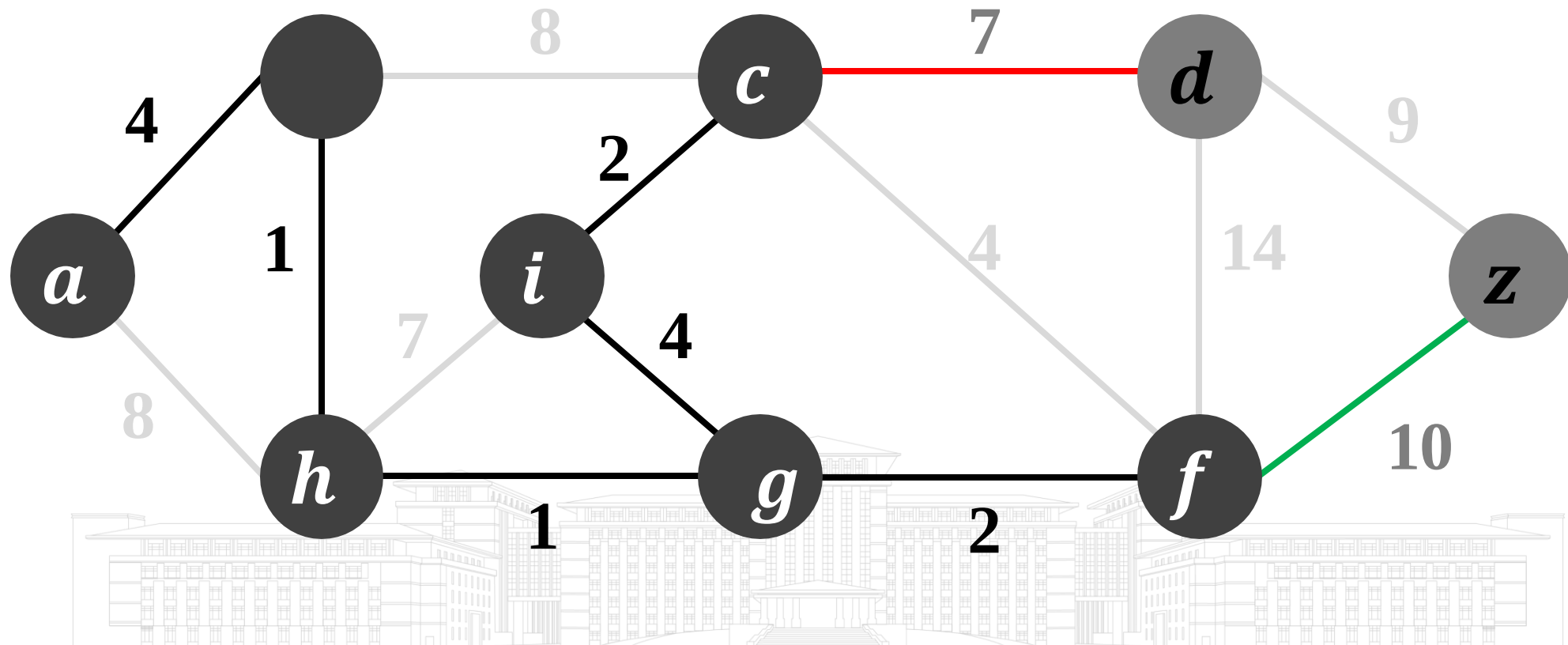
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	W	B	B	B	B	W
<i>dist</i>	0	4	2	7	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



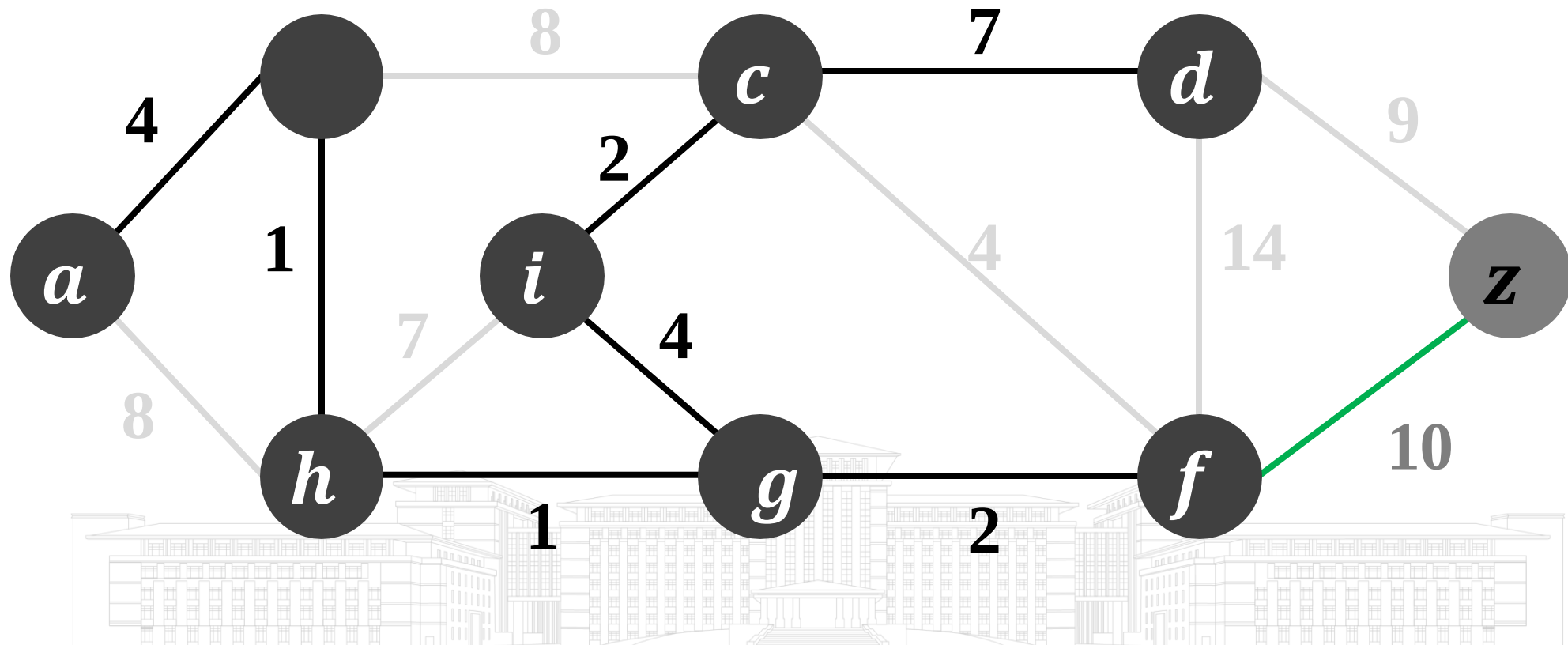
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	W	B	B	B	B	W
<i>dist</i>	0	4	2	7	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



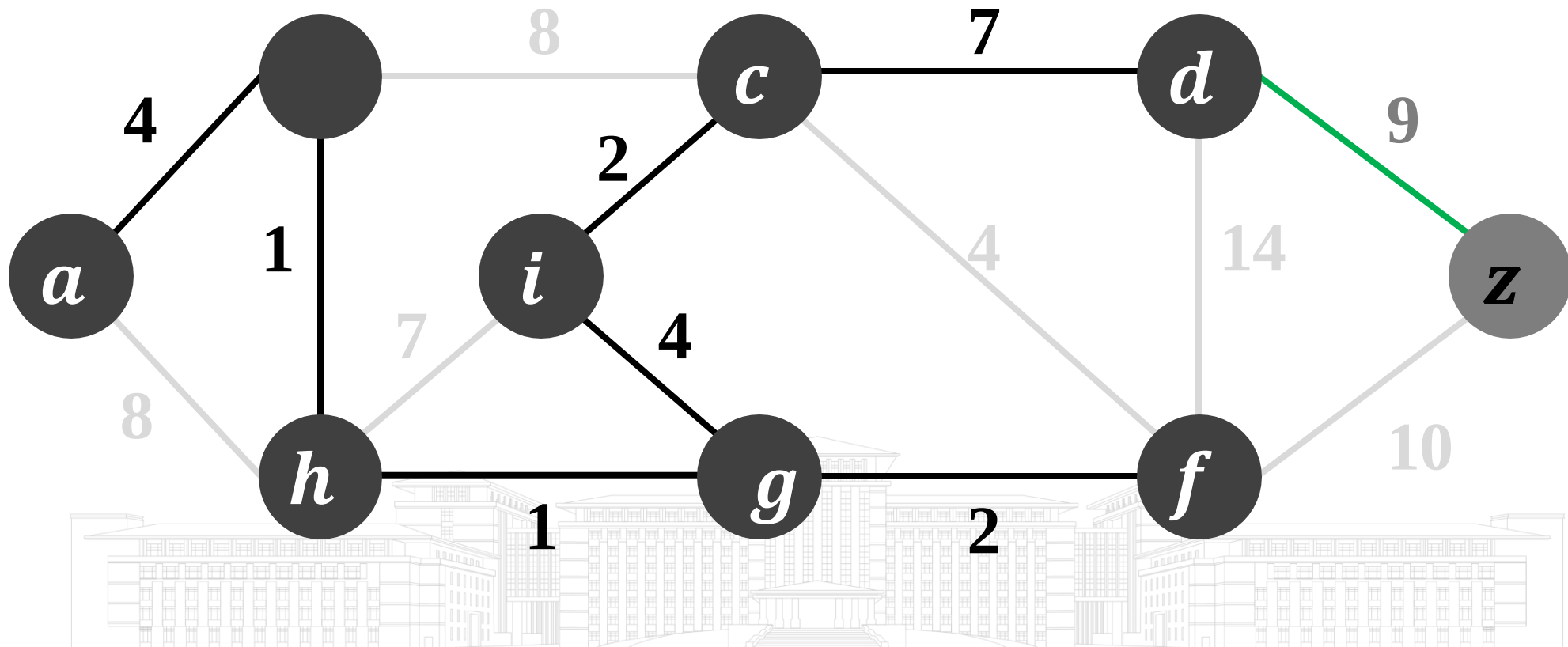
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	B	B	B	B	B	W
<i>dist</i>	0	4	2	7	2	1	1	4	10
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>f</i>



算法实例



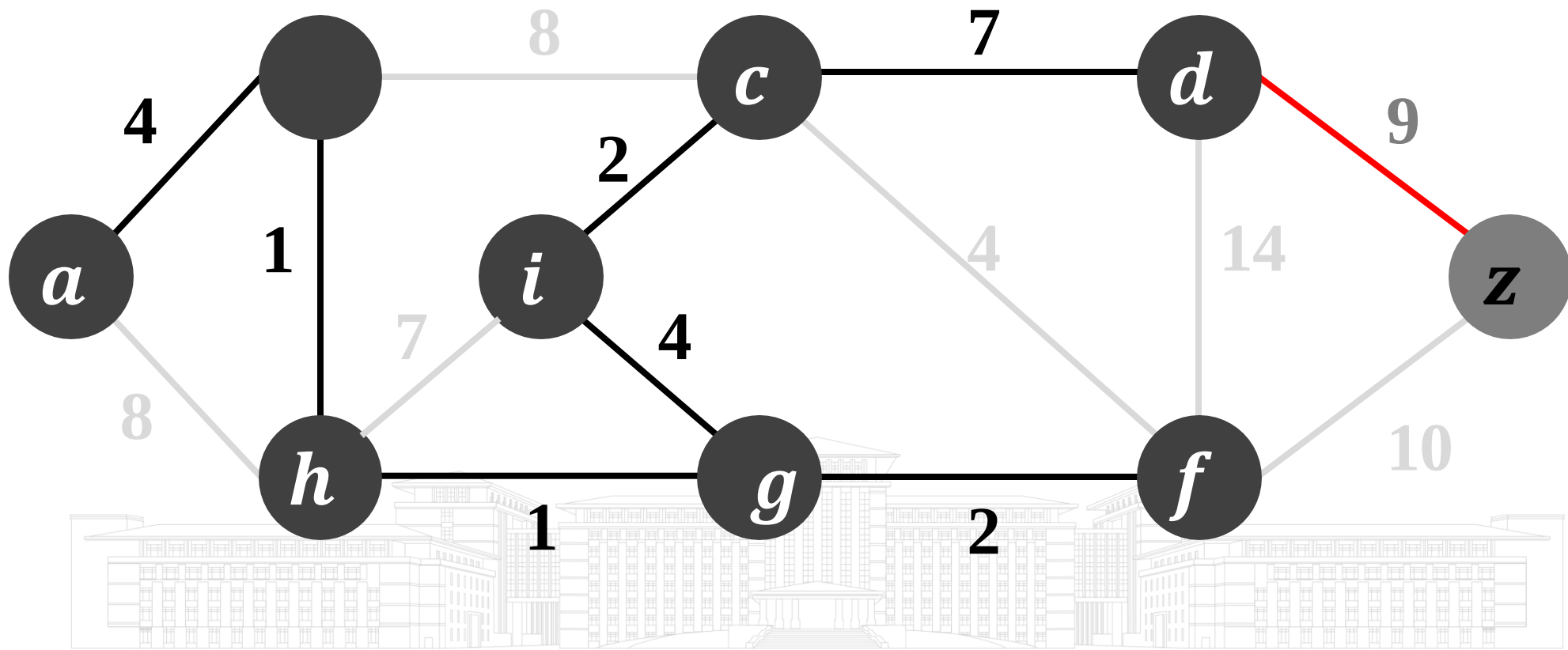
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	B	B	B	B	B	W
<i>dist</i>	0	4	2	7	2	1	1	4	9
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>d</i>



算法实例



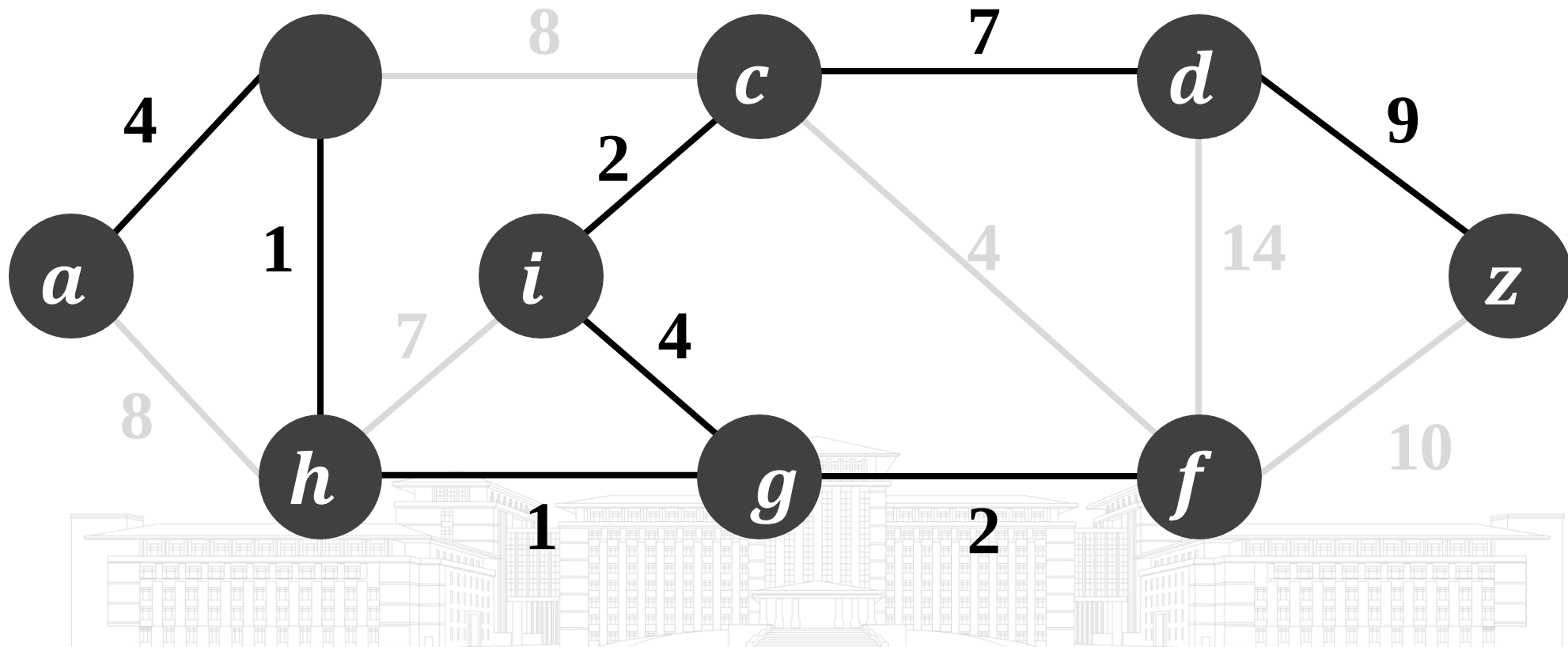
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	B	B	B	B	B	W
<i>dist</i>	0	4	2	7	2	1	1	4	9
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>d</i>



算法实例



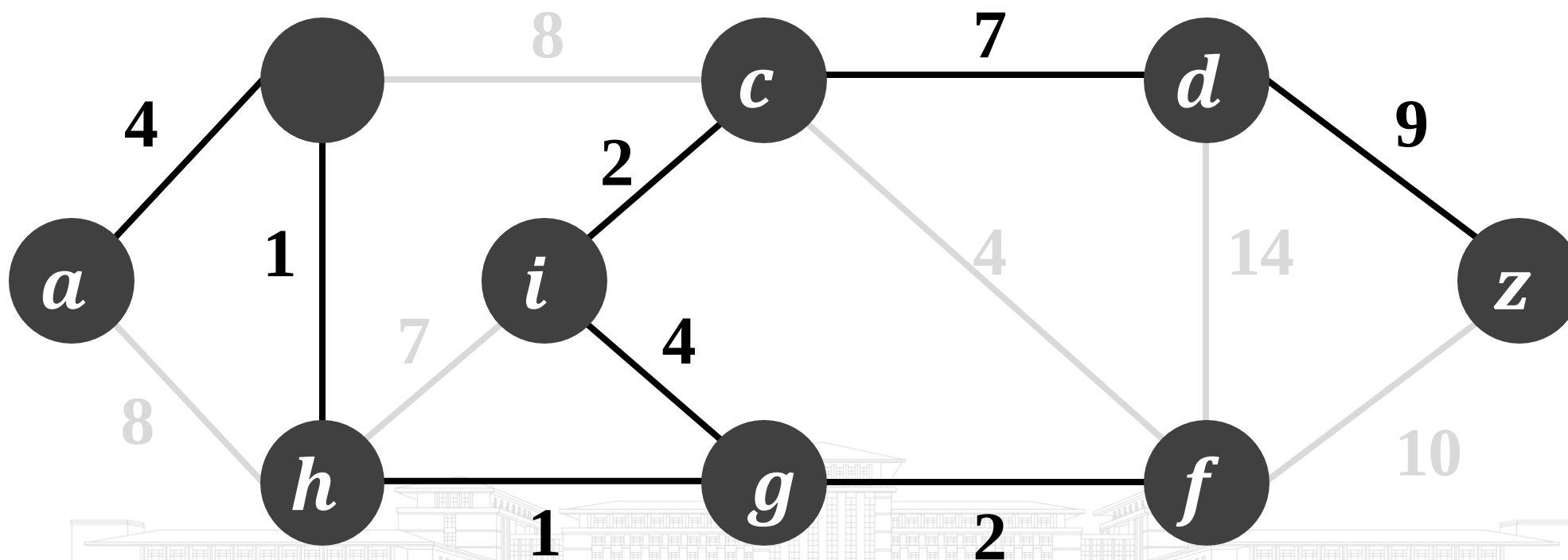
<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	B	B	B	B	B	B
<i>dist</i>	0	4	2	7	2	1	1	4	9
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>d</i>



算法实例



<i>V</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>z</i>
<i>color</i>	B	B	B	B	B	B	B	B	B
<i>dist</i>	0	4	2	7	2	1	1	4	9
<i>pred</i>	N	<i>a</i>	<i>i</i>	<i>c</i>	<i>g</i>	<i>h</i>	<i>b</i>	<i>g</i>	<i>d</i>



$$W(T) = 0 + 4 + 2 + 7 + 2 + 1 + 1 + 4 + 9 = 30$$

伪代码

● MST-Prim(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

//初始化

```
for  $u \in V$  do
     $color[u] \leftarrow WHITE$ 
     $dist[u] \leftarrow \infty$ 
     $pred[u] \leftarrow NULL$ 
```

end

$dist[1] \leftarrow 0$

初始化各个辅助数组



伪代码

• MST-Prim(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

//初始化

for $u \in V$ do

$color[u] \leftarrow WHITE$

$dist[u] \leftarrow \infty$

$pred[u] \leftarrow NULL$

end

$dist[1] \leftarrow 0$

选择任意顶点作为起点



伪代码

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

依次添加其他顶点



伪代码



//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do  
     $minDist \leftarrow \infty$   
     $rec \leftarrow 0$   
    for  $j \leftarrow 1$  to  $|V|$  do  
        if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then  
             $minDist \leftarrow dist[j]$   
             $rec \leftarrow j$   
        end  
    end  
    for  $u \in G.Adj[rec]$  do  
        if  $w(rec, u) < dist[u]$  then  
             $dist[u] \leftarrow w(rec, u)$   
             $pred[u] \leftarrow rec$   
        end  
    end  
     $color[rec] \leftarrow BLACK$   
end
```

记录最小权值



伪代码

//执行最小生成树算法

for $i \leftarrow 1$ **to** $|V|$ **do**

$minDist \leftarrow \infty$

$rec \leftarrow 0$

for $j \leftarrow 1$ **to** $|V|$ **do**

if $color[j] \neq BLACK$ **and** $dist[j] < minDist$ **then**

$minDist \leftarrow dist[j]$

$rec \leftarrow j$

end

end

for $u \in G.Adj[rec]$ **do**

if $w(rec, u) < dist[u]$ **then**

$dist[u] \leftarrow w(rec, u)$

$pred[u] \leftarrow rec$

end

end

$color[rec] \leftarrow BLACK$

end

记录安全边的端点



伪代码

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

记录新增的安全边



伪代码



//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

更新 $dist$ 数组



伪代码

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

标记顶点处理完成

复杂度分析

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

//初始化

for $u \in V$ do

$color[u] \leftarrow WHITE$

$dist[u] \leftarrow \infty$

$pred[u] \leftarrow NULL$

end

$dist[1] \leftarrow 0$

} $O(|V|)$



复杂度分析

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

$O(|V|)$

$O(\deg(u))$





复杂度分析

//执行最小生成树算法

```
for i ← 1 to |V| do
  minDist ← ∞
  rec ← 0
  for j ← 1 to |V| do
    if color[j] ≠ BLACK and dist[j] < minDist then
      minDist ← dist[j]
      rec ← j
    end
  end
  for u ∈ G.Adj[rec] do
    if w(rec, u) < dist[u] then
      dist[u] ← w(rec, u)
      pred[u] ← rec
    end
  end
  color[rec] ← BLACK
end
```

$O(|V|)$

$$O(|V| \cdot |V|) = O(|V|^2)$$

$O(\deg(u))$

$$\sum_{u \in V} \deg(u) = 2|E|$$

复杂度分析

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

$O(|V|)$

$O(|V|^2 + |E|)$

$O(\deg(u))$

$O(|V|^2)$





复杂度分析

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

$O(|V|)$

$O(|V|^2 + |E|)$

$O(\deg(u))$

$O(|V|^2)$

思考：能否进一步优化？



复杂度分析

//执行最小生成树算法

```
for  $i \leftarrow 1$  to  $|V|$  do
   $minDist \leftarrow \infty$ 
   $rec \leftarrow 0$ 
  for  $j \leftarrow 1$  to  $|V|$  do
    if  $color[j] \neq BLACK$  and  $dist[j] < minDist$  then
       $minDist \leftarrow dist[j]$ 
       $rec \leftarrow j$ 
    end
  end
  for  $u \in G.Adj[rec]$  do
    if  $w(rec, u) < dist[u]$  then
       $dist[u] \leftarrow w(rec, u)$ 
       $pred[u] \leftarrow rec$ 
    end
  end
   $color[rec] \leftarrow BLACK$ 
end
```

$O(|V|) \rightarrow O(?)$

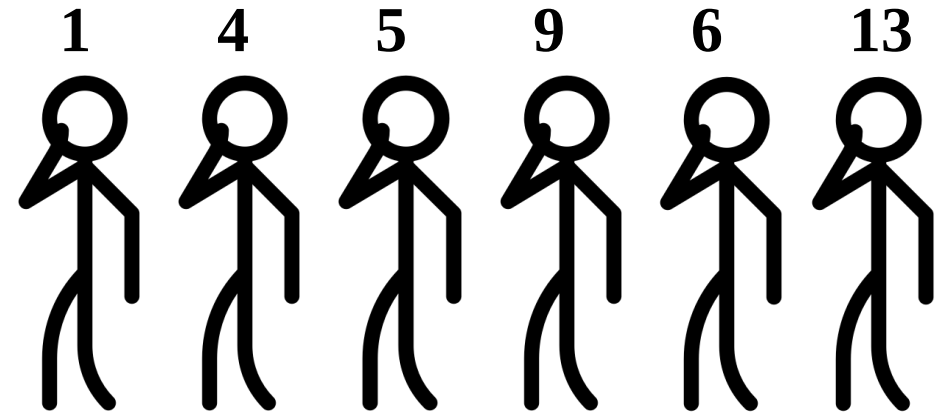
$O(|V|^2)$

数据结构加速最小值查询

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列



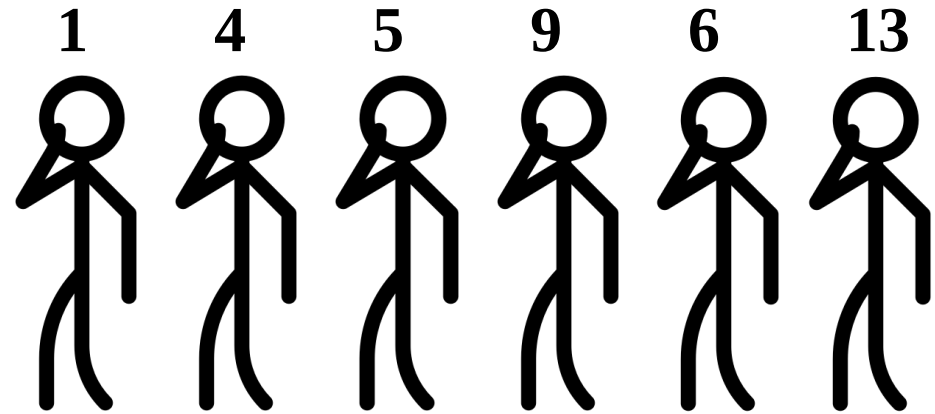
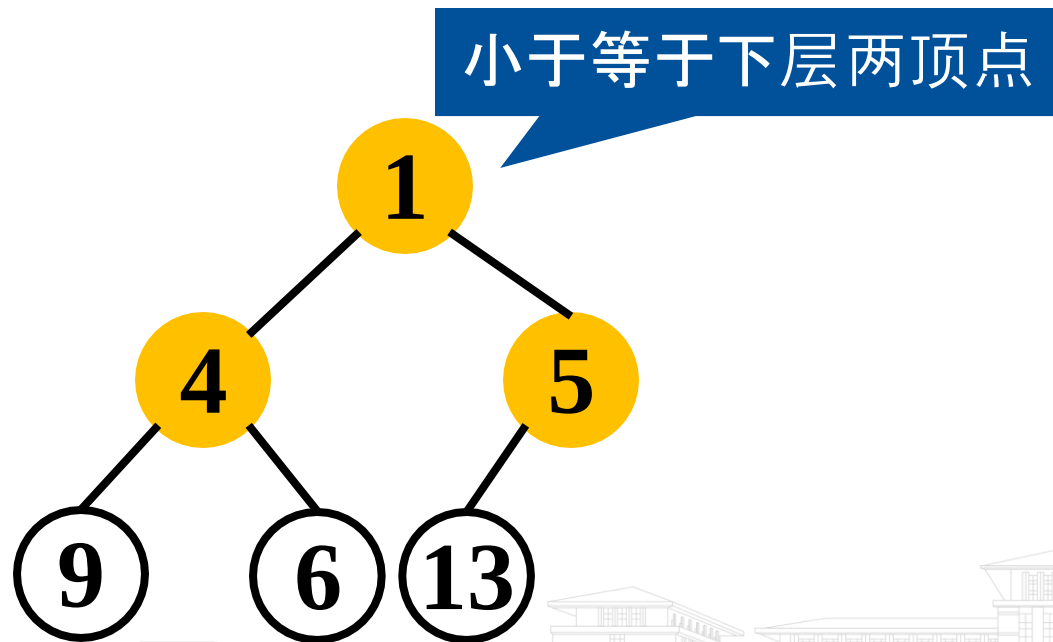
优先队列 Q



数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列

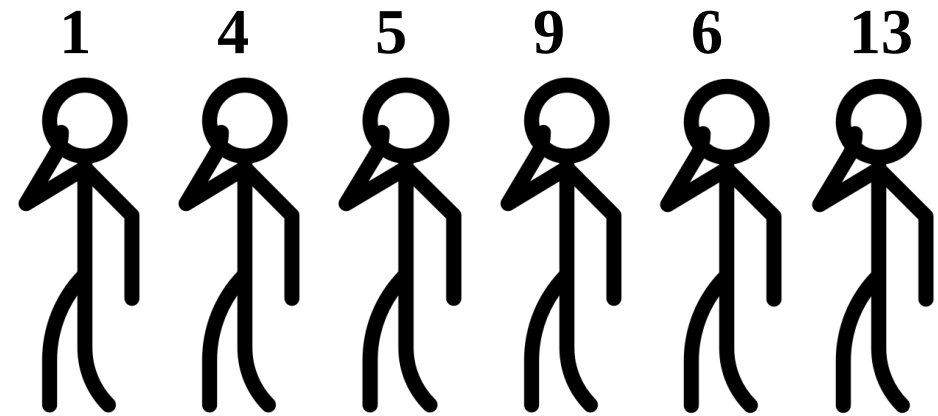
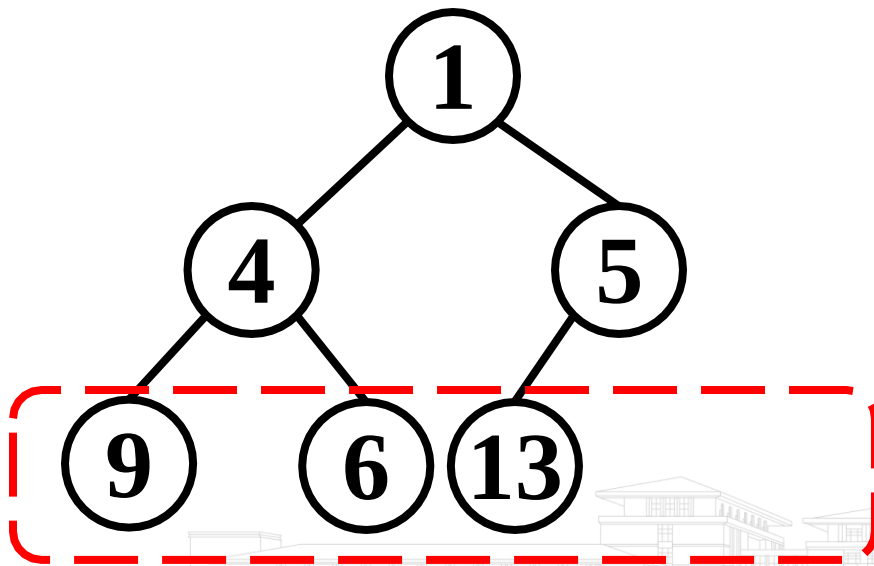


优先队列 Q

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列



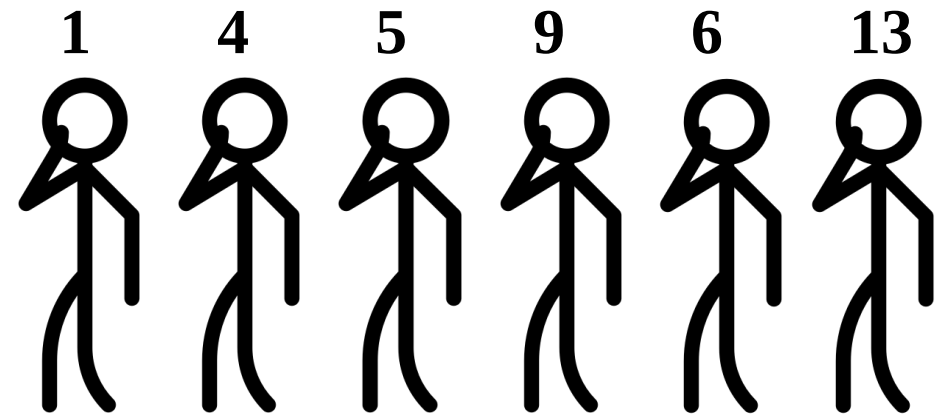
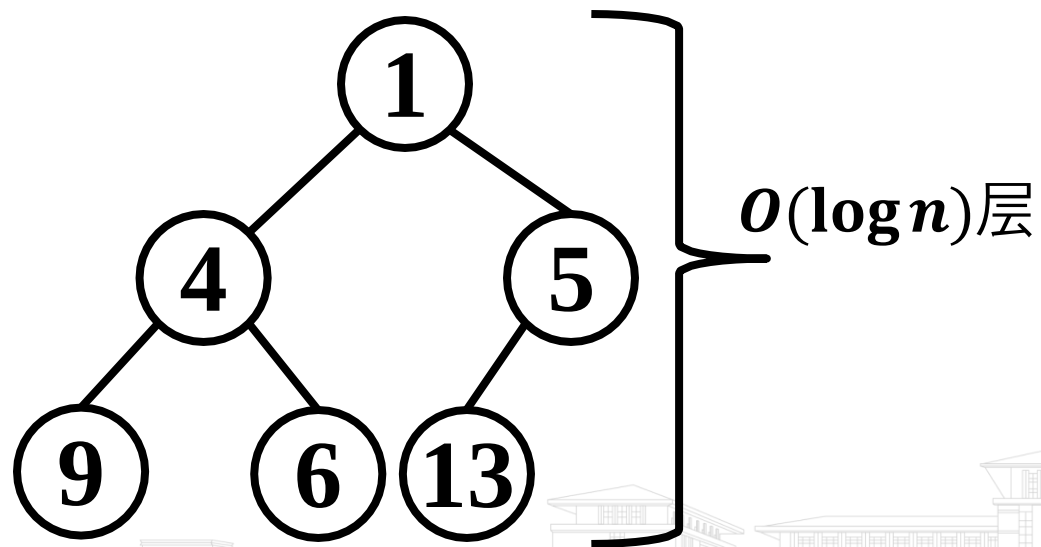
优先队列 Q

除最底层，第 h 层有 2^{h-1} 个顶点

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列



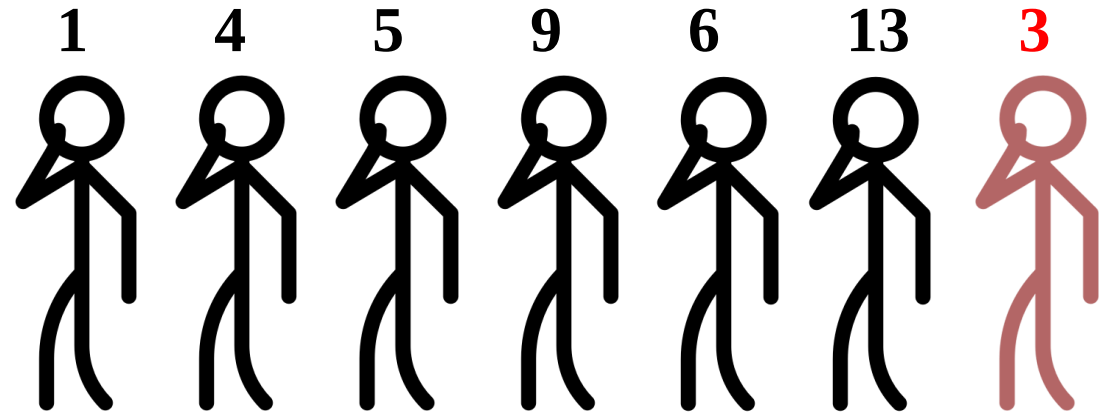
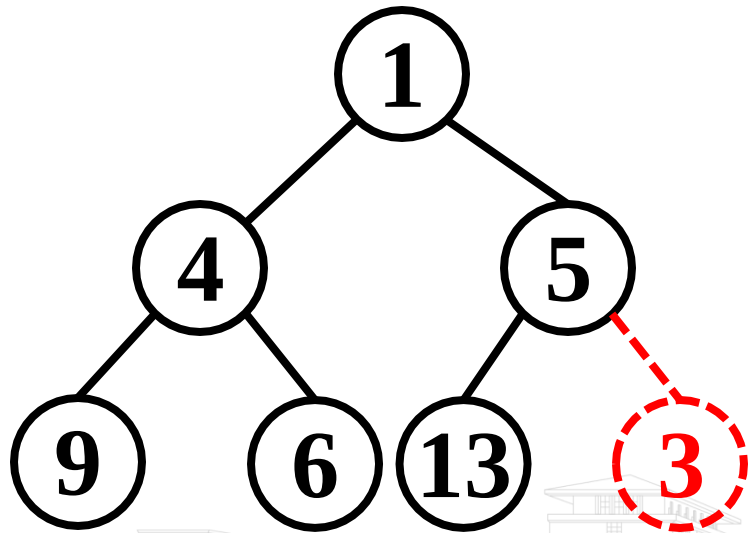
优先队列 Q

除最底层，第 h 层有 2^{h-1} 个顶点

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$

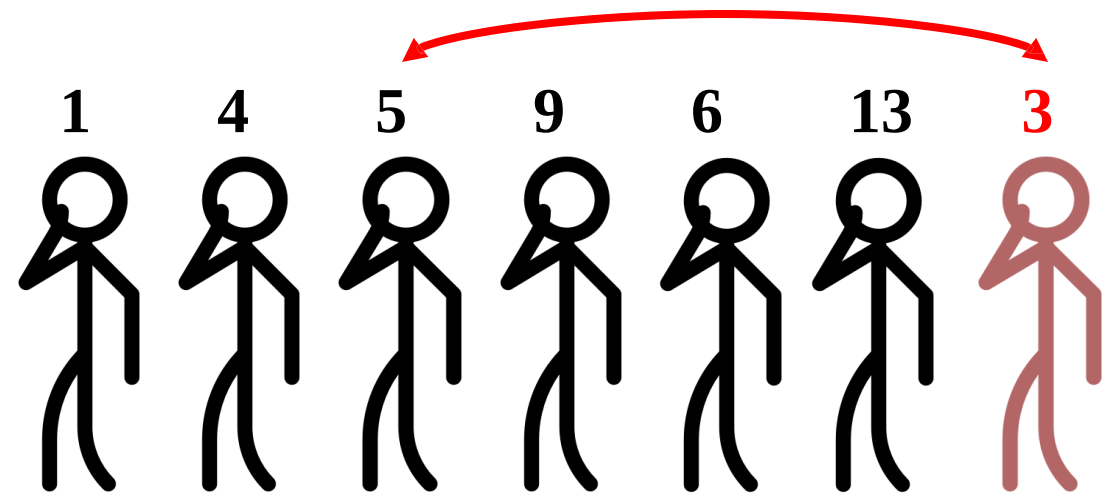
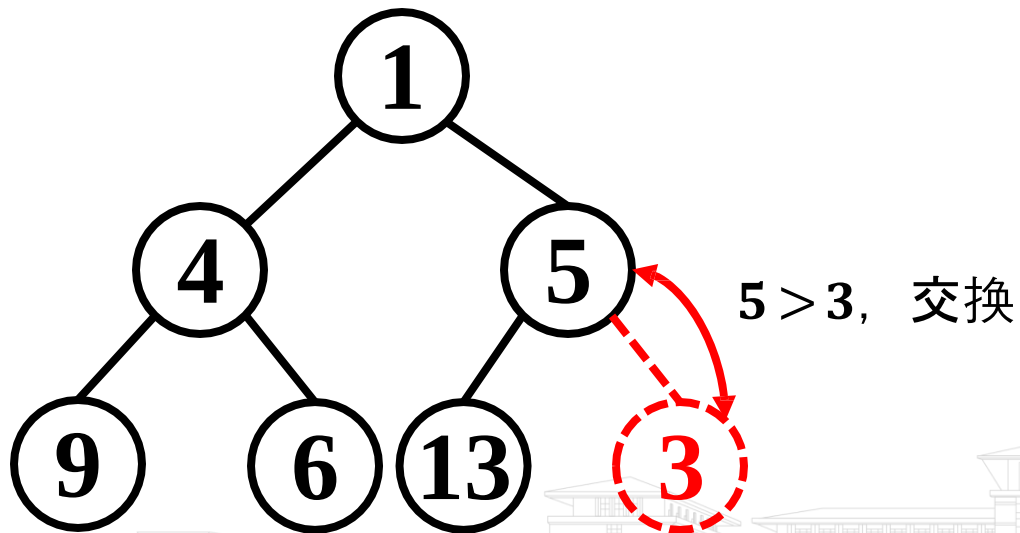


优先队列 Q

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - *Q.Insert(3)*

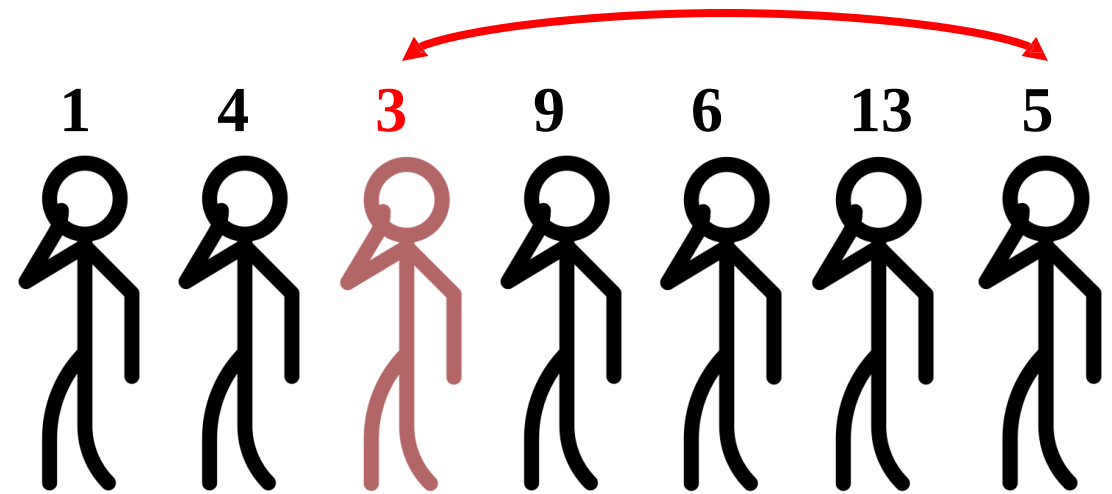
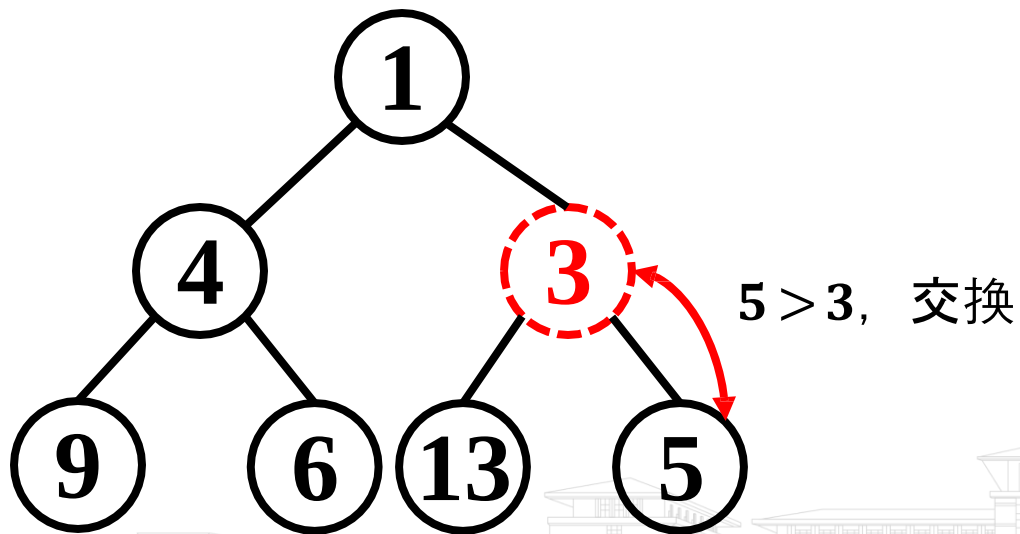


优先队列 *Q*

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - *Q.Insert(3)*

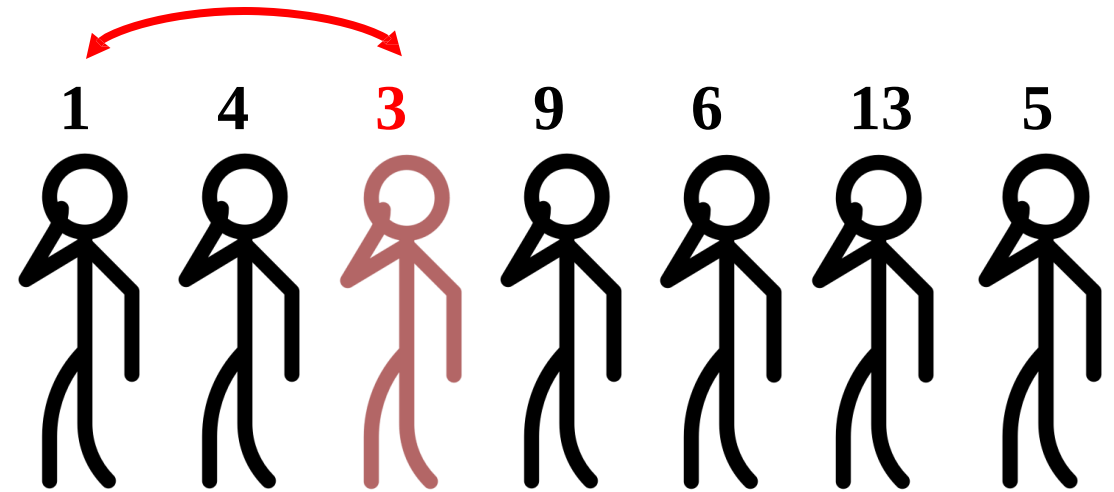
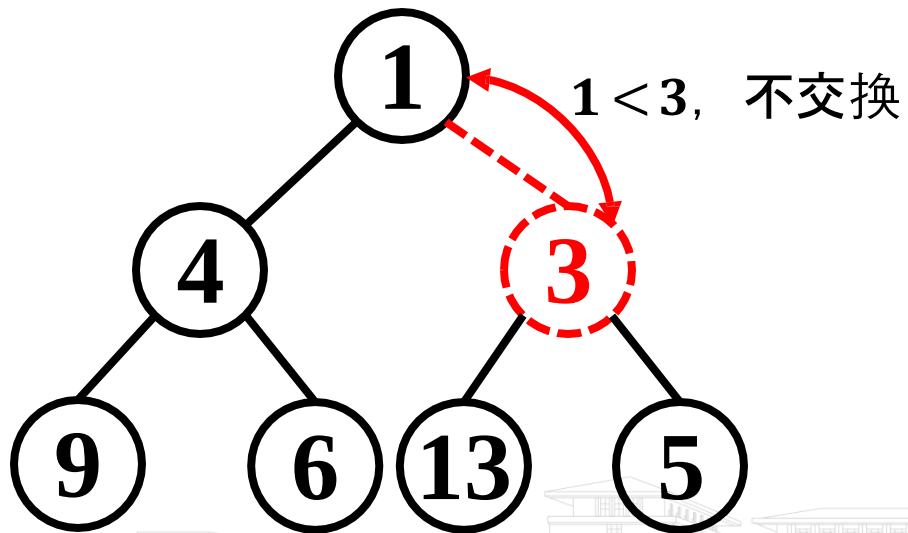


优先队列 *Q*

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - *Q.Insert(3)*

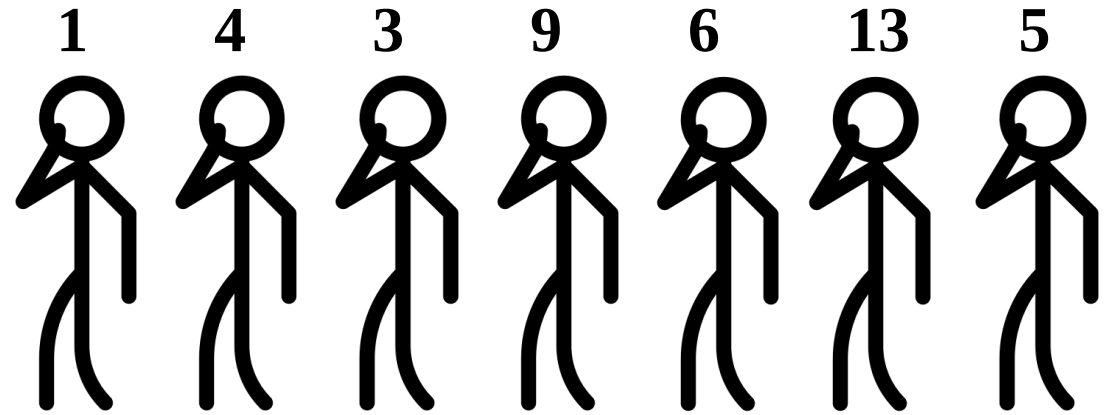
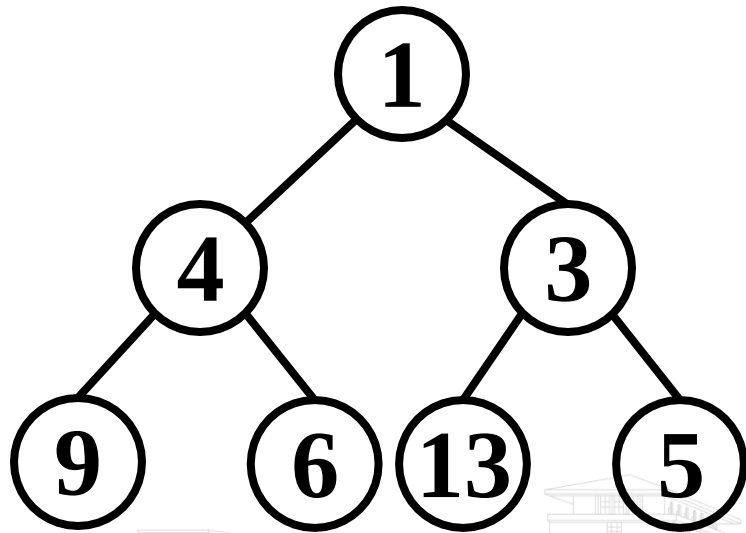


优先队列 Q

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$

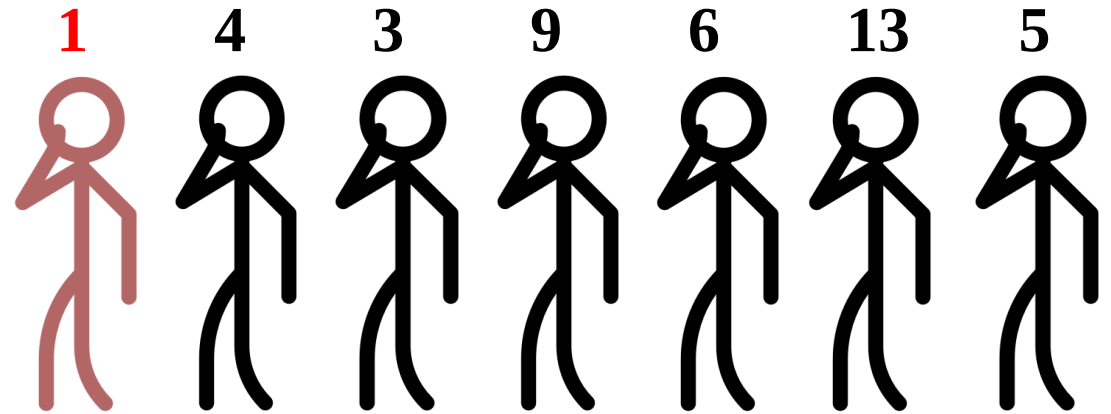
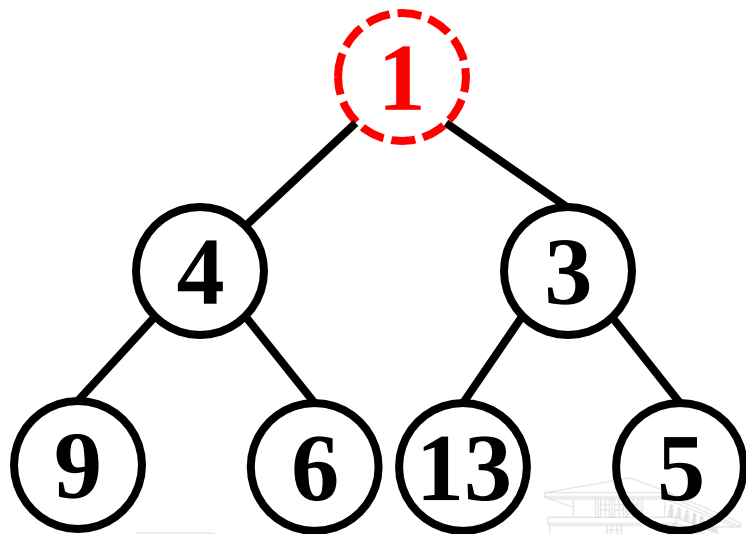


优先队列 Q

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$
 - $Q.ExtractMin()$

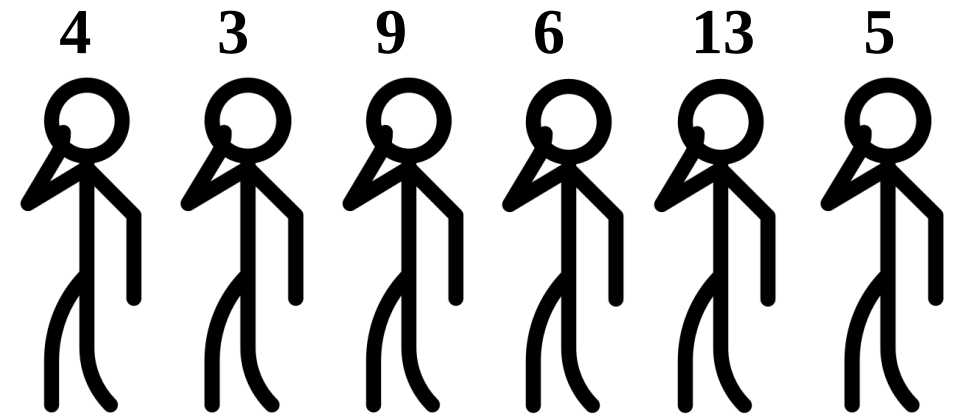
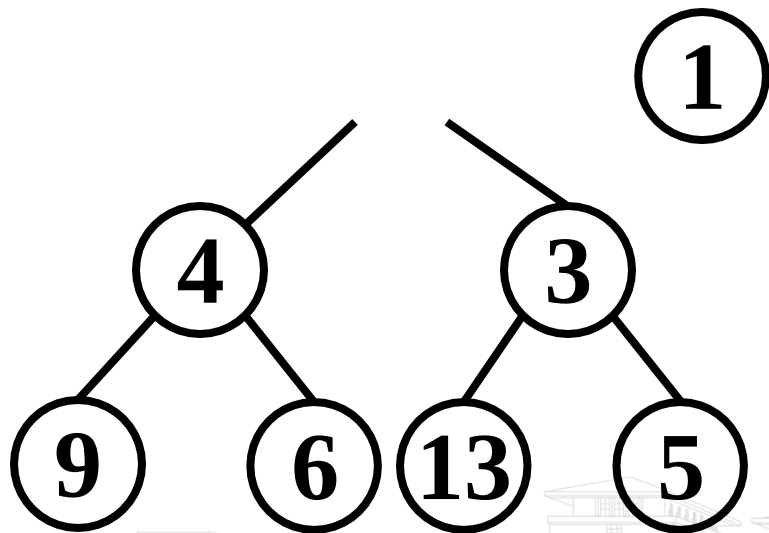


优先队列 Q

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$
 - $Q.ExtractMin()$

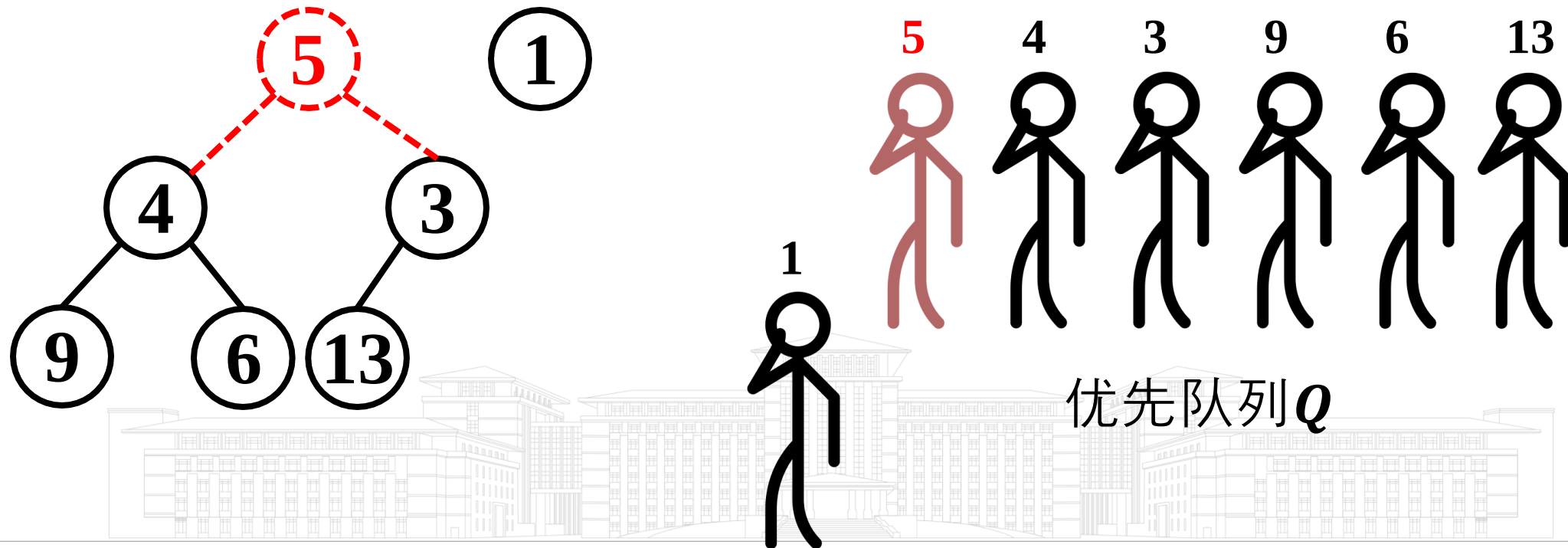


优先队列 Q

数据结构：优先队列

- 优先队列

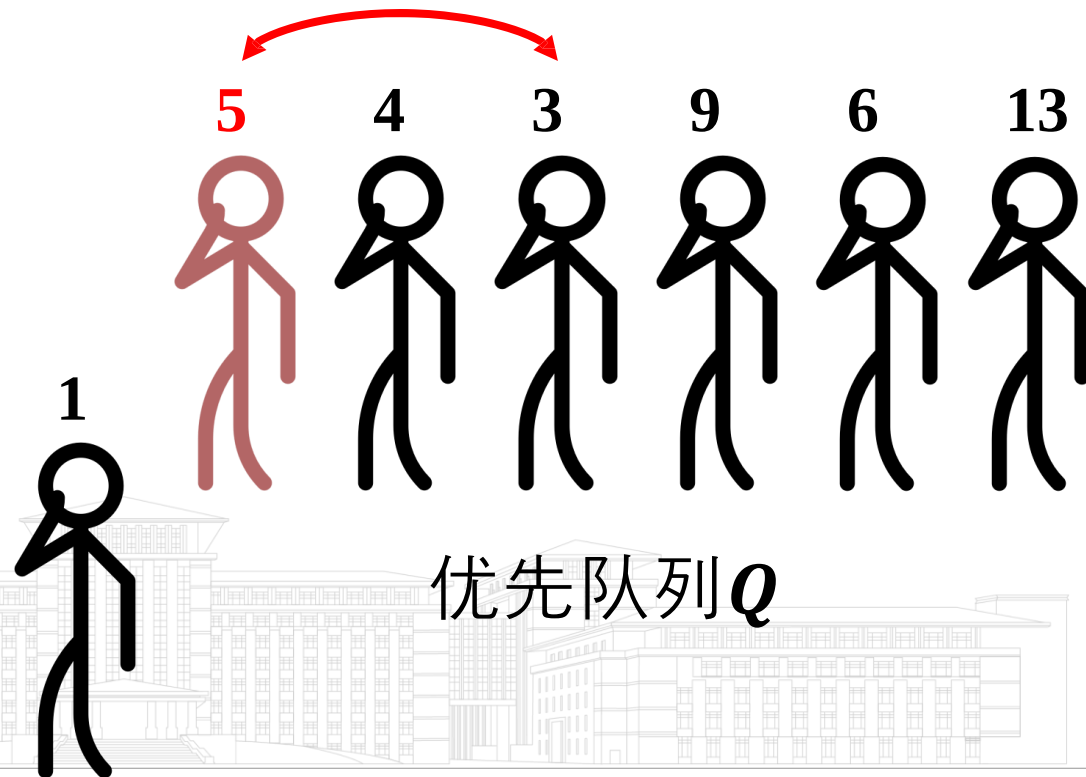
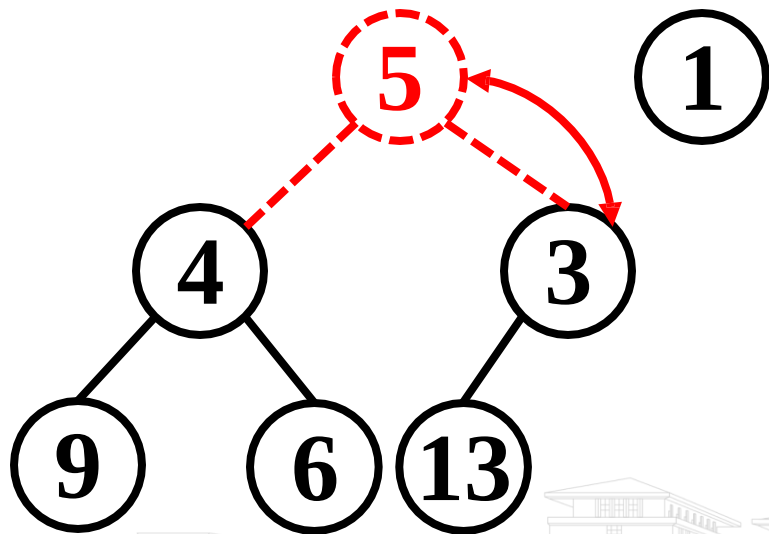
- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$
 - $Q.ExtractMin()$



数据结构：优先队列

- 优先队列

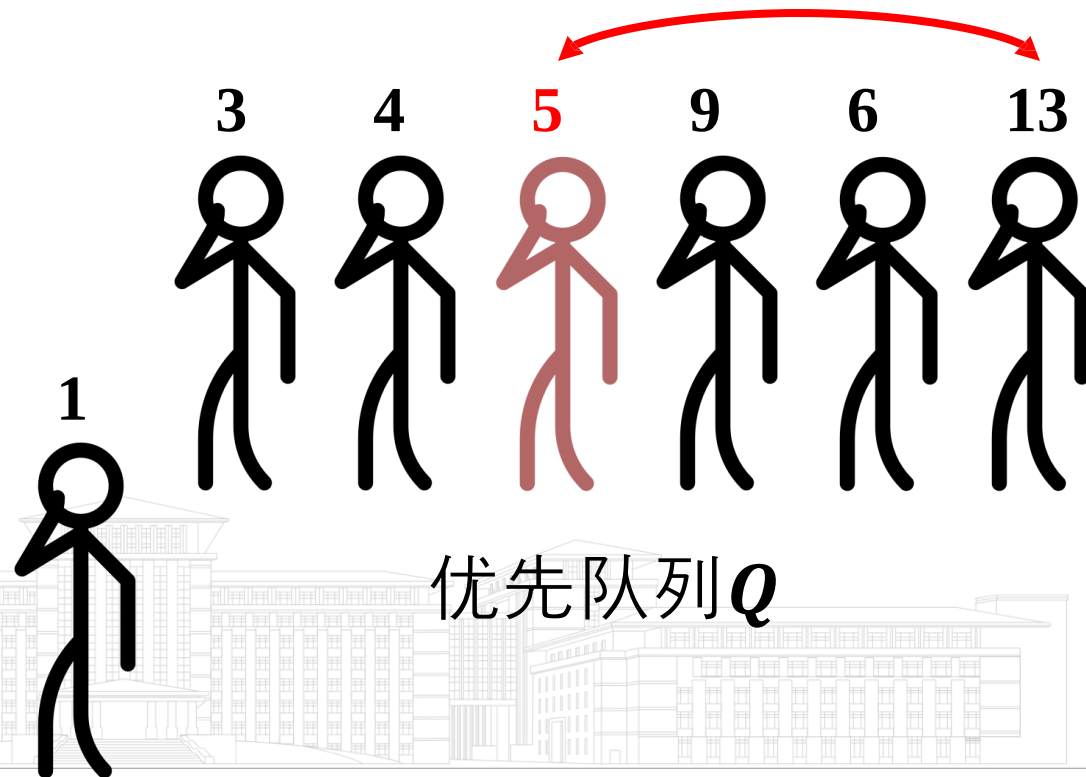
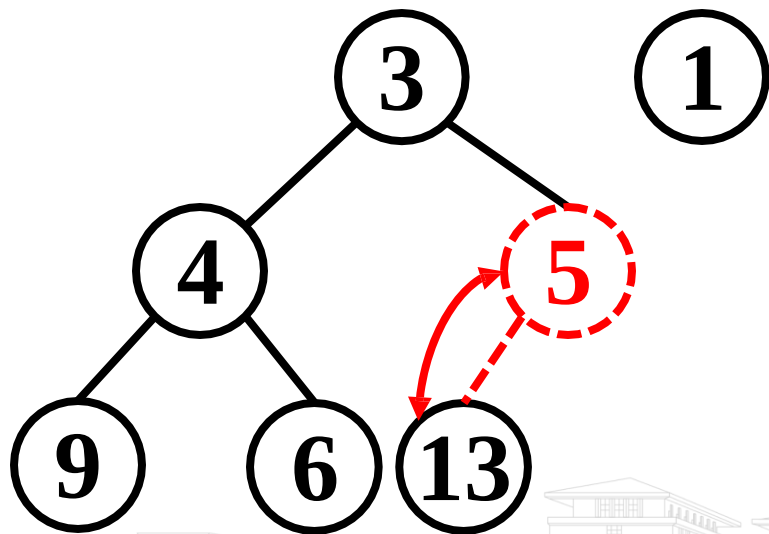
- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$
 - $Q.ExtractMin()$



数据结构：优先队列

- 优先队列

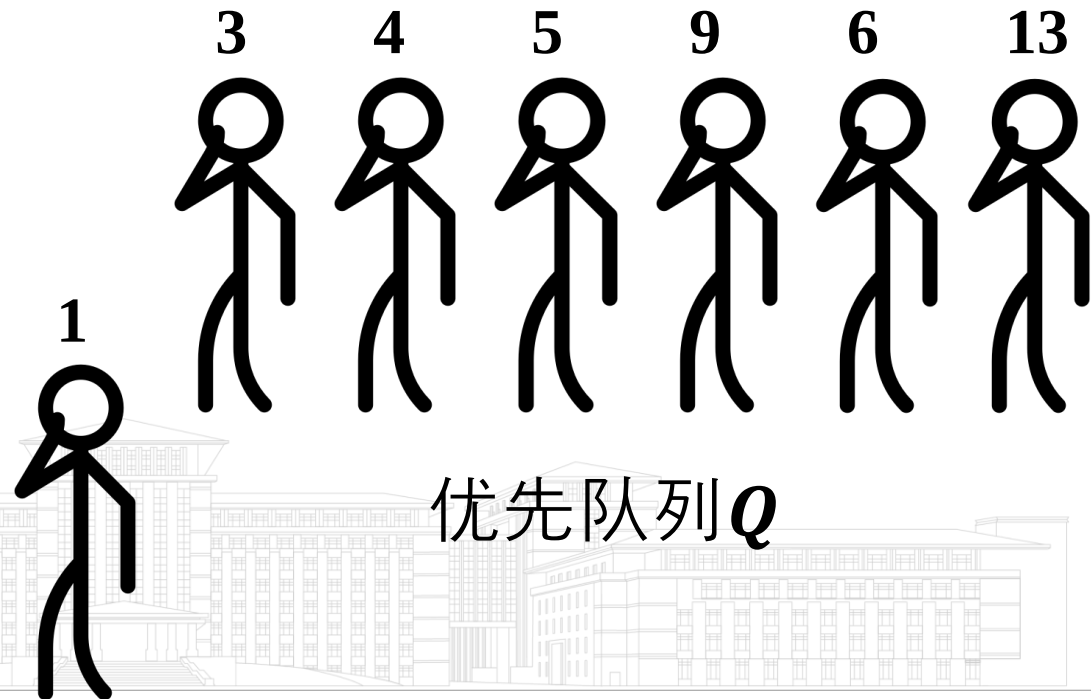
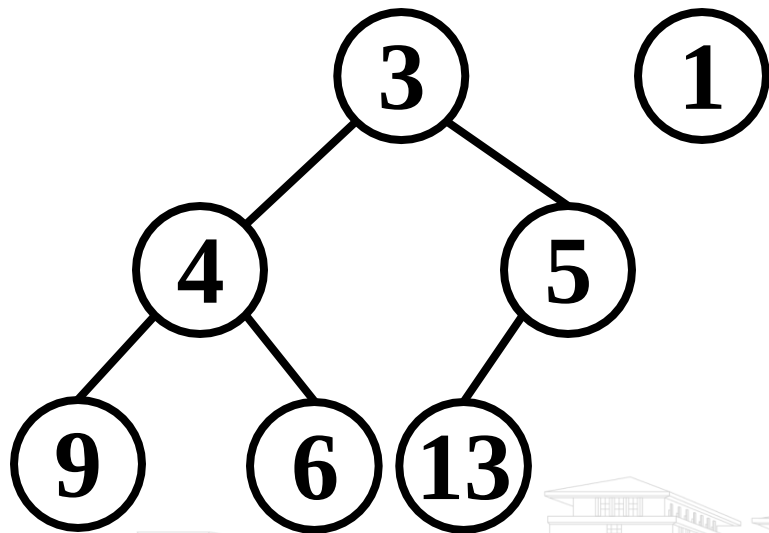
- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - *Q.Insert(3)*
 - *Q.ExtractMin()*



数据结构：优先队列

- 优先队列

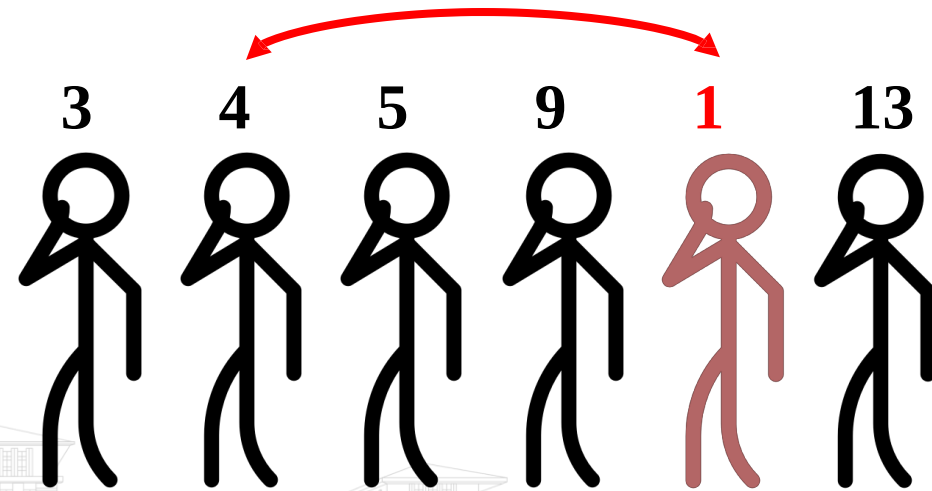
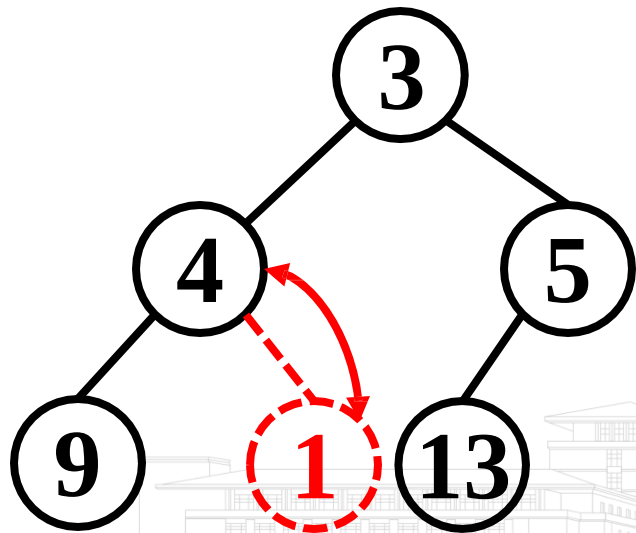
- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$
 - $Q.ExtractMin()$



数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - *Q.Insert(3)*
 - *Q.ExtractMin()*
 - *Q.DecreaseKey(6, 1)*

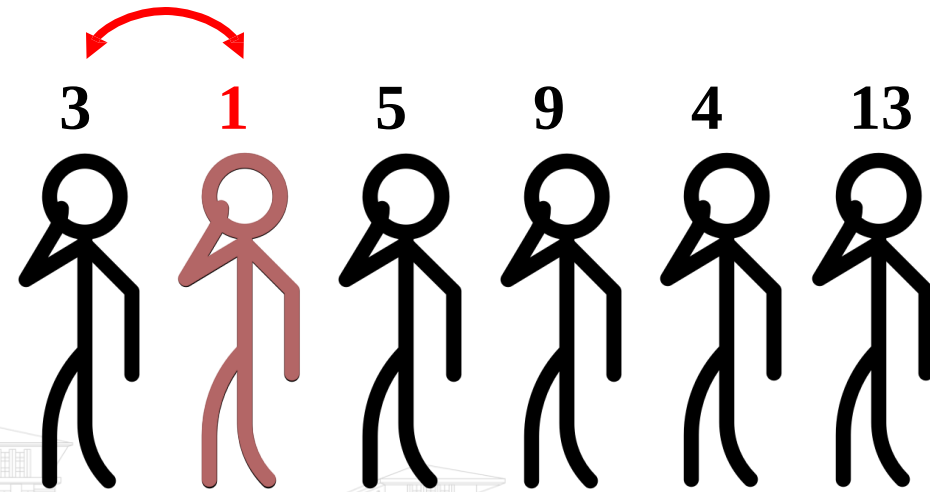
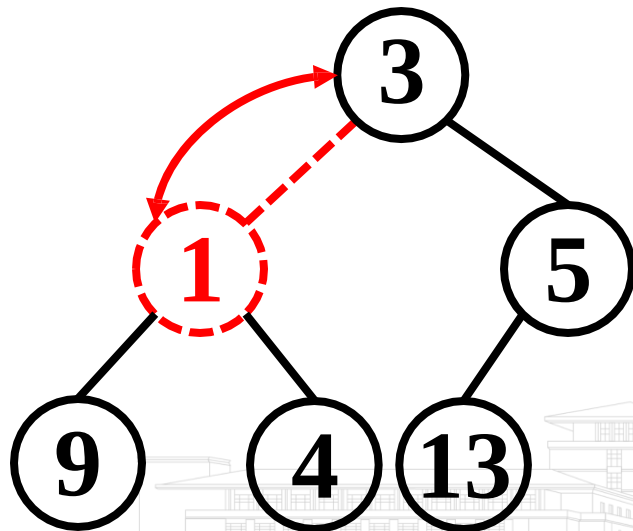


优先队列 *Q*

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - *Q.Insert(3)*
 - *Q.ExtractMin()*
 - *Q.DecreaseKey(6, 1)*

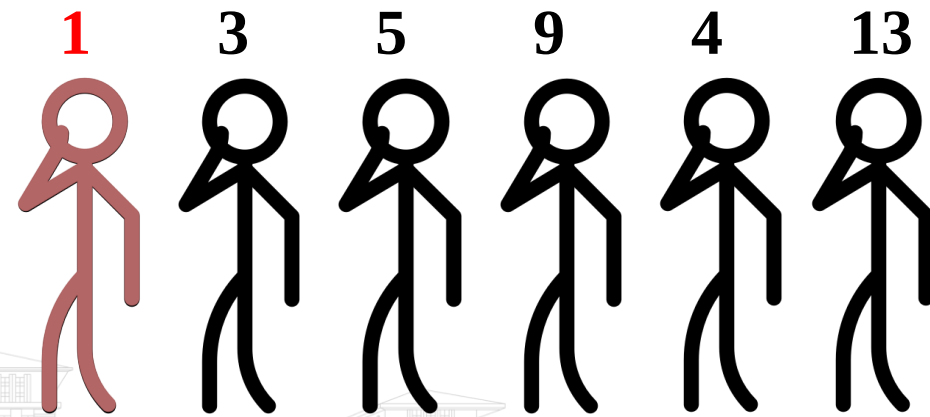
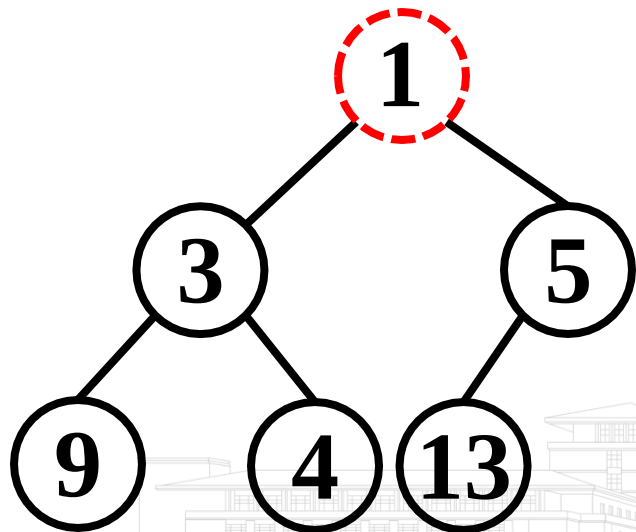


优先队列 *Q*

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$
 - $Q.ExtractMin()$
 - $Q.DecreaseKey(6, 1)$

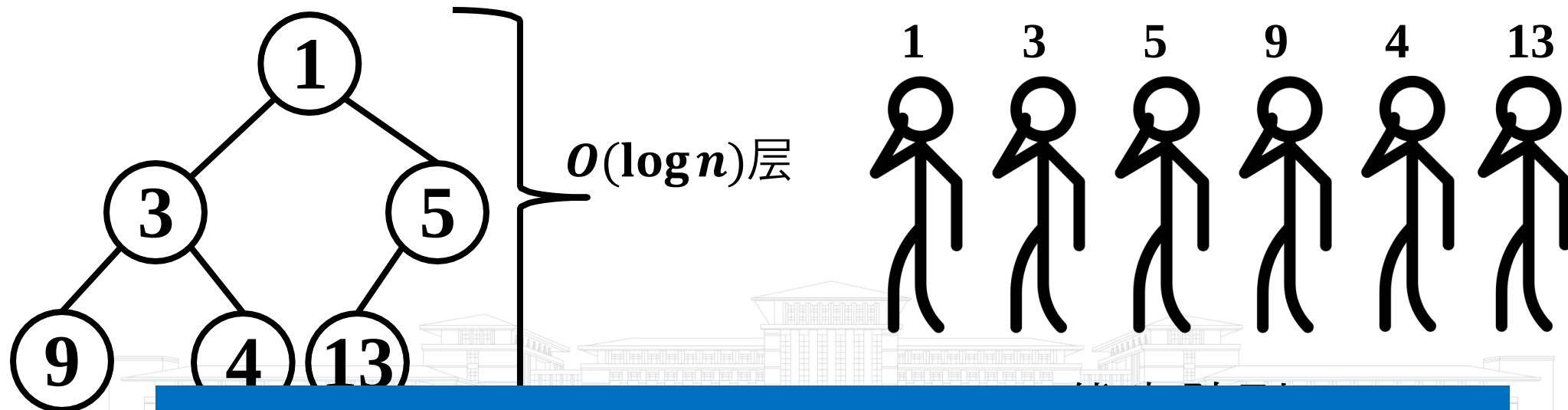


优先队列 Q

数据结构：优先队列

- 优先队列

- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - $Q.Insert(3)$ 时间复杂度 $O(\log n)$
 - $Q.ExtractMin()$ 时间复杂度 $O(\log n)$
 - $Q.DecreaseKey(6, 1)$ 时间复杂度 $O(\log n)$



使用优先队列，高效查找的安全边

伪代码

● MST-Prim-PriQueue(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

新建空优先队列 Q

//初始化

```
for  $u \in V$  do
|    $color[u] \leftarrow WHITE$ 
|    $dist[u] \leftarrow \infty$ 
|    $pred[u] \leftarrow NULL$ 
end
```

初始化辅助数组

$dist[1] \leftarrow 0$

$Q.Insert(V, dist)$



伪代码

● MST-Prim-PriQueue(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

新建空优先队列 Q

//初始化

for $u \in V$ do

$color[u] \leftarrow WHITE$

$dist[u] \leftarrow \infty$

$pred[u] \leftarrow NULL$

end

$dist[1] \leftarrow 0$

$Q.Insert(V, dist)$

选择任意起点



伪代码

● MST-Prim-PriQueue(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

新建空优先队列 Q

//初始化

for $u \in V$ do

$color[u] \leftarrow WHITE$

$dist[u] \leftarrow \infty$

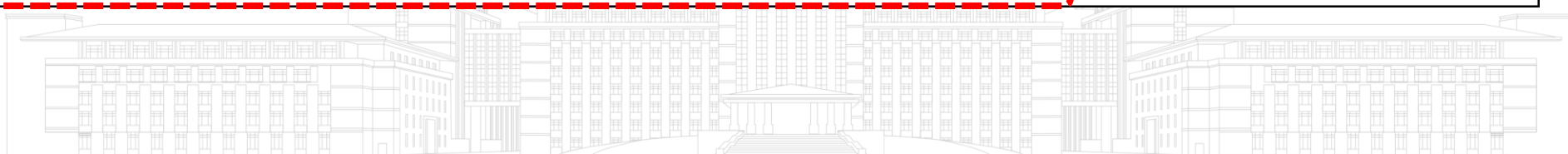
$pred[u] \leftarrow NULL$

end

$dist[1] \leftarrow 0$

$Q.Insert(V, dist)$

初始化优先队列



伪代码

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

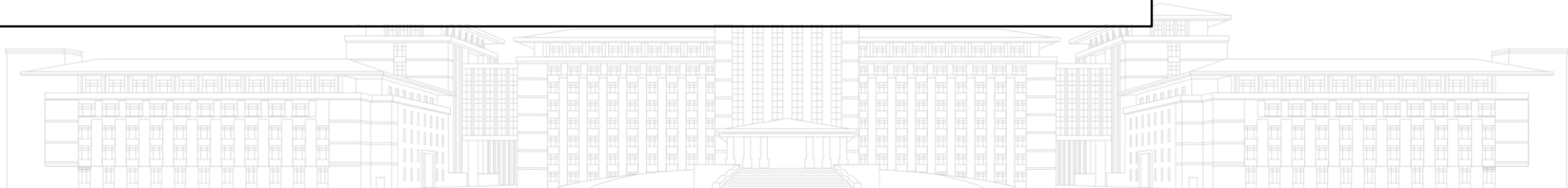
 end

 end

$color[v] \leftarrow BLACK$

end

依次添加其他顶点



伪代码

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

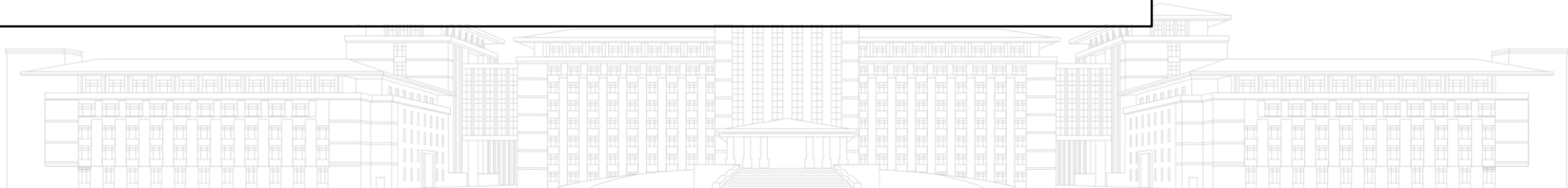
 end

 end

$color[v] \leftarrow BLACK$

end

选择安全边



伪代码

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

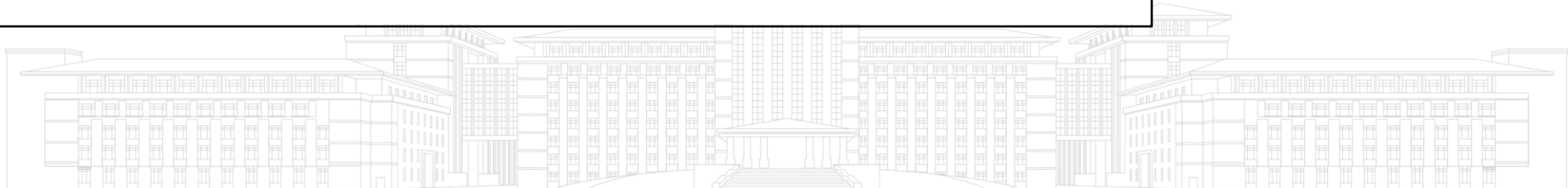
 end

 end

$color[v] \leftarrow BLACK$

end

更新距离数组， 调整优先队列



伪代码

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

 end

 end

$color[v] \leftarrow BLACK$

end

标记顶点处理完成



复杂度分析

● MST-Prim-PriQueue(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 最小生成树 T

新建一维数组 $color[1..|V|]$, $dist[1..|V|]$, $pred[1..|V|]$

新建空优先队列 Q

//初始化

for $u \in V$ do

$color[u] \leftarrow WHITE$

$dist[u] \leftarrow \infty$

$pred[u] \leftarrow NULL$

end

$dist[1] \leftarrow 0$

$Q.Insert(V, dist)$

$O(|V|)$



复杂度分析

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

----- $O(\log|V|)$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$ ----- $O(\log|V|)$

 end

 end

$color[v] \leftarrow BLACK$

end



复杂度分析

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

 end

 end

$color[v] \leftarrow BLACK$

end

----- $O(\log|V|)$

$O(\deg(u)\log|V|)$

----- $O(\log|V|)$



复杂度分析

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

 end

 end

$color[v] \leftarrow BLACK$

end

$O(\log|V|)$

$O(|V|\log|V|)$

$O(\deg(u)\log|V|)$

$O(\log|V|)$



复杂度分析

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

 end

 end

$color[v] \leftarrow BLACK$

end

$O(\log|V|)$

$O(\deg(u)\log|V|)$

$O(\log|V|)$

$O(|V|\log|V|)$

$O(|E|\log|V|)$

$$\sum_{u \in V} \deg(u) = 2|E|$$

复杂度分析

● MST-Prim-PriQueue(G)

//执行最小生成树算法

while 优先队列 Q 非空 do

$v \leftarrow Q.ExtractMin()$

 for $u \in G.Adj[v]$ do

 if $color[u] = WHITE$ and $w(v, u) < dist[u]$ then

$dist[u] \leftarrow w(v, u)$

$pred[u] \leftarrow v$

$Q.DecreaseKey((u, dist[u]))$

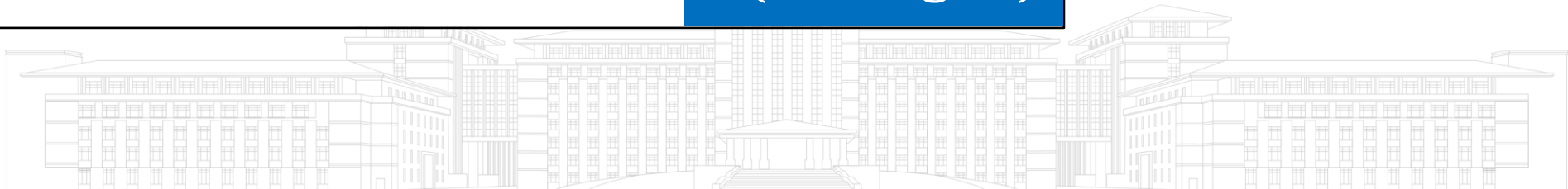
 end

 end

$color[v] \leftarrow BLACK$

end

$O(|E| \cdot \log|V|)$



目录

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 货郎担问题
- 5.5 小结



通用框架

- 生成树是一个连通、无环的生成子图
 - 新建一个空边集 A ，边集 A 可逐步扩展为最小生成树
 - 每次向边集 A 中新增加一条边
 - 需保证边集 A 仍是一个无环图
 - 需保证边集 A 仍是最小生成树的子集

思考：如何有效地实现此贪心策略？

Prim算法

Kruskal算法



Kruskal算法

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 需保证边集 A 仍是最小生成树的子集



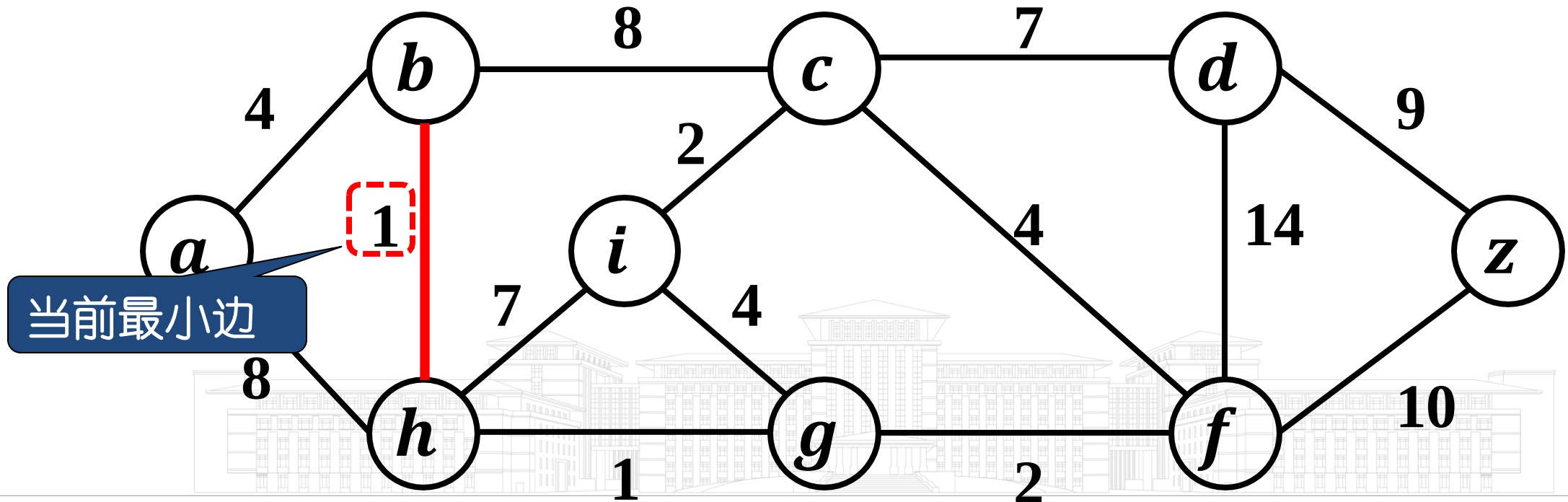
Kruskal算法

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



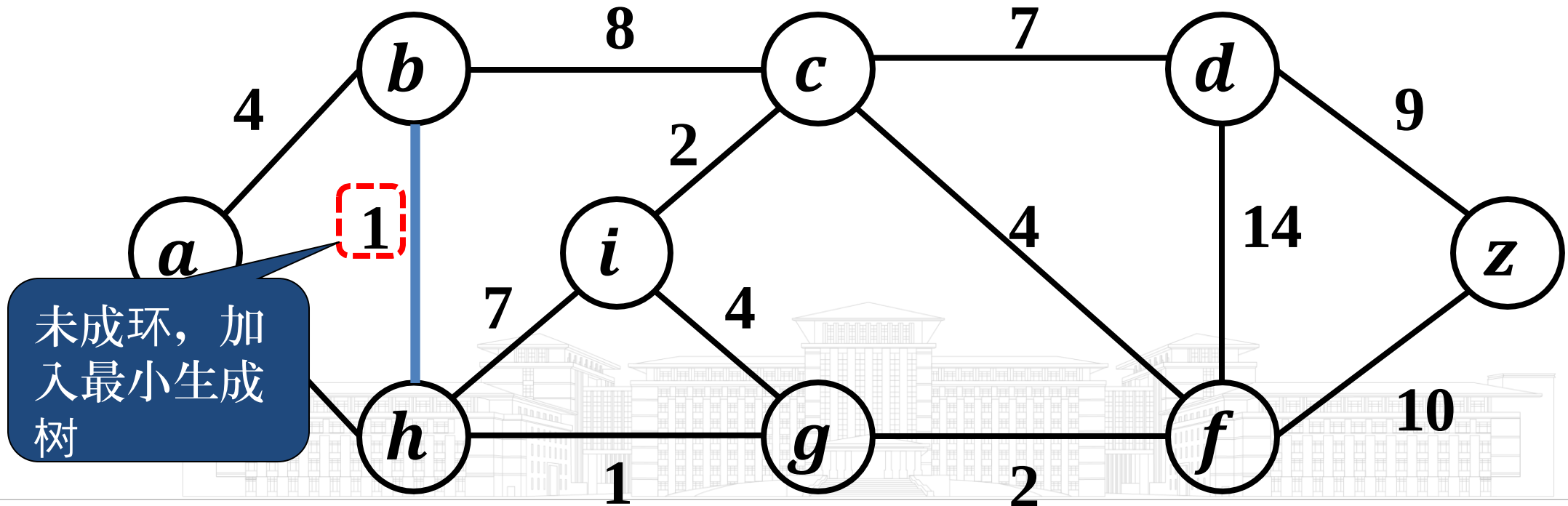
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



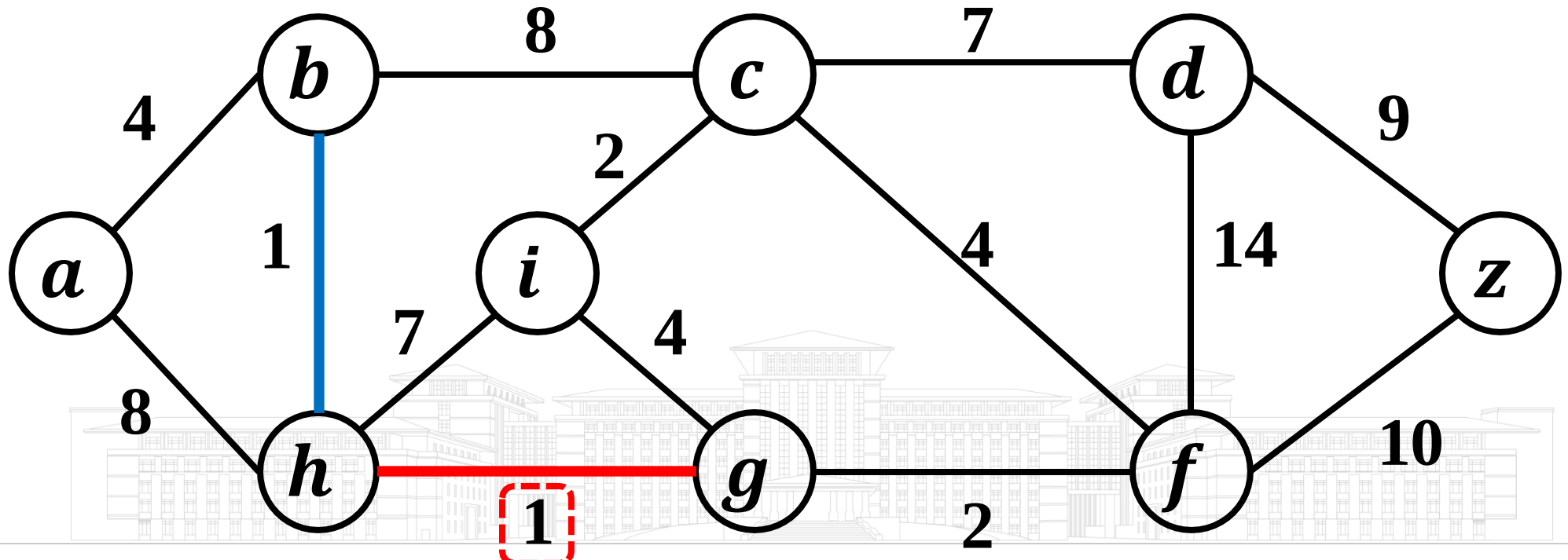
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



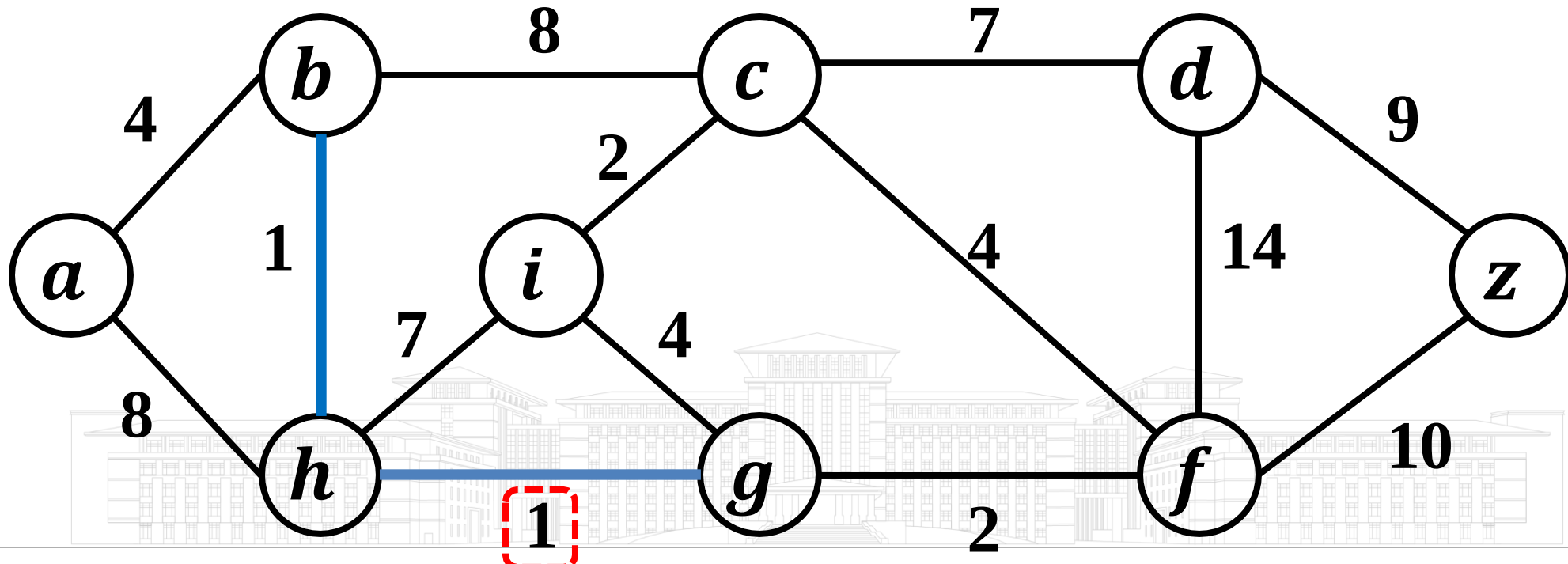
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



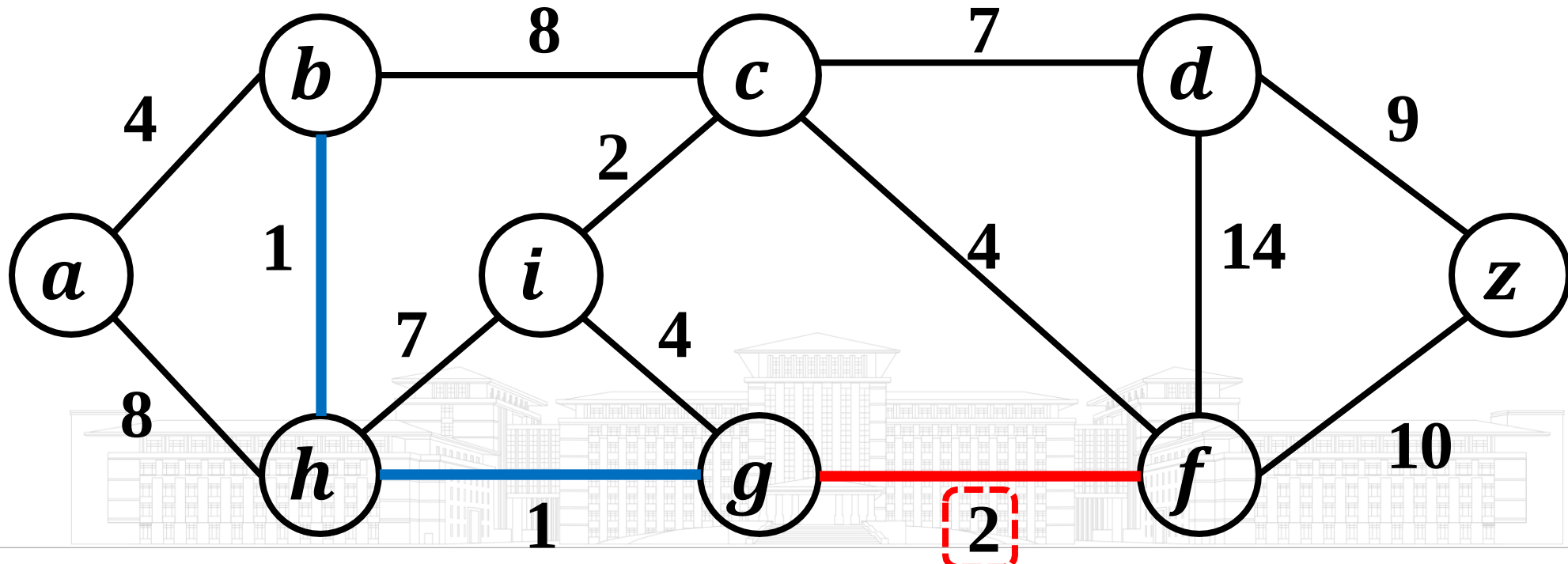
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



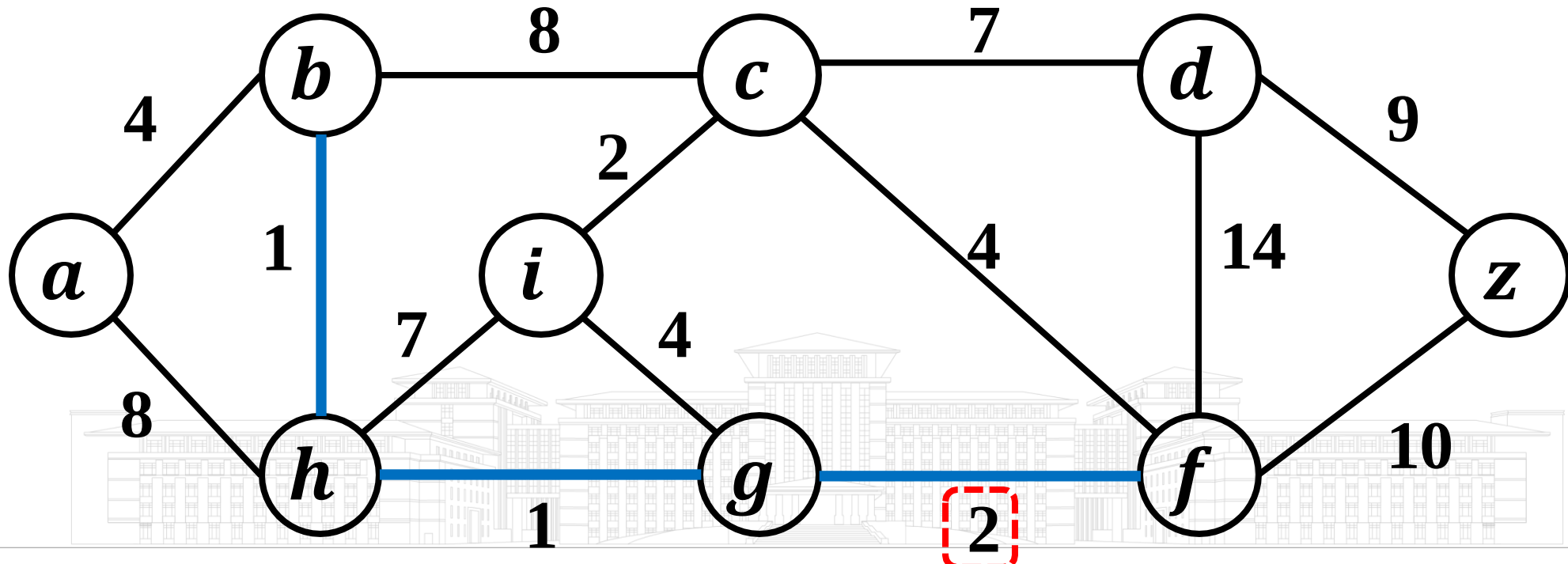
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



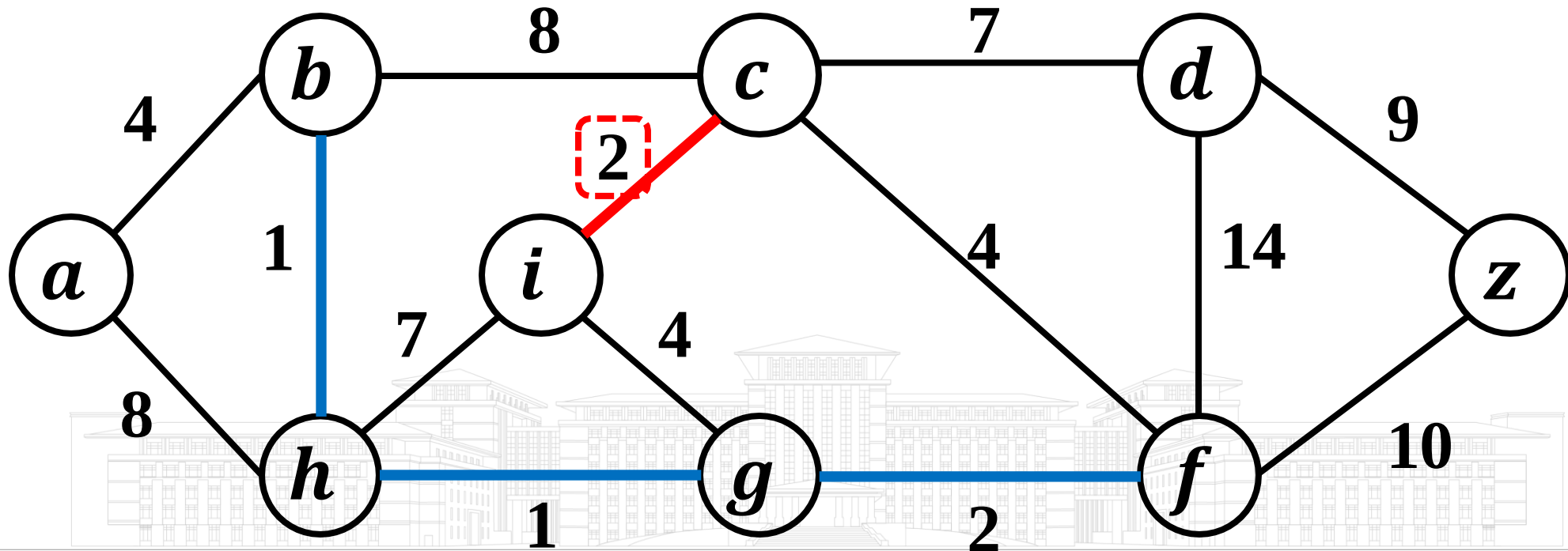
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



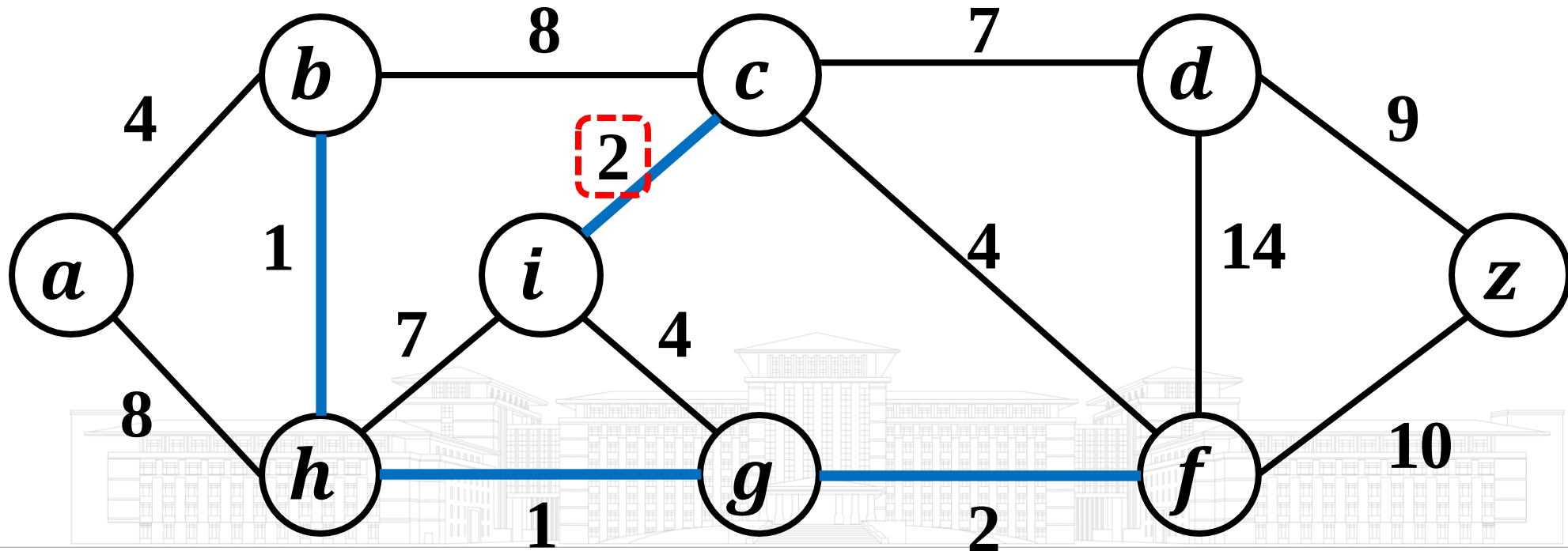
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



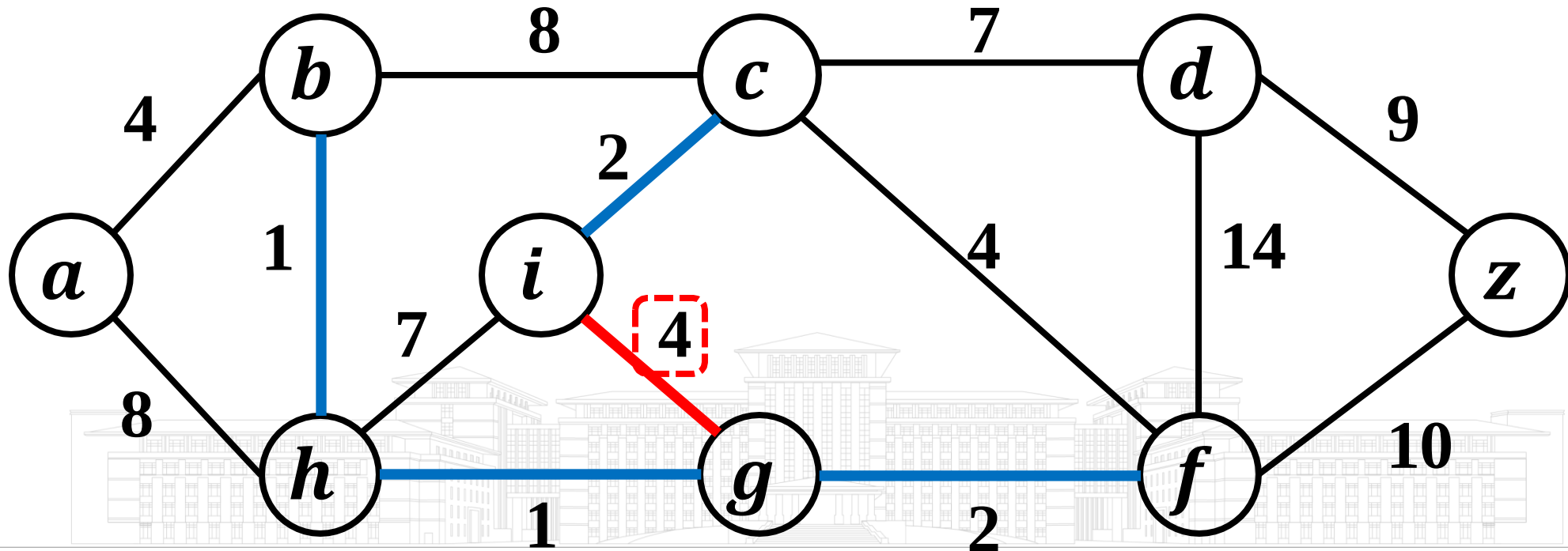
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



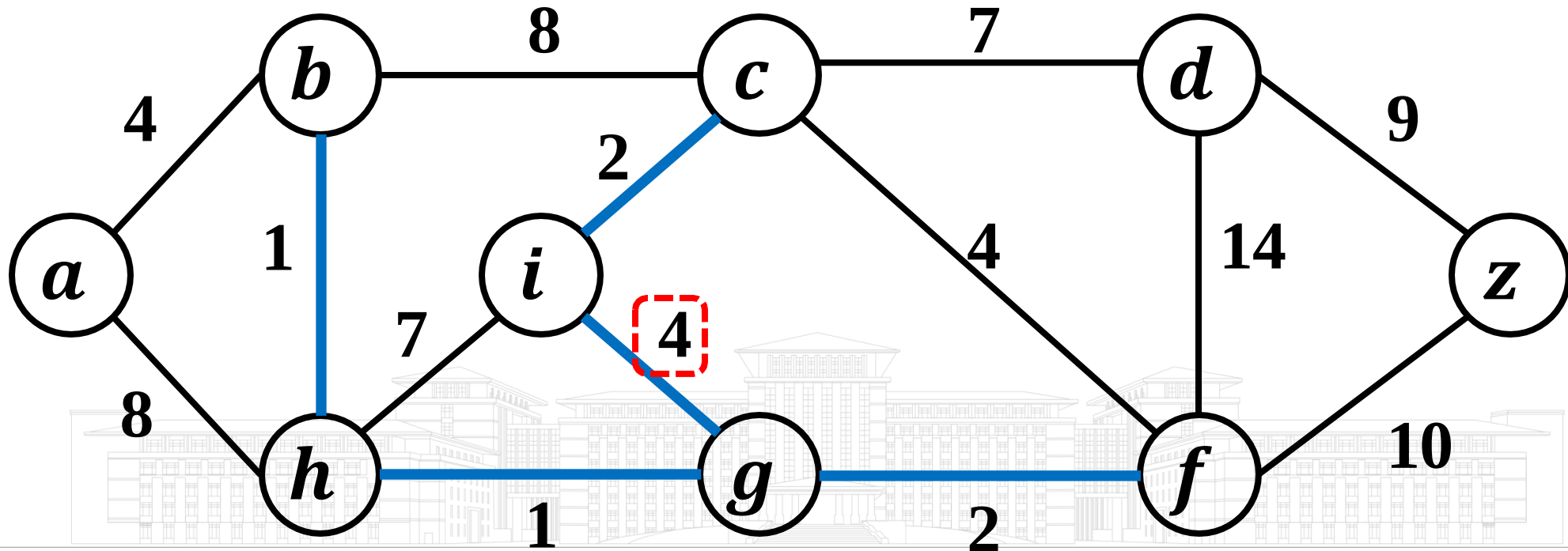
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



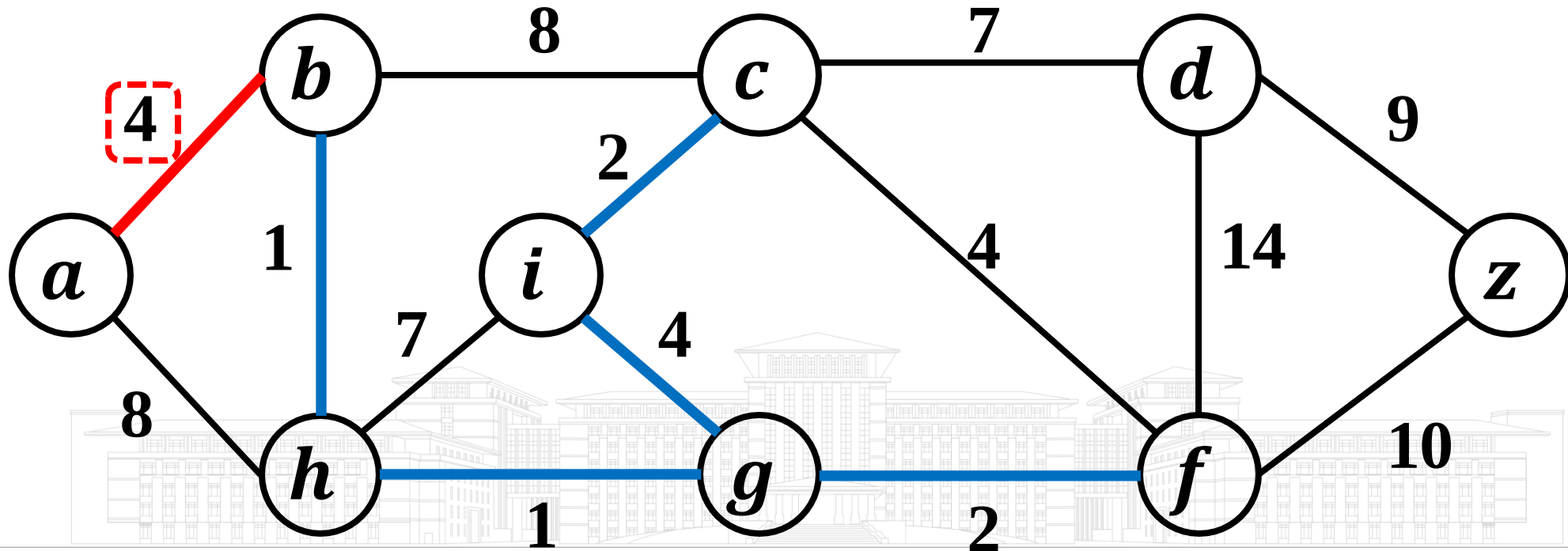
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



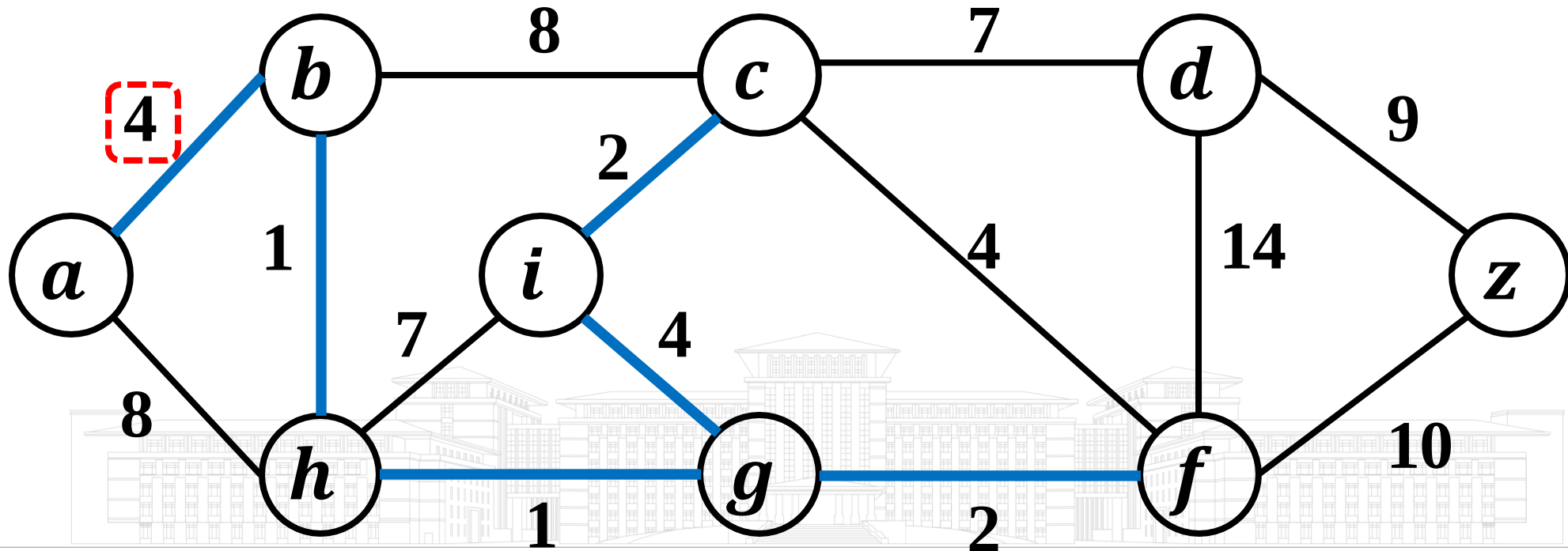
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



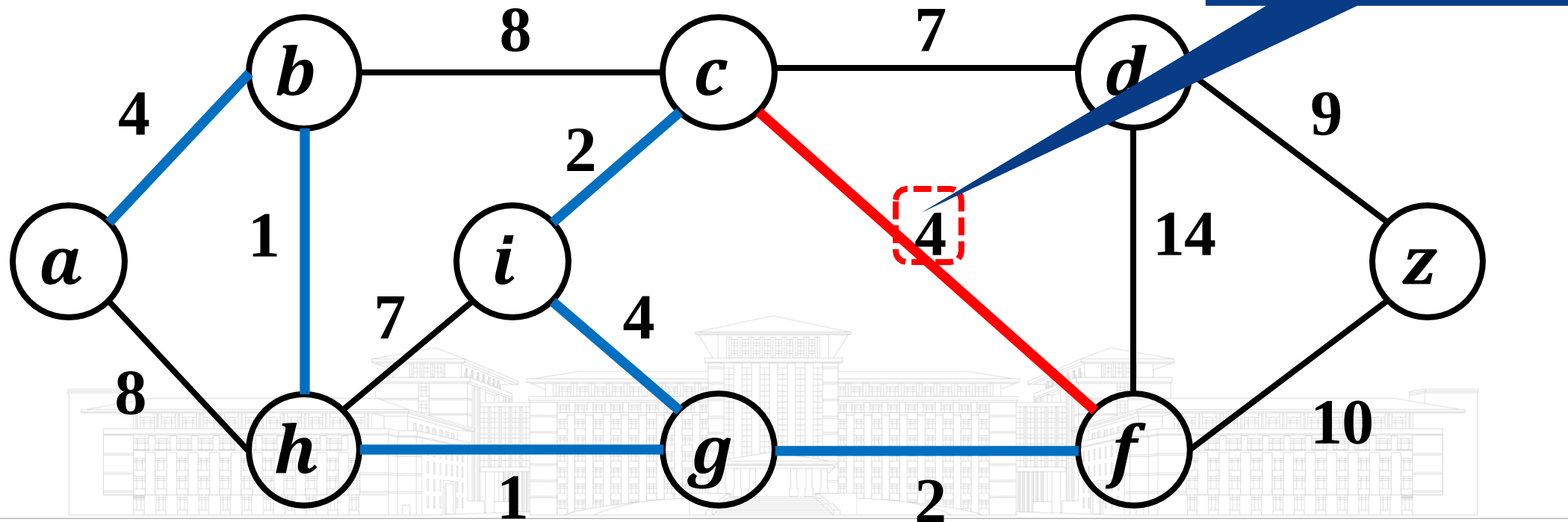
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



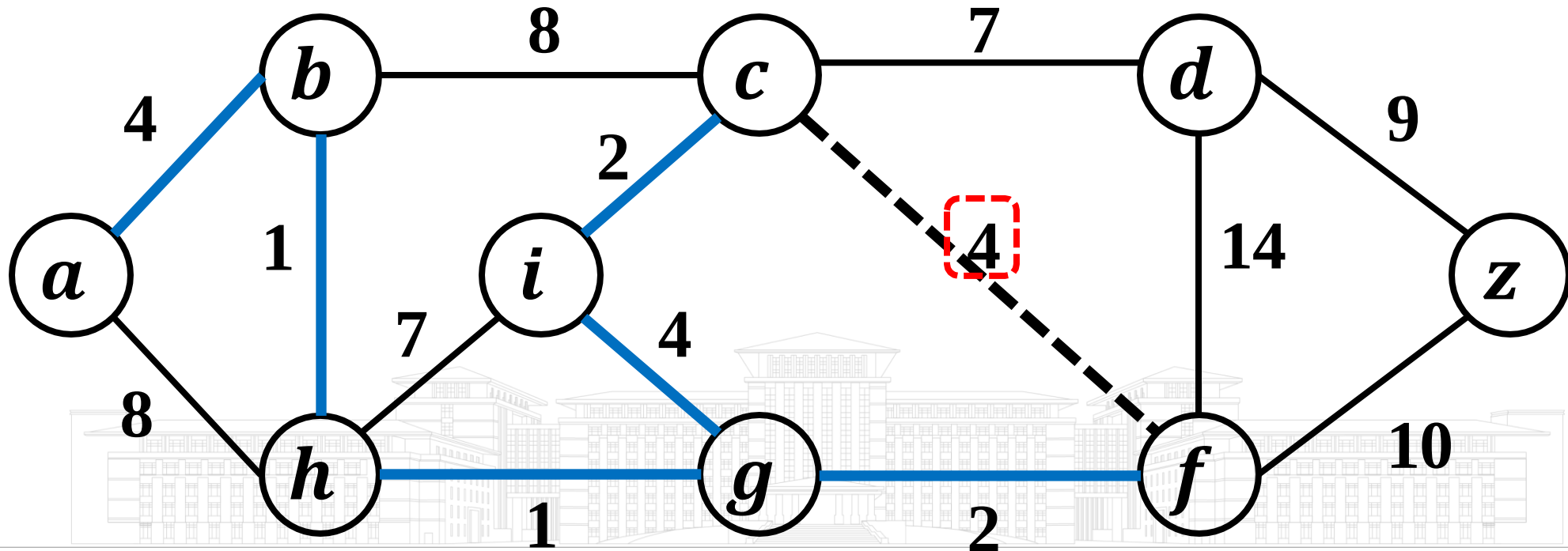
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



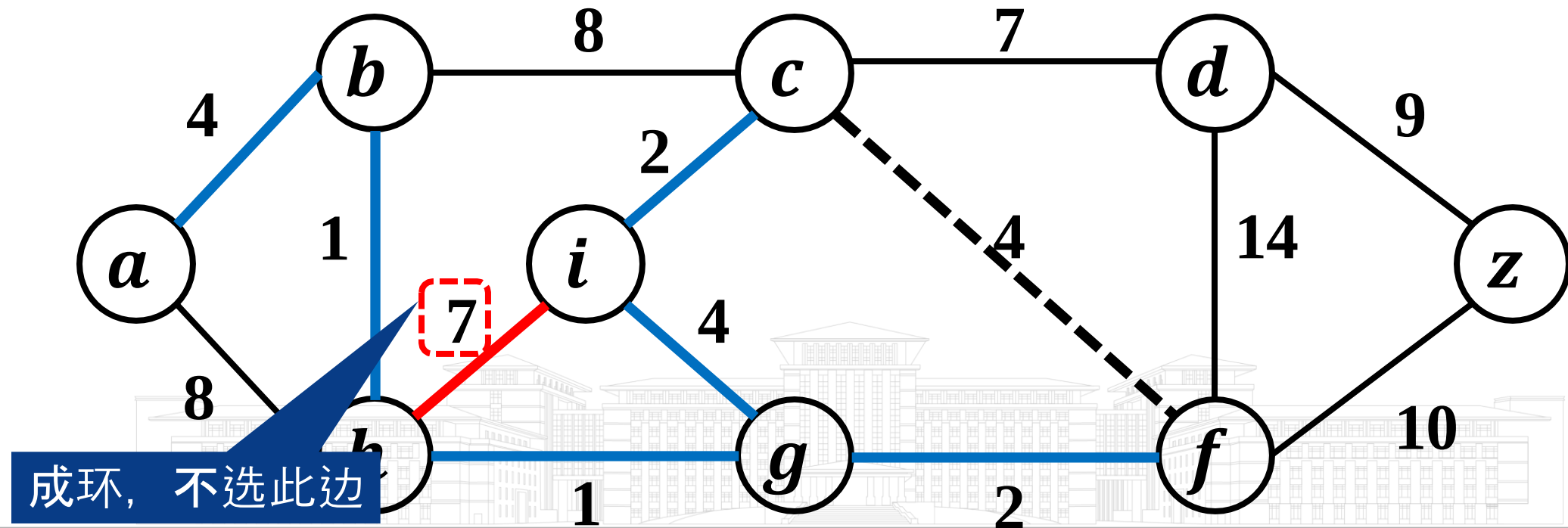
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



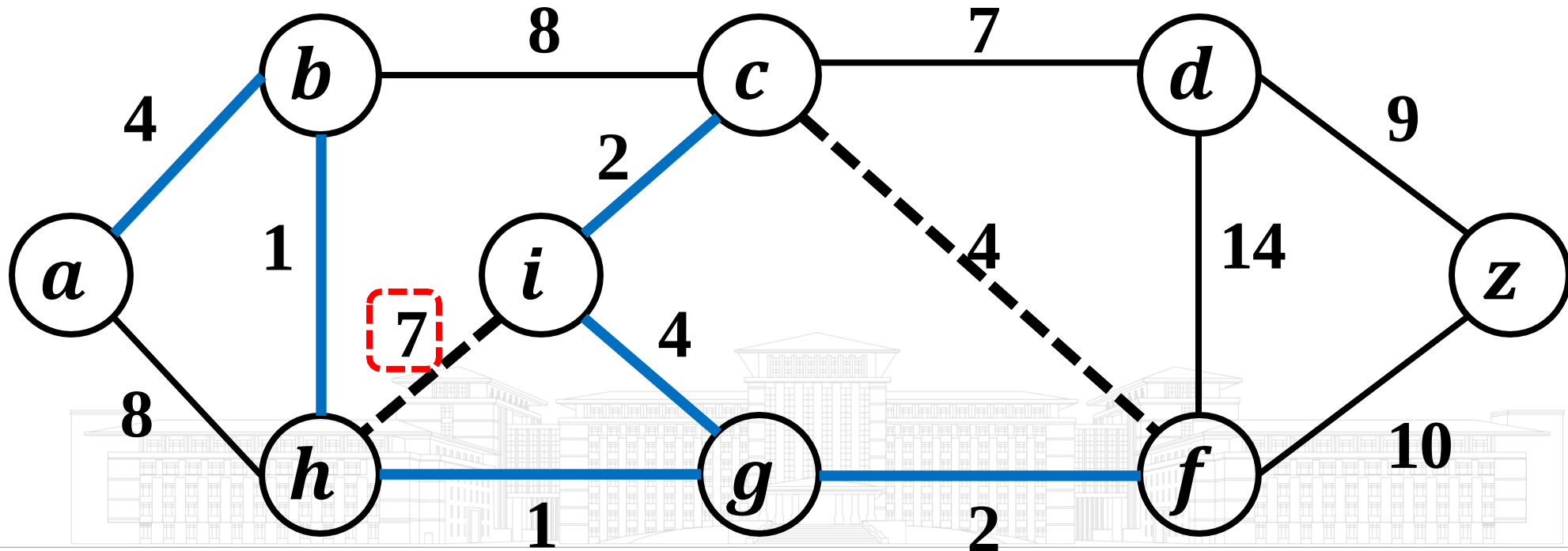
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



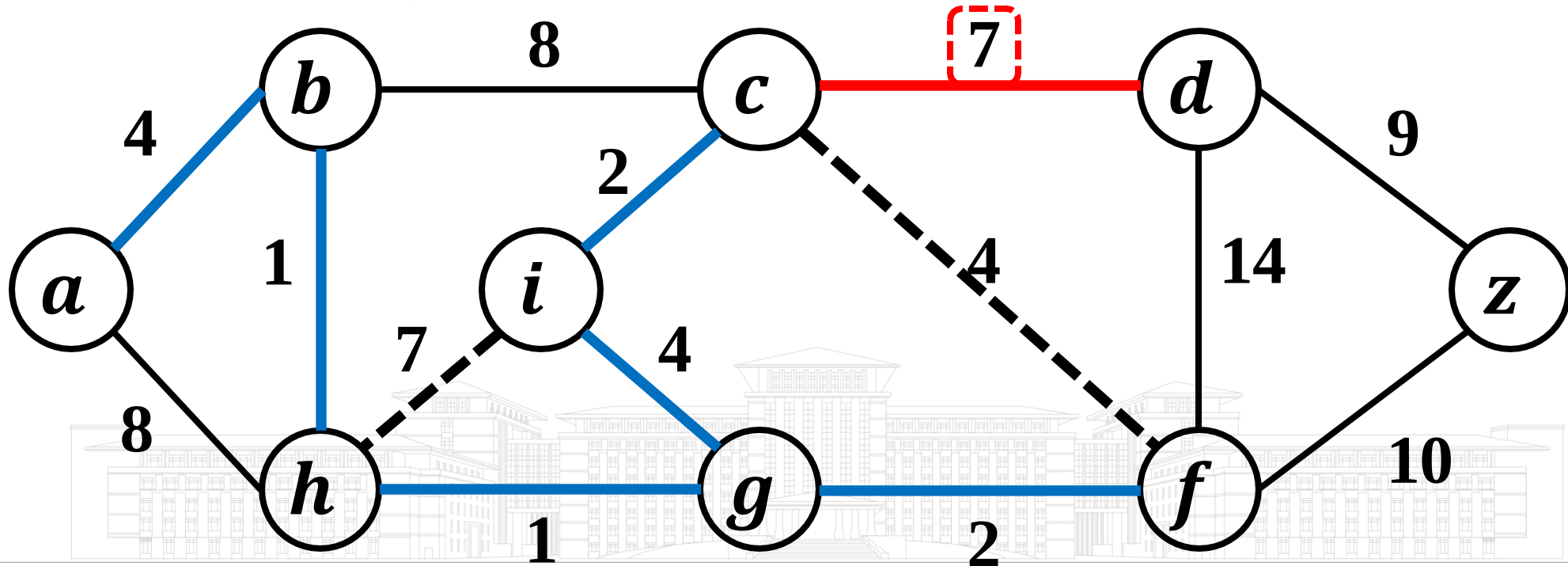
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



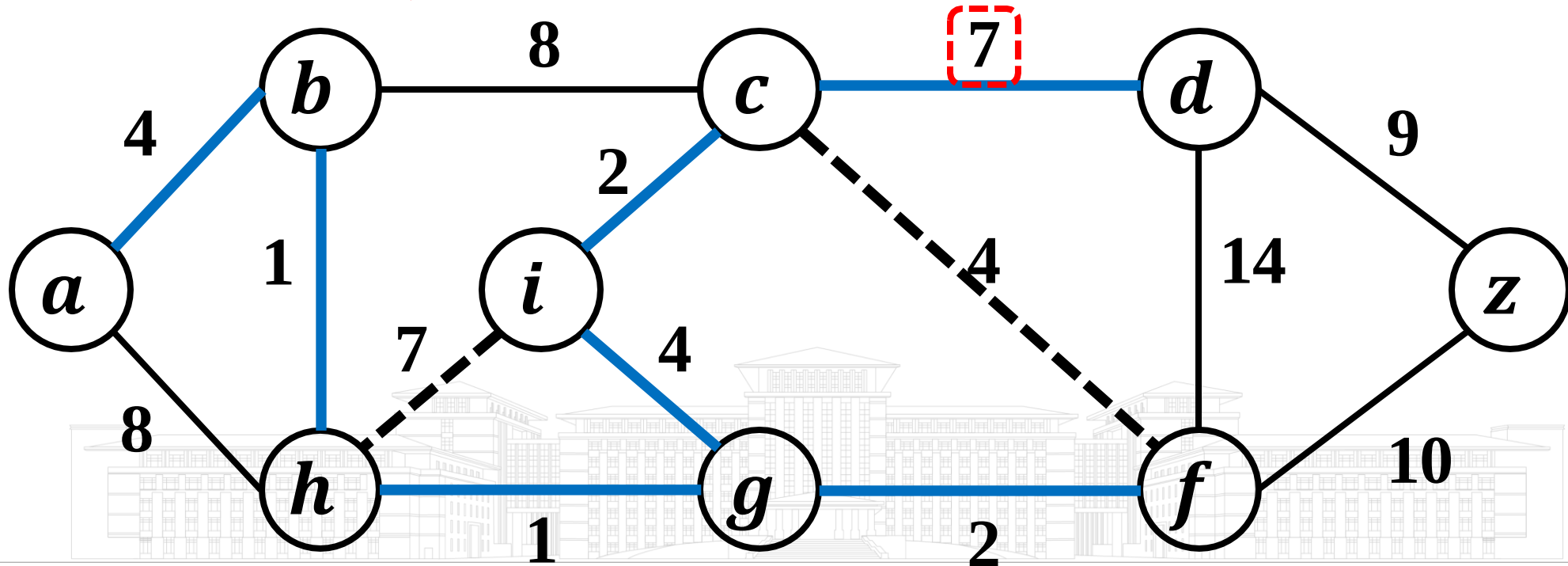
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



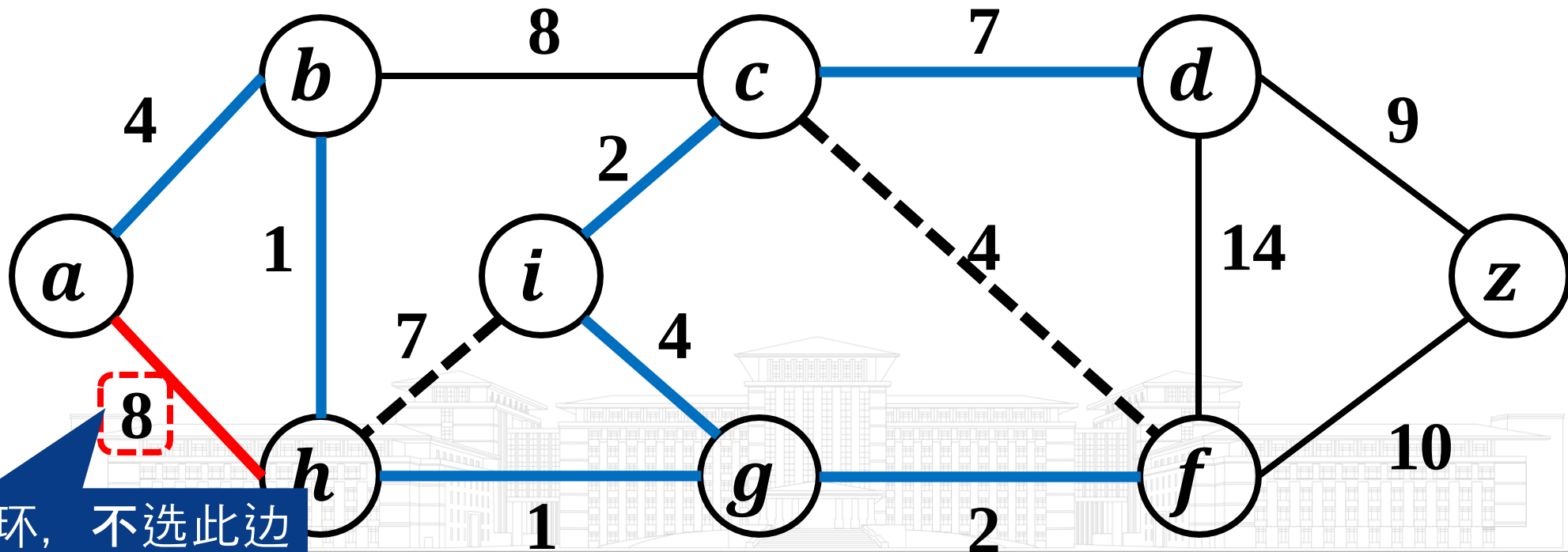
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



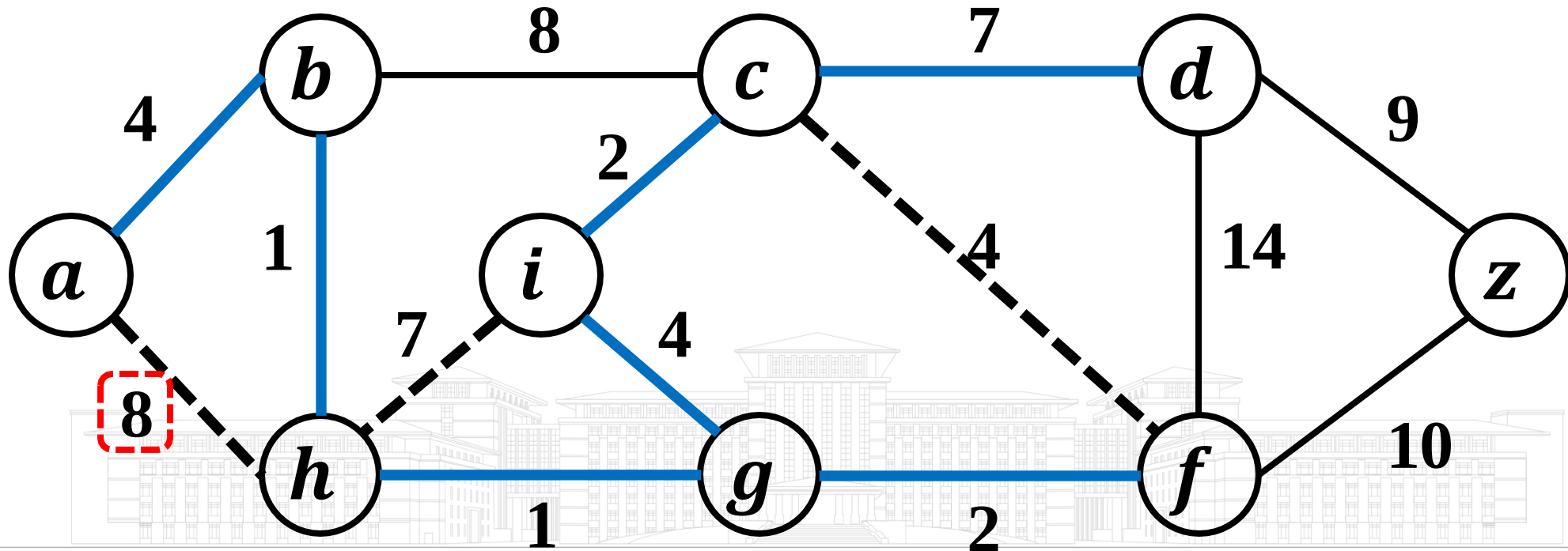
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



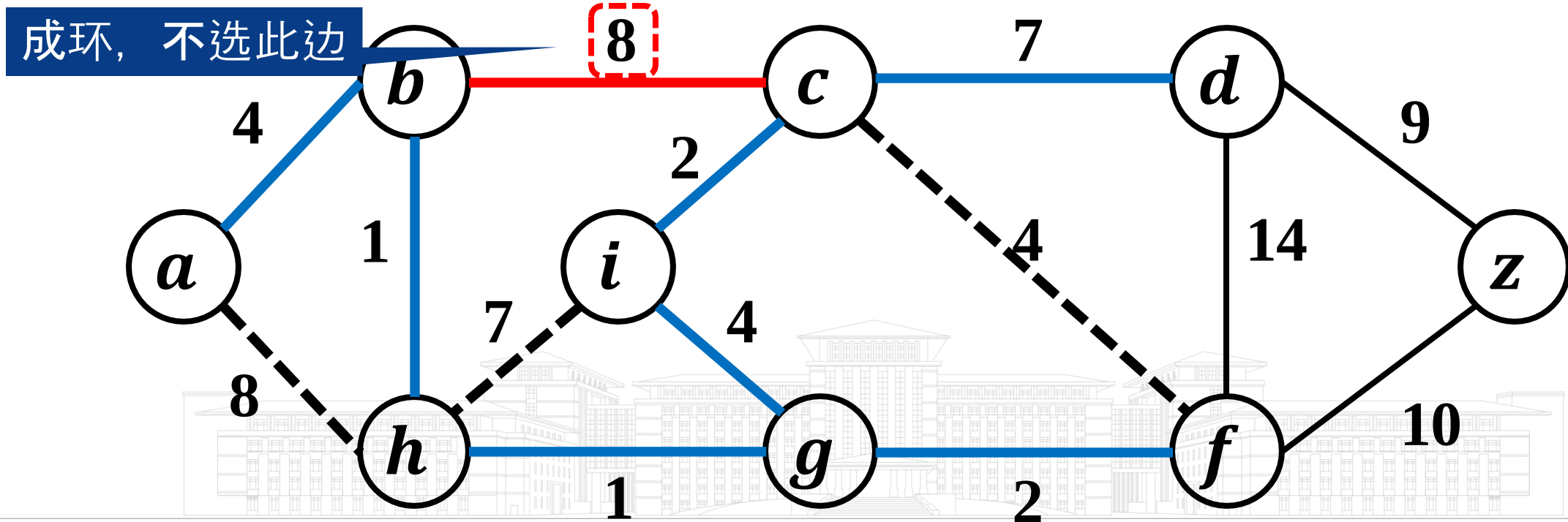
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



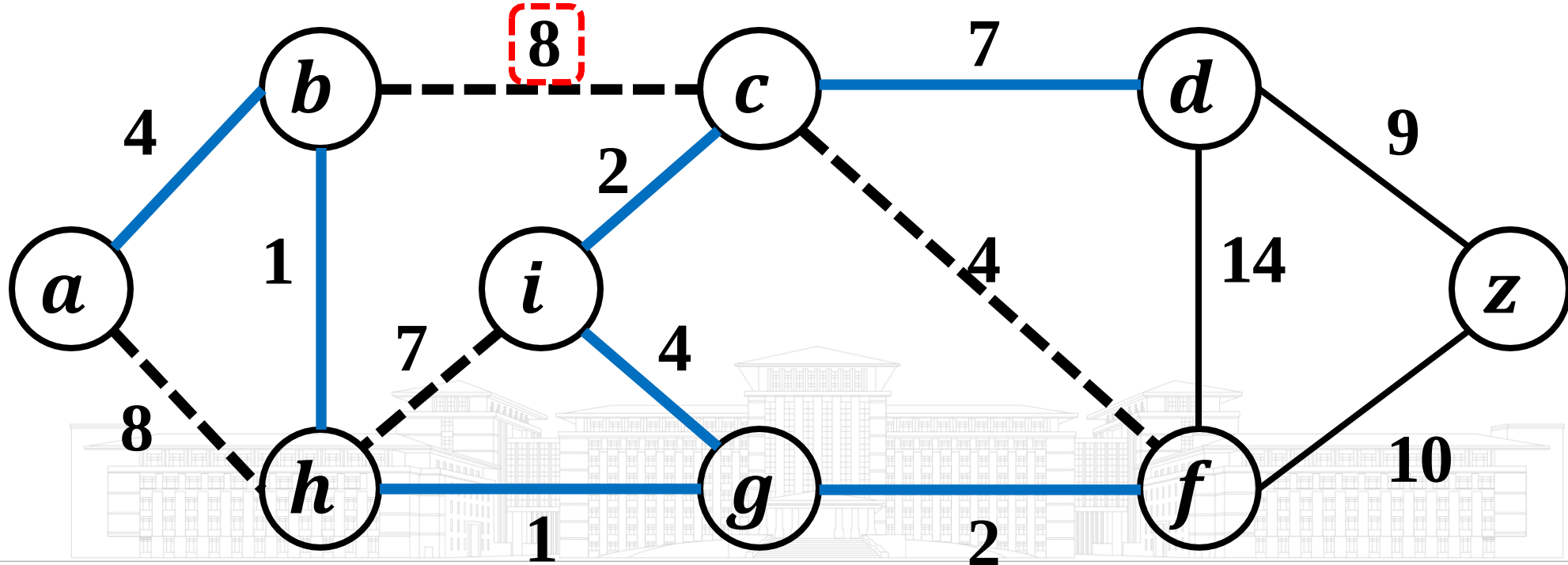
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



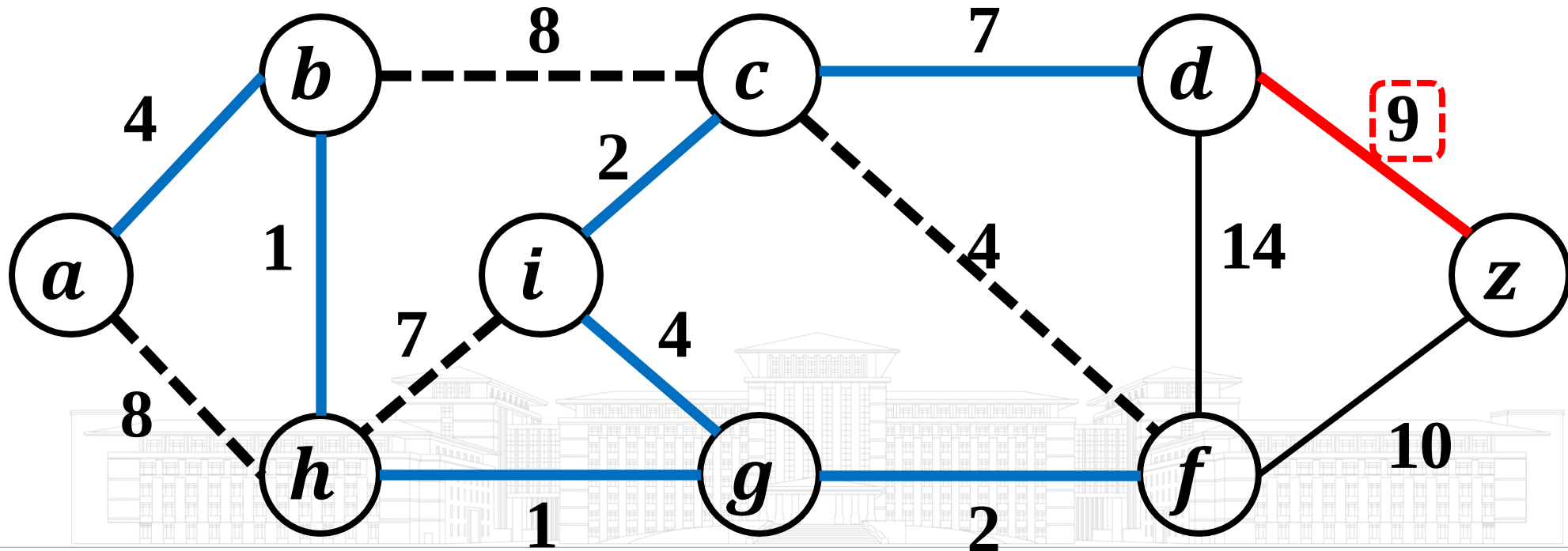
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



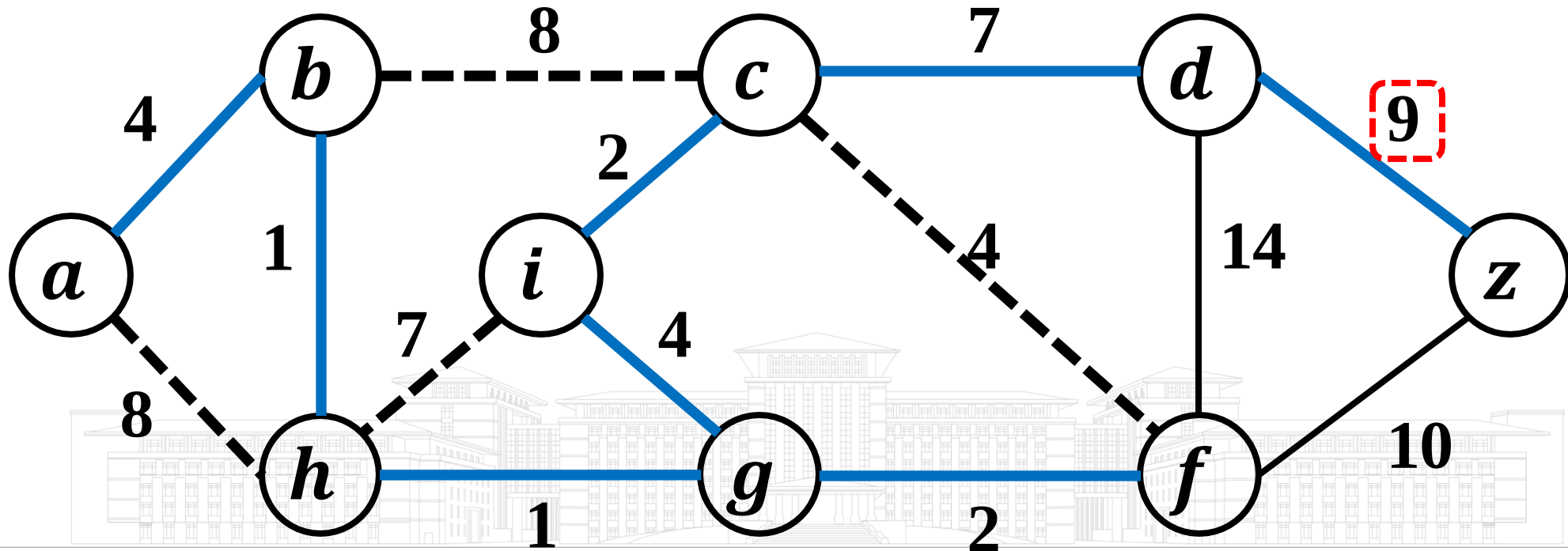
算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



算法实例

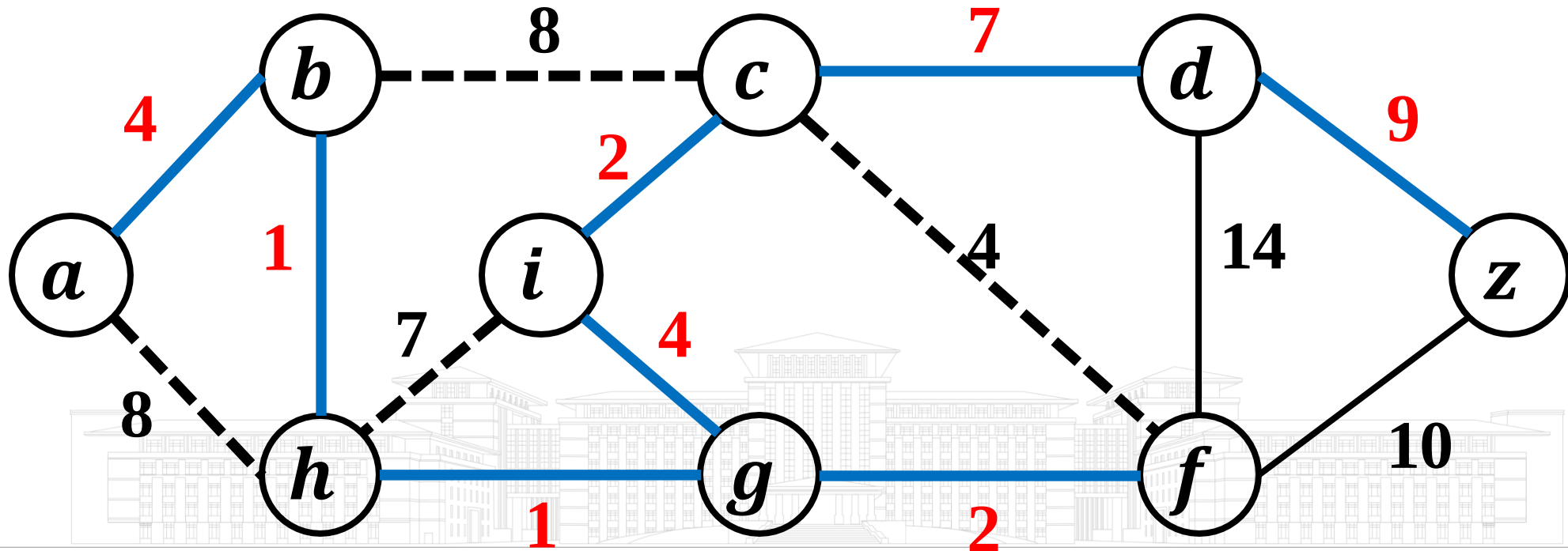
- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边



算法实例

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边

最小生成树权重：30



算法思想

• Generic-MST(G)

```
 $A \leftarrow \emptyset$   
while 没有形成最小生成树 do  
| 寻找  $A$  的安全边  $(u, v)$   
|  $A \leftarrow A \cup (u, v)$   
end  
return  $A$ 
```



不成环的最小边，是一种贪心策略



算法思想

• Generic-MST(G)

```
 $A \leftarrow \emptyset$ 
```

```
while 没有形成最小生成树 do
```

```
    | 寻找 $A$ 的安全边 $(u, v)$ 
```

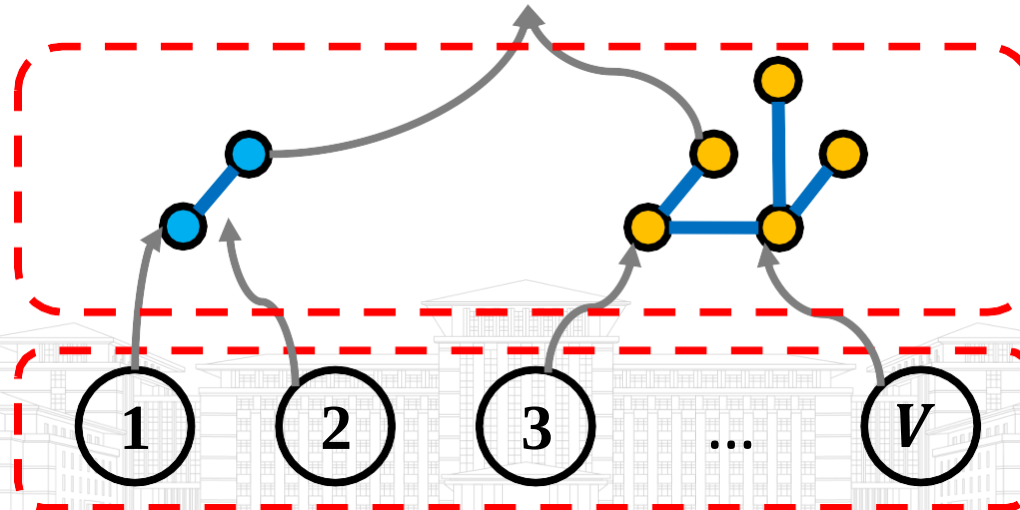
```
    |  $A \leftarrow A \cup (u, v)$ 
```

```
end
```

```
return  $A$ 
```

森林不断合并子树最终形成一棵树
不成环的最小边，是一种贪心策略

最小生成树 T



扩大成为原树

选边保持子树

顶点必是子树

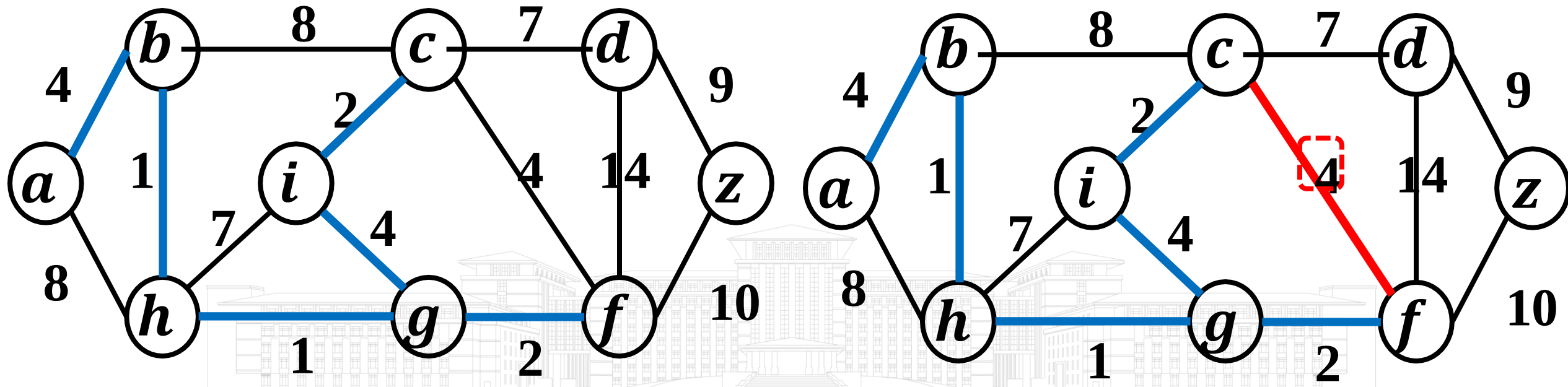

```

A ← ∅
while 没有形成最小生成树 do
    寻找A的安全边(u, v)
    A ← A ∪ (u, v)
end
return A

```

算法正确性的关键

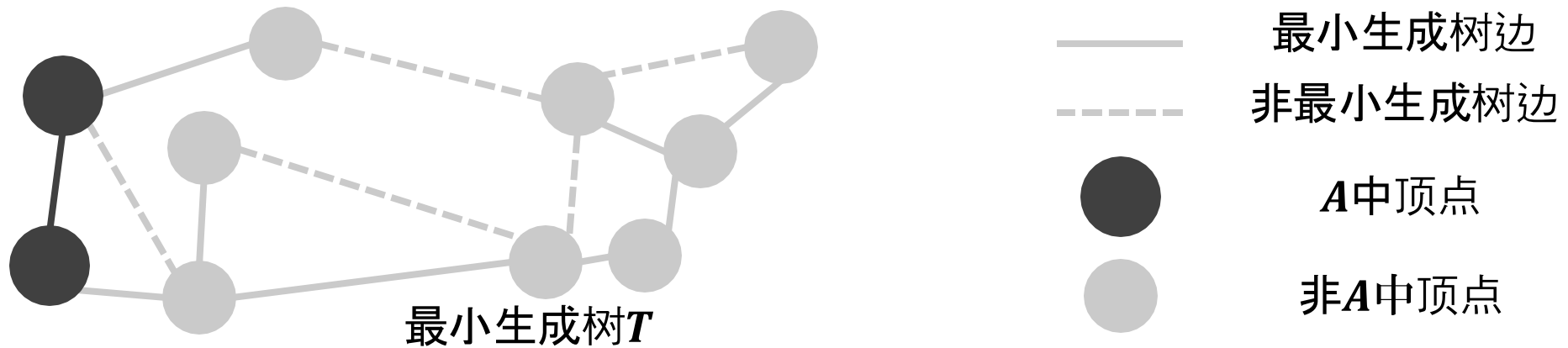
判断所选边的顶点是否在一棵树



安全边定义回顾

- 安全边(Safe Edge)

- A 是某棵最小生成树 T 边的子集, $A \subseteq T$
- $A \cup \{(u, v)\}$ 仍是某棵最小生成树边的一个子集, 则称 (u, v) 是 A 的**安全边**

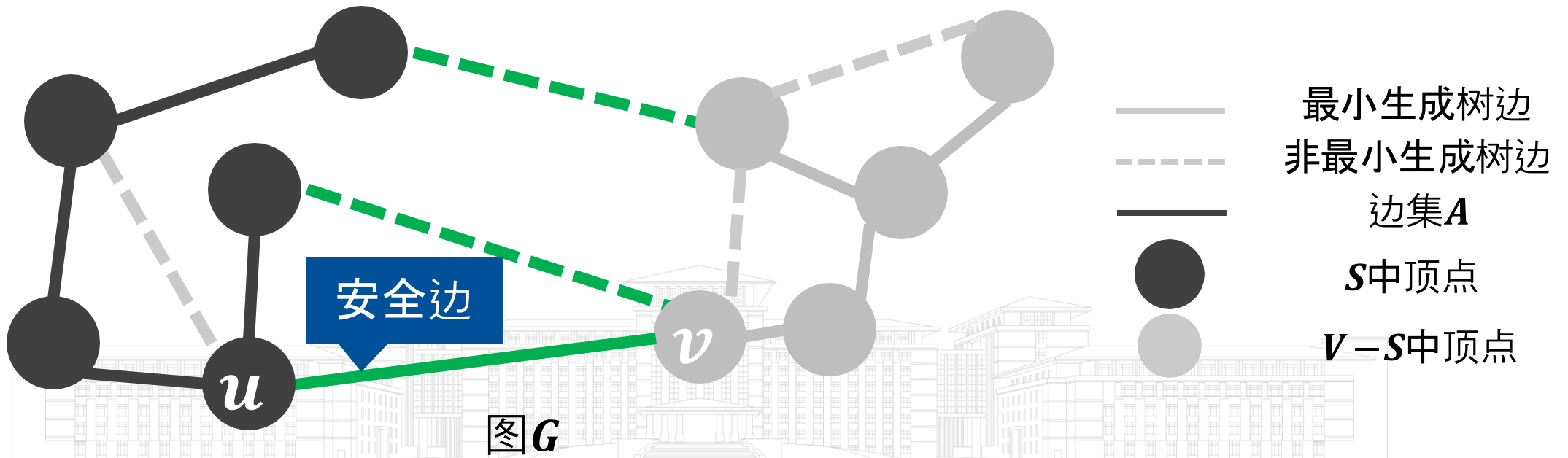


若每次向边集 A 中新增**安全边**, 可保证边集 A 是最小生成树的子集

问题: **Kruskal**算法选边策略能否保证每次都选择了安全边?

安全边辨识定理

- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图，令 A 为边集 E 的一个子集，且 A 包含在图 G 的某棵最小生成树中
 - 若割 $(S, V - S)$ 是图 G 中不妨害边集 A 的任意割，且 (u, v) 是横跨该割的轻边
 - 则对于边集 A ，边 (u, v) 是其安全边





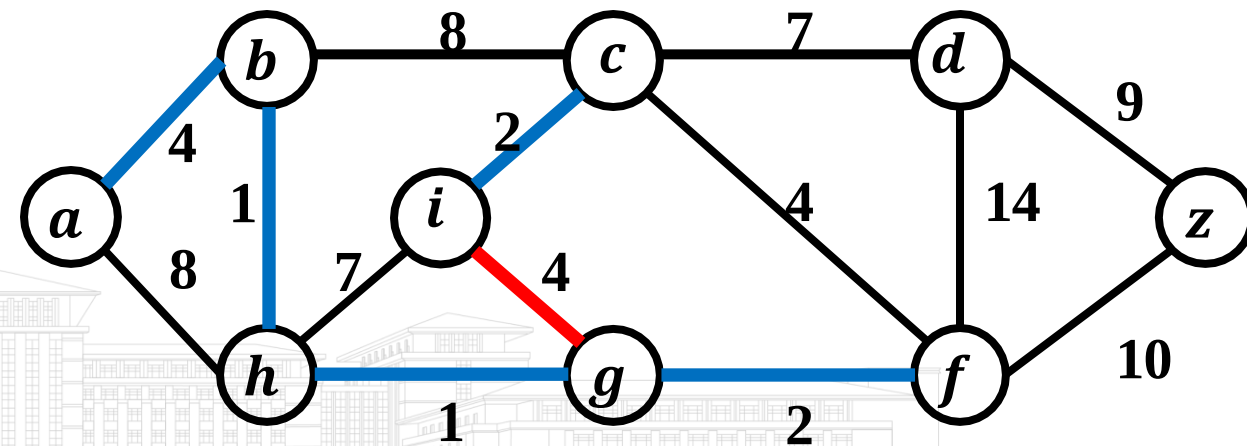
正确性证明

- **Kruskal**算法选边策略能否保证每次都选择了安全边？——能！



正确性证明

- **Kruskal**算法选边策略能否保证每次都选择了安全边？——能！
- 证明
 - 不妨设当前已选边集为***A***，下一条选择的边是(*i*, *g*)



正确性证明

• **Kruskal**算法选边策略能否保证每次都选择了安全边？——能！

• 证明

- 不妨设当前已选边集为 A ，下一条选择的边是 (i, g)
- 已选边集 A 把图分为若干子树，其中 (V', E') 是包含顶点 i 的子树
- 构造割 $(V', V - V')$ ，割不妨害边集 A ，即 A 中的边不横跨 $(V', V - V')$

- 边集 A 不包括横跨边 (i, g)

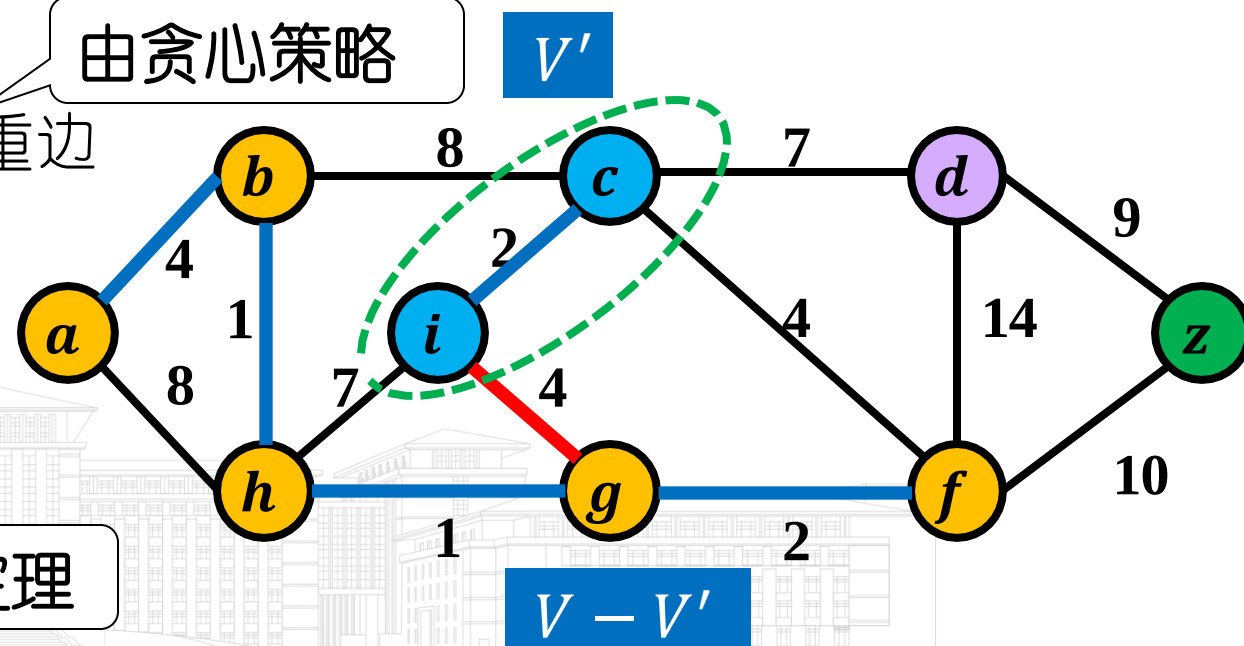
- (i, g) 是横跨 $(V', V - V')$ 的最小权重边

- (i, g) 是割 $(V', V - V')$ 的轻边

- 割 $(V', V - V')$ 不妨害边集 A

- 轻边 (i, g) 是安全边

由贪心策略



由安全边辨识定理

伪代码

● MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

按权重排序

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if u, v 不在同一子树 then

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

 end

end

return T



伪代码

• MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

$T \leftarrow \{\}$

for $(u, v) \in E$ **do**

if u, v 不在同一子树 **then**

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

end

end

return T

按照权重从小到大考察边



伪代码

● MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if u, v 不在同一子树 then

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

 end

end

return T

加入该边不成环



伪代码

● MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if u, v 不在同一子树 then

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

 end

end

return T

加入该边, 更新连通状态



伪代码

• MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if u, v 不在同一子树 then

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

 end

end

return T

思考：如何高效判定和维护所选边的顶点是否在一棵子树？

不相交集合

● MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if u, v 不在同一子树 then

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

 end

end

return T



需要高效查找顶点所属子树

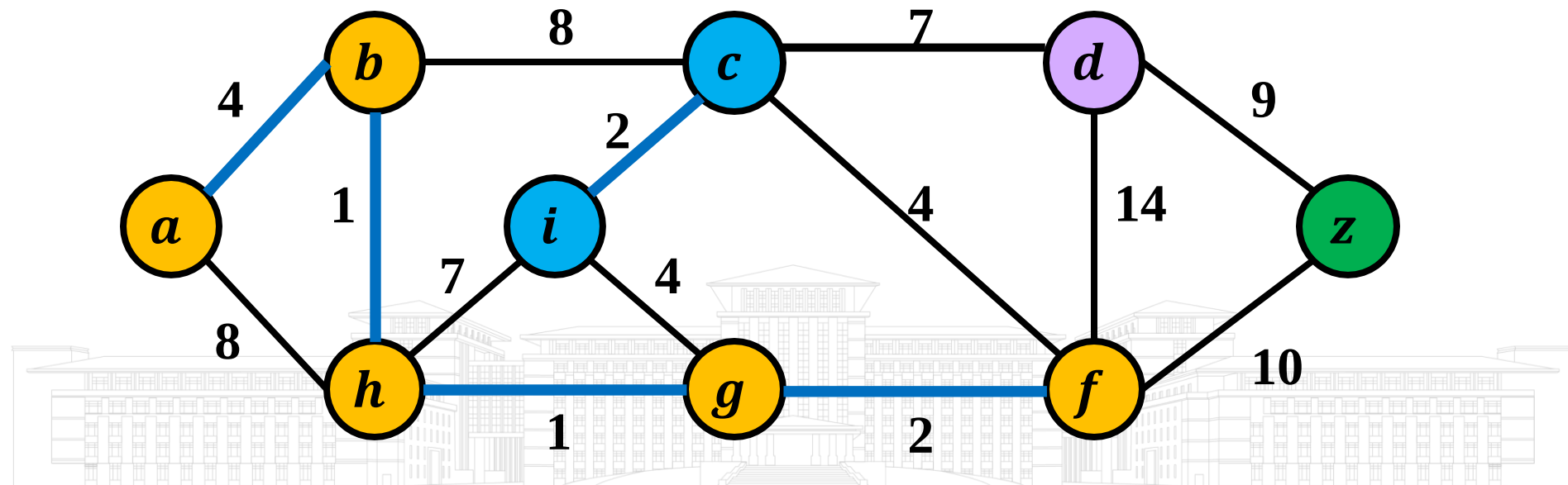


需要高效合并顶点所在子树

同时高效完成两类操作需借助数据结构：不相交集合

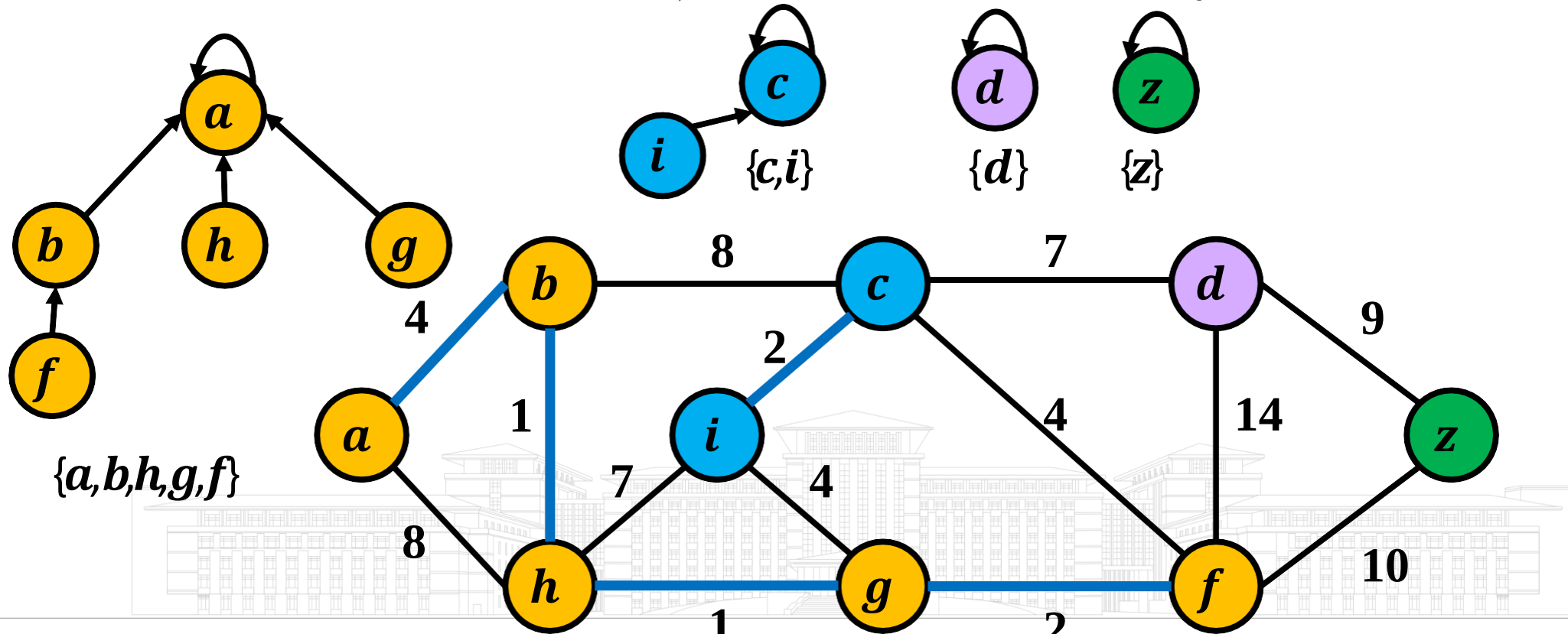
不相交集合

- 把每棵生成子树看作一个顶点集合



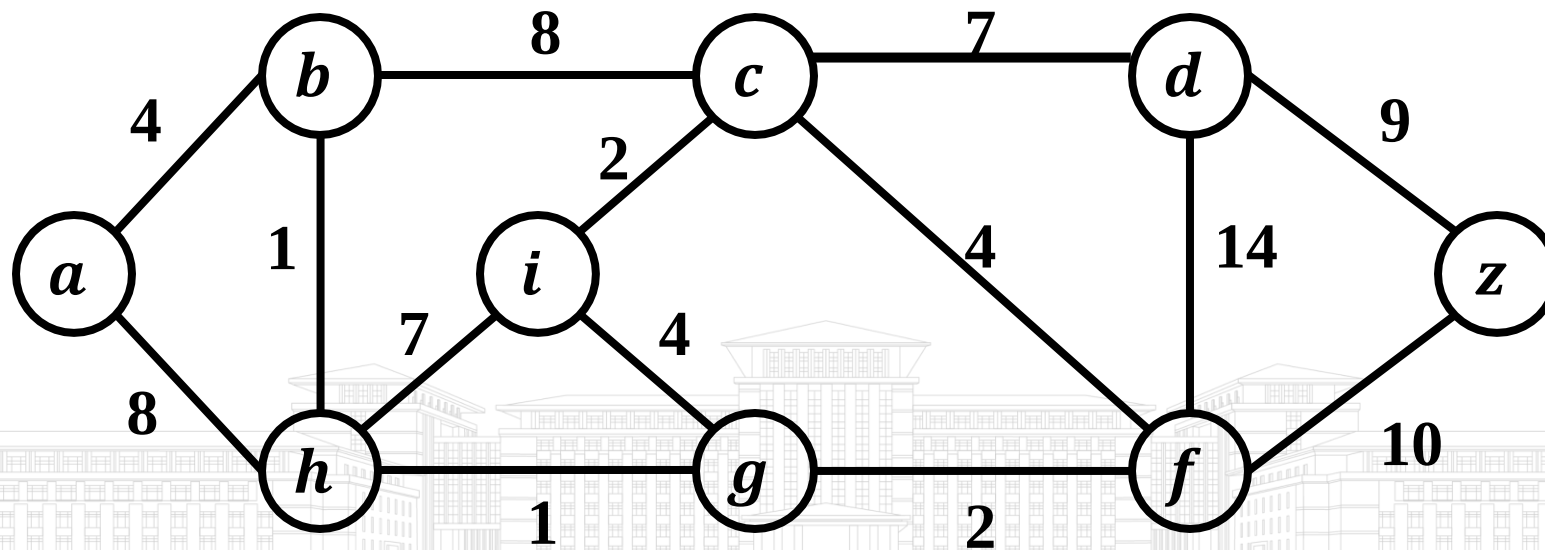
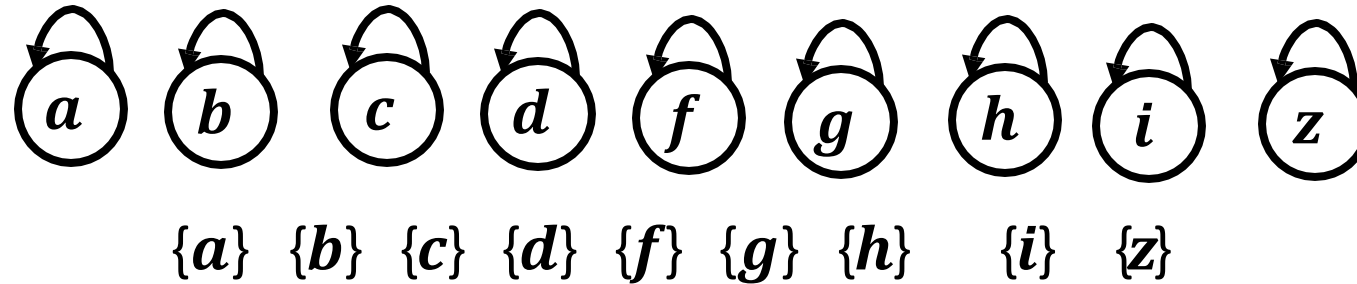
不相交集合

- 把每棵生成子树看作一个顶点集合
 - 每个集合表示为一棵有向树，多个不相交集合构成不相交集合森林
 - 集合元素表示为树结点
 - 树边由子结点指向父结点，根结点有一条指向自身的边



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边



不相交集合：伪代码

● Create-Set(x)

输入: 顶点 x

输出: 并查集

$x.parent \leftarrow x$

return x

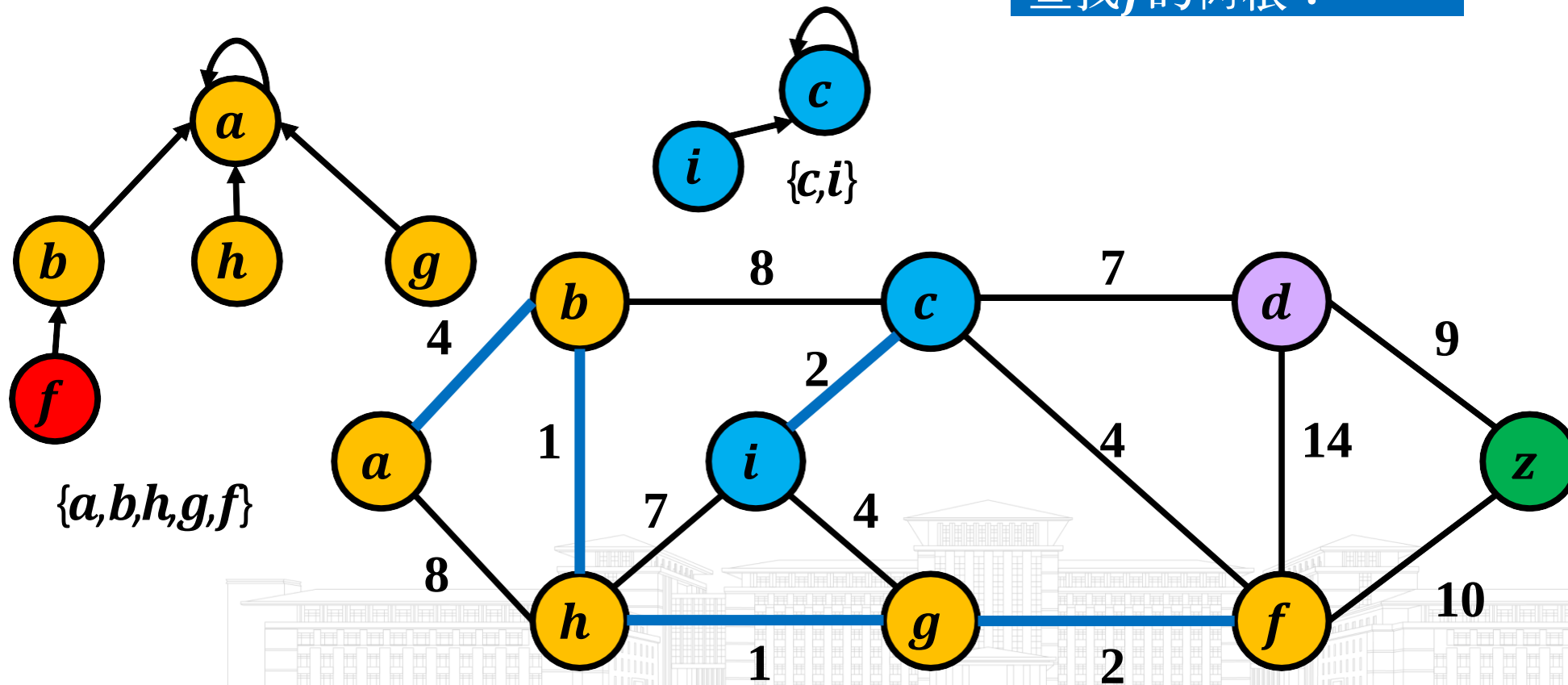
自身为树根



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

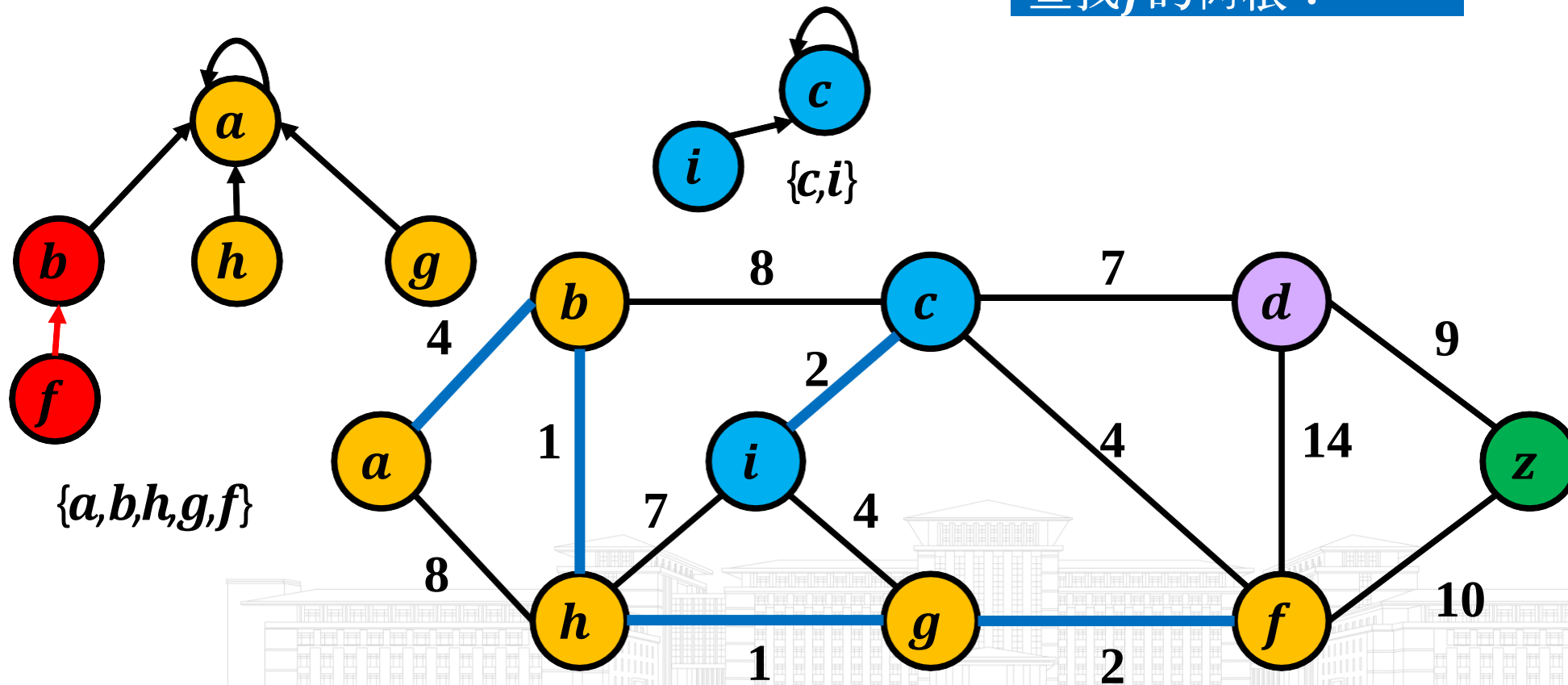
查找 f 的树根：



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

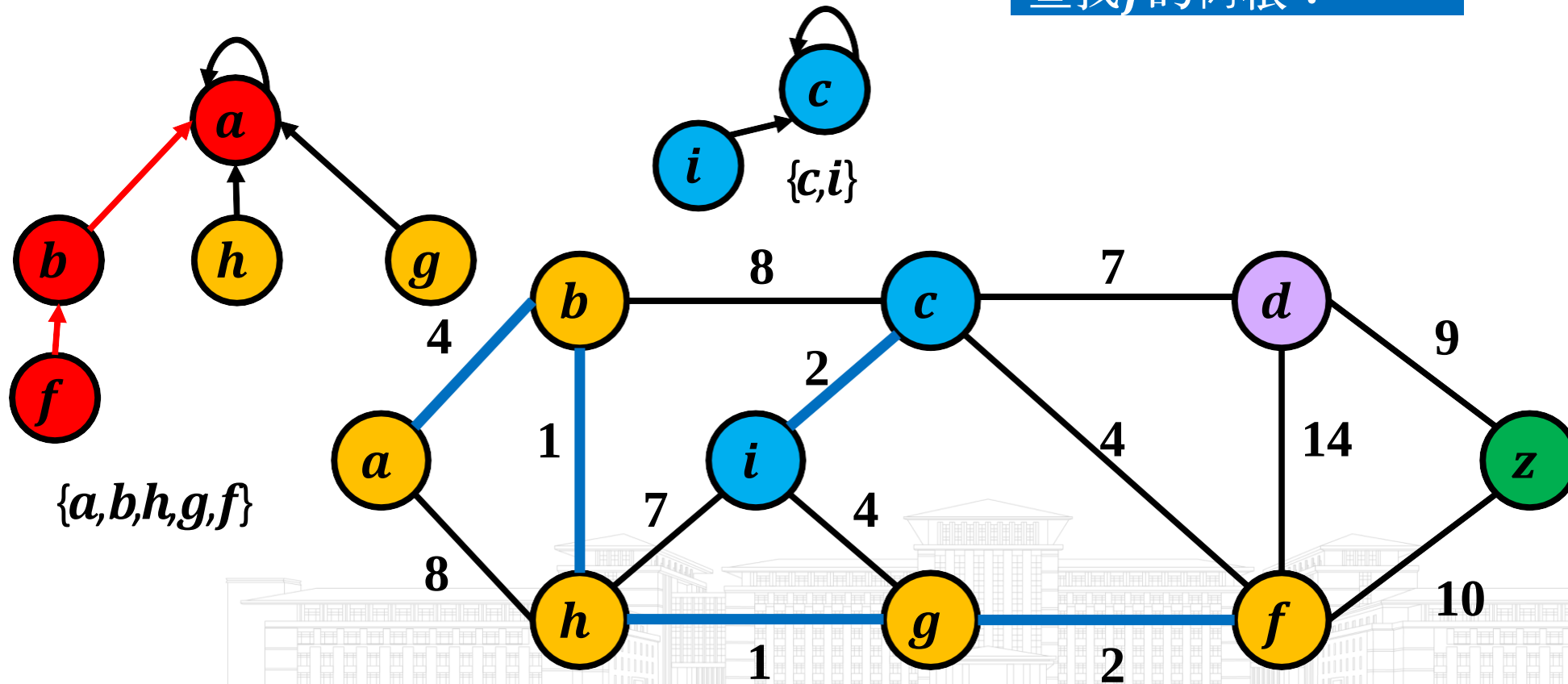
查找 f 的树根：



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

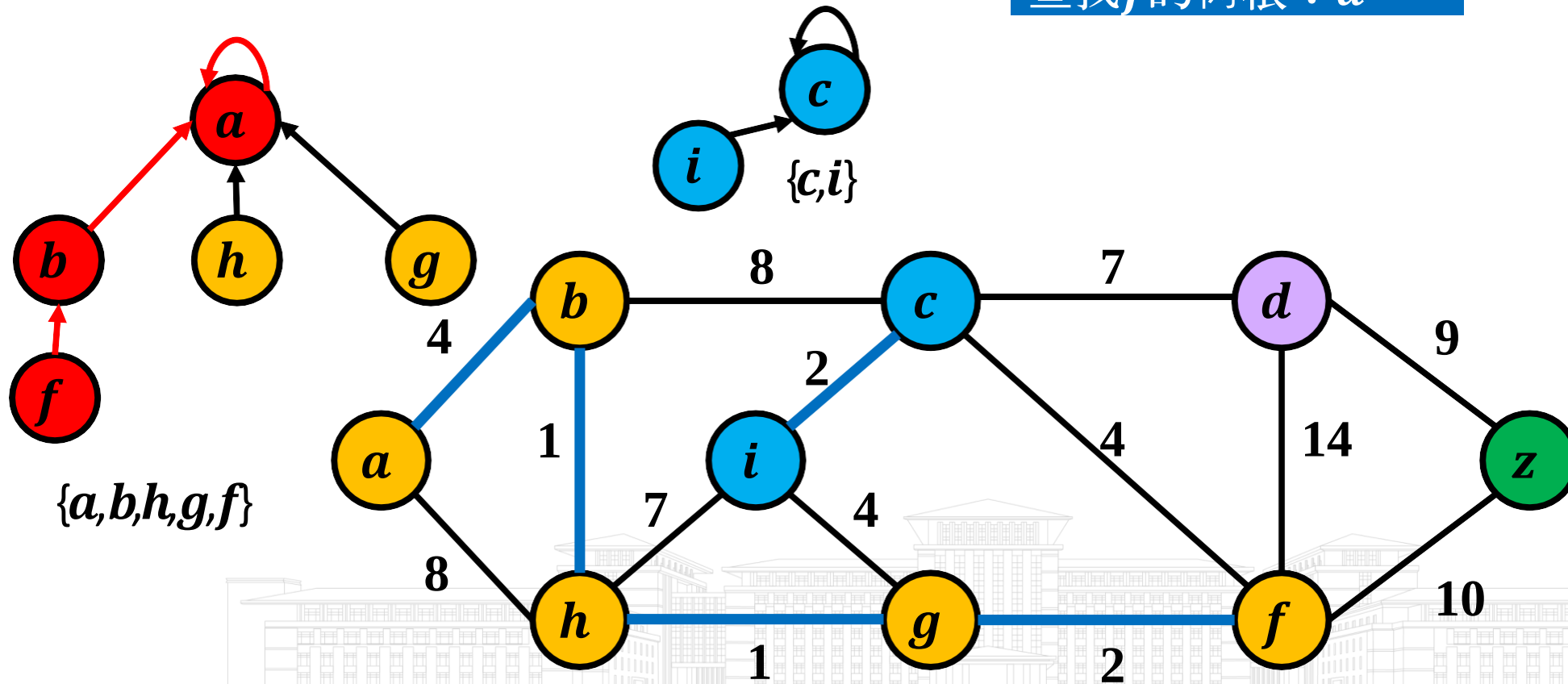
查找 f 的树根：



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

查找 f 的树根： a

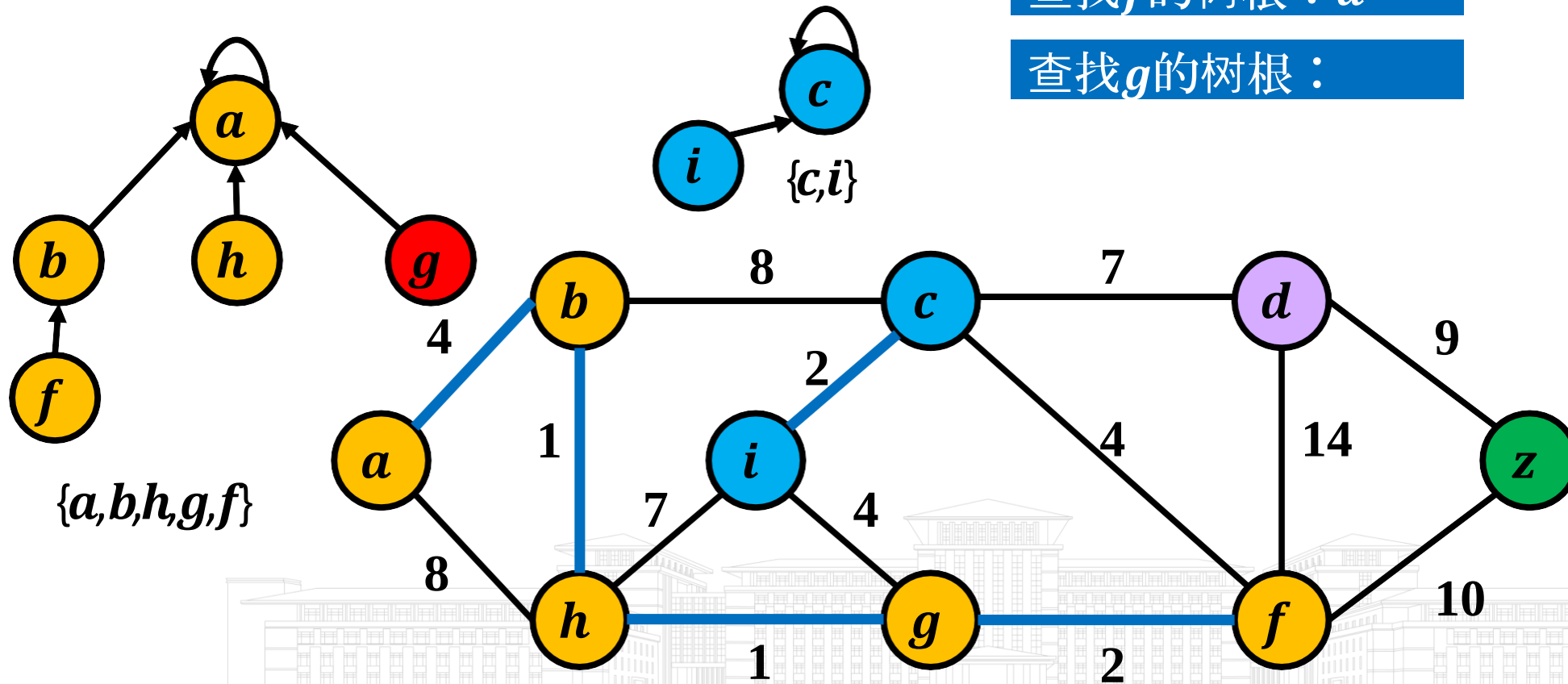


不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

查找 f 的树根： a

查找 g 的树根：

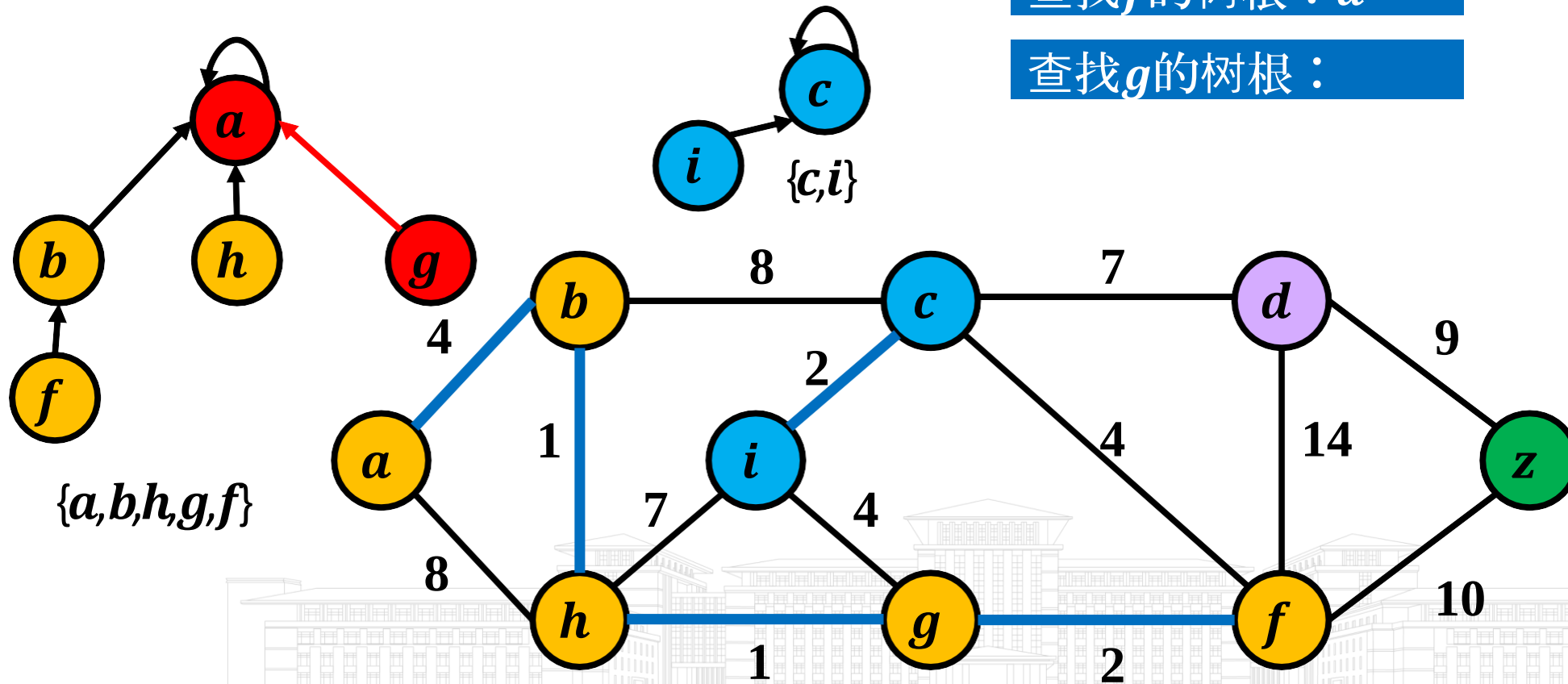


不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

查找 f 的树根： a

查找 g 的树根：

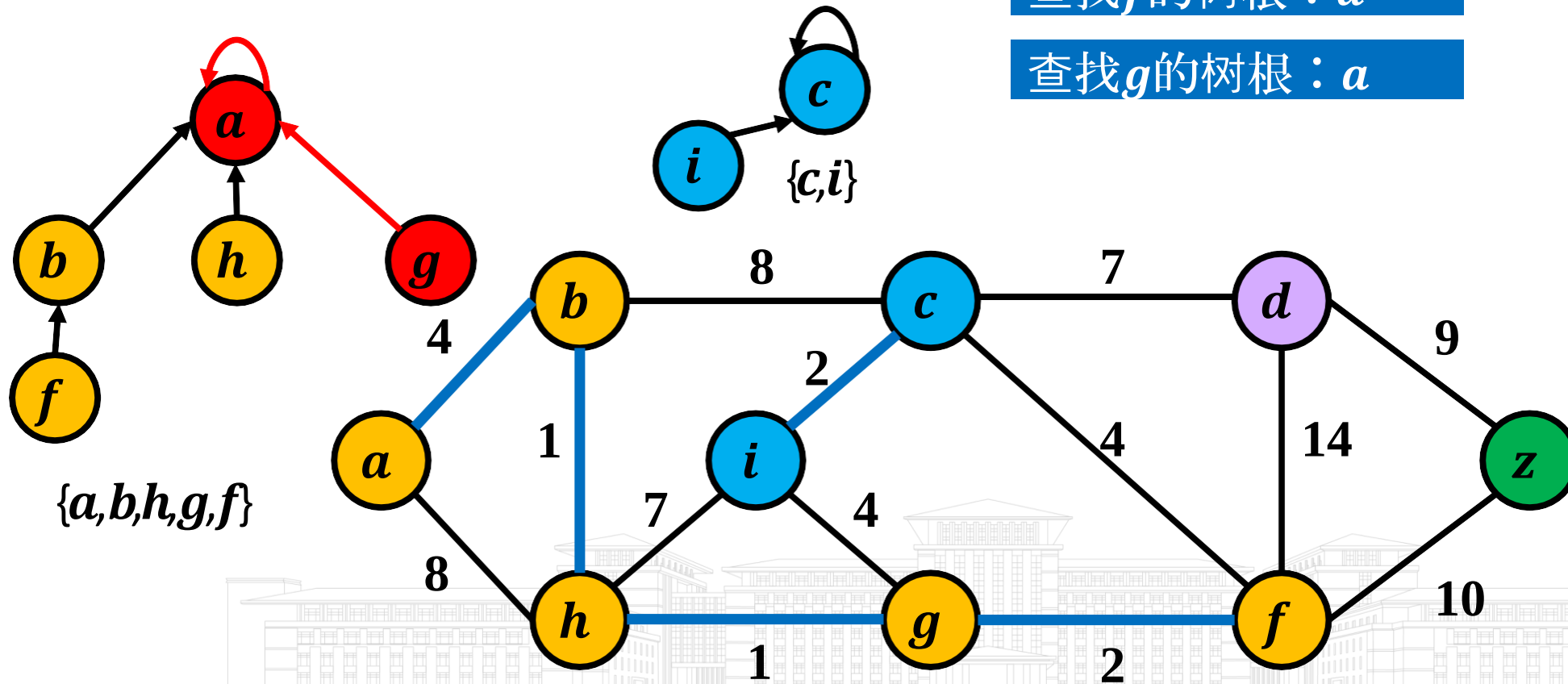


不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同

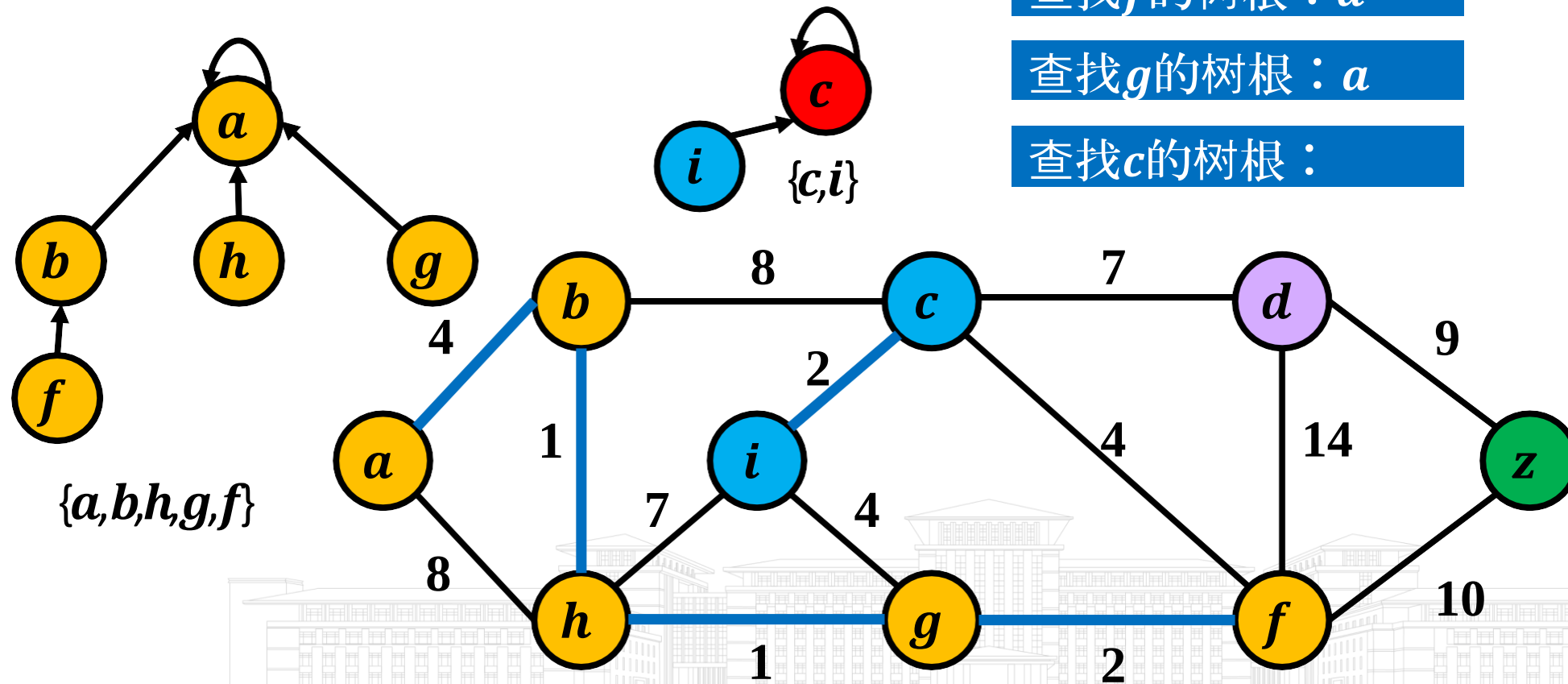
查找 f 的树根： a

查找 g 的树根： a



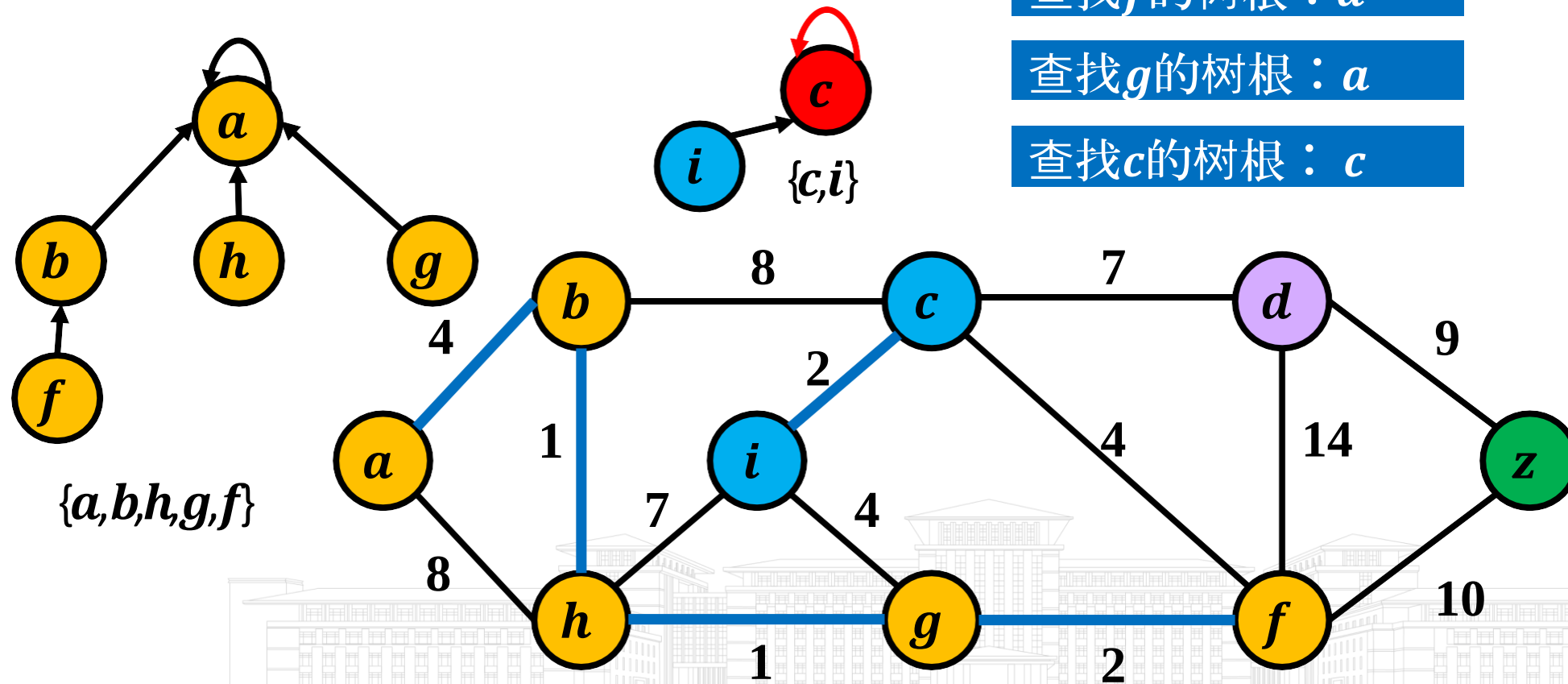
不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同



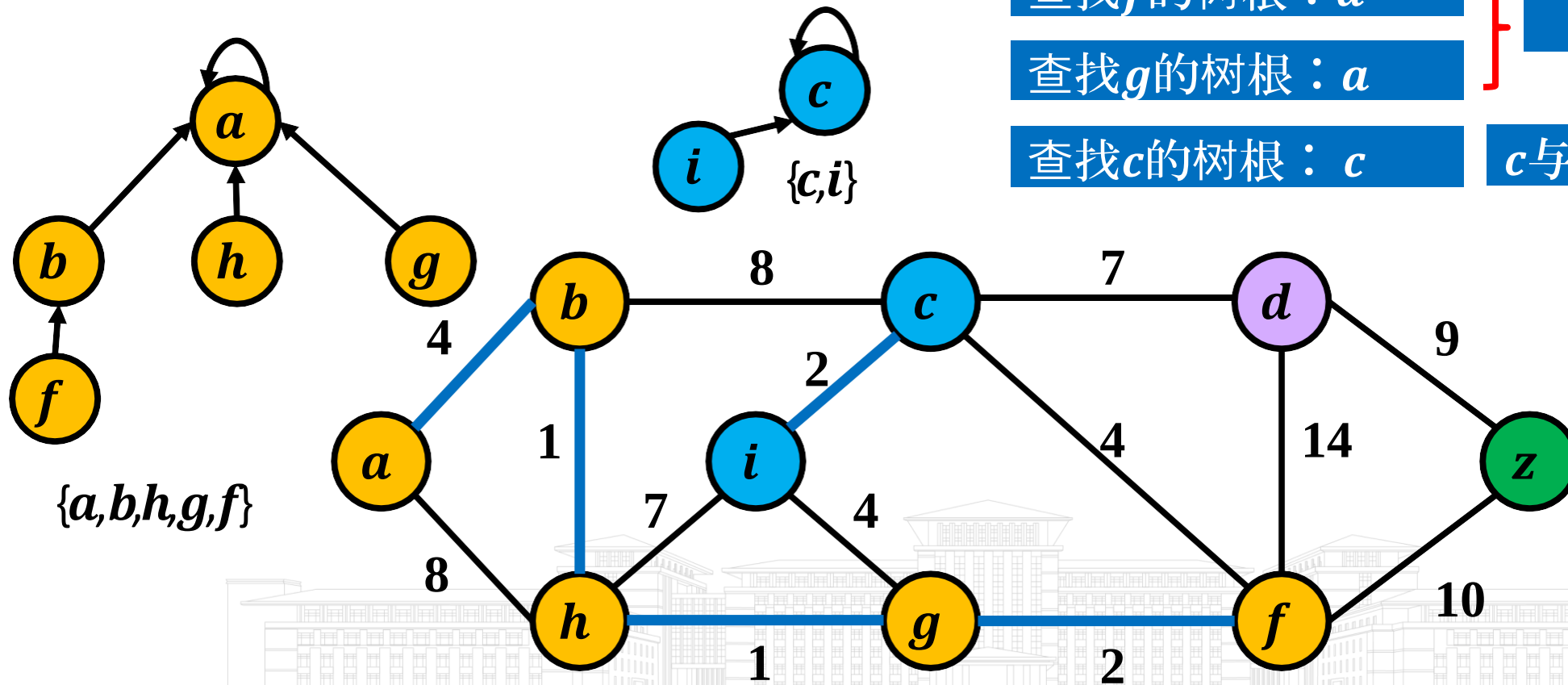
不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同



查找 f 的树根： a

查找 g 的树根： a

查找 c 的树根： c

f 与 g 在同一集合

c 与 f, g 不在同一集合

不相交集合：伪代码

● Create-Set(x)

输入: 顶点 x

输出: 不相交集合树

$x.parent \leftarrow x$

return x

● Find-Set(x)

输入: 顶点 x

输出: 所属连通分量

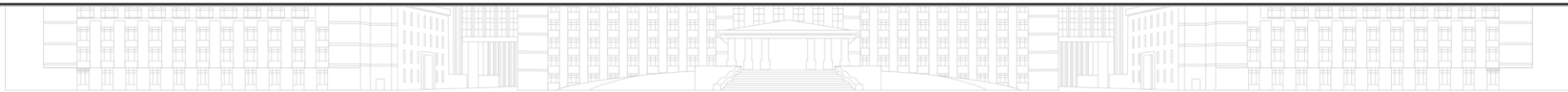
while $x.parent \neq x$ do

| $x \leftarrow x.parent$

end

return x

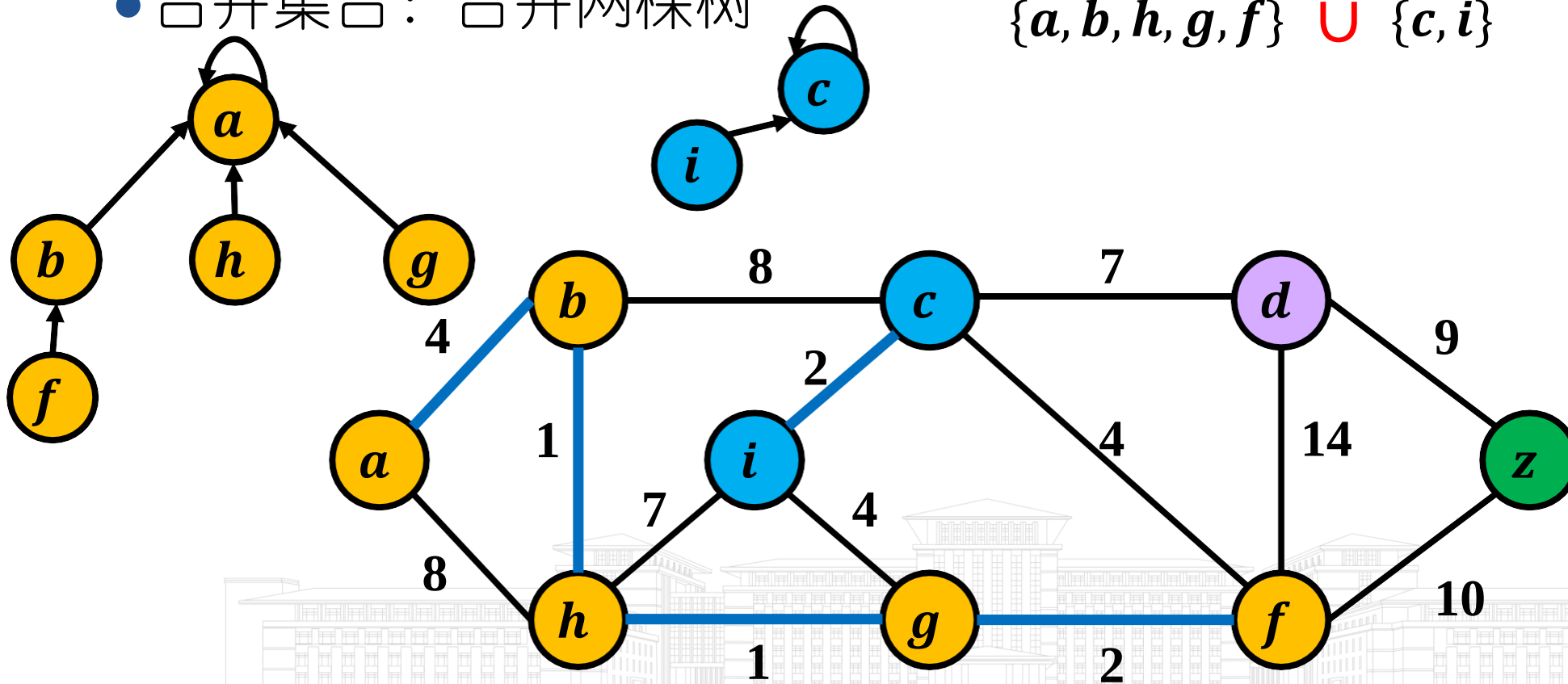
回溯查找



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树

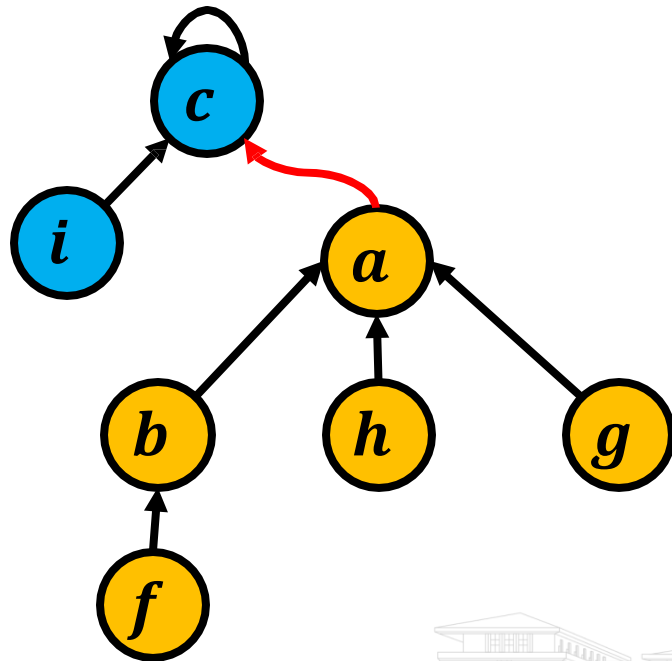
$\{a, b, h, g, f\} \cup \{c, i\}$



不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树

$$\{a, b, h, g, f\} \cup \{c, i\}$$

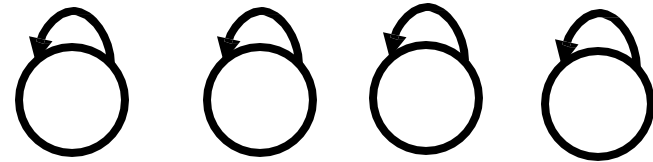
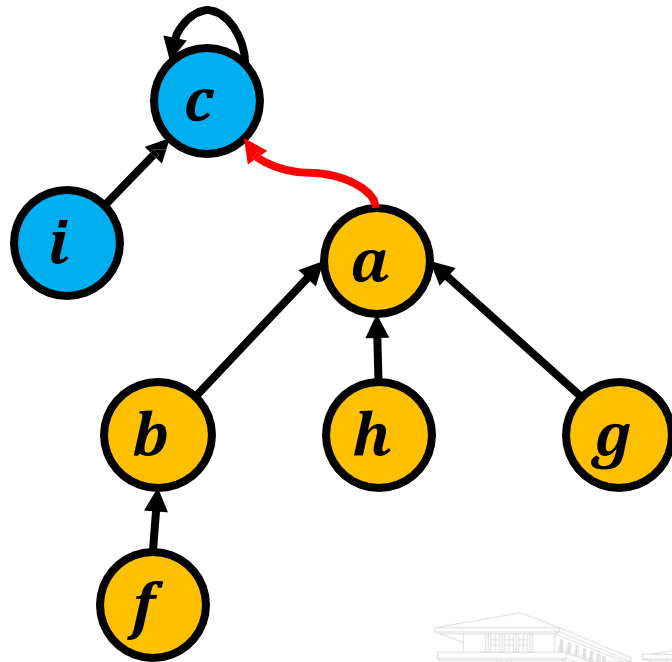


简单实现：找到两树根，任意连接两棵树

思考：是否高效？

不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树

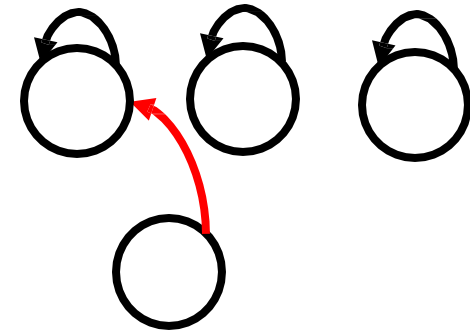
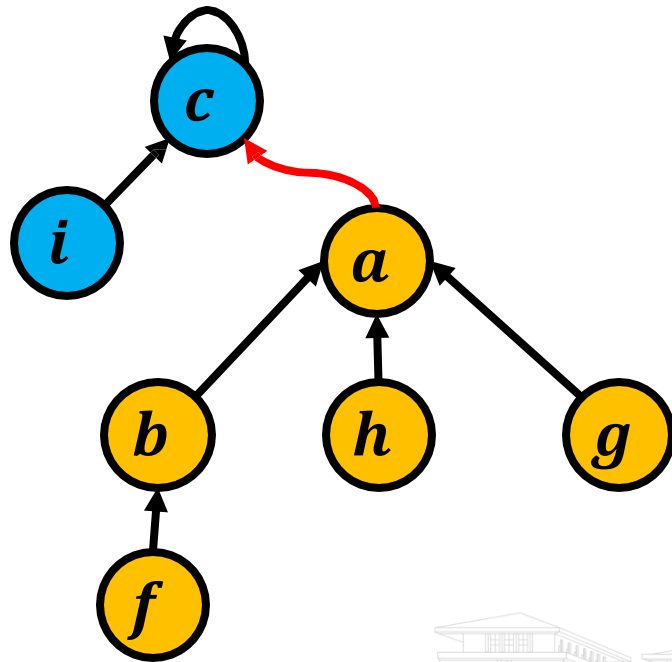


简单实现：找到两树根，任意连接两棵树

思考：是否高效？

不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树

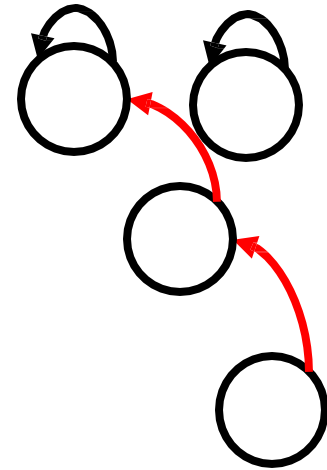
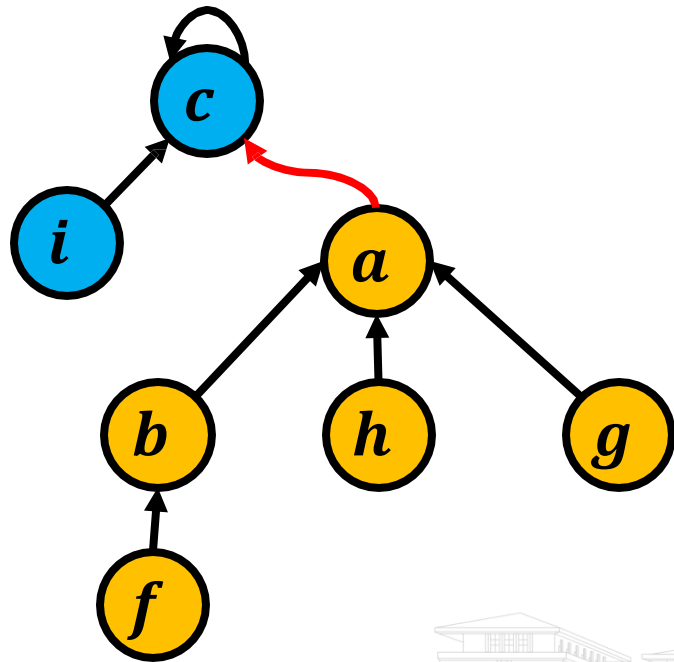


简单实现：找到两树根，任意连接两棵树

思考：是否高效？

不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树

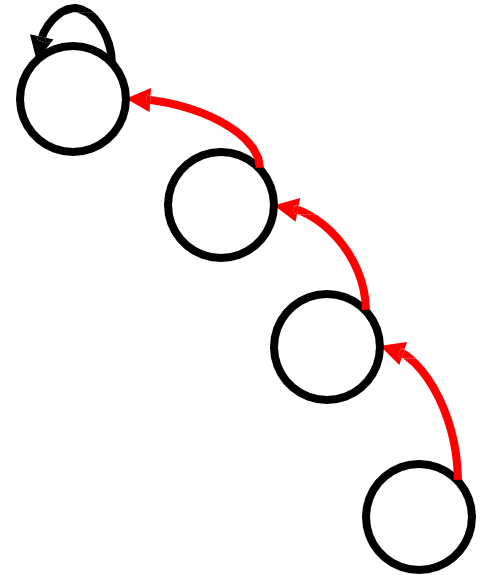
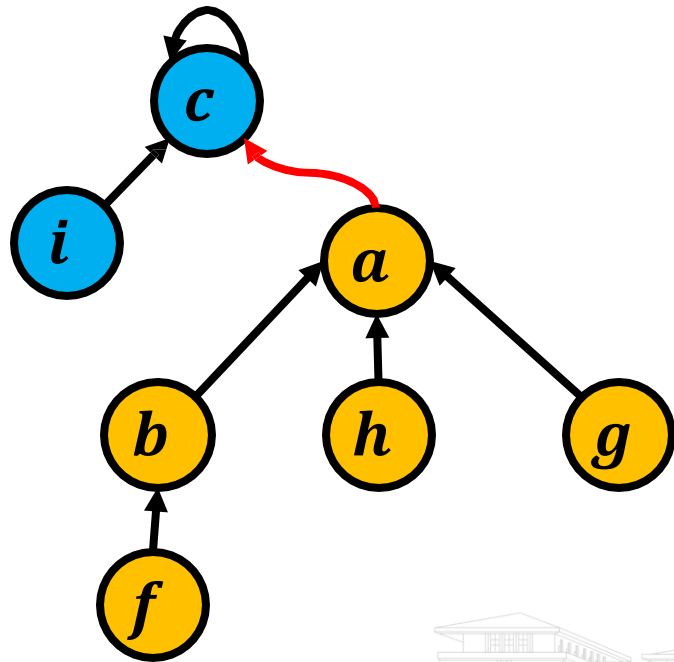


简单实现：找到两树根，任意连接两棵树

思考：是否高效？

不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树

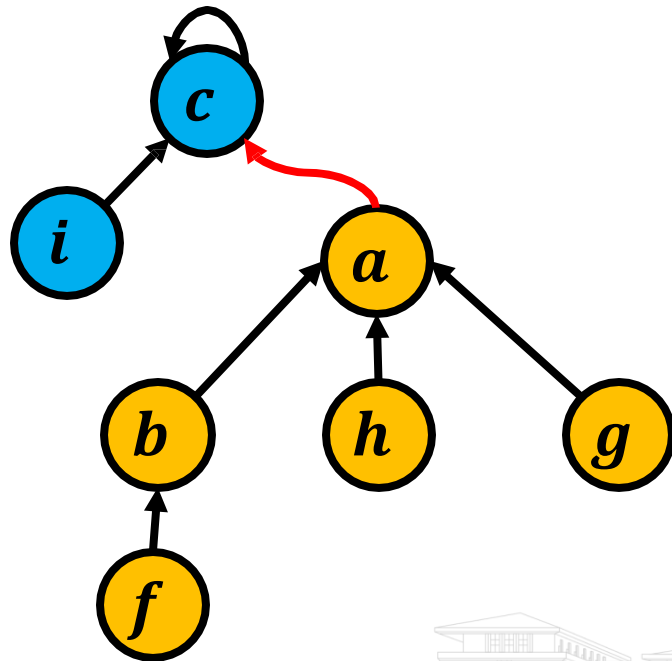


简单实现：找到两树根，任意连接两棵树

思考：是否高效？

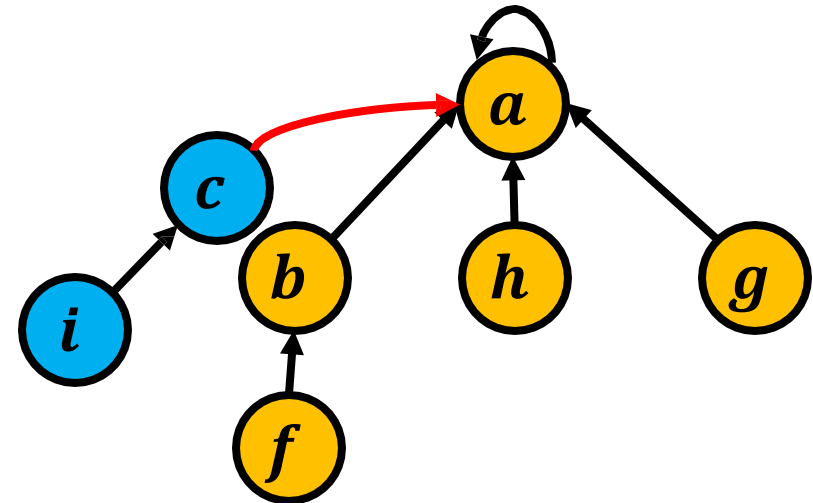
不相交集合

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同
- 合并集合：合并两棵树



简单实现：找到两树根，任意连接两棵树

尽可能降低树高度
提高树根查找效率



高效实现：树高小的树连接到树高大的树上

不相交集集合：伪代码

● Union-Set(x)

输入：顶点 x, y

$a \leftarrow \text{Find-Set}(x)$

$b \leftarrow \text{Find-Set}(y)$

if $a.\text{height} \leq b.\text{height}$ then

 if $a.\text{height} = b.\text{height}$ then

$b.\text{height} \leftarrow b.\text{height} + 1$

 end

$a.\text{parent} \leftarrow b$

end

else

$b.\text{parent} \leftarrow a$

end

找到两树根



不相交集合：伪代码

• Union-Set(x)

输入: 顶点 x, y

$a \leftarrow \text{Find-Set}(x)$

$b \leftarrow \text{Find-Set}(y)$

if $a.\text{height} \leq b.\text{height}$ then

 if $a.\text{height} = b.\text{height}$ then

$b.\text{height} \leftarrow b.\text{height} + 1$

 end

$a.\text{parent} \leftarrow b$

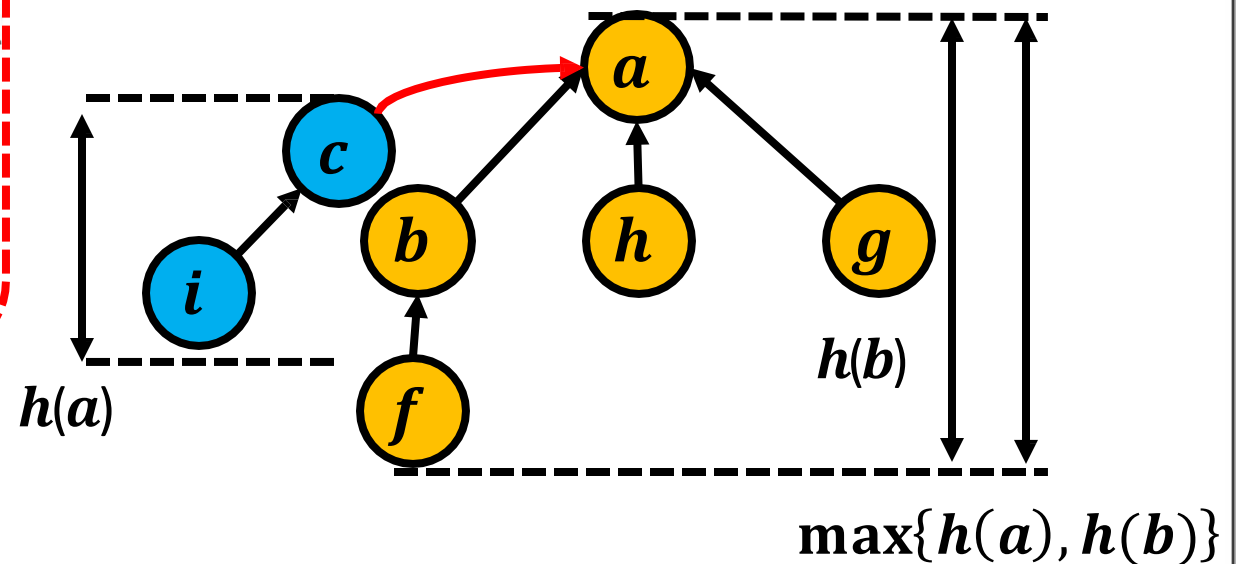
end

else

$b.\text{parent} \leftarrow a$

end

高度小的连到高度大的树



不相交集集合：伪代码

• Union-Set(x)

输入: 顶点 x, y

$a \leftarrow \text{Find-Set}(x)$

$b \leftarrow \text{Find-Set}(y)$

if $a.\text{height} \leq b.\text{height}$ then

 if $a.\text{height} = b.\text{height}$ then

$b.\text{height} \leftarrow b.\text{height} + 1$

 end

$a.\text{parent} \leftarrow b$

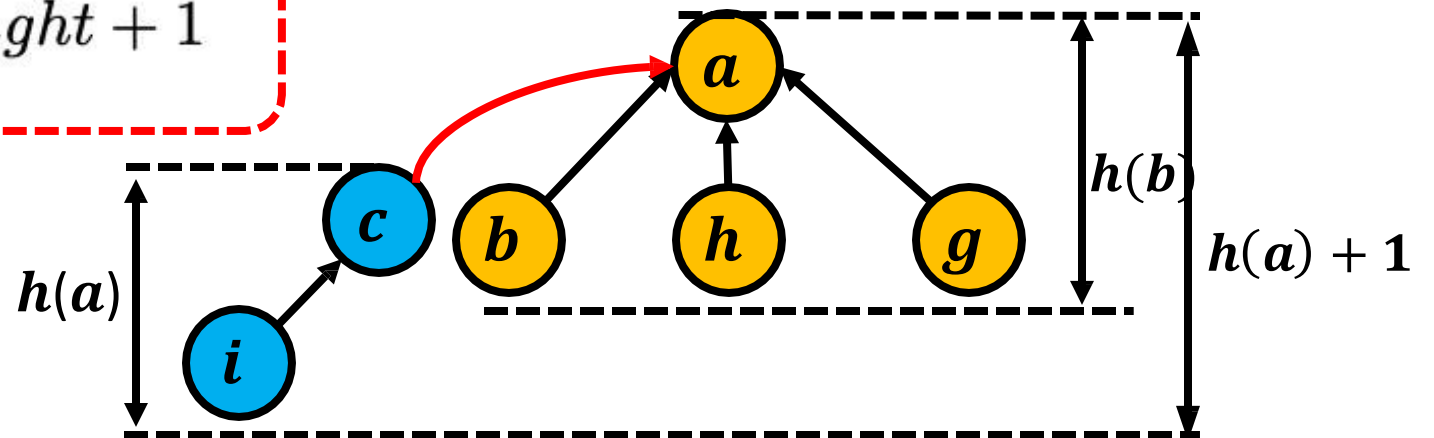
end

else

$b.\text{parent} \leftarrow a$

end

高度相同，需要更新





不相交集合：时间复杂度

● Create-Set(x)

输入: 顶点 x

输出: 不相交集合树

$x.parent \leftarrow x$

return x

时间复杂度 : $O(1)$

● Find-Set(x)

输入: 顶点 x

输出: 所属连通分量

while $x.parent \neq x$ do

$x \leftarrow x.parent$

end

return x

h 为树高

$O(h)$

时间复杂度 : $O(h)$



不相交集合：时间复杂度

● Union-Set(x)

```
输入: 顶点  $x, y$   
 $a \leftarrow \text{Find-Set}(x)$   
 $b \leftarrow \text{Find-Set}(y)$   
if  $a.\text{height} \leq b.\text{height}$  then  
    if  $a.\text{height} = b.\text{height}$  then  
        |  $b.\text{height} \leftarrow b.\text{height} + 1$   
    end  
    |  $a.\text{parent} \leftarrow b$   
end  
else  
    |  $b.\text{parent} \leftarrow a$   
end
```

$O(h)$

$O(1)$

时间复杂度： $O(h)$

思考：树的高度 h 和顶点规模 $|V|$ 有何关系？



不相交集合：时间复杂度

- 问题：树的高度 h 和顶点规模 $|V|$ 有何关系 $\longrightarrow |V| \geq 2^h$
- 归纳法证明！ 思考题
- $|V| \geq 2^h$ ，即 $h \leq \log |V|$
- **Create-Set(x): $O(1)$**
- **Find-Set(x): $O(h) = O(\log |V|)$**
- **Union-Set(x): $O(h) = O(\log |V|)$**

思考：能否继续优化？



伪代码

● MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if u, v 不在同一子树 then

$T \leftarrow T \cup \{(u, v)\}$

 合并 u, v 所在子树

 end

end

return T

问题：如何高效判定和维护顶点所在的子树？

伪代码

● MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

为每个顶点建立不相交集

创建不相交集

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if Find-Set(u) $\neq$ Find-Set(v) then

关系检查

$T \leftarrow T \cup \{(u, v)\}$

 Union-Set(u, v)

合并不相交集

 end

end

return T



复杂度分析

• MST-Kruskal(G)

输入: 图 G

输出: 最小生成树

把边按照权重升序排序

----- $O(|E|\log|E|)$

为每个顶点建立不相交集

----- $O(|V|)$

$T \leftarrow \{\}$

for $(u, v) \in E$ do

 if Find-Set(u) $\neq$ Find-Set(v) then

$T \leftarrow T \cup \{(u, v)\}$

 Union-Set(u, v)

 end

end

return T

$O(\log|V|)$

$O(|E|\log|V|)$

• 时间复杂度

- $O(|E|\log|E| + |E|\log|V|) = O(|E|\log|V|^2 + |E|\log|V|) = O(|E|\log|V|)$



Prim算法和Kruskal算法比较

	Prim算法	Kruskal算法
核心思想	保持一棵树，不断扩展	子树森林，合并为一棵树
数据结构	优先队列	不相交集合
求解视角	微观视角，基于当前点选边	宏观视角，基于全局顺序选边
算法策略	都是采用贪心策略的图算法	

